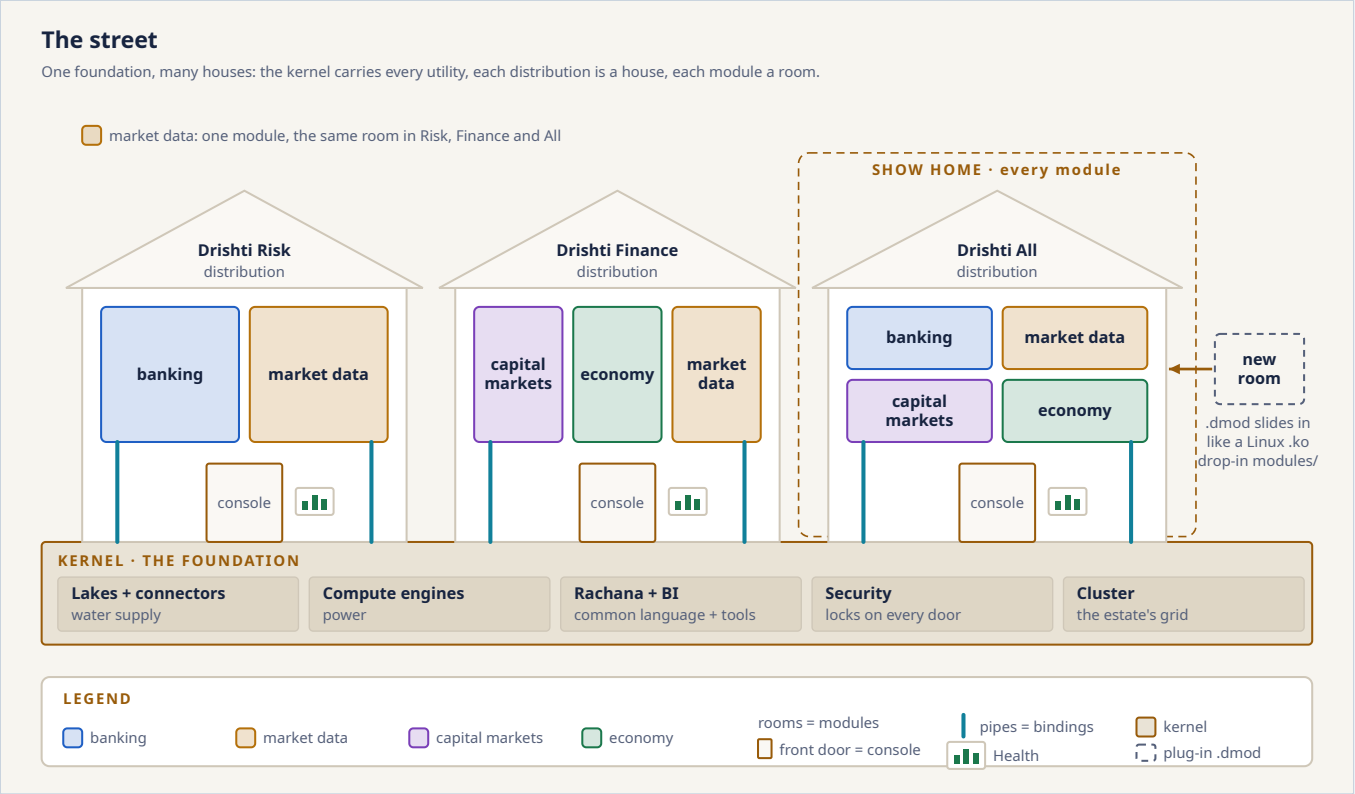
# How Drishti fits together

Drishti · HOW_IT_FITS.md · Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

How Drishti is built, in one document. **Part I** is a short tour for someone new to Drishti (an architect at a bank, a developer, an administrator): one idea per section, one picture per idea, one small real example each. **Parts II to VI** are the full design for those who build on it or run it: the architecture (sections A1 onwards), the data architecture (D1 onwards), the ViewModel contract (V1 onwards), the shared screens (S1 onwards) and the architecture decisions. Also as a PDF (HOW_IT_FITS.pdf).

## The house




Think of Drishti as a street of houses built on one foundation:

| In the house | In Drishti |
|---|---|
| **the foundation and its utilities** (water, power, a common language, locks, the estate's grid) | **the kernel**: lakes and connectors, compute engines, Rachana and BI, security, the cluster |
| **a house** | **a distribution** (Drishti Risk, Drishti Finance; Drishti All is the show home with every room) |
| **a room** | **a module** (banking, economy, capital markets); **market data** is one room shared by every house that needs it |
| **the front door** | **the console**, one per house |
| **a shelf** | **a workspace** (a lens): it arranges the books, never locks them; one book can stand on two shelves |
| **a book** | **a screen** (a page, a dashboard, a Sutra) |
| **a pipe** | **a data binding**: which connector brings each kind of data into the rooms |
| **the buildings on the estate** | **placement**: which nodes of the cluster run which part of a module |
| **a renovation with a permit** | **a deploy**: planned, approved by a second administrator, then done on every placed node together |
| **the manager's office** | **Admin** |
| **the dashboard by the door** | **Health** |
| **the keys** | **access rights** (who may enter which room and read which book) |

**The rule: every function has one room, every room one door, every setting one switch on one wall.** A command word belongs to exactly one module; the module is reached through one console; each setting is switched in one place (its file and its Admin page), never in two.

**The Linux parallel.** The same idea in software everyone knows:

| Drishti | Linux (and Tomcat) |
|---|---|
| the kernel | the Linux kernel |
| a module (`.dmod`) | a driver (`.ko`), built apart and checked against the kernel it loads into |
| a first-party module | an in-tree driver, shipped with the kernel's own source |
| a third-party module | an out-of-tree driver |
| a distribution | a distro: one kernel, a chosen set of packages, its own name |
| the `modules/` drop-in folder | Tomcat's `webapps/`: drop a package in and it deploys |
| the console | the desktop you sign in to |

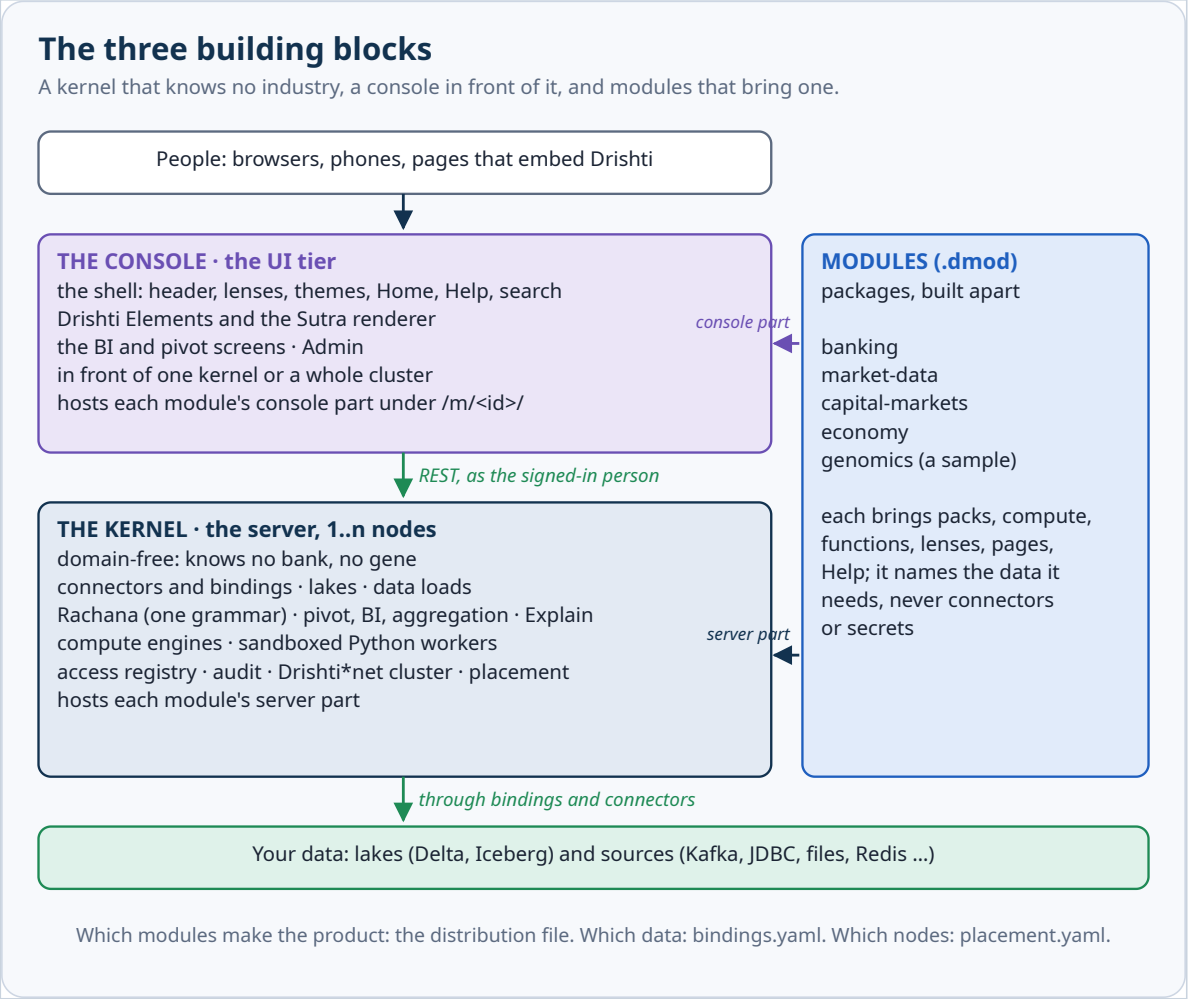
## Contents


1. The three building blocks
2. A request's journey
3. Modules
4. Data: bindings
5. Where things run: placement
6. A module's life
7. Distributions (white label)
8. The files and how they relate
9. Where to read more


The design in depth:

- Part II · Architecture: the view pipeline, one request end to end, Rachana and inference, packs, live updates, business dates, the API, the console, security, testing, one grammar and one engine, virtual tables, drishti-native, the pivot engine, compute engines and BI on the table layer
- Part III · Data architecture: live updates, history queries, drishti-iceberg, reactive connectors, the content store, the one Drishti database, loading, the lake file, the in-memory lake, the parallel loader, Arrow Flight, computed attributes and VaR sharding
- Part IV · The ViewModel contract: the JSON the server returns for every entity screen, its panels, live frames, masks, versions and JSON Schema
- Part V · Shared screens: one Build home, one Approvals queue, one Explain, the trader home, the controls screens and the editors
- Part VI · Decisions: the architecture decisions, one paragraph each

## 1. The three building blocks



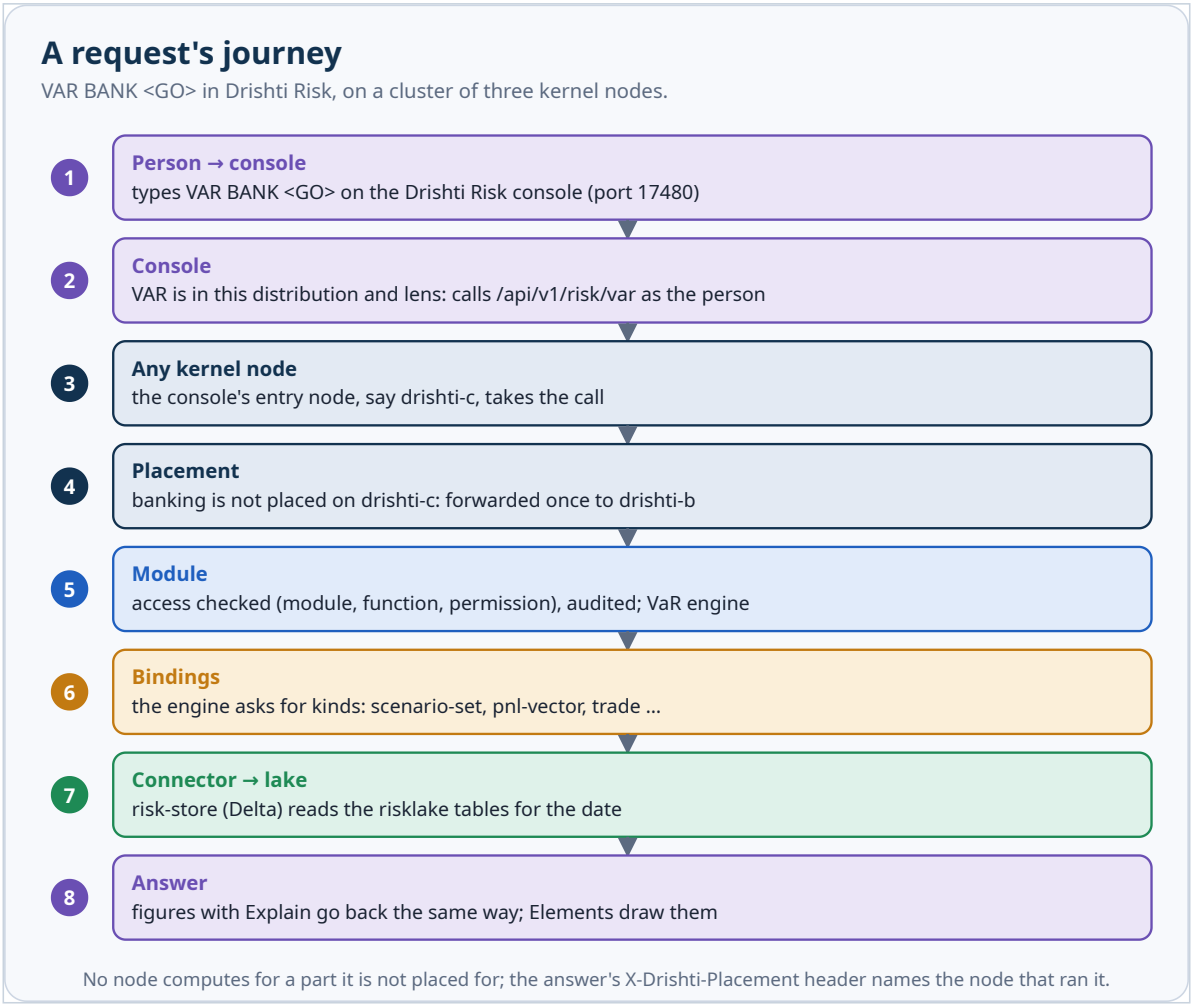

| Block | In one line | What it holds |
|---|---|---|
| **The kernel** | the server (Java 21, Spring Boot); one node or a cluster of many | connectors and lakes, data loads, Rachana (the one grammar), the pivot, BI and aggregation engines, Explain, the access registry, audit, the cluster (Drishti*net) and placement; it runs the **server part** of each module |
| **The console** | the UI tier, in front of one kernel or a whole cluster | the shell (header, lenses, themes, Home, Help, search), Drishti Elements and the Sutra renderer, the BI and pivot screens, Admin; it shows the **console part** of each module under `/m/<id>/` |
| **Modules** | `.dmod` packages, built apart from both | an industry: packs, compute, functions (command words), lenses, pages, Help |

The kernel knows no industry: it has no idea what a trade or a gene is. Modules bring that knowledge:

| Module | Brings |
|---|---|
| `banking` | VaR and ES, counterparty exposure and the pre-deal check, sensitivities, FRTB-SA, SIMM; the trading and risk packs |
| `market-data` | prices, curves, volatilities and FX; needed by banking |
| `capital-markets` (id `capitalmarkets`) | US equities (filings, facts, indices), funds and ETFs |
| `economy` | economies, central banks, official statistics, public-sector data and news |
| `genomics` | a sample third party's module: variants, genes, cohorts; it proves the kernel carries no finance |

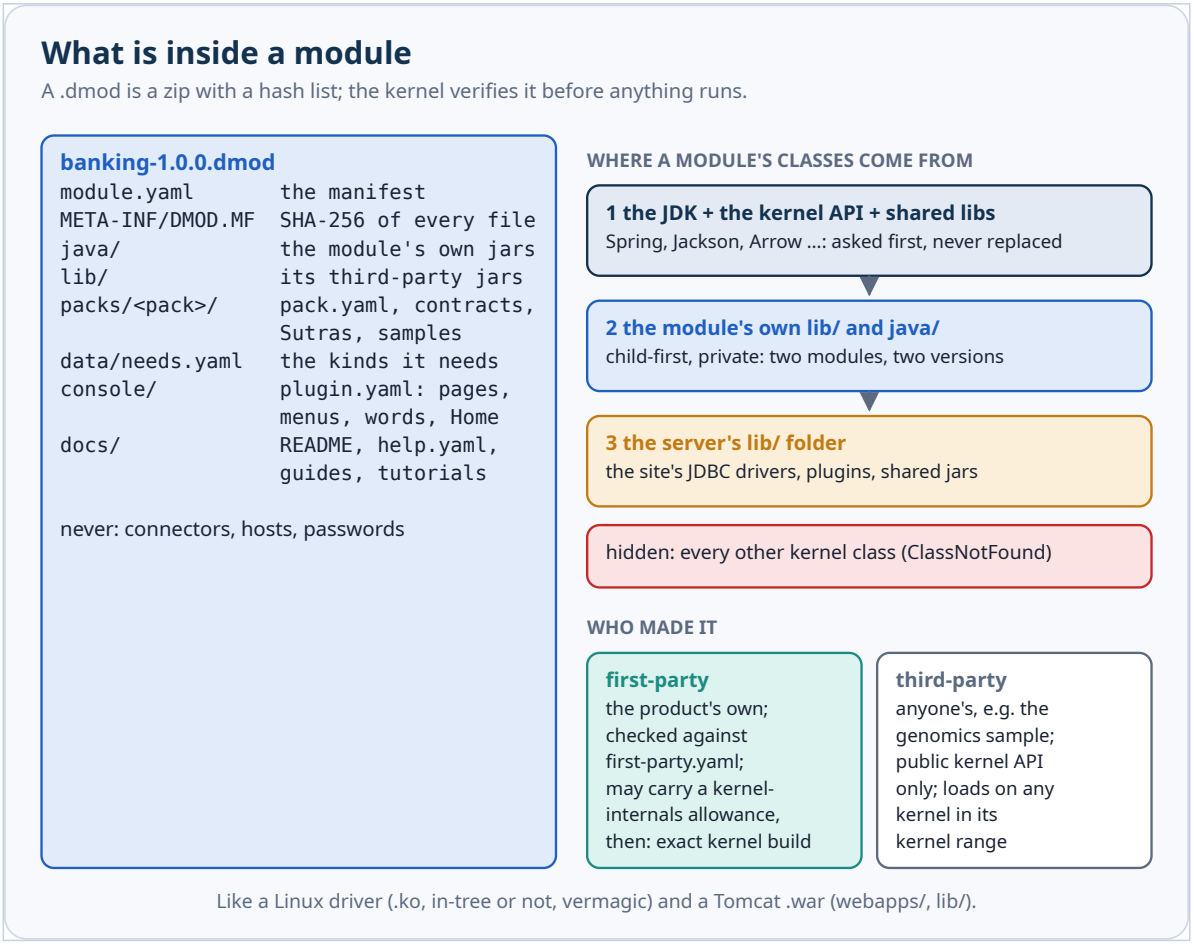
**Example.** `VAR BANK <GO>` is typed into a console, answered by banking's VaR engine running inside a kernel, from trade-level P&L the bank's own systems loaded into a lake.

## 2. A request's journey




1. **Person → console.** A risk manager types `VAR BANK <GO>` on the Drishti Risk console.
2. **Console → kernel.** The console knows `VAR` belongs to banking and is offered in this distribution and lens. It calls `/api/v1/risk/var` on its kernel entry node, **as that person** (the kernel decides what the person may see, never the console).
3. **Any node.** Any kernel node of the cluster may take the call; it need not be the one that computes.
4. **Placement.** If banking is not placed on this node, the call is **forwarded once** to a node where it is placed and up. The answer's `X-Drishti-Placement` header names the node that ran it. A node never computes for a part it is not placed for.
5. **Module.** The kernel checks that the module is on, the function is on and the person may use it, writes the audit line, and hands the call to banking's VaR engine.
6. **Bindings.** The engine asks for **kinds** (`scenario-set`, `pnl-vector` or `scenario-pv`, `trade` ...), never for a database. The kernel's bindings say which connector serves each kind.
7. **Connector → lake.** The connector (`risk-store`, a Delta connector) reads that day's tables of the lake (`risklake`).
8. **Answer.** The figures, with how each was calculated (Explain), go back the same way; the console draws them with Drishti Elements.

## 3. Modules




**What a `.dmod` is.** A zip with a fixed layout that anyone can produce:

| Entry | Holds |
|---|---|
| `module.yaml` | the manifest: id, version, origin, the kernel range it needs, the modules it requires, the kinds it provides, its functions, engines, packs, API prefix, console plugin |
| `META-INF/DMOD.MF` | the SHA-256 of every file; one changed byte refuses the whole module |
| `java/` and `lib/` | the module's own jars, and the third-party jars it carries (listed under `java.lib`) |
| `packs/<pack>/` | its packs: `pack.yaml`, the contracts of its kinds, Sutras (screens in YAML), samples, guides |
| `data/needs.yaml` | the kinds it reads, which are required and which optional |
| `console/plugin.yaml` | its console part: pages, menus, command words, Home cards |
| `docs/` | its README, Help sections, guides and tutorials |

**What a `.dmod` never holds:** a connector, a host name, a URL of a data source, a user name or a password. The packer refuses them.

**Checked before anything runs.** The kernel verifies the hash list and the manifest and runs fifteen conformance checks (C1–C15: no kernel internals, declared jars only, contracts for its kinds, nothing leaked after unloading, and the rest). A refused module is absent, never half loaded.

**First-party and third-party.**

- A **first-party** module is the product's own (banking, market-data, capital-markets, economy). Its claim is **verified, never trusted**: the server jar carries `first-party.yaml`, the id and hash-list hash of each, and only a module that matches counts as first-party.
- Only a first-party module may carry a **kernel-internals allowance** (`kernel-internals:` in its manifest): a short list of kernel packages it may still reach, which only ever shrinks. Banking has one. A module with an allowance loads only on the **exact kernel build** it was packed for (*built for kernel X, this is Y: install the matching module*).
- A **third-party** module (the genomics sample, or a vendor's) uses the public kernel API only and loads on any kernel inside its `kernel` range.

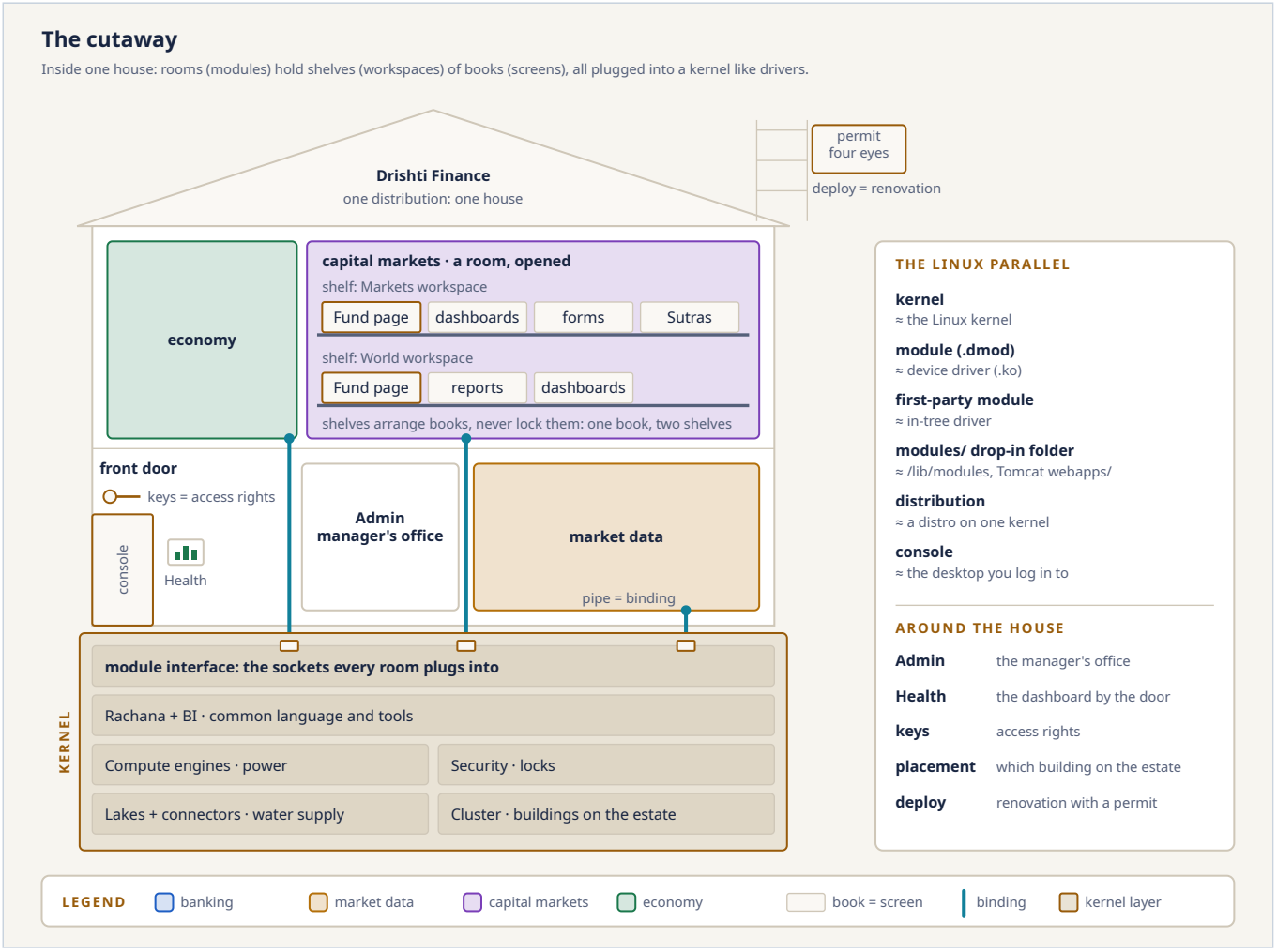
**Where a module's classes come from**, in this order:

1. **The JDK, the kernel API and the shared libraries** (Spring, Jackson, SLF4J, Arrow …): always asked first, never replaced by a module.
2. **The module's own jars** (`java/`, `lib/`): private to it, so two modules may carry two versions of one library.
3. **The server's `lib/` folder** (`drishti.lib.path`): jars the site adds for everyone (a JDBC driver, a connector plugin, a company library).

Every other kernel class is hidden from a module, except a verified first-party module's allowance.

**The Linux parallel, in detail** (the table is in The house): the driver's `vermagic` here is the module's kernel range, or the exact build for an allowance; the server's `lib/` folder is Tomcat's `lib/` (shared jars).

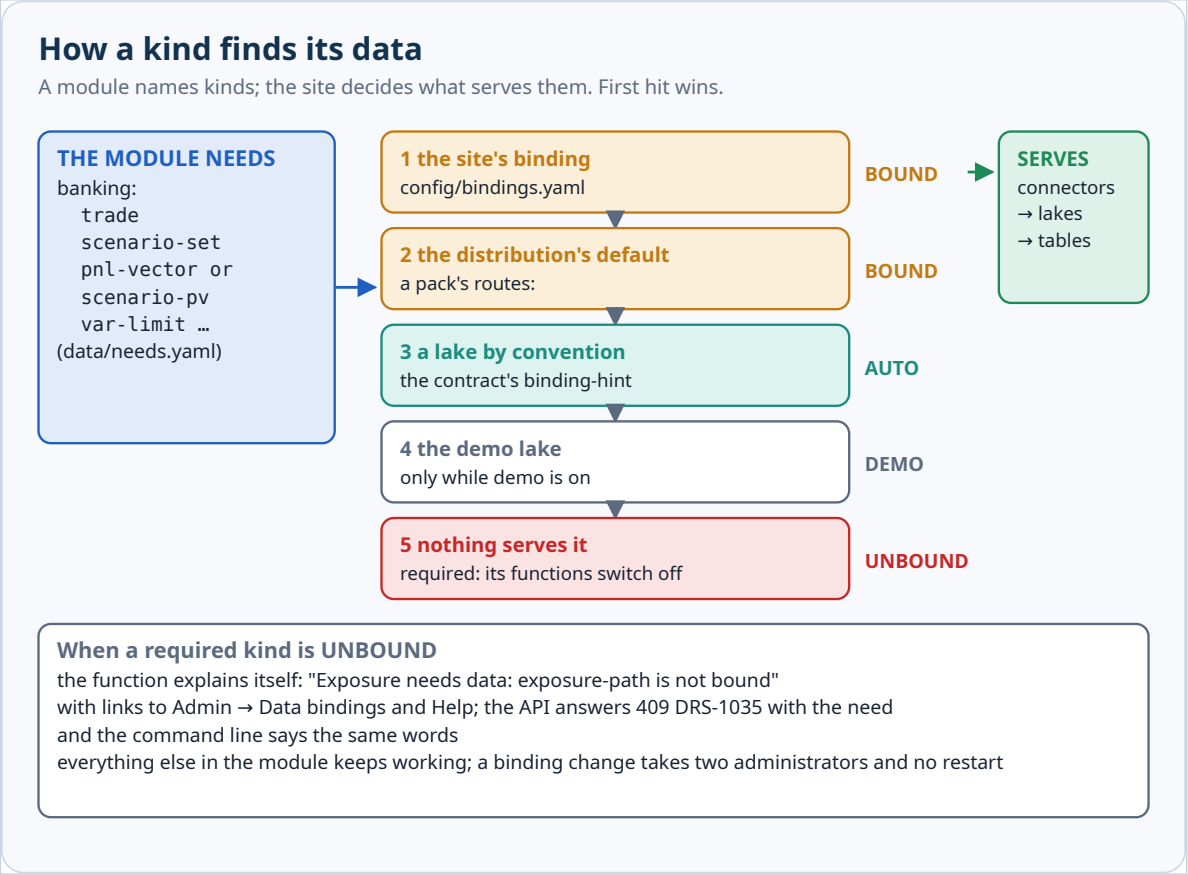
## Inside one house: rooms, shelves and books




A module (a room) brings its **screens** (books): pages, dashboards, forms and Sutras, in its packs and its console part. A **workspace** (a lens, a shelf) only arranges screens for one kind of person; it never owns or locks them, so the fund page stands on the Markets shelf and the World shelf at once. A screen gets its data through the room's **bindings** (pipes) from the kernel, and the person sees it only with the **keys**: the access rights checked on every call.

**Example.** `banking-1.0.0.dmod` says `origin: first-party`, `requires: market-data`, function `VAR` at `/api/v1/risk/var`, engines `var`, `exposure`, `frtb-sa`, `simm` and the packs `trading`, `market-risk`, `counterparty-risk`. The genomics sample says `origin: third-party` and brings its own VCF reader, kinds `variant`, `sample`, `gene`, `cohort` and a lollipop visual.

## 4. Data: bindings




A module says **what** data it needs (kinds); the site decides **where** it comes from. That decision is a **binding**, one per kind, shared by every module that needs the kind, in `config/bindings.yaml`:

```
format: 1
bindings:
  trade:
    read: [trade-endur, trade-calypso]        # connectors (config/connectors/<name>.yaml), in read order
    union: true                               # rows from all of them: one trade store
    live: trade-kafka                         # live changes from a Kafka connector
  scenario-pv:
    read: [risk-store]                        # a Delta connector on the risklake lake
  var-official:
    write: { lake: risklake, table: banking.var-official }   # where a produced kind is written
  pnl-vector:
    demo: true                                # the demo lake's samples, on purpose
```

A binding names connectors and lakes only. Hosts, URLs and credentials stay in the connector files and the secret store, and no module ever sees them.

**How a kind finds its data**, first hit wins:

| # | Source | State |
|---|---|---|
| 1 | the site's binding in `config/bindings.yaml` | `BOUND` |
| 2 | the distribution's default (a pack's `routes:`) | `BOUND` |
| 3 | a configured lake that holds the contract's `binding-hint` table | `AUTO` (confirm it) |
| 4 | the demo lake, only while demo is on | `DEMO` |
| 5 | nothing | `UNBOUND` |

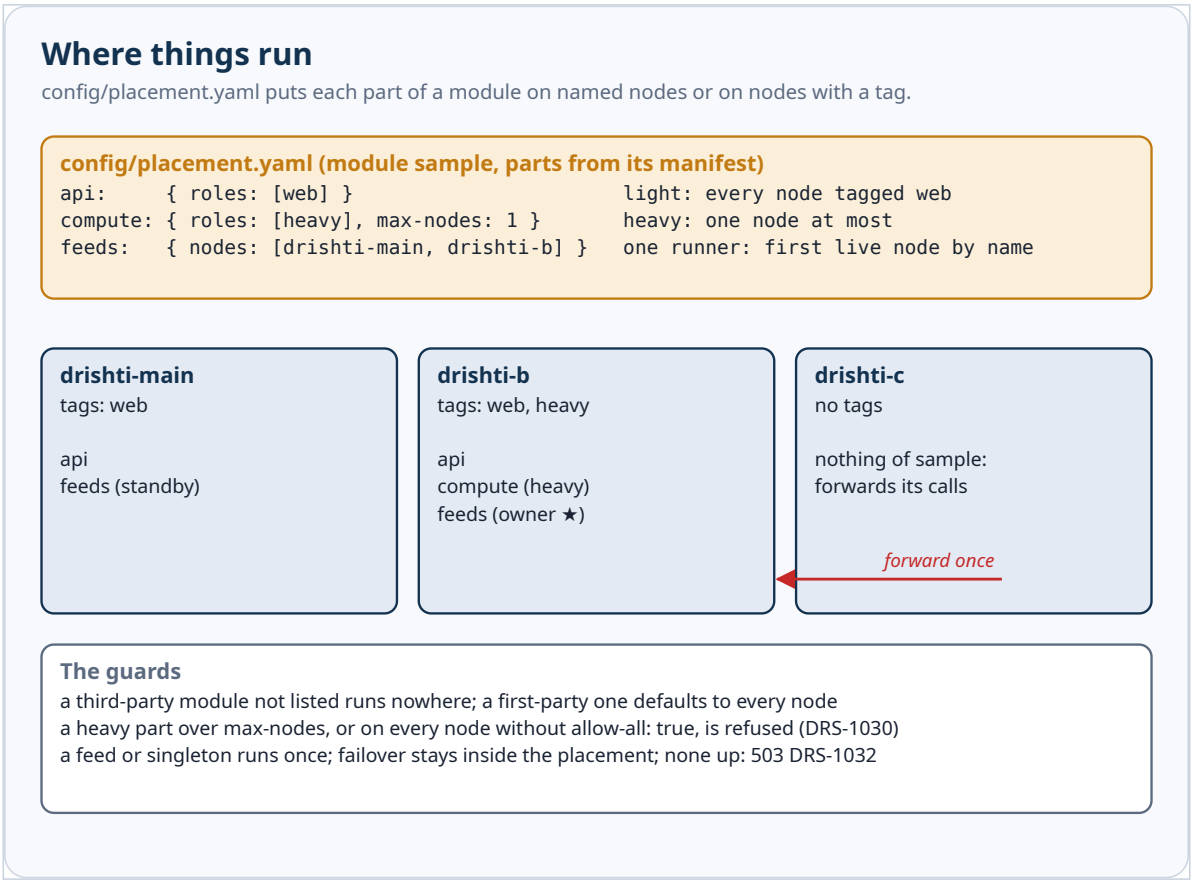
A bound kind with no load inside its contract's cadence is `STALE`; one whose source breaks the contract is `MISMATCH` (treated as unbound); a kind a module computes itself (`position`, `var-result`) is `COMPUTED` and never bound.

**Gated functions explain themselves.** When a **required** kind is unbound, only the functions that need it switch off, and they say why: the page is *Exposure needs data: exposure-path is not bound* with links to Admin → Data bindings and Help, the API answers `409 DRS-1035` with the need, the command line says the same words. The rest of the module keeps working. An optional kind that is unbound only hides its features (backtesting without `desk-pnl`).

**Changing a binding**: Admin → Data bindings or the file; one administrator proposes, a second approves; it applies without a restart.

---

## 5. Where things run: placement



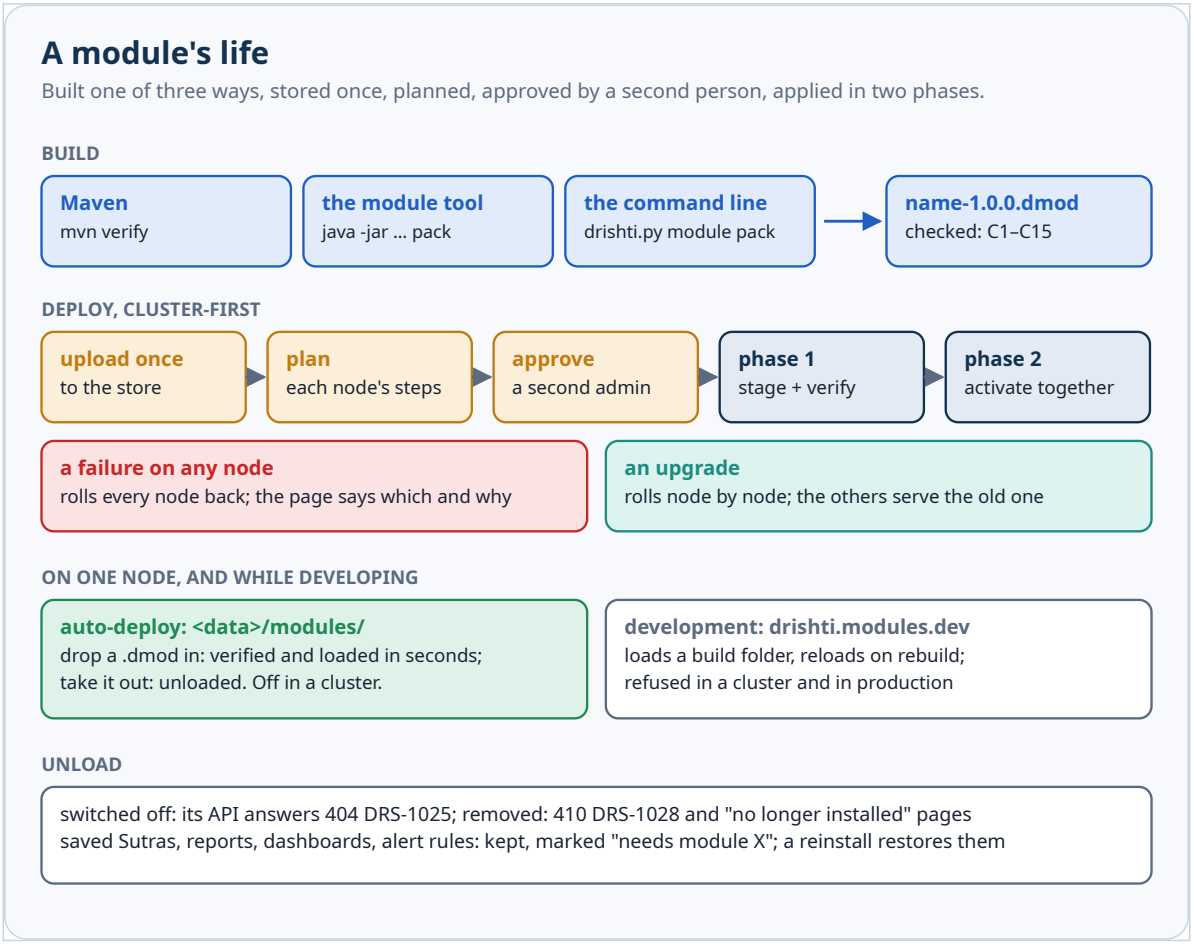

On a cluster, `config/placement.yaml` says which nodes run each **part** of a module. The module's manifest declares its parts and their **resource class** (`light`, `heavy` with its memory, `feed`, `singleton`); the site decides where they go, by node name or by node tag (`tags:` in `config/net/node.yaml`). A module that declares no parts has one, `api`, light.

```
format: 1
modules:
  sample:
    parts:
      api:     { roles: [web] }                    # every node tagged web
      compute: { roles: [heavy], max-nodes: 1 }    # heavy: one node at most
      feeds:   { nodes: [drishti-main, drishti-b] } # one runner, elected among these two
```

- **Not listed, not run.** A third-party module or part that the file does not name runs nowhere (*not placed*). A first-party module the file does not name runs on every node, as it always has.
- **Singletons run once.** A `feed` or `singleton` part has exactly one owner: the first live placed node by name. When it stops, the next placed node takes over, never one outside the placement.
- **Heavy work does not spread by accident.** A heavy part over its `max-nodes`, or on every node without `allow-all: true`, is refused (`DRS-1030`).
- **Forwarding.** A node where a part is not placed forwards the call once, for the same person, to a placed node that is up. With none up, the call answers `503 DRS-1032` (*unavailable*, not *absent*: nothing saved is touched).
- **Changing it**: Admin → Cluster → Modules: one administrator plans, a second applies; every member takes it at once, without a restart.

On a single node (no cluster) every part runs on that node: a module the file does not name is placed there by default, and Health notes *single node: placed here*.

## 6. A module's life




**Build, three ways, one packer.** The same packer and the same fifteen checks, reached from:

| Way | For | Command |
|---|---|---|
| Maven | a project made from the archetype `drishti-module-archetype` | `mvn verify` → `target/dmod/<id>-<version>.dmod` |
| The module tool | Gradle, Bazel, a script, a plain folder | `java -jar drishti-module-tool.jar pack <folder>`, then `check` and `inspect` |
| The command line | Linux, macOS and Windows with Java and Python only | `python3 tools/drishti.py module pack <folder>`, `module check`, `module inspect` |

**Deploy, cluster-first.**

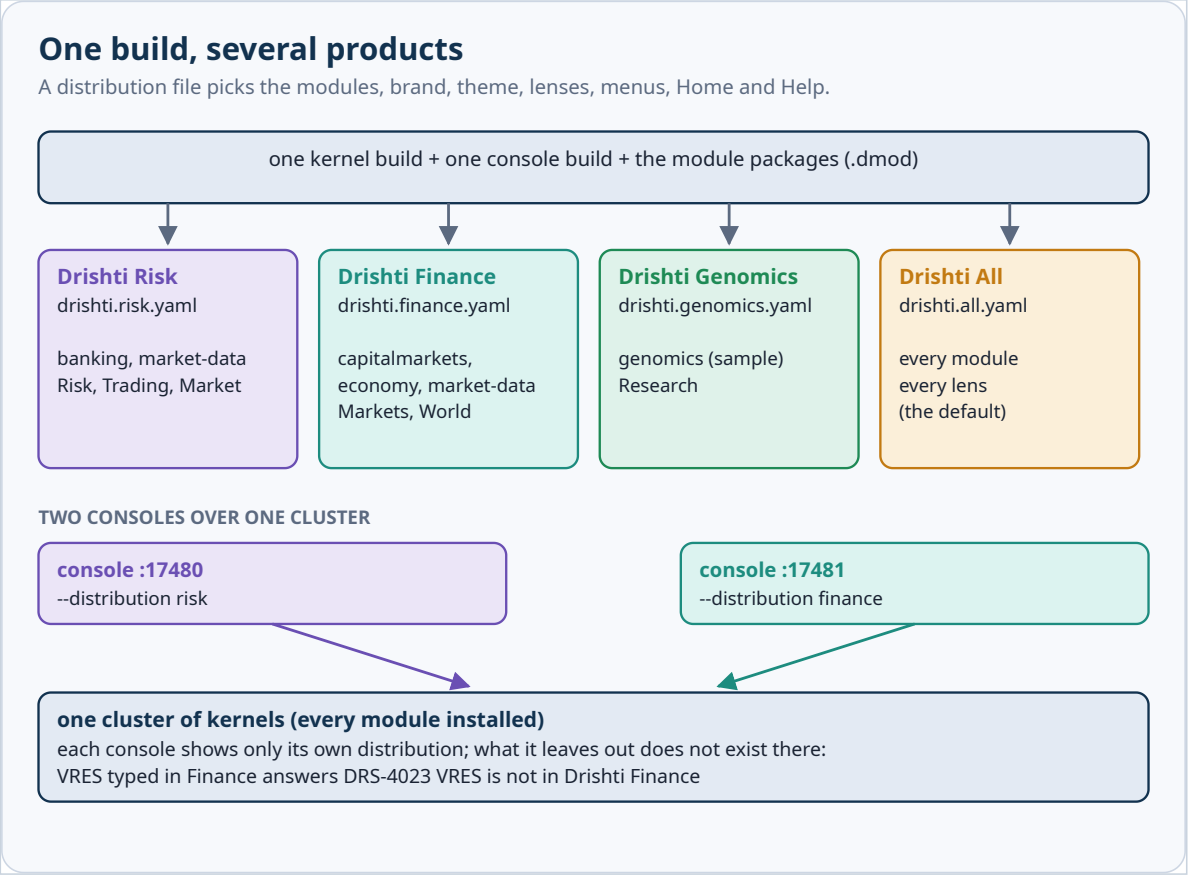
1. **Upload once** to the module store (Admin → Modules → Deploy, or `drishti.py module deploy <file>`). It is checked there once and kept by id and version; a version is never rebuilt with other bytes.
2. **Plan**: what each node does now and after; nodes where nothing is placed receive nothing.
3. **Approve and apply**: a **second administrator** applies the plan (`drishti.py module apply <id>`); the proposer cannot.
4. **Two phases**: every placed node receives the package, checks it itself and stages it; then all activate together. A failure on any node rolls every node back, and the page says which node and why.
5. **Upgrade**: a newer version is staged everywhere first, then switched **node by node** while the others serve the old one.

**Auto-deploy on one node.** Drop a `.dmod` into `<data folder>/modules/` and within seconds it is verified, installed and loaded; take the file out and the module is removed. It is on for a single node and off in a cluster, where deploys go through plan and apply.

**Development folders.** With `drishti.modules.dev` a developer's own server loads a module's build folder unpacked and reloads it on every rebuild; Health marks it *development (unverified)*. It is refused in a cluster and under the production profile.

**Unload and "needs module X".** Switched off, a module's API answers `404 DRS-1025`; removed, `410 DRS-1028` and its pages say it is no longer installed. Saved Sutras, reports, dashboards, baskets, searches and alert rules that use it are **kept**, marked *needs module X*; reinstalling restores them.

## 7. Distributions (white label)




A **distribution** is one file that turns the kernel into a product: its modules, brand, theme, lenses, menus, Home and Help. One kernel build and one console build make all of them.

| Distribution | For | Modules | Start |
|---|---|---|---|
| **Drishti Risk** | traders, risk analysts and managers | banking, market-data | `bin/start-all.sh --distribution risk` |
| **Drishti Finance** | investment and research analysts, economists | capital-markets, economy, market-data | `bin/start-all.sh --distribution finance` |
| **Drishti Genomics** | the sample that proves the kernel is domain-free | genomics | `bin/start-all.sh --distribution genomics` |
| **Drishti All** | developers, QA, demos; the default | every module | `bin/start-all.sh` |

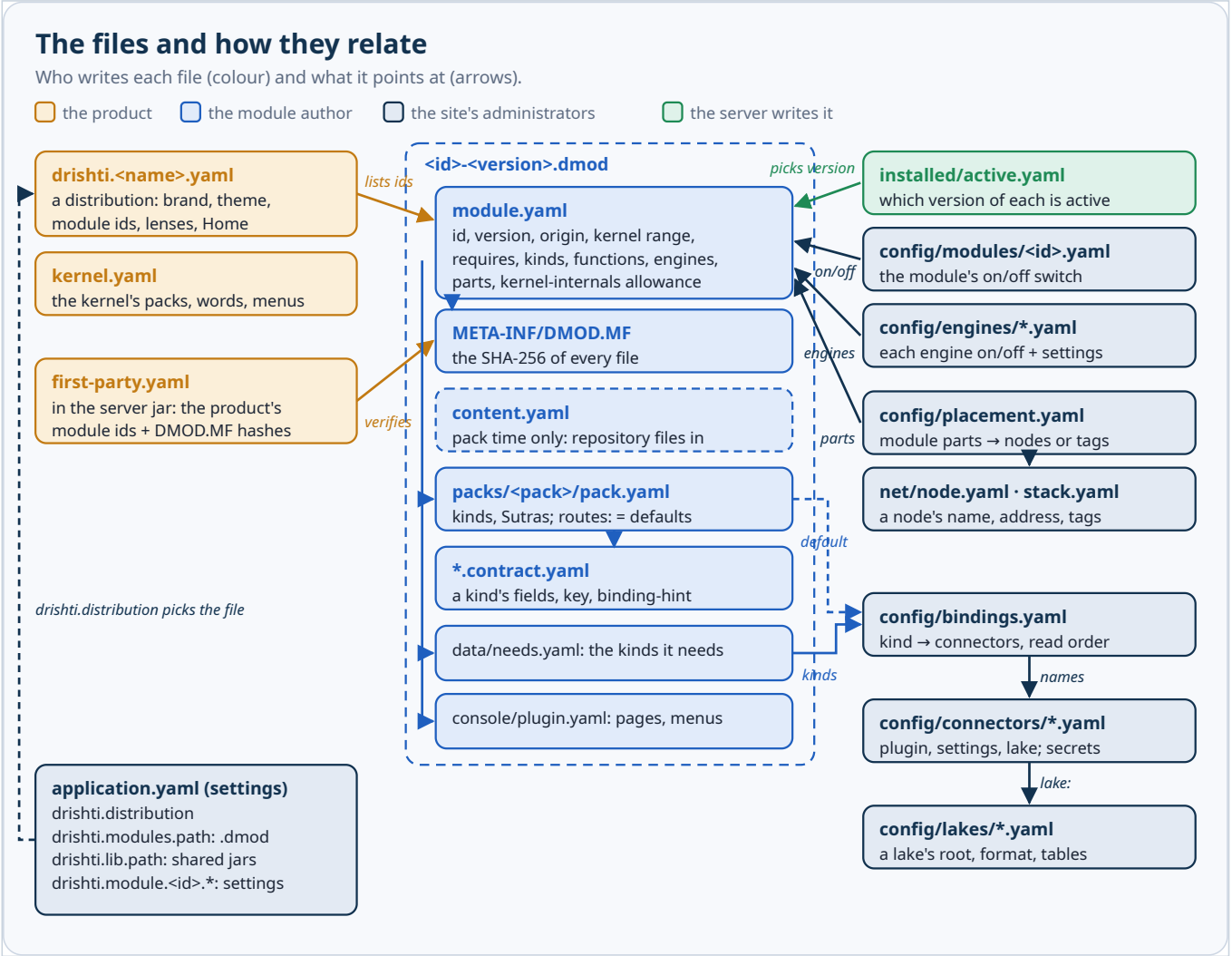
`config/distributions/drishti.risk.yaml`, abridged:

```
format: 1
distribution: drishti.risk
brand:
  product: Drishti Risk
  tagline: VaR, exposure and limits from your own pricing data
theme: { tokens: terminal-blue, mode: dark, density: compact }
modules:
  - { id: banking,     required: true }
  - { id: market-data, required: true }
lenses:
  offered: [risk, trading, market]
  default: risk
home:
  cards: [VAR BANK, VAR DESK RATES, "VRES DESK:RATES", "EXP CPTY:CP-ALDERSHOT"]
docs: { title: Drishti Risk Help }
```

What a distribution leaves out **does not exist in it**: not on the command line, in menus, Home, Help or the catalogue the AI reads. `VRES` typed in Drishti Finance answers *DRS-4023 VRES is not in Drishti Finance*.

- **Choosing one**: `--distribution <name|path>` on `bin/start-all`, `start-server` and `start-console` (`.sh`, `.ps1`, `.cmd`), else `DRISHTI_DISTRIBUTION`, else `drishti.distribution`. On Windows: `bin\start-all.ps1 -Distribution risk`.
- **Two consoles over one cluster**: a console started with its own distribution keeps it whatever the servers run. A stack file's `consoles:` list (`{ name: risk, port: 17480, distribution: risk }`, `{ name: finance, port: 17481, distribution: finance }`) gives Drishti Risk and Drishti Finance side by side over the same kernels.
- **Your own brand**: copy a shipped file to a new `drishti.acme.yaml` beside it in `config/distributions`, change `distribution:`, `brand:`, `theme:`, `modules:`, `lenses:`, `menus:`, `home:` and `docs:`, and start with `--distribution acme`. A distribution names modules; it never contains code, connectors or credentials, and it cannot widen the licence. Admin → White label shows every distribution side by side.

## 8. The files and how they relate




### 8.1 Every file, in one table

Who writes it, when it is read, what it decides and where an administrator sees it.

| File | Written by | Read | Decides | Admin page |
|---|---|---|---|---|
| **Inside a `.dmod`** | | | | |
| `module.yaml` | the module author | at verification and load; by the distribution reader | id, version, origin, kernel range, required modules, kinds, functions, engines, packs, API prefix, parts, the allowance | Modules (detail), `module inspect` |
| `META-INF/DMOD.MF` | the packer | at every verification, on every node | the SHA-256 of every file in the package | Modules (verify report) |
| `content.yaml` | the module author | **at pack time only**, by the packer | which repository files (packs without their connector templates, lenses, tutorials) go into the package | none |

| File | Written by | Read | Decides | Admin page |
|---|---|---|---|---|
| `packs/<pack>/pack.yaml` | the module author | when packs load | the pack's kinds and Sutras; its `routes:` are the distribution's default bindings | Packs |
| `packs/<pack>/contracts/*.contract.yaml` | the module author | when packs load; at binding checks | a kind's fields, key, cadence and `binding-hint` | Data bindings (state) |
| `data/needs.yaml` | the module author | at load; at every binding resolution | the kinds the module needs, required or optional | Data bindings |
| `console/plugin.yaml` | the module author | by the console when the module is active | the module's pages, menus, command words, Home cards | Modules |
| **The product** | | | | |
| `first-party.yaml` (inside the server jar) | the product build (`modules/dist`) | at verification | which modules are the product's own (id + hash-list hash), so which may use an allowance | Health, `module inspect` (origin) |
| `config/distributions/drishti.<name>.yaml` | the product; a site for its own brand | at start, by the kernel and the console | brand, theme, modules, lenses, menus, Home, Help title | White label (read only) |
| `config/distributions/kernel.yaml` | the product | at start | the kernel's own packs, words and menus, in every distribution | White label |
| **The site** | | | | |
| `config/bindings.yaml` | the data administrator (four eyes) | at start, then re-read (`drishti.bindings.refresh-seconds`, 30) | which connectors or lake table serve each kind | Data bindings |
| `config/placement.yaml` | the cluster administrator (plan, then a second applies) | at start; at once on apply, on every member | which nodes run each part of each module | Cluster → Modules |
| `<data folder>/modules/installed/active.yaml` | **the server** (deploy, auto-deploy, upgrade, rollback) | at start and at each activation | which installed version of each module is active | Modules |
| `config/modules/<id>.yaml` | the administrator | at start | the module's on/off switch (`drishti.modules.<id>.enabled` wins over it) | Modules (switch off / on) |
| `config/connectors/*.yaml` | the data administrator; pack templates write first copies | watched; a change applies without a restart | a source: plugin, settings, lake; credentials only as secret references | Connectors |
| `config/lakes/*.yaml` | the data administrator | with the connectors that use the lake | a lake's format, root and tables | Lakes |
| `config/engines/*.yaml` | the product's defaults; the administrator | at start; the switch from the Admin page | each compute engine on or off, and its settings (VaR confidence, window …) | Engines |
| `config/net/node.yaml` | the administrator of each node | watched | this node's name, real address, tags (the `roles:` placement matches) | Cluster → Files |
| `drishti-stack/stack.yaml` | the operator | by `bin/start-cluster.sh` | the nodes to start, their ports and heaps, the consoles and each one's distribution | `bin/status.sh` |
| `application.yaml` and settings | the operator | at start | `drishti.distribution`; `drishti.modules.path` (the `.dmod` folder), `drishti.modules.autodeploy`, `drishti.modules.dev`; `drishti.lib.path` (shared jars); `drishti.module.<id>.*` (a module's own settings) | Configuration (Help → Configuration) |

## 8.2 How they point at each other

- A **distribution** lists **module ids** → each id resolves to an installed `.dmod`, and `active.yaml` picks its version.
- `first-party.yaml` verifies the module's origin against its `DMOD.MF`, which hashes every file; only then may the `module.yaml` allowance be used.
- `module.yaml` names its **packs** (whose **contracts** define the kinds), its `data/needs.yaml` and its `console/plugin.yaml`.

- The kinds in `needs.yaml` are mapped by `bindings.yaml` to **connectors** (or, unbound, by a pack's `routes:` default), and each connector names its **lake**.
- `placement.yaml` names the **parts** declared in `module.yaml` and puts them on **nodes** known by their `node.yaml` names and tags (in a local cluster, the **stack file** creates those nodes).
- `config/modules/<id>.yaml` switches the module, and `config/engines/*.yaml` switches each engine it declares.
- `application.yaml` chooses the distribution and the folders everything else is found in.

### 8.3 One trace: `VAR BANK` in Drishti Risk

Every file the request touches, from start to answer:

| # | File | What it contributes |
|---|---|---|
| 1 | `drishti-stack/stack.yaml` or `bin/start-all.sh -- distribution risk` | starts the server and a console with the distribution `risk` |
| 2 | `config/distributions/drishti.risk.yaml` (+ `kernel.yaml`) | brand *Drishti Risk*, modules `banking` and `market-data`, the Risk lens, the Home card `VAR BANK` |
| 3 | `<data folder>/modules/installed/active.yaml` | banking `1.0.0` is the active version (the full server jar installs the shipped `.dmod` at the first start) |
| 4 | `META-INF/DMOD.MF` + `first-party.yaml` | the package is intact and is the product's own, so banking's allowance is honoured on this exact kernel build |
| 5 | `config/modules/banking.yaml` | `enabled: true` |
| 6 | banking's `module.yaml` | requires `market-data`; function `VAR` at `/api/v1/risk/var`; engine `var`; packs `market-risk` and the others |
| 7 | `config/engines/var.yaml` | the VaR engine is on; confidence 0.99, ES 0.975 |
| 8 | `config/placement.yaml` | banking is first-party: not named, so it runs on every node; named, only where placed (others forward) |
| 9 | banking's `console/plugin.yaml` | the console's `VAR` page (route `var`, function `VAR`) |
| 10 | banking's `data/needs.yaml` | VaR needs `scenario-set`, `trade` and `pnl-vector` or `scenario-pv` |
| 11 | `packs/market-risk/contracts/scenario-pv.contract.yaml` | the kind's key (`SCENARIO_PV_ID`), cadence (daily), `binding-hint` (`risk.scenario-pv`) |
| 12 | `config/bindings.yaml`, else `market-risk`'s `routes:` | `scenario-pv` → the connector `risk-store` |
| 13 | `config/connectors/risk-store.yaml` | plugin `delta`, its lake |
| 14 | `config/lakes/risklake.yaml` | format `delta`, root `<data folder>/delta/risklake`, the tables |

The answer goes back through the kernel to the console, which draws it; Explain on any figure shows how it was calculated.

## 9. Where to read more

| To understand | Read |
|---|---|
| the full design of the kernel, modules, loading, data needs, placement and the distribution file | KERNEL_AND_MODULES.md |
| each distribution, what is shared, and how to start it | DISTRIBUTIONS.md |
| building a module: the SDK, the archetype, the three ways to pack, the genomics sample step by step | DEVELOPER_GUIDE.md section 1.6 |
| modules on or off, the `modules/` and `lib/` folders, data bindings, white label, Admin → Modules and Deploy | ADMIN_GUIDE.md section 22 to section 25 (docs/guides/ADMIN_GUIDE.md#25-modules-and-deploy) |
| several servers as one, placement and deploying across a cluster | CLUSTER_GUIDE.md section 1.9 and 1.10 (docs/guides/CLUSTER_GUIDE.md#110-deploying-a-module) |
| lakes, connectors and the bindings file in detail | DATA_LAKE_GUIDE.md, Data bindings |
| installing, starting and checking each part on Windows, Linux and macOS | DEPLOYMENT_AND_QA_GUIDE.md |
| every setting | CONFIGURATION.md |

## Part II · Architecture

> **Any data. Any domain. One grammar.** Drishti (दृष्टि, "sight") turns any record from any system into a dense, keyboard-driven, live terminal view — without a hand-built screen per product.

**New to Drishti?** Section A5 follows one command from the keyboard to a live screen, and ends with where to change what. Section A7a shows, with real outputs, how every screen is drawn by the same generic code.

## A1. Problem and intent

Front-office, risk and operations staff look at hundreds of entity kinds: interest rate swaps, FX swaps, listed futures, netting sets, CSAs, counterparties, curves. Each one traditionally gets a bespoke screen, written once and rotting forever. The data changes shape faster than screens can be rebuilt.

Drishti inverts this. A screen is **not code** — it is the result of applying a layout written in the declarative screen grammar **Rachana** (a **Sutra**) to a data document, with an **inference engine** filling whatever the grammar does not say. A brand-new product type with no Sutra at all still renders a usable view on day one; a Sutra makes it beautiful.

The four reference mockups (`docs/requirements/drishti-*.png`) define the target. Their entities and Sutras ship in the `finance` pack (`examples/packs/finance/`), so they open only on a server that enables it (`DRISHTI_PACKS_EXTRA_DIRS=examples/packs DRISHTI_PACKS=finance`; it is a retired example pack):

| Mockup | Command | Sutra | Source |
|---|---|---|---|
| Interest rate swap | `TRD IRS-48213 <GO>` | `irs-vanilla v3` + inference | `aero-risk`, live |
| FX swap | `TRD FXS-20931 <GO>` | `fx-swap v2` + inference | `aero-fx`, live |
| Commodity future | `TRD CFT-77120 <GO>` | `listed-future v1` + inference | feed file `eod-futures` |
| Netting set | `NSET NS-NORTH-01 <GO>` | `netting-set v1` | `aero-risk` exposure |

### A1.1 Goals

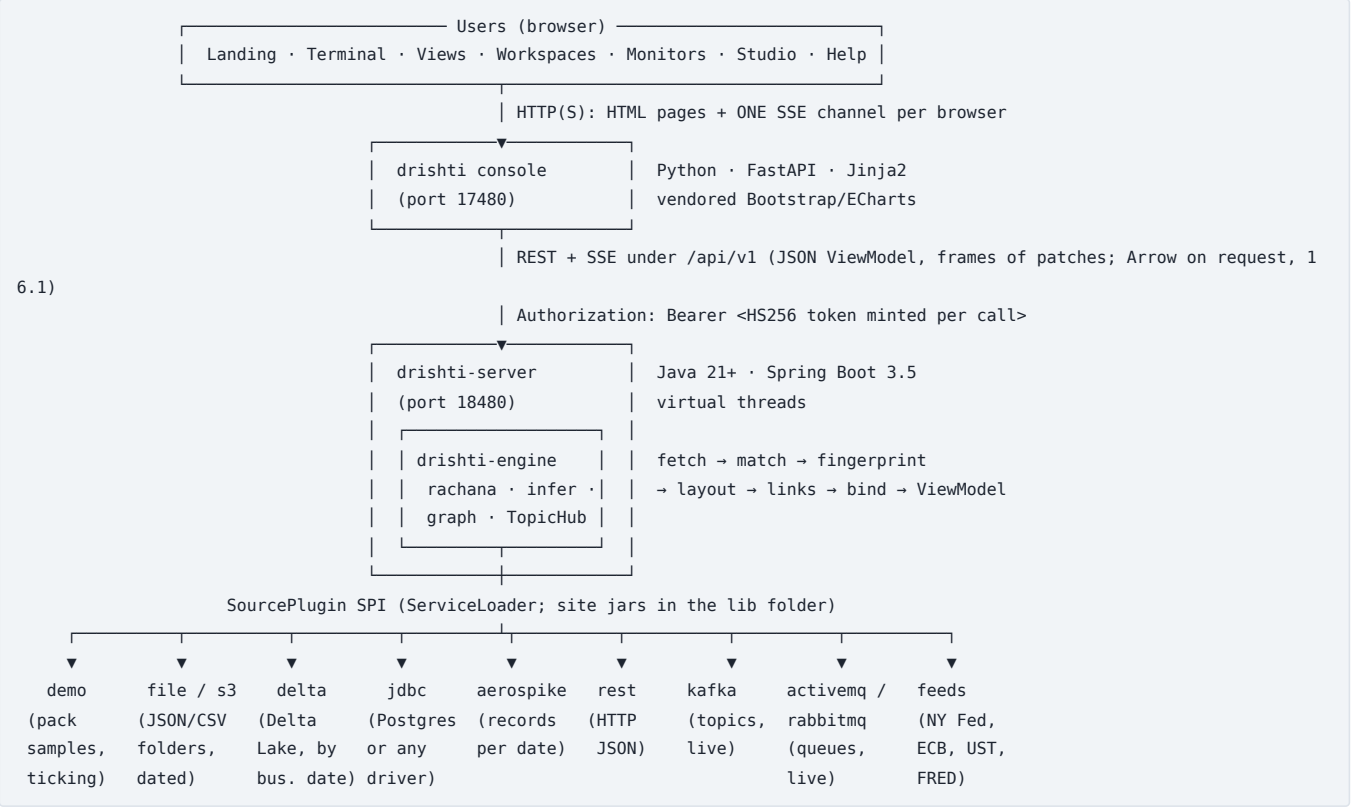
1. **G1 One grammar** — every view in the product is a Sutra (explicit or inferred).
2. **G2 Any source** — data arrives through a plugin SPI (Delta Lake, JDBC, Aerospike, REST, files, S3, Kafka, ActiveMQ, RabbitMQ, public data feeds).
3. **G3 Linked** — every identifier that names another entity is navigable (`Linked entities`, breadcrumbs, `Alt+←`).
4. **G4 Live** — views tick; p99 end-to-end view latency is measured and shown in the top bar.
5. **G5 Provenance** — every view shows *how it was built*: Sutra + version, data fingerprint, source + generation.
6. **G6 Keyboard first** — command line (`<MNEMONIC> <ID> <GO>`), function keys per view, `F9 Raw JSON` always.

### A1.2 Non-goals (v1)

- Trade capture / booking (Drishti is read-only).
- Pricing or risk calculation on the server — values (MTM, DV01, PFE) come from sources. A user may compute on a view in the browser with Calc (USER_GUIDE.md); a server-side QuantLib service is proposed, not built (AI_GUIDE.md).
- A free-form page designer. Screens stay Sutras; the Build workbench and the Sutra Studio edit them with a preview, and the BI report designer lays out report pages (HOW_IT_FITS.md §A28).

---

## A2. System context



Two things sit beside this picture. With **Drishti*net** switched on (section A13a), several such servers gossip with each other, each with its own sources, and any of them answers any request; the console is given one server and learns the others. And every Python run (Transforms, python visuals, Python-driven tables, Notebook cells) is the server's, on its compute pool `python` in a strict sandbox (BI_GUIDE.md section N19); the console runs none.

**Deployment model — a departure from Pravaha.** The Drishti backend is always a **Spring Boot application**; it is never shipped as an embeddable library. The Maven modules are internal building blocks of that one application, each contributing its beans through a module `@Configuration` class and `@ConfigurationProperties` records. Domain classes (grammar, inference, pipeline) remain plain, constructor-injected Java objects so they unit-test without a Spring context, but Spring is a first-class, permitted dependency everywhere. The UI is a separate **Python console** laid out the way MAYA's web layer is.

---

## A3. Core concepts

| Concept | Meaning |
|---|---|
| **Entity** | Any addressable record: `(kind, id)` → e.g. `(trade, IRS-48213)`, `(netting-set, NS-NORTH-01)`. |
| **Mnemonic** | Command-line alias for a kind: `TRD`, `NSET`, `CPTY`, `CSA`, `AGR`, `CRV`, `BOOK`. Declared by packs (`mnemonics:` in `pack.yaml`); sites add more under `drishti.commands.mnemonics`. |
| **DataNode** | Immutable, source-neutral tree (object / array / scalar) holding the entity document. |
| **Source** | A plugin (`SourcePlugin`) that can `fetch`, and optionally `subscribe`, `search` and `reverse`; every document carries its *generation* (monotonic version). A **connector** is a named instance of a plugin with its own settings (`market-store` is a `delta` connector); it is a site resource kept as one file, `config/connectors/market-store.yaml`, that the registry watches and applies live, one connector at a time (see ../guides/DATA_GUIDE.md section 9). |
| **Fingerprint** | Stable hash of a document's **shape** (keys + types, not values or array lengths), shown as e.g. `b7df…1372`. Keys the layout cache. |
| **Rachana** | The declarative screen grammar (रचना, *composition*): so no product ever gets its own coded screen. |
| **Sutra** | One layout written in Rachana, versioned (`irs-vanilla v3`): header strip, panels, bindings, formats, keys, links. |
| **Inference** | Rule engine that derives panels from shape when a Sutra is absent or partial ("Sutra X + inference"). |
| **Layout** | Resolved, data-free plan = Sutra ⊕ inferred panels. Cached per `(Sutra version, kind, fingerprint)`. |
| **ViewModel** | Layout bound to values: what the console renders. Serialisable JSON. |
| **Patch** | Minimal change to a ViewModel: one strip cell (`op: strip`), one whole panel (`op: panel`) or the provenance. Patches travel in **frames** over SSE when the source ticks. |
| **Link** | A reference from a field to another entity, resolved through the entity graph. |

---

## A4. The view pipeline

Every view, live or not, is built by `ViewPipeline` (`drishti-engine`) in the same stages. The stages work on immutable inputs, so they are safe to run concurrently and their results cache well.

```
"TRD MX-20000001 <GO>"
       |
① Command    CommandParser + Mnemonics → EntityRef(kind=trade, id=MX-20000001)              ~µs
② Fetch      SourceRouter → EntityDocument (DataNode + Provenance)                      I/O, virtual thread,
                                                                                       drishti.sources.fetch-timeout (10s)
③ Match      SutraMatcher → highest-priority Sutra of the kind whose `where` holds (or none)   ~µs
④ Shape      ShapeFingerprinter → Fingerprint, cached per (entity, generation, date)     ~µs–ms
⑤ Layout     LayoutMerger: cache[(Sutra version, kind, fingerprint)] else Sutra ⊕ inference   cold ms, warm µs
⑥ Links      ReferenceCatalog.discover + chart sources → SourceRouter.fetchAll            I/O in parallel,
                                                                                       drishti.graph.link-budget (40ms)
⑦ Bind       Binder: every panel bound in parallel on the bind pool; strip and title     ~ms
       |
  ViewModel {ref, mnemonic, title, strip, panels, keys, provenance, timings}
       └─► REST response  /  the first `view` event of a live stream
```

The ViewModel's every field, panel kind and live patch: Part IV · The ViewModel contract.

Notes on each stage, checked against the code:

- **Fetch order.** `SourceRouter` tries the plugin routed for the kind (`drishti.sources.routes`), then `default-route`, then every other plugin that serves the kind. For **Live**, plugins that stream come first (and a real stream before the `demo` samples); for a **picked business date**, dated plugins come first. The first one that holds the entity answers. A dated plugin that holds the date (`SourcePlugin.coverage` is `HELD`) is authoritative for it: an entity it does not list is not held and the plugins behind it are not asked (searches list the kind only up to it). Nobody serving the kind is `DRS-1002`; nobody holding it is `DRS-1001`; a source error is `DRS-1003`; running out of time is `DRS-1004`; a *known at* read of a dated plugin without time travel (`SourcePlugin.timeTravel`) that may hold the date is `DRS-1007`.

- **Links before binding.** Linked entities (counterparty, netting set, the curve behind a chart) are fetched in parallel within `link-budget`. Those that miss it are marked `pending` in the `Linked entities` panel; a later rebuild (the next tick, or a reload) fills them in.
- **Failures stay local.** An expression that fails in the strip shows a dash; a failing panel binding shows an inline error for that panel only. The view still renders.
- **Timings.** Each ViewModel reports how long it took, per stage, in milliseconds (`"timings": {"fetch": 0.11, "layout": 0.14, "links": 0.15, "bind": 0.26, "total": 0.66}`).

---

## A5. Walk through one request

This follows a single command from the keyboard to the screen and then through one live tick. The outputs are real, captured from a running server with the `trading` pack enabled (security off, so no token is needed).

**1. The user types `TRD MX-20000001` and presses Enter.** While typing, `drishti_command.js` asks the console for suggestions (debounced 60 ms, stale requests cancelled); the console forwards to the server:

```
curl -s 'http://localhost:18480/api/v1/command/suggest?q=TRD%20T-1000&limit=2'
```

```
[{"type":"entity","mnemonic":"TRD","kind":"trade","id":"MX-20000001","title":"MX-20000001",
  "subtitle":"Trade · Interest rate swap (fixed/float) · Meridian Reinsurance Ltd · AUD 242m","complete":"TRD MX-20000001"}, …]
```

**2. The console resolves the command.** The form goes to the console's `GET /go?q=TRD MX-20000001`. The console calls the server's `POST /api/v1/command` with `{"text": "TRD MX-20000001"}`, receives `{"ref": {"kind": "trade", "id": "MX-20000001"}, "mnemonic": "TRD"}`, and redirects the browser to `/v/trade/MX-20000001`.

**3. The console renders the first paint.** For `/v/trade/MX-20000001` the console calls `GET /api/v1/views/trade/MX-20000001`. Each call carries `X-Drishti-User`, `Authorization: Bearer <token>` (when sign-in is on; the console mints a fresh HS256 token for every call) and `X-Drishti-As-Of` when a business date is picked. On the server, `TokenFilter` turns the token into a `Principal`, `Entitlements` checks the role and pack may open `trade`, `AsOfResolver` resolves the business date, and `ViewPipeline.view` runs the stages of section A4:

```
curl -s http://localhost:18480/api/v1/views/trade/MX-20000001 | python3 -m json.tool | head -40
```

Abbreviated:

```
{
  "ref": {"kind": "trade", "id": "MX-20000001"},
  "mnemonic": "TRD",
  "title": {"pill": "Rates · Interest rate swap (fixed/float)", "id": "MX-20000001",
            "with": {"text": "Meridian Reinsurance Ltd",
                     "link": {"kind": "counterparty", "id": "CP-MERIDIAN-RE", "mnemonic": "CPTY"}}},
  "strip": [{"label": "Notional", "text": "AUD 242,000,000", "path": "$.currency"},
            {"label": "Direction", "text": "Receive fixed", "path": "$.direction"}, …],
  "panels": [{"id": "terms", "kind": "kv", "title": "Terms", "key": "F2", "area": "main", "inferred": false,
              "data": {"fields": [{"label": "Fixed rate", "text": "4.0829%", "path": "$.terms.fixedRate"}, …]}},
             {"id": "legs", "kind": "tabs", …}, {"id": "schedule", "kind": "ladder", …},
             {"id": "built", "kind": "provenance", …}, {"id": "marketData", "kind": "line", …},
             {"id": "sensitivities", "kind": "hbar", …}, {"id": "pnl", "kind": "line", …},
             {"id": "refs", "kind": "links",
              "data": {"links": [{"label": "Netting set", "text": "NS-MERIDIAN-RE-NY",
                                  "link": {"kind": "netting-set", "id": "NS-MERIDIAN-RE-NY", "mnemonic": "NSET"},
                                  "badge": "PFE 46.0m", "status": "resolved"}, …]}}],
  "keys": [{"key": "F2", "label": "Terms", "action": "panel", "panel": "terms"}, …],
  "provenance": {"layout": "Sutra irs-fixfloat v1 + inference", "fingerprint": "b7df…1372",
                 "source": "murex-rates", "generation": 1701, "live": true, "businessDate": null, …},
  "timings": {"fetch": 0.11, "layout": 0.14, "links": 0.15, "bind": 0.26, "total": 0.66}
}
```

Reading it: the router found the trade in the `demo` plugin (whose sample's `_meta` names the source system `murex-rates`); `SutraMatcher` chose `irs-fixfloat v1` from `config/packs/banking/trading/sutras/murex/`; inference added what the Sutra did not say ("+ inference"); the counterparty, netting set, book and trader were fetched as links and given badges from the pack's `badges:` expressions. The console renders the panels with the Jinja macros in `web/templates/_macros/panels.html`, main-area panels on the left and `area: right` panels on the right.

**4. The page goes live.** The page does not open its own stream. `drishti_live.js` subscribes `view:trade/MX-20000001` on the browser's single channel (`drishti_channel.js` → the hub in `drishti_live-hub.js`, a SharedWorker shared by every tab → console `GET /api/channel?s=view:trade/MX-20000001&s=alerts`). The console opens the server stream for that subscription and relays its events, each wrapped with its subscription (`{"ch": "view:trade/MX-20000001", "d": …}`). Directly against the server:

```
curl -sN http://localhost:18480/api/v1/views/trade/MX-20000001/stream
```

```
event:view
id:0
data:{"ref":{"kind":"trade","id":"MX-20000001"},"mnemonic":"TRD","title":{…},"strip":[…],"panels":[…],…}

event:frame
id:1
data:{"seq":1,"generation":1710,"patches":[{"op":"strip","index":4,"cell":{"label":"MTM (USD)","text":"+1,625,979","tone":"pos","em
phasis":true,"path":"$.mtm"}},{"op":"panel","panel":{"id":"built","kind":"provenance",…}}]}
```

Press Ctrl+C to stop. Every stream starts with the full view (`view`), then sends `frame` events.

**5. A tick arrives.** The demo plugin moves the trade's MTM and bumps the generation. `TopicHub` (one topic per entity, one source subscription however many people watch) puts the new document in a latest-wins slot and delivers it at most once per 50 ms frame. The `ViewStream` for this client rebuilds the view (layout from cache, so only binding runs), `PatchDiffer` compares it with what the client holds, and the result goes into the client's one-slot `FrameMailbox`. The client's writer thread sends it as the `frame` above: only the MTM strip cell (index 4) and the provenance panel changed, so only they travel.

**6. The browser applies it.** The console turns non-chart panel patches into ready HTML (same macro as first paint) before relaying them; `drishti_live.js` replaces the strip cell's text and tone, swaps the panel, moves chart data in place, and flashes what changed. The top bar's `Live, p99 N ms` comes from `GET /api/v1/health/live`:

```
curl -s http://localhost:18480/api/v1/health/live
```

```
{"streams":1,"topics":2,"frames":11684,"p50Ms":1.304,"p99Ms":1.761}
```

### Where to change what

Each row names what to change for a common need and the document that explains it.

| I want to… | Change | Read |
|---|---|---|
| Read a new store (a database, a topic, a folder, a service) | A connector: a file `config/connectors/<name>.yaml` with `plugin:` and `settings:` (or Admin → Connectors); the pack names it in `routes:` for the kinds it serves, and may carry a `connectors:` template for it | ../guides/DATA_GUIDE.md section 9, DEVELOPER_GUIDE.md, CONFIGURATION.md |
| Add a domain (new kinds, commands, links) | A pack folder `config/packs/<domain>/<name>/` with `pack.yaml`, `sutras/`, `samples/`; enable it with `DRISHTI_PACKS` | BUILD_GUIDE.md |
| Change how one screen looks | Its Sutra (`*.sutra.yaml`), or a Design in the Build workbench, proposed and approved | RACHANA_REFERENCE.md, RACHANA_REFERENCE.md |
| Change how a figure is formatted | `fmt:` in the Sutra; named formats in the pack's `config/formats.yaml` | RACHANA_REFERENCE.md |
| Change how inference reads field names | The pack's `config/semantics.yaml` | RACHANA_REFERENCE.md |
| Add a panel kind | The engine (`Binder` and the panel model) and a console renderer (`_macros/panels.html`) | RACHANA_REFERENCE.md, BUILD_GUIDE.md (docs/guides/DEVELOPER_GUIDE.md) |
| Decide who may see what | `roles:` in `pack.yaml` (kinds, `raw`), `drishti.security.redact` (fields shown as `•••`), the Admin pages | ADMIN_GUIDE.md |
| Add a command word | `mnemonics:` in `pack.yaml` (or under `drishti.commands` for a site) | BUILD_GUIDE.md |
| Add a link between kinds | `graph.fields` in `pack.yaml` and `link(...)` in the Sutra | BUILD_GUIDE.md |
| Change the sentence the About drawer says about a kind | `kinds.<kind>.about` in the pack's `config/about.yaml` (a `${...}` template over the document); for a generated pack, the about source in `tools/packgen/` | BUILD_GUIDE.md |
| Explain a field (term, meaning, unit, sign) | `glossary.<field path>` in `about.yaml`, or a shared `vocabulary.<name>` entry in the lowest pack that owns the concept (the glossary layer and field hints are step 4 of RACHANA_REFERENCE.md › About this page: the design of context help; section A3.9 says what is in today) | BUILD_GUIDE.md |
| Change how the drawer looks or behaves | `drishti-console/web/templates/_macros/about.html` and `terminal/_about.html`, `static/js/drishti_about.js`, `static/css/drishti_about.css` | RACHANA_REFERENCE.md |

| I want to... | Change | Read |
|---|---|---|
| Change what the explanation says about the layout and the data | `ExplainService`, `PageContext`, `EmptinessReason` (`drishti-engine`, package `explain`), `MatchTrace` (`SutraMatcher.explain`); the endpoint is `ExplainController`; limits under `drishti.explain.*` | API_REFERENCE.md, CONFIGURATION.md |
| Decide who may share, comment or be mentioned | the role power `roles.<role>.collaborate` (on unless removed); `drishti.collab.packs.<pack>.share-enabled`; `mentionable-roles`, `directory.scope`, `share.max-recipients`, `limits.*` under `drishti.collab` | CONFIGURATION.md, ADMIN_GUIDE.md |
| Change what a note or comment may contain, or what is masked in it | `drishti.security.redact` (the fields), `drishti.collab.text.on-masked-copy` and `deny-patterns`; the logic is `NoteText`, `Entitlements.maskedValues`, `CommentRenderer` | CONFIGURATION.md, §3.10 |
| Switch on email, or change what it says | `spring.mail.*`, `drishti.collab.console-url`, `email.enabled`; per pack `packs.<pack>.email.content` (`comment`, `title`, `link-only`); templates in `drishti.collab.email.templates-dir`; the code is `EmailNotifier`, `OutboxDispatcher`, `MailRenderer` | OPERATIONS_GUIDE.md |
| Add another channel (a chat bridge) | one more `Notifier` bean: `ShareService` and `ThreadService` hand every bean the audience, so neither service changes | HOW_IT_FITS.md §A31 (phase 2) |
| Keep, hold or export what people wrote | `drishti.collab.retention.*` and `packs.<pack>.retention-days`; legal holds and the export through `/api/v1/admin/collab` (`CollabPurge`, `HoldService`, `ExportService`, `ChainVerifier`) | OPERATIONS_GUIDE.md |
| Change how the Share dialog, the inbox or the Discussion tab look or behave | `drishti-console/web/templates/terminal/_share.html`, `inbox.html`, `_discussion.html`; `static/js/drishti_share.js`, `drishti_inbox.js`, `drishti_alerts.js`, `discussion*.js`; `static/css/drishti_collab.css` | HOW_IT_FITS.md §A31 (console design) |
| Let people search a kind by filling in a form | the `form:` section of `pack.yaml` (fields, types, operators), or a Design's Form tab; people open it with `FORM <pack>` | BUILD_GUIDE.md, USER_GUIDE.md |
| Let people choose what a panel shows | `controls:` in the Sutra (dropdowns, sliders, date ranges), read by panels as `ctl.<id>` | RACHANA_REFERENCE.md |
| Tell Drishti a batch of data landed (refresh, verify, tell people, flag late data) | `POST /api/v1/packs/{pack}/loads` from your ETL; what a pack expects is its `loads:` section | DATA_GUIDE.md |
| Reach a source over TLS or through a proxy | `tls.*` and `proxy:` in the connector file | DATA_GUIDE.md section 10, DATA_GUIDE.md section 10 (proxy) |
| Make a dataset, report or board (business intelligence) | YAML files in `config/bi/` (watched) or the BI pages (workspaces, designer, Model view); the query runs on the server over DuckDB with the reader's masks and row rules | BI_GUIDE.md, BI_GUIDE.md, BI_GUIDE.md, HOW_IT_FITS.md §A28 |
| Show a view inside another web application | register the application in Admin → Embedding; the page uses `<drishti-view>` | EMBED_GUIDE.md |
| Use a language model, or an MCP tool server | a provider file in `config/ai/` or `config/mcp/` and `drishti.ai.enabled` / `drishti.mcp.enabled` (both off by default) | AI_GUIDE.md, AI_GUIDE.md |
| Run several servers as one (Drishti*net) | the files of `config/net/` (`node.yaml` with `enabled: true` and the cluster key, a seed in `peers/`), or Admin → Cluster → Files; members advertise a real address, never a loopback one | OPERATIONS_GUIDE.md (docs/guides/CLUSTER_GUIDE.md#1-several-servers-as-one-drishtinet), tutorial 22 (docs/tutorials/22-a-cluster-of-three.md) |
| Lay a report out on paper (a statement, one per recipient) | `layout: statement` on a report page and the report's `statement:`; BI Studio's Paper tab | STATEMENTS.md (docs/guides/BI_GUIDE.md#r12-statements-paginated-reports-page-exact-one-per-reader) |
| Make something that failed (a pack, a connector, a provider) try again | nothing: it is retried with backoff; Retry now in Admin | OPERATIONS_GUIDE.md |

## A6. Rachana — the screen grammar — and Sutras written in it

Rachana is the grammar; each Sutra is one YAML document written in it (ADR-017, which supersedes ADR-011's Markdown Sutras). The first key, `rachana: 1`, is the language version, so the grammar can evolve without misreading old files; `description`, `notes` and a per-panel `description` carry the prose. Files are named `<name>.v<N>.sutra.yaml` and are found by scanning, recursively, the site

directories (`drishti.rachana.dirs`, default `./sutras`) and every enabled pack's `sutras/` folder (`config/packs/<domain>/<pack>/sutras/<area>/…`). `SutraBuilder` validates each one at load against the panel kinds' required and allowed options and reports problems with line and column (codes `DRS-2001`, `DRS-2002`; the detailed checks are 2009–2012 and 2020–2039; the About files add 2040–2047, search forms 2060). Any other file in a Sutra folder (a `.sutra.md`, a plain `.yaml`) is reported as `DRS-2004` with the fix. Because the grammar is plain YAML, `RachanaSchema` generates a JSON Schema of it from `PanelKind`, the formats and the served kinds, served at `GET /api/v1/rachana/schema`; Studio's editor and any schema-aware editor complete and check Sutras with it. With `hot-reload` on, file events reload changed files (debounced 250 ms); every watcher in the JVM (Sutras, packs, connectors, BI, AI, MCP) shares one `WatchService` and one dispatcher thread through `SharedWatchService` in `drishti-common`, so a process holds one inotify instance however many contexts or folders it watches; an invalid edit keeps the last good version and is listed at `GET /api/v1/sutras/problems`. Bindings use a small, compiled, side-effect-free expression language (**Rachana-EL**) — not scripting.

A real Sutra, abbreviated (`examples/packs/finance/sutras/rates/irs-vanilla.v3.sutra.yaml`, after its copyright comment):

```
rachana: 1
sutra: irs-vanilla
version: 3
description: Vanilla fixed/float interest rate swap (mockup drishti-irs.png).
match: { kind: trade, where: "$.productType == 'IRS' && size($.legs) == 2", priority: 10 }
title: { pill: "Trade · Interest rate swap", id: $.tradeId, with: "link($.counterparty.id, 'counterparty', $.counterparty.name)" }
strip:                                  # header key figures (at most 8)
  - { label: Notional (USD), bind: $.notional, fmt: amount0 }
  - { label: Direction, bind: "$.direction == 'PAY_FIXED' ? 'Pay fixed' : 'Receive fixed'" }
  - { label: MTM (USD), bind: $.mtm, fmt: signed0, tone: sign, emphasis: true }
  - { label: DV01 (USD), bind: $.dv01, fmt: signed0, tone: sign }
panels:
  - id: legs
    kind: tabs
    title: Legs
    key: F2
    each: $.legs
    tabTitle: "'Leg ' + @.leg + ' · ' + @.label"
    body: { kind: kv, columns: [ { label: Currency, bind: "@.currency" }, … ] }
  - id: cashflows
    kind: table
    title: Cashflows · Leg 1
    key: F3
    rows: $.legs[0].cashflows
    totalLabel: Total
    columns:
      - { label: Amount (USD), bind: "@.amount", fmt: signed2, tone: sign, total: true }
      - …
```

**Panel kinds** (the `PanelKind` enum, twenty-two): `kv`, `table`, `tabs`, `line`, `area`, `hbar`, `ladder`, `links`, `status`, `provenance`, `markdown`, `gauge`, `surface`, `waterfall`, `histogram`, `scatter`, `candlestick`, `graph`, `timeline`, `pivot`, `metric`, `map`. The chart and aggregate kinds are bound by `ChartBinder` under the `drishti.panels` limits and drawn by `static/js/drishti_charts.js`; RACHANA_REFERENCE.md describes every kind in depth. The full grammar is in RACHANA_REFERENCE.md. A new kind is added to the enum (with its required and allowed options) and to the binder on the Java side, plus a Jinja macro in `_macros/panels.html` and its client-side handling on the console (section A7a maps each kind to its macro).

**Formats** (`fmt`) are named and config driven: the core set is in `drishti-rachana/src/main/resources/formats.yaml` (`text`, `amount0`, `amount2`, `signed0`, `signed2`, `pct0`, `pct2`, `price2`, `compact` (4.1m), `date`, `dmy`), a site file can be named in `drishti.rachana.formats-file`, and packs add their own (the finance pack's `config/formats.yaml` adds `pct4`, `rate5`, `pips1`, `df4`, `bp1`). **Tones** (`sign`, `threshold`, `status`) colour values through theme tokens only, and always pair colour with a glyph or sign (WCAG, as MAYA requires).

Versioning: a view records `Sutra name vN` in its provenance; every version stays loadable. With review on (`drishti.governance.enabled`, ADR-013), a Studio save becomes a proposal that an approver makes live.

---

## A7. Inference engine

Inference (`drishti-inference`) is a **scored rule system** over the document's shape — deterministic, explainable, and run on the cold path only (results are cached by fingerprint). Each rule proposes panel candidates with a score; `InferenceEngine` packs the winners into areas (`main`, `right`) under density limits, and builds the header strip itself from the top-scoring scalars (at most 8). A `Linked entities` panel (`refs`) is always added.

| Rule (`Rules.java`) | Detects | Proposes |
|---|---|---|
| `LegsRule` | 2–4 structurally similar objects with many fields (legs, tranches); or 1–8 records that hold lists of their own (legs with cashflows with fixings) | `tabs` of `kv`; or `tabs` with `panels:` (a `kv` and a table or tree table per list) |

| Rule (`Rules.java`) | Detects | Proposes |
|---|---|---|
| `HomogeneousArrayRule` | array of objects sharing most keys | `table`; numeric columns right-aligned; totals for amount-like columns |
| `TermStructureRule` | array keyed by tenor-like values (`1M`, `5Y`, dates) with one numeric | `line` chart |
| `DistributionRule` | small map/array label → number | `hbar` |
| `TimeSeriesRule` | array of `{date, value…}` sorted by date | `ladder` |
| `NestedObjectRule` | object of scalars | `kv` panel, column count by field count |

The strip comes from top-level scalars ranked by semantic weight (money, rate, date); references (`NS-*`, `CSA-*`) are found by the entity graph, not by a rule (section A8). Semantic hints come from field names and value shape (`*notional*`, `*rate*`, ISO dates, currency codes, tenor strings) and from config: the core `drishti-inference/src/main/resources/inference/semantics.yaml` plus each pack's `semantics:` file. Every inferred panel carries `"inferred": true`, and the provenance label says `+ inference`; Sutra Studio can show the inferred layout as a Sutra to start from (`GET /api/v1/studio/inferred/{kind}/{id}`).

---

## A7a. How a screen is made: generic, not bespoke

A Sutra is **not** compiled to JavaScript or HTML, and no code is generated from it. It is YAML that the server reads and interprets: `SutraBuilder` turns it into a model, `LayoutMerger` combines it with what inference proposes into a **data-free layout**, and `Binder` (with `ChartBinder` for the chart kinds) binds that layout to one document into a **JSON ViewModel**. The console draws every ViewModel with the same page template and **one Jinja macro per panel kind**, and the browser runs the same generic scripts on every page: charts, tables, tabs, keys and live patches. Every entity of every mnemonic, in every pack, goes through this one path. There is no screen per product, per kind or per mnemonic.




What is code and what is configuration:

| Layer | Written once, used by every kind | Written per pack or kind |
|---|---|---|
| Server | `SutraBuilder`, `SutraMatcher`, `InferenceEngine`, `LayoutMerger`, `Binder`, `ChartBinder`, `PatchDiffer` | nothing |
| Console | `terminal/view.html`, `_macros/view.html`, `_macros/panels.html`, `api_routes._panel_html` (live patches) | nothing |
| Browser | `drishti_view.js`, `drishti_charts.js`, `drishti_geomap.js`, `drishti_tables.js`, `drishti_tree-rows.js`, `drishti_pivot.js`, `drishti_live.js` | nothing |
| Pack | — | YAML only: Sutras (or none), fragments, `form:`, `columns:`, `pivot:`, `semantics:`, formats |

The default packs hold 219 Sutras (`config/packs/*/sutras/`) and the UI kit and packs 36 fragments; all of them are drawn by the one page template and the one macro file. Search, query by form and the pivot are generic in the same way (examples 3 and 4). The examples below were captured from a server started from the repository with the default packs and `DRISHTI_LAKE_ENABLED=false`, so the `demo` connector's committed samples answered, and a console pointed at it; the commands use the quick-start port 18480.

### Example 1: Sutra → ViewModel → HTML

**The Sutra.** `TRD MX-20000001` matches `config/packs/banking/trading/sutras/murex/irs-fixfloat.v1.sutra.yaml` (generated by `tools/packgen/banking/make_sutras.py`). One strip cell, and the first panel, which it takes from a fragment:

```
match: { kind: trade, where: "$.SOURCE_SYSTEM == 'MUREX' && $.PRODUCT_TYPE == 'IRS_FIXFLOAT'", priority: 10 }
strip:
  - { label: Notional, bind: "$.CURRENCY + ' ' + fmt(first($.LEGS).NOTIONAL, 'amount0')", emphasis: true }
panels:
  - { use: trading/deal-ticket@1, with: { rate-fmt: pct4, product: "Interest rate swap (fixed/float)" } }
```

The fragment (`config/packs/banking/trading/sutras/common/deal-ticket.v1.fragment.yaml`) is a `kv` panel, still only YAML:

```
panels:
  - id: ticket
    kind: kv
    title: Deal ticket
    code: DES
    key: F2
    columns:
      - { label: Trade id, bind: $.TRADE_ID }
      - { label: Transaction, bind: "nonblank($.TRANSACTION_ID, '—')" }
      - { label: Version, bind: "nonblank($.TRADE_VERSION, '—')" }
      - …                                  # 31 fields in all
```

**The ViewModel.** The server binds it to the document; abbreviated:

```
curl -s http://localhost:18480/api/v1/views/trade/MX-20000001
```

```
{
  "ref": {"kind": "trade", "id": "MX-20000001"}, "mnemonic": "TRD",
  "strip": [ …, {"label": "Notional", "text": "AUD 242,000,000", "emphasis": true, "path": "$.CURRENCY"}, … ],
  "panels": [
    {"id": "ticket", "kind": "kv", "title": "Deal ticket", "code": "DES", "key": "F2", "area": "main", "inferred": false,
     "data": {"fields": [
        {"label": "Trade id", "text": "MX-20000001", "link": {"kind": "trade", "id": "MX-20000001", "mnemonic": "TRD"}, "path": "$.
TRADE_ID"},
        {"label": "Transaction", "text": "20000001002", "path": "$.TRANSACTION_ID"},
        {"label": "Version", "text": "2", "path": "$.TRADE_VERSION"}, … ]},
     "empty": false},
    {"id": "legs", "kind": "tabs", …}, {"id": "flows", "kind": "table", …}, {"id": "events", "kind": "timeline", …}, …],
  "provenance": {"layout": "Sutra irs-fixfloat v1 + inference", "sutra": "irs-fixfloat", "source": "murex", …}
}
```

The ViewModel is the whole contract between server and console: labels, formatted text, tones, links and the JSON path of every value. Nothing in it says "trade" or "swap" to the console.

**The macro.** For each panel the page calls `panel(p)` in `drishti-console/web/templates/_macros/panels.html`; it draws the frame (`<section>`, header, code, key, download and help) and dispatches on `p.kind` in `pbody`:

```
{% elif p.kind in ['kv', 'status'] %}{{ fields(p.data.fields, 4 if p.area == 'main' else 2) }}
…
{% macro fields(list, cols=4) -%}
<dl class="kv kv-{{ cols }}">
  {% for f in (list or []) %}<div class="kv-item"><dt>{{ f.label }}</dt><dd class="mono{{ tone(f) }}"{% if f.path %} data-path="{{
f.path }}"{% endif %}>{{ cell_text(f) }}</dd></div>{% endfor %}
</dl>
{%- endmacro %}
```

**The HTML.** What the console sent for `/v/trade/MX-20000001` (abbreviated; the download menu left out):

```
<section class="pnl" id="p-ticket" data-panel="ticket" data-kind="kv" data-key="F2" data-span="12" data-height="0" …>
  <header class="pnl-h"><h3 id="h-ticket">Deal ticket</h3>
    <span class="pnl-code">DES · F2 … <a class="pnl-help" href="/help/rachana-reference#kv" …></a></span></header>
  <div class="pnl-b">
  <dl class="kv kv-4">
  <div class="kv-item"><dt>Trade id</dt><dd class="mono" data-path="$.TRADE_ID"><a class="lnk" href="/v/trade/MX-20000001">MX-20000
001</a></dd></div>
  <div class="kv-item"><dt>Transaction</dt><dd class="mono" data-path="$.TRANSACTION_ID">20000001002</dd></div>
  <div class="kv-item"><dt>Version</dt><dd class="mono" data-path="$.TRADE_VERSION">2</dd></div> …
```

A field with a `link` becomes an anchor to `/v/{kind}/{id}`; `data-path` is what the About drawer, the live flash and the export read. When a tick changes the panel, the console renders it again with the same `panel` macro (`routes/api_routes.py`, `_panel_html`) and `drishti_live.js` swaps it in (section A10).

**Panel kind → macro → browser script.** One to one, for all twenty-two kinds of the `PanelKind` enum:

| kind | Macro in _macros/panels.html | Drawn or enhanced in the browser by |
|---|---|---|
| `kv`, `status` | `fields` | none (plain HTML) |
| `provenance`, `markdown`, `metric`, `gauge` | inline in `pbody` | none |
| `table`, `ladder` | `table` (through `pivotable`; `drill` when the Sutra has `levels:`) | `drishti_tables.js`, `drishti_colfilter.js`, `drishti_tree-rows.js`, `drishti_pivot.js` (the Pivot tab) |
| `tabs` | `tabs` (each tab's body is again `pbody`) | `drishti_view.js` |
| `line`, `area` | `chart` (`data-chart` JSON) | `drishti_view.js` (ECharts) |
| `hbar` | `bars`, or `donut` with `shape: donut` | none; `drishti_charts.js` for the donut |
| `surface` | `surface` (`data-surface` JSON) | `drishti_view.js` (heatmap or 3D) |
| `links` | `links` | none |
| `waterfall`, `histogram`, `scatter`, `candlestick`, `graph` | the macro of the same name (`xchart`, `data-xchart` JSON) | `drishti_charts.js` |
| `timeline` | `timeline` | none |
| `map` | `geomap` | `drishti_geomap.js`, `drishti_charts.js` |
| `pivot` | `pivot`, `pivot_table` | `drishti_tree-rows.js`, `drishti_pivot-grid.js` |

**Example 2: a kind with no Sutra**

No Sutra in the default packs matches `var-limit` (`VLIM`, a hierarchy node's VaR limit; it is normally read from the `risk-store` lake). To show what inference alone makes of it, the first line of the committed sample `config/packs/banking/market-risk/samples/var-limit.jsonl` is put in the `file` connector's folder of a scratch server (`<data>/feeds/var-limit/VL-LL~BANK.json`). The document is flat UPPER_SNAKE fields: ids, a number, a currency and dates.

```
curl -s 'http://localhost:18480/api/v1/views/var-limit/VL-LL~BANK'
```

Abbreviated:

```
{
  "title": {"pill": "Var limit", "id": "var-limit", "with": null},
  "strip": [{"label": "Limit ID", "path": "$.LIMIT_ID"}, {"label": "Hierarchy", "path": "$.HIERARCHY"},
            {"label": "Node ID", "path": "$.NODE_ID"}, {"label": "Limit VaR", "path": "$.LIMIT_VAR"}, …],
  "panels": [
    {"id": "built", "kind": "provenance", "title": "How this view was built", "inferred": true, …}, …],
  "provenance": {"layout": "inference only", "sutra": null, "source": "feed-file", …}
}
```

Reading it: `provenance.layout` is `inference only` and `sutra` is `null`; every panel says `"inferred": true` and why (`explanation`); the strip is the top-ranked scalars. The identifier rules look for camelCase names (`varLimitId`, `id`, `code`, `name`), so the title falls back to the kind's name; a Sutra with `title: { id: $.LIMIT_ID }` fixes it. The ViewModel has the same shape as example 1, so the same template and macros draw it. To start that Sutra from what inference made:

```
curl -s 'http://localhost:18480/api/v1/studio/inferred/var-limit/VL-LL~BANK?name=var-limit-row'
```

```
# Started from what inference makes of var-limit VL-LL~BANK. Edit freely.
rachana: 1
sutra: var-limit-row
version: 1
match: { kind: var-limit, priority: 1 }
title: { pill: "Var limit", id: "'var-limit'" }
strip:
  - { label: "Limit ID", bind: "$.LIMIT_ID" }
  - …
```

RACHANA_REFERENCE.md explains the rules and the step from here to a Sutra.

### Example 3: query by form

**The YAML.** A pack lists the fields people search a kind by; from `config/packs/banking/trading/pack.yaml`:

```
form:
  trade:
  - path: SOURCE_SYSTEM
    label: Source system
    type: enum
    group: Identity
    order: 2
    help: 'ENDUR, CALYPSO or MUREX: the booking system the trade comes from.'
  - path: CURRENCY
    label: Currency
    type: enum
    group: Where
    order: 8
  - path: QUANTITY
    label: Quantity
    type: amount
    group: Money
    order: 12
    operators: ['>=', '<=', between, '=']
```

**The page.** `FORM trading <GO>` goes to `/form/trading` (`drishti-console/routes/form_routes.py`, `form_page`). The route asks the server for the form (`GET /api/v1/forms/trading?kind=trade`: the fields with their operators and the values the data holds, e.g. `"values": ["CALYPSO", "ENDUR", "MUREX"]`), and `core/forms.py` groups the fields for `web/templates/terminal/form.html`, which has one branch per field **type** (`text`, `number`/`amount`, `date`, `enum`, `id`, `bool`), never per pack or field. The address holds the criteria:

```
/form/trading?kind=trade&op.SOURCE_SYSTEM=in&v.SOURCE_SYSTEM=MUREX&op.CURRENCY=in&v.CURRENCY=EUR&v.CURRENCY=USD&op.QUANTITY=%3E%3D&
v.QUANTITY=100m&run=1
```

**The search it writes.** `forms.criteria_from` reads the criteria from the address and the server compiles them to the search grammar:

```
curl -s -X POST -H 'Content-Type: application/json' http://localhost:18480/api/v1/forms/trading/trade/compile \
  -d '{"criteria":[{"path":"SOURCE_SYSTEM","op":"in","values":["MUREX"]},
                   {"path":"CURRENCY","op":"in","values":["EUR","USD"]},
                   {"path":"QUANTITY","op":">=","values":["100m"]}],"limit":100}'
```

```
{"command":"TRD where SOURCE_SYSTEM = 'MUREX' and (CURRENCY = 'EUR' or CURRENCY = 'USD') and QUANTITY >= 100m limit 100"}
```

The route runs that command as any search (`GET /api/v1/search?q=…`, example 4) and the page shows it with Copy and Edit as text, then the results table (`127 of 765 trades` on the demo samples). Masks, kinds per role and packs per user apply because it is an ordinary search. A pack adds a form by adding YAML; no form has code of its own. The reference is USER_GUIDE.md.

### Example 4: search and the pivot

**Plain words.** `murex trades in eur over 100m <GO>` is not a command, so `/go` sends it to `/s?words=murex%20trades%20in%20eur%20over%20100m`; the console asks the server what the words mean:

```
curl -s 'http://localhost:18480/api/v1/phrase?text=murex%20trades%20in%20eur%20over%20100m'
```

```
{"query": "TRD where SOURCE_SYSTEM = 'MUREX' and CURRENCY = 'EUR' and QUANTITY > 100000000",
 "kind": "trade",
 "steps": [{"words": "trades", "meaning": "Trade (TRD)"}, {"words": "murex", "meaning": "source system is MUREX"},
           {"words": "eur", "meaning": "currency is EUR"}, {"words": "over 100m", "meaning": "quantity > 100000000"}],
 "ignored": [], "problem": null, "command": null}
```

The page says *Your words make this search:* and links the query; following it runs the structured search.

**The search.** Typed directly (`TRD where … <GO>`) or reached from words or a form, it is one call:

```
curl -s "http://localhost:18480/api/v1/search?q=TRD%20where%20SOURCE_SYSTEM%20%3D%20'MUREX'%20and%20CURRENCY%20%3D%20'EUR'%20and%20
QUANTITY%20%3E%20100000000%20limit%20100"
```

Abbreviated:

```
{"query": "TRD where SOURCE_SYSTEM = 'MUREX' and CURRENCY = 'EUR' and QUANTITY > 100000000 limit 100",
 "kind": "trade", "mnemonic": "TRD",
 "condition": "lower($.SOURCE_SYSTEM) == lower('MUREX') && lower($.CURRENCY) == lower('EUR') && $.QUANTITY > 100000000",
 "columns": ["$.SOURCE_SYSTEM", "$.CURRENCY", "$.QUANTITY", "$.PRODUCT_TYPE", …],
 "labels": {"$.SOURCE_SYSTEM": "Source system", "$.CURRENCY": "Currency", …},
 "rows": [{"ref": {"kind": "trade", "id": "MX-20000002"}, "title": "MX-20000002",
           "values": {"$.SOURCE_SYSTEM": "MUREX", "$.CURRENCY": "EUR", "$.QUANTITY": 110000000.0, …}}, …],
 "scanned": 765, "matched": 21, "elapsedMs": 7.72,
 "pivot": {"rows": ["PRODUCT_TYPE"], "columns": ["SOURCE_SYSTEM"], "values": [{"field": "QUANTITY", "agg": "sum"}], …}}
```


**The results page.** `/s` (`routes/terminal_routes.py`, `search`) renders `web/templates/terminal/search.html` for every kind: `21 of 765 trades match`, a table whose columns come from the pack's `columns:` for the kind (each row links to `/v/{kind}/{id}`), a chart of the rows, and, because the trading pack declares `pivot:` for `trade`, a **Table | Pivot** switch drawn by `drishti_pivot.js` from the `pivot` block above. A kind without `columns:` gets columns from the condition and the documents; a kind without `pivot:` gets the table alone. The grammar is in USER_GUIDE.md, Search by value and Asking in plain words (docs/guides/USER_GUIDE.md#asking-in-plain-words).

### When code is needed

YAML covers new kinds, packs, screens, forms, searches and pivots. Code is needed only to give the grammar or the platform a new capability, and then it serves every pack:

| Need | Code to write | Where |
|---|---|---|
| A new panel kind | the enum entry with its required and allowed options; its binding; its macro; its client script if it draws anything; its help entry | `PanelKind` (`drishti-rachana`), `Binder` or `ChartBinder` (`drishti-engine`, package `bind`), `drishti-console/web/templates/_macros/panels.html` (`pbody` and a macro), `drishti-console/web/static/js/drishti_view.js` or `drishti_charts.js`; RACHANA_REFERENCE.md, BUILD_GUIDE.md (docs/guides/DEVELOPER_GUIDE.md) |
| A new kind of store (a source plugin) | a `SourcePlugin` (`fetch`, optionally `subscribe`, `search`, `reverse`) in a plugin module; after that each store is a connector file | `SourcePlugin` (`drishti-api`), a module under `plugins/` such as `drishti-plugin-file`; DEVELOPER_GUIDE.md |
| A new computed measure | the computation: a Java engine (as VaR and exposure in `drishti-compute`, packages `var` and `ccr`) or a Python engine behind a `reactive` connector; its result is then an ordinary kind, drawn by the generic view | REACTIVE_CONNECTOR.md (docs/connectors/types/reactive/README.md), VAR_GUIDE.md, Part III |

### The hand-built pages and why

Some pages are hand-built. Each is a product feature that works across packs and kinds, built on the same server APIs, entitlements and masks as the generic view; none is a screen for one product or one mnemonic.

| Page | Routes | Templates | Why it is its own page |
|---|---|---|---|
| VaR explorer | `/var` (`VAR <scope> <GO>`) | `terminal/var.html`, `static/js/drishti_var-explorer.js` (drishti-risk's grids and charts) | Any scope (hierarchy node, book, basket, trade) computed on demand by the VaR engine (`/api/v1/risk/var`), with drill-down, captures and compare; the computed kinds stay ordinary kinds on `/v/…` too. VAR_GUIDE.md |
| Exposure explorer | `/exposure` (`EXP <scope> <GO>`) | `terminal/exposure.html`, `static/js/drishti_exposure-explorer.js` (drishti-risk's grids and charts) | The same for counterparty exposure (`/api/v1/risk/exposure`); `exposure-result` is still a kind. EXPOSURE_GUIDE.md |
| BI Studio, report viewer and the BI pages | `/bi/…` | `templates/bi/` | Reports and models over datasets (DuckDB on the server), a different job from one entity's view. BI_GUIDE.md |
| Admin | `/admin/…` | `templates/admin/` | Running the system: users, packs, connectors, lakes, Drishti*net |
| Sutra designer and Build | `/designer`, `/build/…`, `/studio` | `templates/designer/`, `templates/build/`, `templates/studio/` | Tools that write Sutras; their previews are the generic view's macros. RACHANA_REFERENCE.md |
| Data map | `/datamap` (`DATAMAP <GO>`) | `terminal/datamap.html`, `static/js/drishti_datamap.js` | The packs' kinds and joins, checked against the data (`/api/v1/datamap`). USER_GUIDE.md |
| Landing and Home | `/`, `/t` | `landing.html`, `terminal/home.html` | Entry pages: the command line, recent entities, the watch list |
| Workspaces and Monitors | `/w/…`, `/m/…`, `/alerts` | `templates/workspace/`, `templates/monitor/` | Arrangements of generic views and searches, saved per user; a workspace pane is a `/v/…` view. USER_GUIDE.md |
| Help centre | `/help/…`, `/what` | `templates/help/` | Renders the repository's guides and the packs' Knowledge pages |

## A8. Entity graph and navigation

- **Reference catalog** (`drishti-graph`, `ReferenceCatalog`) maps identifier patterns and field names to kinds. Packs declare them in `pack.yaml` (`graph: {id-patterns, fields, badges}`), sites add more under `drishti.graph.id-patterns` and `drishti.graph.fields`, e.g. `^NS-[A-Z]+-\d+$ → netting-set`, `nettingSet → netting-set`.
- The `Linked entities` panel shows label, target, a **badge** computed from the target by a Rachana-EL expression (`drishti.graph.badges`, e.g. `PFE 46.0m`), and a status (`resolved`, `pending`, or denied with the reason).
- Navigation keeps a **breadcrumb trail per browser tab** (`drishti_view.js`); `Alt+←` goes back. Reverse links (netting set → member trades) and **F8 Impact** (`/api/v1/impact/{kind}/{id}`) are answered by plugins that implement `SourcePlugin.reverse`, rolled up along `drishti.graph.impact.follow`.

---

## A9. Command suggestions (type-ahead)

As in the Bloomberg terminal, the command line suggests while the user types, in a dropdown under the input:

```
┌ TRD MX-2000█ ──────────────────────────────────────────────────────────────┐
│ TRD  MX-20000001   Trade · Interest rate swap (fixed/float) · Meridian Reinsurance Ltd │  ← highlighted
│ TRD  MX-20000002   Trade · Interest rate swap (fixed/float) · Halcyon Shipping plc     │
└────────────────────────────────────────────────────────────────────────────┘
```

- **What it matches**, by the state of the input:
- an empty or partial first token matches **mnemonics** (`T` → `TRD Trade`, ...);
- after a mnemonic, it matches **entities of that kind** by identifier or name;
- a bare identifier matches any kind, with the mnemonic filled in;
- recently viewed entities (`drishti.commands.recent-size`, 20 per user) rank first.
- **Engine:** `SuggestionService` merges the mnemonic table, the per-user recent list and `SourcePlugin.search`. Plugins are queried in parallel on virtual threads within `drishti.commands.suggest-budget` (30 ms); late answers are dropped. Each plugin answers from an in-memory `HitIndex` (`drishti-api`), so most queries take microseconds.
- **API:** `GET /api/v1/command/suggest?q=TRD%20T-1000&limit=10` → `[{type, mnemonic, kind, id, title, subtitle, complete}]` (see section A5 for a real answer).
- **Console:** `drishti_command.js` debounces by 60 ms and cancels stale requests with `AbortController`. ↑/↓ move, `Tab` completes, `Enter` (`<GO>`) opens, `Esc` closes; matched characters are highlighted; ARIA `combobox`/`listbox`.

### A9.1 Several business days

A connector keeps the newest business day's ids by default. With `index.days` above 1 it keeps that many days, and a type-ahead for a past business date answers from that day's own ids, never from another day's.

- **Date keys everywhere.** The index is a map of business date to day segment, for every query, today included. "Live" or "current" is resolved to a concrete business date (the kind's newest date) before the lookup and the day is found by that date; there is no "empty" or "current" key. A kind that has no dates uses the explicit key `UNDATED`.
- **Structure** (`DatedHitIndex`, `drishti-api`). Per kind: one dictionary of ids, each stored once, sorted for prefix binary search, with title and subtitle; and one compact bitmap per business day marking which ids existed that day. A prefix search is a binary search in the dictionary and a membership test in the day's bitmap.
- **Tiers.** The newest day and, by default, the closed days are on the heap. When the connector's `index.max-memory` is reached (or `index.storage: mmap` is set) a closed day is written once to a **segment file** (`DaySegment`): ids sorted and compressed in blocks of 128, read with **positional reads** of an open file channel (no memory map: a mapped file's blocks are freed only when the garbage collector unmaps it, so `df` would keep counting a deleted day). The block index (first id and offset per block) is on the heap; a prefix search binary-searches it, reads and decodes one or two blocks, and keeps a small bounded LRU of decoded blocks per segment (`index.block-cache`, 16). **Release is deterministic**: every lookup holds the segment (a reader count); a roll, a disk-cap eviction, a reload swap or a stale or corrupt file *retires* it, which removes the file and closes the channel as soon as the last running lookup lets go, so the disk is free at once and a lookup racing a retirement never fails (it looks again at the current segment). A day neither tier holds is built on demand: the connector answers as it always did (the newest day's ids) while the day is built in the background.
- **Files.** Under `drishti.index.dir` (`DRISHTI_INDEX_DIR`; default `<data dir>/index`, which is `./data/index`; never the temporary folder), one folder per connector, capped by `index.max-disk`. A file is written under a temporary name in its folder and moved into place atomically, ends in a CRC, and records the **watermark** of its source (a load time, a table version, a snapshot, a folder's modification time). After a restart a file is reused only if the source's watermark is unchanged; otherwise the day is rebuilt. A truncated or corrupt file is deleted and treated as absent, and temporary files left by a crash are removed at the next start.
- **Freshness.** The newest day updates (streams on arrival; tables on `refresh-seconds`). A data-load signal (the loader's load time, a Delta commit, an Iceberg snapshot, a replaced DuckDB file, a changed folder or file) rebuilds the kept past days in the background. On a business-date roll the new date's segment is added, the "today" pointer moves (one immutable snapshot swap, so readers are lock-free and never see a half-rolled index), the oldest day's bitmap or file is dropped, and dictionary ids no kept day holds are purged: memory and disk stay flat. `max-memory` moves the oldest closed days to disk (or drops them without a disk) early; `index.max-disk` drops the oldest segments.

- **Reload (admin and automatic).** `DatedHitIndex.reload(kind, day)` rebuilds chosen days from the source on a background thread: the new day is read in full, written (a heap day's bitmap; a disk day's temporary file, then an atomic move), and swapped in in one step, so the old day keeps answering until the new one replaces it and a reader never sees a mix. A day that cannot be read keeps its old content and shows the error. **One job per connector**: a manual reload and the automatic check share the same slot (a second request is refused with `409 DRS-5032`; an automatic cycle that finds a job running is skipped), so they never overlap. The automatic check (`index.reload-every`, default `10m`, `0` = off; per kind with `index.kinds.<kind>.reload-every`) runs on one shared daemon scheduler: each connector's first check is jittered, and a cycle compares every *past* day (the newest day has its own faster refresh) with the watermark recorded when it was built (load time, Delta commit, Iceberg snapshot, file modification time) and rebuilds only the days whose watermark changed; a source with no reliable watermark (S3, a table without a load time) has each past day rebuilt once per cycle. Rebuilds of one cycle are spaced by a random pause so a lake is not hit all at once. This is what catches a **past day restated with no signal**. Admin: `GET` and `POST /api/v1/admin/connectors/{name}/index[/reload]` (API_REFERENCE.md); the status gives, per kind and day, ids, tier, bytes, build time, whether it is rebuilding and the last error, and the automatic check's last check, next check and last change found.
- **Warm-up.** After start, past days are built in the background, newest first; start-up is not delayed. `index.warm: lazy` builds a day on the first search for it instead.
- **`index.any-recent-day: true`.** A bare id on the newest date may also suggest ids seen only on earlier kept days, marked `last seen <date>` in the subtitle.
- **Health.** Admin, Health and the connector page show per connector: `type-ahead index: 5 days (4 on disk, 0.1 MB), 1,200 ids, 0.3 MB, warm`. The connector's cache statistics add `indexDays`, `indexDaysOnDisk`, `indexIds`, `indexMemoryMb`, `indexDiskMb` (the segment files' real size), `indexWarm`, `indexReloadEvery`, `indexLastCheck`, `indexNextCheck`, `indexLastChange`, `indexReloading`.

The settings are in CONFIGURATION.md. Streaming connectors keep the newest day only. Measured: a prefix lookup over 100,000 ids a day for five days takes 0.002 ms on the heap and 0.04 ms from a segment file read with positional reads (548 KB on disk for 100,000 ids) (CLUSTER_GUIDE.md).

---

## A10. Live updates

```
SourcePlugin.subscribe ──► TopicHub (one topic per EntityRef, one source subscription, single writer)
                               │  latest-wins slot; at most one delivery per 50 ms frame (leading edge)
                               ▼
                           ViewStream (one per client view): rebuild with cached layout, PatchDiffer
                               │  rebuilds never overlap; a change during one causes exactly one more
                               ▼
                           FrameMailbox (one slot per client, pending frames merge)
                               │
                               ▼
                           SSE writer (virtual thread): `view` once, then `frame` events, `: hb` every 15 s
                               │
                               ▼
                           console channel (ONE per browser tab) ──► drishti_live.js applies patches
```

- **Rebuild and diff.** Each rebuild uses the cached layout and is diffed against what the client holds; only changed strip cells, changed panels and the provenance travel. (Expressions also report their paths, so path-targeted re-binding is possible; measurement showed rebuild-and-diff is fast enough. See Part III · Data architecture.)
- **Chart sources tick too.** A view also listens to the topics of the entities its `line` charts read from (the curve behind a swap).
- **Static views.** A picked business date, or a source that does not stream, gets one `view` event and the stream ends; no live slot is held.
- **Capacity.** `drishti.live.max-streams` (20,000) caps view and monitor streams together; the slot is taken atomically before any work. Over the cap the request is refused with `DRS-5001` ("too many live streams on this server").
- **One channel per browser.** Browsers allow six connections per site over HTTP/1.1, shared by all its tabs, so the number of live connections must not grow with tabs or panes (a channel per tab froze a seventh tab, UX-01). No page opens its own `EventSource`: views, the alerts bell, monitors and every workspace pane subscribe through `drishti_channel.js` to one hub per browser (`drishti_live-hub.js`, in a SharedWorker; without one, in a tab elected with a Web Lock that relays over a BroadcastChannel), which holds the browser's single channel (`/api/channel?s=view:trade/MX-20000001&s=alerts&s=monitor:<name>`, at most `live.max_subscriptions`, 32). A key held by several tabs is subscribed once; later subscriptions are added to the open channel with `POST /api/channel/{id}`. Frames are still built per signed-in user: the hub only serves tabs whose session fingerprint (`<meta name="drishti-live">`) matches the channel's (`who`). A tab hidden for 10 s gives its subscriptions back and repaints from fresh data when shown. Page loads never wait for the channel. The console closes each upstream server stream within seconds of the browser going away. Details in Part III · Data architecture.
- **Latency.** The top bar's `Live, p99 N ms` is a rolling (30 s) HdrHistogram of source tick → frame built, published at `GET /api/v1/health/live` and as the Prometheus gauge `drishti_live_latency_p99_milliseconds`.
- **Alerts and monitors.** The alert engine (`drishti-server`, `AlertEngine`) subscribes to the topics of every entity an enabled rule watches, evaluates each rule's Rachana-EL condition on every frame and fires on the false → true edge. A monitor stream holds one `ViewStream` per row and multiplexes their strip patches as `row` events.

---

## A11. Business dates and history (ADR-012)

Every read is for a business date. The console keeps the user's choice (**Live**, or a picked date) and sends it as `X-Drishti-As-Of` (and `X-Drishti-Known-At` for "as known at"); it never reads data itself. The server's `AsOfResolver` and `BusinessDates` resolve it on the configured calendar (`drishti.business-date.calendar`: `USNY`, `GBLO`, `EUTA`, `JPTO`, `CATO` (Toronto), or joint such as `USNY+GBLO`), rolling weekends and holidays back to the previous business day, and the `AsOf` value travels through the pipeline, links, impact and suggestions to every plugin. Live streams; a picked date is a static snapshot.

Dated sources stamp `Provenance.businessDate` and are tried first for a picked date:

| Plugin | How it is dated |
|---|---|
| `delta` | Delta Lake through Delta Kernel: `<root>/<domain>/<kind>/business_date=yyyy-MM-dd/`; `root` may be local or object storage (`s3a://…`, through `LakeStore`); time travel answers "as known at" |
| `file`, `s3` | dated folders `<root or prefix>/<yyyy-MM-dd>/<kind>/<id>.json`, newest on or before the date |
| `jdbc` | a query that uses `:asOf` |
| `aerospike` | a record per entity per business date (`kind/id/yyyyMMdd`), found through the entity's index record listing its dates (AEROSPIKE_CONNECTOR.md (plugins/drishti-plugin-aerospike/README.md)) |
| `feeds` | keeps dated observations of public feeds |

```
console ──X-Drishti-As-Of──▶ AsOfResolver ─▶ BusinessDates ─▶ ViewPipeline ─▶ SourceRouter ─▶ dated connectors first
                                                                                          └─▶ undated (demo, rest, …)
```

Named connectors (one file each in `config/connectors/`, written at the first start from a pack's `connectors:` template when absent; `drishti.sources.connectors` still works but is deprecated) give each domain its own lake connector, e.g. `market-store` and `trading-store`. Lake retention, compaction and vacuum run outside the server (`tools/lake/`, `deploy/lake-maintenance.yaml`; see OPERATIONS_GUIDE.md).

---

## A12. Domain packs

The core carries no industry. A **domain pack** (`config/packs/<domain>/<name>/pack.yaml`) holds everything specific to one: kinds, Sutras, mnemonics, identifier patterns, reference fields and link badges, roles, semantic hints and formats, connectors, starter workspaces, help guides and sample data.

- **Loading.** `drishti.packs.enabled` (`DRISHTI_PACKS`, default the seven shipped packs) lists the packs, from `drishti.packs.dir` (`./config/packs`). Before any bean is built, `PackEnvironmentPostProcessor` loads them and adds their content as the **lowest-precedence** property source, so site configuration always wins. No core module depends on `drishti-packs`; they only read ordinary properties.
- **Inheritance (ADR-015).** A pack may say `extends: [parent, …]`. Enabling a pack loads its ancestors too: the running server enabled with `DRISHTI_PACKS=market-risk,counterparty-risk,…` also loads `core`, `market-data` and `trading`. The order is a C3 linearisation (`PackLineage`, as Python orders classes): where related packs define the same mnemonic, field, badge, role, route or connector, the child wins over its parents and the rightmost parent over the others; every override is reported (`drishti.packs.overrides`). Unrelated packs defining the same thing stop start-up and name both packs, and a kind always belongs to exactly one pack.

| Pack | extends |
|---|---|
| `core`, `world-economy`, `news-and-markets` | – |
| `market-data` | `core` |
| `trading` | `core`, `market-data` |
| `market-risk` | `market-data`, `trading` |
| `counterparty-risk` | `trading` |

The retired example packs under `examples/packs/` (`finance`, `logistics`, `genomics`, `retail-banking`, `climate-risk`, `operational-risk`, `liquidity-risk`) keep their own `extends` and load with `DRISHTI_PACKS_EXTRA_DIRS=examples/packs`. - **Per-user packs.** Installed packs are what the server runs; an admin **assigns** packs to users (default `drishti.packs.default-for-users`), and each user chooses which assigned ones are **active**. A kind owned by an inactive pack cannot be opened (`PackAccess`). - **Shipped packs** (`config/packs/`): `core`, `market-data`, `trading` (production-shaped UPPER_SNAKE trades from ENDUR, CALYPSO and MUREX), `counterparty-risk`, `market-risk`, `world-economy` (real public data) and the sample `news-and-markets`; the non-financial ones prove the core is neutral. The example packs (above) are examples and test fixtures.

`GET /api/v1/packs` lists the loaded packs; the console reads their console content from the same folders. See BUILD_GUIDE.md and ADR-010.

---

## A13. Module layout (Java)

Maven multi-module reactor on `spring-boot-starter-parent`, `groupId com.ash.drishti`, Java 21 or newer (production runs 21; 25 also supported), packages `com.ash.drishti.<module>…`, `package-info.java` everywhere.

| Module | Responsibility | Contributes |
|---|---|---|
| `drishti-api` | Plugin SPI: `SourcePlugin`, `PluginManifest`, `SourceCapabilities`, `DataNode`, `EntityRef`, `EntityDocument`, `Provenance`, `AsOf`, `HitIndex`, `Subscription` (no Spring, so plugins stay light) | – |
| `drishti-common` | Error codes (`ErrorCode`, `DRS-nnnn`), `DrishtiException`, JSON codec, `ShapeFingerprinter`, `BusinessCalendar` | `CommonConfiguration` |
| `drishti-rachana` | Sutra model, YAML parser and validator, the language's JSON Schema (`RachanaSchema`), Rachana-EL compiler, formats and tones, `SutraRegistry` + hot reload, `SutraMatcher` | `RachanaConfiguration` |
| `drishti-inference` | Shape analysis, semantic hints, rules, scorer, packer, `LayoutMerger` (Sutra ⊕ inference) | `InferenceConfiguration` |
| `drishti-graph` | `ReferenceCatalog` (identifier patterns, reference fields), badges | `GraphConfiguration` |
| `drishti-engine` | `ViewPipeline`, `Binder`, caches, `SourceRouter`, `SourceRegistry`, `PluginDiscovery`, `TopicHub`, `ViewStream`, `PatchDiffer`, suggestions, structured search, impact, business dates | `EngineConfiguration` |
| `drishti-identity` | Users, roles, passwords, lockout, audit, per-user preferences (workspaces, settings); the collaboration records (`com.ash.drishti.identity.collab`: shares, inbox, outbox, threads, holds, `HashChain`); stored in a JPA database (section A18) | `IdentityConfiguration` |
| `drishti-packs` | Domain packs: reads `config/packs/<domain>/<name>/pack.yaml`, resolves inheritance, contributes lowest-precedence properties (an `EnvironmentPostProcessor`) | `PackRegistry` |
| `drishti-ai` | The LLM provider layer and MCP client, no Spring: `LlmGateway`, `LlmRegistry`, providers (`openai`, `anthropic`, `mock`), `LlmPolicy` (masking, limits, audit), `McpGateway`, `McpRegistry`; off by default (AI_GUIDE.md, AI_GUIDE.md) | – |
| `drishti-diskcache` | `DiskCache`: a size-bounded, daily-cleared RocksDB cache on local disk for live connectors | – |
| `drishti-messaging` | `MessageStateSource`: the shared half of the ActiveMQ and RabbitMQ connectors (latest document per entity in a persistent RocksDB store plus a memory cache) | – |
| `drishti-lakeio` | Lake file access without Hadoop, shared by the native Delta and Iceberg engines: local disk (Windows included) and S3, byte ranges, Parquet page codecs, the Parquet footer cache; other schemes through a registered `StorageProvider` (`StorageProviders`) | – |
| `drishti-tables` | The table layer, plain Java (BI_GUIDE.md, section 24): the catalogue of virtual tables and models (`config/tables`, stored items), their definitions and checks, preview, lineage, the materialisation's API, `tables migrate`; on Rachana and the pivot engine, never on BI | read by the server (`tables` on the command line); BI reads it with the switch |
| `drishti-memlake` | The in-memory lake, plain Java (IN_MEMORY_LAKE.md): the `mem://` stores (the `mem` storage provider of `drishti-lakeio`), tables, replenish jobs, eviction, compaction, the owner binding, the guard (`Retention`), the host checks | wired by `server.memlake.MemoryLakeConfiguration` |
| `drishti-iceberg` | Drishti's own Apache Iceberg engine without Hadoop (reads, writes, path-based and REST catalogs), used by the `iceberg` (engine `native`) and `iceberg-external` connectors (Part III · Data architecture) | – |
| `drishti-deltalake` | A Delta Kernel engine without Hadoop (local disk, Windows included, and S3), used by the Delta connector | – |
| `drishti-lakeload` | The parallel lake loader: JSON Lines (folder or S3) into Delta or Iceberg tables of the connectors' layout, every stage multi-threaded and memory bounded, one commit per business date; `drishti.py lake build --engine java` (PARALLEL_LOADER.md) | `drishti-lakeio`, `drishti-deltalake`, `drishti-iceberg` |
| `drishti-net` | Drishti*net's core, no Spring (CLUSTER_GUIDE.md): the member table and SWIM-style gossip over HTTP (`MemberTable`, `Gossiper`, `PeersFile`), the cluster key and on-behalf-of tokens (`ClusterKey`, `OnBehalfOf`), the `config/net/` folder and its schema (`NetFolder`, `NetSchema`), signed HTTP between nodes (`PeerHttp`) | – |
| `drishti-registry` | The access registry, plain Java (ACCESS_MODEL.md): licence → module → lens → function → permission decided in one place (`AccessRegistry`, `EmbeddedAccessRegistry`), over the YAML store (`AccessStore`, `YamlAccessStore`: the modules' manifests, `config/modules/`, `config/access/`); nothing else reads the access model | wired by `server.modules.ModuleConfiguration` |

| Module | Responsibility | Contributes |
|---|---|---|
| `drishti-server` | **The** Spring Boot application, the core: `DrishtiApplication`, REST controllers, SSE, security (`TokenFilter`, OIDC), governance, alerts, actuator, OpenAPI; the SPIs a domain module fills (`PivotSourceProvider`, `CacheGroup`, `CapacityFigures`, `HealthFigures`, `ApiAlias`, `@ConditionalOnModule`) | controllers, filters |
| `drishti-compute-banking` | The banking domain's maths, no Spring (DOMAIN_MODULES.md): VaR and ES, parametric, components, backtesting, the scope algebra, exposure, FRTB-SA, SIMM, beta and correlation (`com.ash.drishti.banking.compute`) | – |
| `drishti-banking` | The banking domain module (DOMAIN_MODULES.md): the VaR, exposure, FRTB-SA, SIMM, sensitivities, MR cube and official-results services, storage and API under `/api/v1/risk` (`com.ash.drishti.banking`); its manifest and lenses; loaded only while the module is on | `BankingAutoConfiguration` |
| `drishti-app` | The runnable jar: the core with the domain modules this build ships, at `drishti-server/target/drishti-server-<version>-exec.jar`; no code | – |
| `plugins/drishti-plugin-*` | Source plugins: `demo`, `file`, `rest`, `jdbc`, `delta`, `aerospike`, `cassandra`, `redis`, `mongodb`, `iceberg`, `iceberg-external`, `duckdb`, `feeds`, `news`, `yfinance`, `kafka`, `activemq`, `rabbitmq`, `s3` (ServiceLoader; all are on the server's class path, extra jars from `drishti.sources.plugin-dir` get their own class loader) | – |
| `drishti-testkit` | Shared contract tests for plugins (`DatedSourceContract`, `MessageSourceContract`, `BrokerOutage`) | – |
| `drishti-it` | Architecture (ArchUnit: the layers, and the domain rules: the core never imports a domain, domains never import each other, only the registry reads the access model), licence-header and file-size tests | – |
| `drishti-benchmarks` | JMH benchmark of the hot path (`HotPathBenchmark`) | – |

**Inside `drishti-server`** (packages of `com.ash.drishti.server`): `api` (the controllers), `security`, `governance`, `collab` (shares, threads, compliance, bridges), `connectors` (connector files and the manager), `deploy` and `registry` (pack archives, the signed registry), `design` (the Build workbench's Designs), `explain` (About this page and Ask), `embed` (Drishti Elements), `loads` (data loads), `reports` (scheduled reports), `pivot`, `calc`, `layout`, `ai` (the AI and MCP endpoints), `cli` (the Java `sutra` tool), `net` (Drishti*net: the node, its endpoints, the call filter that checks the cluster key) and `bi` (Rūpaka: workspaces and catalogue, datasets and the DuckDB query engine, reports, boards, governance, live tables).

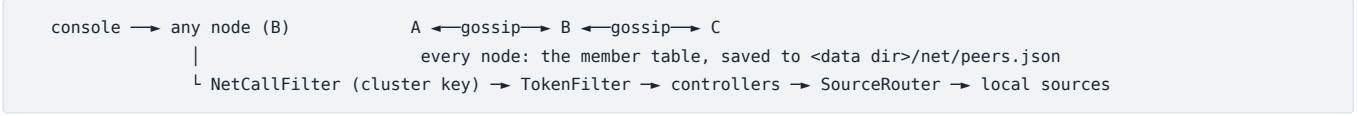
**Naming** (Pravaha convention): `XxxController`, `XxxProperties` (records bound via `@ConfigurationProperties`), DTO holders `ApiDtos`, `ApiExceptionHandler`, `XxxRegistry` / `XxxCatalog` instead of "Repository" (identity's Spring Data repositories in `identity.db` are the exception).

---

## A13a. Several servers as one (Drishti*net)

How a production cluster is deployed (roles, counts, sharding of VaR and exposure, high availability, load balancing, zero downtime) is CLUSTER_GUIDE.md, the deployment reference.

Off by default (CLUSTER_GUIDE.md, OPERATIONS_GUIDE.md (docs/guides/CLUSTER_GUIDE.md#1-several-servers-as-one-drishtinet)). With `config/net/node.yaml` switched on, a server is a **node**: `NetService` keeps the member table and runs a gossip round every second on a virtual thread (push-pull with three random members over `POST /api/v1/net/gossip`, probes, suspicion, death), saves `<data dir>/net/peers.json` and rejoins from it after a restart. `NetCallFilter` (before the token filter) checks the cluster key on every call between nodes. What each node offers, and the routing of a request to the node that can answer it, sit behind the source router (`SourceRouter`), so nothing above it learns about the cluster. `IdentitySync` and `StateSync` copy users, roles and user state between nodes over signed batches; a peer's change that would leave a node with no enabled administrator is repaired as it is applied (audited `last-admin-kept`). A member advertises only a real address (`HostAddress`: `advertise-url`, else this machine's primary IPv4 with the port; loopback and wildcard addresses are refused in the files and never kept from gossip).




## A14. Concurrency and performance

| Concern | Approach |
|---|---|
| Request I/O (fetch, links, search) | Spring MVC on virtual threads (`spring.threads.virtual.enabled`); the engine's `drishtiVirtualExecutor` runs one virtual thread per fetch, with a deadline (`orTimeout`) |
| CPU work (bind) | Bounded `ForkJoinPool` (`drishti.engine.bind-parallelism`, 0 = one per core); panels bind in parallel |

| Concern | Approach |
|---|---|
| Live state | One topic per `EntityRef`, one source subscription each (connected exactly once, even when subscribers arrive together); ticks land in a latest-wins slot and a single flush per frame delivers them; frames are timed by a small platform-thread scheduler (`drishti-frame-*`); one drainer per view stream, failed rebuilds retried with backoff |
| Caching | Caffeine in the engine: layouts (`layout-cache-size`, 10,000, keyed by Sutra version, kind and fingerprint), fingerprints (`fingerprint-cache-size`, 100,000, keyed by entity, generation and date), compiled expressions (`drishti.rachana.expression-cache-size`, 10,000). Connectors keep their own caches (Delta partitions by size, `cache-mb`; Kafka documents; RocksDB disk caches). The Delta plugin's loads that do I/O use async caches loading on virtual threads, so no lock is held during I/O. A Sutra change clears the layout cache. Admins see and purge the engine's, history's and the connectors' caches at **Admin → Caches** (`GET /api/v1/admin/caches`); VaR and exposure have their own `reload`, BI its dataset refresh. Every cache and memory-mapped file, its bound and the proposals to bound them all are in CLUSTER_GUIDE.md |
| Locks | `ReentrantLock` around anything that can block (file I/O, network, subscribes): Java 21 pinned a virtual thread blocked in `synchronized` to its carrier (Java 24 and later do not; the rule stays, so locks that may block remain explicit). `synchronized` remains only around short in-memory sections. Read-modify-write of shared records happens under the record's lock on the *current* value |
| Publication | State that readers need together is published in one volatile write of an immutable snapshot (Sutra registry, search index); readers never lock |
| Native resources | RocksDB generations are reference counted: a call enters the current generation and leaves after, and a retired generation closes only when its last caller has left, so a nightly clear or a purge never frees memory under a running read |
| Schedulers | The plugins' shared scheduler runs on virtual-thread workers, so a slow refresh or scan never delays another plugin's ticks or the nightly cache clearing |
| Back-pressure | One coalescing `FrameMailbox` per live client (pending frames merge, newest value wins), bounded alert queues; a slow client costs one slot, never a growing queue |
| Proof | Race tests reproduce each hazard with many threads and fail on the unsafe version: lockout under parallel guessing, admin changes against late sign-ins, the last-admin rule, simultaneous topic subscribers, index rebuilds during search, the stream cap, and RocksDB clear/close under 32 threads |
| Targets | warm view p99 < 50 ms, cold (first fingerprint) p99 < 150 ms, tick → screen p99 < 40 ms, 10k concurrent SSE subscribers per node (see CLUSTER_GUIDE.md for measured numbers) |

---

## A15. Configuration (config driven everywhere)

Precedence (low → high): pack content → `application.yaml` → `application.local.yaml` (git-ignored, imported from the working directory) → environment variables → `--key=value` arguments. `${VAR:default}` interpolation; no secrets in tracked files. The full key reference is CONFIGURATION.md.

| File | Owns |
|---|---|
| `drishti-server/src/main/resources/application.yaml` | server port, actuator, every `drishti.*` default: `sources`, `business-date`, `security`, `identity`, `rachana`, `governance`, `engine`, `packs`, `commands`, `live`, `graph` |
| `config/packs/<domain>/<name>/pack.yaml` | a pack's kinds, mnemonics, references, badges, roles, the names of its connectors with their routes (and connector templates), Sutra and sample folders |
| `config/connectors/<name>.yaml` | one connector of the site: plugin, switch, kinds and settings; watched and applied without a restart |
| `config/packs/<domain>/<name>/config/*.yaml` | the pack's formats, semantic hints, help and starter workspaces |
| `drishti-rachana/src/main/resources/formats.yaml` (+ `drishti.rachana.formats-file`) | core named number/date formats |
| `drishti-inference/src/main/resources/inference/semantics.yaml` | core semantic hints and density limits |
| `./sutras/**` (`drishti.rachana.dirs`) | site Sutras, in addition to the packs' |
| `drishti-console/config/application.yaml` | console host and port, server URL, default theme, sign-in and OIDC, help docs folder |

---

## A16. REST and streaming API (`/api/v1`)

The main endpoints; API_REFERENCE.md lists every one with examples. OpenAPI is served at `/api/docs` (Swagger UI at `/api/docs/ui`).

| Method | Path | Returns |
|---|---|---|
| `POST` | `/command {text}` | `{ref, mnemonic}` for `<MNEMONIC> <ID>` |
| `GET` | `/command/suggest?q=&limit=` | type-ahead entries (mnemonics, recents, entity hits) |
| `GET` | `/views/{kind}/{id}` | `ViewModel` |
| `GET` | `/views/{kind}/{id}/stream` | SSE: one `view` event, then `frame` events |
| `GET` | `/entities/{kind}/{id}/raw` | source document + provenance (F9), field masks applied for roles without `raw` |

| Method | Path | Returns |
|---|---|---|
| `GET` | `/search?q=` | structured search across entities |
| `GET` | `/impact/{kind}/{id}` · `/history/{kind}/{id}/diff` | F8 impact · what changed between dates |
| `GET` | `/sutras` · `/sutras/{name}/{version}` · `/sutras/problems` | registry listing · one Sutra · load problems |
| `GET` | `/rachana/schema` | the JSON Schema of a Sutra (Rachana language 1), with this server's kinds and formats, for editors |
| `POST` | `/studio/preview` · `/sutras` | ViewModel from an unsaved Sutra · save (a proposal when review is on) |
| `GET` | `/sutras/proposals` (+ `approve/reject/withdraw`) | Sutra governance |
| `GET` | `/sources` · `/packs` · `/about` · `/business-date` | plugins and health · packs · build info · resolved date |
| `GET` | `/health/live` · `/admin/health` · `/admin/caches` | live latency · everything's health · caches |
| `GET/PUT` | `/me/workspaces`, `/me/monitors`, `/me/settings`, `/alerts/rules` | per-user state |
| `GET/POST` | `/alerts`, `/approvals` | the one alert engine (the alerts raised for you) and the one Approvals queue |
| `POST` | `/auth/login` · `/auth/oidc` · `/admin/users…` | identity (see ADMIN_GUIDE.md) |
| `GET/POST` | `/forms/{pack}` · `/forms/{pack}/{kind}/compile` | query by form |
| `POST` | `/packs/{pack}/loads` | the batch-landed signal (DATA_GUIDE.md) |
| `GET/PUT/POST` | `/admin/connectors…` · `/admin/packs…` · `/admin/registry…` | connectors as files, packs, deploy, the signed registry |
| `GET/POST` | `/threads…` · `/shares…` · `/me/inbox…` | discussion and sharing (HOW_IT_FITS.md §A31) |
| `GET/POST/PUT` | `/bi/…` | Rūpaka: workspaces, datasets, models and the model query, Drishti SQL, reports (format schemas, themes; boards are report pages), Python, governance |
| `GET/POST/PUT` | `/net/…` | Drishti*net between nodes (signed with the cluster key: gossip, digest, fetch, search, group, subscribe, state) and for administrators (members, files, capabilities, access, security, explain, traffic) (OPERATIONS_GUIDE.md (docs/guides/CLUSTER_GUIDE.md#1-several-servers-as-one-drishtinet)) |
| `POST` | `/ai/chat` · `/ai/mcp/{server}/tools/{tool}` | the AI and MCP layers (off by default) |
| `POST` | `/embed/token` and the `embed:` scopes | embedded views for other web applications (EMBED_GUIDE.md) |

Errors are RFC 7807 `application/problem+json` with a `code` (`DRS-nnnn`) member, for example:

```
{"type":"about:blank","title":"entity not found","status":404,
 "detail":"DRS-1001 no source holds trade/IRS-48213","instance":"/api/v1/views/trade/IRS-48213","code":"DRS-1001"}
```

### A16.1 Data formats on the wire: JSON by default, Arrow where it pays

Every answer is **JSON** unless a client asks otherwise. A client that sends `Accept: application/vnd.apache.arrow.stream` gets **Arrow IPC** where the server offers it, cell for cell the same table, with the rest of the answer in the schema metadata. Anything Arrow cannot carry falls back to JSON, and nothing changes for a client that does not ask. Where each format is used, and why (measured 2026-10-30; DATA_GUIDE.md › Data formats: JSON Lines is the standard, section 6):

| Boundary | Format | Why |
|---|---|---|
| Lake | Parquet (Delta or Iceberg) | Arrow's partner on disk: compressed columns, statistics to skip files |
| VaR hot files | Drishti's own float32 rows in hierarchy order, memory-mapped, float64 node totals | summing rows runs near memory bandwidth (about 70 GB/s on 24 threads); Arrow would not be faster |
| Shard partials between servers | raw vectors of doubles | already minimal; Arrow would add framing |
| Server ↔ Python compute workers | Arrow IPC files, memory-mapped | no copy, no parse (BI_GUIDE.md) |
| Server → console → browser: **VaR contributions** by trade or position | **Arrow** on request; Arrow JS in the browser (vendored) | 61–65% fewer bytes, faster fetch |
| BI queries | Arrow on request (API clients); JSON in the console's own pages | mostly text: Arrow 8–14% smaller after gzip, parsed slower, painted no sooner |
| VaR P&L paths, entity pages, ViewModels | JSON | small answers; Arrow is no smaller |

| Boundary | Format | Why |
|---|---|---|
| Search results | HTML drawn on the server | no table reaches the browser; drawing it in the browser belongs with the client-side renderer |

Every front-end library, Arrow JS included, is vendored under `drishti-console/web/static/vendor/` and listed in its `VENDOR.md`; no page loads anything from a CDN.

### A16.2 Choosing a data mechanism

Drishti has five ways to hold and serve data. Each new kind or measure uses the **lightest one that meets its need**; the heaviest, the hot file with sharding, is kept for the few measures that cannot do without it.

| Shape of the need | Mechanism | Examples |
|---|---|---|
| Entity documents read one at a time or searched | **The lake** (Delta or Iceberg) through connectors, drawn by Sutras | trades, reference data, market data, the public-data packs |
| A figure per entity derived from its inputs | **Computed (derived) kinds**, computed on demand, never stored (HOW_IT_FITS.md section D12) | NSMTM, CPMTM, SSPNL, the Checks panels |
| Heavy or non-linear models per request | **Python compute workers**, Arrow in and out (BI_GUIDE.md) | pricing, Monte Carlo, what-ifs |
| Aggregates for dashboards and reports | **The BI engine**: rollups and caches (HOW_IT_FITS.md §A28) | desk reports, live exposure reports |
| Large vectors per position, re-aggregated over any scope, interactively, beyond one server | **The hot file and sharding** (HOW_IT_FITS.md section D13) | VaR |

**The mechanism check.** A measure uses the hot file and sharding only if **all four** hold, answered with sizes at bank scale in its design document before anything is built:

1. **Vectors per position:** each position carries a long array (scenarios, time steps, paths), not a handful of fields.
2. **Any scope, on demand:** people ask for any desk, book, basket or "bank minus X", not only fixed reports.
3. **Partials combine simply:** each server sums its part and the coordinator adds the parts before the final non-linear step (a quantile, a maximum, a percentile). When a group must be formed first (exposure's netting sets), the shard key keeps each group on one server.
4. **Too big for one server, or growing toward it,** and needed in under a second.

If all four hold, the measure plugs into the measure-agnostic machinery: it brings a `ShardedMeasure<P>` (its name, key order, bytes per row, partial type and how two partials combine) and reuses the shard map, ownership, replicas, handover and scatter-gather (HOW_IT_FITS.md section D13). If not, it uses one of the lighter mechanisms. Likely future users of the heavy one: exposure paths (sharded by netting set), stress P&L across hundreds of scenarios, FRTB internal-model ES and DRC simulations, climate scenario losses per position.

### A16.3 How memory-mapped data follows the lake

A hot file is a **copy** of lake data made for speed, so it must never be used after the lake has changed. Three things keep it honest:

1. **The data-load signal (the designed path).** A loader's last step announces "kind K landed for business date D" (DATA_GUIDE.md). Drishti drops the connectors' caches for that kind, and the engines that hold the kind react: a full load of D reloads that date (a new hot file), an intraday batch is applied on top of the morning's without a reload, and the official records follow. The same signal fires alerts and tells people.
2. **The inputs' digest (the safety net).** A hot file is named after a digest of its inputs: the **data versions** of the tables it was built from (for Delta, a stamp of the date's files in the table's current version, read from its `_delta_log`, so a commit to another date leaves it unchanged; for Iceberg, the date's data files in the current snapshot), the scenario set, the hierarchies and the position rules (VAR_GUIDE.md section 26.2). When a date is opened or reloaded and any version has moved, the digest no longer matches: the file is not used, a new one is built from the lake, and the old one is deleted. A file is never patched in place to follow a restatement.
3. **Provenance on every answer.** Every figure names the data version, batch and scenario set it came from, so anyone can see which state of the lake an answer reflects.

**Without a signal** (a table changed by hand, a loader that skipped its last step), the connectors see the new table version within `refresh-seconds` (10 s) and BI on its next query; an **engine's open date** notices it too, through the **version watch** (O9): on a request, at most every `drishti.var.version-check` (30 s; exposure's `drishti.ccr.exposure.version-check`) the date's lake version is compared with the one it was loaded from; when it moved, the date is reloaded in the background and swapped in, and the answers meanwhile carry the warning "the lake's data of D changed at T; a new version is loading". A VaR date with an intraday layer is left to its signal. Staleness without a signal is so bounded by the interval plus a load; the signal still makes the change visible at once, which is why it stays part of every production loader. Detail in CLUSTER_GUIDE.md section 4 and section 14.

---

## A17. Console (UX)

Python 3.12, FastAPI + Jinja2: `core/` (app, settings, auth, backend client, business date), `routes/<area>_routes.py`, `web/templates/<area>/`, `web/templates/_macros/`, `web/static/{css,js,img,vendor}`. All libraries are **vendored** (Bootstrap 5, Bootstrap Icons, ECharts, ECharts GL, CodeMirror, Vega and Vega-Lite, and Pyodide for Calc; vanilla JS, no build step); no CDN, no inline script (strict CSP); static assets are fingerprinted.

**Themes** — tokens only in `web/static/css/drishti_tokens.css` (gradient chrome in `drishti_gradients.css`); ten themes selected by `data-theme` on `<html>`: `terminal` (default, the navy/amber of the mockups), `light` (parchment), `wallstreet` (Bloomberg Terminal colours: black ground, amber data, yellow commands, green/red ticks, blue highlight — colours only, not fonts), `blue`, `green`, `crimson`, `crimson-dark`, and `terminal-green`, `terminal-crimson` and `terminal-blue` (the Terminal look in one dominant hue: phosphor green with lime, crimson with blush, azure with periwinkle). The choice is saved to the user's account and mirrored in `localStorage['drishti.theme']` for first paint. Contrast is checked by `drishti-console/tests/test_contrast.py` (≥ 4.5:1 for every token used as text — ink, muted, faint, link, accent, accent-strong, pos, neg, ok, warn, bad — on bg, bg-2, surface and surface-2, and for `--d-on-accent` on accent fills; ≥ 3:1 non-text). Layout is checked at 390, 1600 and 2560 px by `drishti-console/tests/test_page_widths_browser.py` (no page scrolls sideways; images fit their column).

### Pages

| Route | Page |
|---|---|
| `/` | Landing: hero animation, stats strip, "one grammar" section, capabilities, mockup showcase |
| `/t` | Terminal home: command line, recent entities, watch list |
| `/go?q=` | Resolves a command and redirects to its view (or to search) |
| `/v/{kind}/{id}` | Entity view (server-rendered first paint, then live patches over the tab's channel) |
| `/s?q=`, `/s?words=` | Structured search results (one generic page for every kind, with the Pivot tab where the pack declares `pivot:`); plain words shown as the search they make |
| `/p/{name}` | A pack at a glance (its code typed alone, e.g. `MKT`): its kinds, mnemonics, counts and README |
| `/v/{kind}/{id}/history` | History query: the entity over time (`TRD <id> 30`) |
| `/compare/{kind}/{id}` | History: what changed in the entity between two business dates, or since a "known at" time |
| `/impact/{kind}/{id}` | F8 Impact: dependents by reverse lookup, rolled up along pack-configured fields, with summed measures |
| `/var` | VaR explorer (`VAR <scope>`): any scope computed on demand, drill-down, captures and compare (hand-built, section A7a) |
| `/exposure` | Exposure explorer (`EXP <scope>`): counterparty exposure computed for any scope (hand-built, section A7a) |
| `/datamap` | Data map (`DATAMAP`): the packs' kinds and their joins, checked against the data |
| `/func` | Functions (`FUNC`): the command directory for everyone; administrators switch commands on and off there (`/admin/commands` redirects here) |
| `/what` | Knowledge (`WHAT <term>`): what the words mean |
| `/m`, `/m/{name}` | Monitors: live watchlists |
| `/alerts`, `/alerts/all` | Alert rules (Rachana-EL, evaluated server-side on every tick, edge-triggered), pack suggestions, history; a bell and toasts on every page |
| `/w`, `/w/{name}` | Workspaces: 2–4 embedded views with follow-the-selection between panes, saved per user on the server |
| `/studio`, `/studio/reviews` | Sutra Studio (editor + live preview) and the review queue |
| `/designer`, `/designer/new`, `/designer/grid`, `/designer/outline` | The Sutra designer and its modes (`ui.designer`, off by default) |
| `/build`, `/build/d/{id}`, `/build/new`, `/build/pack/new` | The Build home (every kind a person builds, My drafts), the workbench: one design (Sutra, samples, panels, controls, form, About), New pack from schemas |
| `/approvals` | Designs waiting for review; write-back changes waiting for an approver |
| `/forms`, `/form/{pack}` | Query by form: the forms home and one pack's form |
| `/reports`, `/reports/all` | Saved and scheduled reports, and BI reports |
| `/bi`, `/bi/catalogue`, `/bi/workspaces`, `/bi/models`, `/bi/d/{id}`, `/bi/r/{id}`, `/bi/designer`, `/bi/design/theme/{id}`, `/bi/m/{dataset}` | Rūpaka: home, catalogue, workspaces, models, datasets, report viewer (boards are report pages; `/bi/b/{id}` opens the viewer), BI Studio's report and model designers and the theme designer, Model view |
| `/bi/datasets`, `/bi/w/{workspace}`, `/bi/environments`, `/bi/run/{id}` | BI datasets, one BI workspace, environments (export and import a workspace), where a BI run opens |
| `/bi/notebook`, `/bi/transform` | the Notebook (SQL cells run by the server, Python cells on its compute pool), the Transform editor (Python data prep step by step) |

| Route | Page |
|---|---|
| `/inbox`, `/share/{id}` | The inbox and the page a shared link opens |
| `/help`, `/help/{slug}`, `/help/search`, `/help/examples` | Help centre: guides and the repository's docs rendered in-app; F1 opens help for the current screen; the Rachana examples |
| `/about` | Version and build, Java, uptime, loaded Sutras, source health, licence and notices; for administrators, the link to the competitive analysis (Help, `competitive-analysis`; `/about/competitive` leads there) |
| `/login`, `/logout`, `/account` | Sign-in and sign-out; own profile, settings and password |
| `/asof` | Sets the business date (and "known at") for the session, then returns to the page |
| `/servers` | One console, many servers: the list of servers and the one this browser works with |
| `/admin/users`, `/admin/roles`, `/admin/tokens`, `/admin/audit`, `/admin/access`, `/admin/health`, `/admin/caches`, `/admin/edits` | User administration, roles, API tokens, audit log, who looked at what, health of everything, caches, write-back edits |
| `/admin/packs`, `/admin/packs/{name}`, `/admin/connectors`, `/admin/connectors/{name}`, `/admin/loads` | Packs (switch, load, deploy, registry), connectors, data loads |
| `/admin/embedding`, `/admin/collab`, `/admin/ai`, `/admin/bi`, `/admin/bi/runs`, `/admin/bi/usage`, `/admin/compute` | Host applications for embedding, collaboration compliance, AI and MCP, BI settings, BI runs and usage, compute pools and requests |
| `/admin/lakes`, `/admin/artifacts`, `/admin/net` | Delta lakes and their maintenance, installed artifacts (install and uninstall safely), Drishti*net (members, files, capability map, security, access, routing) |
| `/api/channel` | The browser's single live channel, shared by its tabs (section A10); `/api/channel/who` the session it carries |

**Phones and tablets.** The console is responsive down to 360 px, with no separate app: views stack into one column and the strip shows two figures per row; tables scroll inside their panel; F-keys become a swipeable row of tap targets; inputs are 16 px so iOS does not zoom on focus; safe-area insets respect the notch; a web-app manifest lets people add Drishti to the home screen.

**Hero animation** (`web/static/js/drishti_landing.js`, canvas 2D, no library): raw JSON fragments drift in, are drawn into the `{ ◉ }` eye, and emerge as assembled terminal panels that then tick. Colours come from CSS tokens at runtime; it plays once on view, has a Replay button, shows a static final frame under `prefers-reduced-motion`, and pauses when hidden.

**Renderer contract** — each panel kind has a Jinja macro (`_macros/panels.html`) for first paint. Live patches reuse it: the console renders a changed non-chart panel with the same macro and sends the HTML; `drishti_live.js` swaps that one panel, updates strip cells in place, and hands chart panels new data — so a tick never re-renders the whole view. The kind-to-macro table and worked examples are in section A7a.

---

## A18. Security and identity

- **Sign-in.** Users, roles, preferences and the audit trail live in the **server** (`drishti-identity`), in a JPA database: SQLite by default (one file) or PostgreSQL. The console verifies a sign-in by calling `POST /api/v1/auth/login` with its own service token and never sees password hashes (PBKDF2). It then keeps a signed, expiring session cookie. Details, storage and administration: ADMIN_GUIDE.md.
- **Token boundary.** For every server call the console mints a short-lived HS256 token (`auth.token_secret` = server `drishti.security.secret`, `DRISHTI_TOKEN_SECRET`; 300 s). With `drishti.security.enabled`, `TokenFilter` requires it on every `/api/**` request (algorithm pinned, constant-time check) and answers 401 problem+json otherwise. Filters decide on the decoded path the dispatcher serves (`RequestPaths.routed`), and `PathGuard` first refuses, with `400 DRS-5001`, any request line under `/api` or `/actuator` that does not spell that path plainly (path parameters, needless `%`-escapes, dot or empty segments), so `/api/v1;x/…` or `/api/%761/…` cannot skip the token check (QA SEC-02). With security off (local development), the caller is taken from `X-Drishti-User` with every role.
- **Single sign-on** (OIDC, ADR-014): the console runs the authorization code flow with PKCE; the server verifies the ID token itself against the provider's keys (`RS256`/`ES256`, no `none` or HMAC), maps groups to roles and provisions the user. The token boundary is unchanged.
- **Entitlements** per role and kind (`drishti.security.roles`) and per user pack (section A12). Links to entities the user cannot open stay visible but disabled, with the reason (MAYA rule: *visible, not hidden*).
- **Field masks.** For roles without `raw`, the fields in `drishti.security.redact` read `•••` on every path where their value could be seen or inferred: raw JSON (F9), search, CSV, compare, history, Calc, pivots, views (strip, title, tables and totals, panel records, values derived from them), the live stream, monitors, Studio previews, Impact, the type-ahead, phrase search and alert rules. One function (`Entitlements.redactor`) masks the documents before anything reads them; Rachana-EL carries the mask through (a value computed from it is masked, a condition on it is never true), so a masked field can be neither seen nor probed (CONFIGURATION.md).
- **Collaboration** (share with a note, comment threads, the inbox; package `com.ash.drishti.server.collab`, records in `drishti-identity`). What travels is a person's words and a pinned link, never data: every share, comment, notice and email is rendered **for the reader, when they look**, from what they may see now (`Entitlements.mayReach`, `maskedValues`, `CommentRenderer`), and the records are tamper-evident (`HashChain`). The whole path is worked through in HOW_IT_FITS.md §A31, Share and Discussion end to end, the design and its as-built notes are HOW_IT_FITS.md §A31, the decision is ADR-020.
- **Console** CSP without `unsafe-inline`; all assets same-origin.

## A19. Observability

- **Health.** `GET /api/v1/admin/health` (Admin → Health) reports each connector's state, traffic, latency and last error, each pack's connectors and Sutras, live streaming and the server, with an overall `OK`, `DEGRADED` or `DOWN`. Spring actuator serves `/actuator/health` (with liveness/readiness probes), `/actuator/info`, `/actuator/metrics` and `/actuator/prometheus`.
- **Metrics** (Micrometer → Prometheus): `drishti_view_seconds` (time to build a view, p50/p99), `drishti_live_streams`, `drishti_live_topics`, `drishti_live_frames_total`, `drishti_live_latency_p99_milliseconds`, plus the standard JVM, HTTP and executor metrics. Cache sizes and hit rates are on Admin → Caches. A Grafana dashboard ships in `deploy/grafana/`.
- **Per-view timings** are in every ViewModel (`timings`), and Admin → Health shows per-connector latency.

---

## A20. Quality rules

- **Every source file ≤ 2000 lines** (UX templates, styles and scripts excepted) — enforced by `SourceFileSizeTest` (`drishti-it`) and `drishti-console/tests/test_assets_policy.py`.
- **Copyright header on every file** — enforced by `LicenseHeaderTest` (`drishti-it`); `tools/license_headers.py` adds missing ones.
- **ArchUnit** (`ArchitectureRulesTest`): `drishti-api` is free of Spring and of every internal module; `common`, `rachana`, `inference`, `graph` and `identity` never see `engine` or `server`; `engine` never sees `server`; `@RestController` only in `drishti-server`; no field injection; no `Serializable`.
- **Nothing unbuilt looks built:** every figure, panel, menu item, help page or decision that depends on unbuilt work is in the under-construction registry (`config/under-construction.yaml`, UNDER_CONSTRUCTION.md) and shows *Under construction*, or is hidden; one mechanism (the matched Sutra is rewritten; the console reads the same file), enforced by `UnderConstructionTest` (drishti-rachana) and `test_under_construction.py` (`tools/under_construction.py --check`).
- Golden ViewModel tests for the reference entities (`ViewPipelineTest`, `ReferenceSutrasTest`); plugin contract tests from `drishti-testkit`; a JMH benchmark of the hot path in `drishti-benchmarks`.

## A21. Key decisions

| # | Decision | Why |
|---|---|---|
| D1 | Backend is one Spring Boot application (never an embedded library); Python console | Single deployable, Spring wiring/config/observability everywhere; domain objects still testable without a context |
| D2 | Layout = Sutra ⊕ inference, cached by shape fingerprint | Unknown data renders immediately; cost paid once per shape |
| D3 | Sutras are YAML files (`*.sutra.yaml`, `rachana: 1`) with a compiled, side-effect-free EL (ADR-003, ADR-017) | Reviewable, diffable, safe; ordinary YAML tooling and a served JSON Schema; no scripting in layouts |
| D4 | Server returns ViewModel, not HTML | Same model feeds console, API clients and golden tests |
| D5 | SSE (not WebSocket) for live, one channel per browser (shared by its tabs through a SharedWorker or an elected tab) | One-way and proxy-friendly; a reconnect simply receives a fresh `view` event, so nothing is replayed; one channel per browser leaves five of the browser's six connections to pages, however many tabs are open |
| D6 | Vendored front-end assets, no build pipeline | Air-gapped desks; MAYA/Pravaha practice |
| D7 | Industries are packs that inherit; users live in the server | Neutral core; any number of consoles share users (ADR-009, ADR-010, ADR-015) |

The full records are in `docs/architecture/adr/`.

## A22. Testing architecture

**The pyramid, from the bottom.**

| Layer | What | Where |
|---|---|---|
| Unit and module suites | JUnit 5 in every Maven module (about 900 test classes): the grammar, inference, the engines, the stores, the connectors' contract tests from `drishti-testkit`, golden ViewModels of the reference entities (`ViewPipelineTest`, `ReferenceSutrasTest`); container suites (Testcontainers) for the JDBC, MongoDB, Redis, Cassandra and Aerospike plugins | `*/src/test/java` |
| Rule tests | ArchUnit (`ArchitectureRulesTest`, `OneGrammarRulesTest`) and the repository rules of `drishti-it`: file size, licence headers, branding literals, caches weighed in bytes, no blocking under `synchronized`, connection discipline, schema parity; `TokenScopeGuardTest` | `drishti-it`, `drishti-server` |
| Parity and regression checks | the BI parity recorder and `BiParityTest` (BI's figures against recorded ones), the render parity suites (the console's macros against the client renderer, every pack Sutra), the VaR regression check of a release (bit-exact on the Java path against the previous release, within 1e-12 on OpenBLAS; BENCHMARK.md section 17) | `drishti-server`, `drishti-console/tests` |
| Console, other lane | about 220 pytest files run in-process against a stand-in server (`conftest.py`'s `FakeBackend`): routes, templates, the help centre, the docs checks | `drishti-console/tests` |
| Console, browser lane | about 160 pytest files that drive Chromium through Playwright, against a stand-in console, a real server (`wb_live`) or a stack of their own (`*_stack.py`) | `drishti-console/tests/*_browser.py` |

| Layer | What | Where |
|---|---|---|
| Tool tests | the `unittest` and pytest suites of `tools/` | `tools/` |

**The drill** (`tools/drill.sh`, BUILD_GUIDE.md section 2.6 (docs/guides/DEVELOPER_GUIDE.md#26-the-drill-toolsdrillsh)) is how work reaches `main`. Stage 1, the gate: the static checks, the docs and help console tests, the quick file checks (every test marked `static`: it reads files and starts nothing, so it runs here and never in a lane) and one Java 21 build that runs only the rule tests, stopping at the first failure. Stage 2, at once: the console's browser lane and other lane, the tool tests, and the Java 21 and Java 25 verifies each in a copy of the committed tree. A failed console test runs once more, file by file; one that passes then is logged as a flake. Nothing is pushed unless everything passes.

**How the tests share the machine.** The other lane runs on up to eight pytest-xdist workers, a file per worker. The browser lane runs on as many workers as the memory available holds (2.5 GB each, after the Java verify's share; at most the CPU threads less two and at most twelve), handed out **by test**: a file that keeps a server or state between its tests is one group on one worker, and a small independent test marked `spread` is a unit of its own that any worker takes. Each worker starts its expensive things once and reuses them: one Chromium (its contexts reused per width, reset between tests), one stand-in console, a pool that keeps at most one idle real server, and the pack ViewModels bound once per run for every worker. Each worker's scratch ports are a block of their own. The lane's memory is sampled through the run.

**Rules for writing tests.** Small and independent (a test that can run alone on any worker is marked `spread`); no fixed pause over 100 ms in a browser test: wait on the page's settled signal (`Drishti.settled`: charts drawn, nothing aria-busy, no request unanswered, the page quiet) or on the condition itself; real servers only through the pool, never one per test; 300 s per browser file, or it is split or listed with its reason (the drill flags any other); a page is reloaded once, and only when Chromium cancelled its requests because the network changed (`net::ERR_NETWORK_CHANGED`); browser tests never run beside the Docker-based suites, whose containers change the network under Chrome. `test_browser_rules.py` checks the rules on the source. The full design, its measurements and the numbered rules: TEST_HARNESS.md.

## A23. One grammar, one engine

Every expression in Drishti is **Rachana**, and every aggregation is the **pivot engine's**. A sum over a trade's legs has one implementation, one treatment of nulls, decimals, currencies, units and masks, and one spelling, whether a Sutra, a BI measure, a virtual table, an alert or the pivot screen asks for it. The server computes; the console displays.

| Layer | Module | Owns | Never does |
|---|---|---|---|
| **The grammar** | `drishti-rachana` | the one expression language: paths, operators, filters, templates, links and the scalar functions (`fmt`, `upper`, `days`, `coalesce` ...); the parser and the type checker (checked mode) every caller uses | implement an aggregate |
| **The aggregation engine** | `drishti-bicompute` | every aggregate (built-in, once-per-parent, sketch, custom), grouping, pivots, totals, windows and the filter-context functions; the one function catalogue with each function's documentation | parse a language of its own |
| **Measures with their own engines** | the compute engines (VaR, exposure, FRTB-SA, SIMM) | their figures, exactly; reached through their pivot providers and through `measure(...)` | be summed by anyone |
| **Callers** | Sutras, BI, the table layer, the pivot screen, alerts, playgrounds, the API, the command line | write Rachana and ask the engine | carry their own parser or aggregate |

### A23.1 How Rachana reaches the engine

`drishti-rachana` keeps one function table (`rachana.el.Functions`) open to providers (`rachana.el.FunctionProvider`), found once by `ServiceLoader` when the table is built. `drishti-bicompute` provides every aggregate (`bicompute.rachana.EngineFunctions`, listed in `META-INF/services`): a list aggregate in an expression (`sum($.LEGS[*].NOTIONAL)`, `percentile(@.PNL, 0.99)`) turns the list's values into a `Batch` and runs the very `AggFunction` a pivot cell runs (`ListAggregates`); `group(list, key, aggregate)` is a Rachana form (it binds `@`) that hands the grouping to the engine (`ListGrouping`). The dependency is one way: the engine depends on Rachana for paths and expressions, never the reverse. Without the engine on the class path an aggregate compiles and is refused with the reason when evaluated (`DRS-2102`), never computed another way.

| Function | Means | Example |
|---|---|---|
| `sum`, `count`, `avg`, `min`, `max`, `median`, `percentile`, `distinct` ... over a list | the engine's built-ins on the list's values | `sum($.LEGS[*].NOTIONAL)`, `count($.EVENTS[?@.EVENT_TYPE == 'BOOKED'])` |
| `group(list, key, aggregate)` | grouping inside one document | `group($.LEGS[*], @.CURRENCY, sum(@.NOTIONAL))` |
| `measure(name, scope?, options?)` | a computed measure from its own engine, exactly (never summed); inside a model, a model measure | `measure('VAR', 'DESK:DESK-COMM')` |
| `cell(view, field, member, ...)` | one cell of a saved pivot view, as the view computes it | `cell('ann/desk-pnl', 'DESK', 'DESK-COMM', 'month', '2026-09')` |
| a Sutra **pivot panel** | groupings as Rachana paths, the value an engine aggregate | `by: "@.BOOK"`, `across: "@.CURRENCY"`, `value: sum(@.MTM)` |

`measure` and `cell` go through the engine's SPI `bicompute.spi.FigureSource`, which the server implements (`server.pivot.RachanaFigures`): `measure` asks the compute engine whose pivot provider offers the name for one cell; `cell` opens a saved view the person may open. Both answer for the person of the request (entitlements and masks) and are refused with the reason when the pivot engine is switched off.

### A23.2 One compiler: Java and SQL agree

BI's measures, a virtual table's columns and filters, row rules and the pivot's formulas are Rachana, checked by Rachana's **checked mode** (the type checker run wherever types are known: a table's source contract, a model's tables, a typed pivot; it only refuses more, never means something else) and compiled by one compiler in the engine (`bicompute.lang`: `Compiler`, `SqlLowering` for scans, `RowsEvaluator` for rows in memory, `ScanEvaluator` for SQL over a scan). A row expression becomes the engine's row expression (DuckDB SQL for a scan); an aggregate an `AggFunction` call (a formula of built-ins the engine's `FormulaFunction`, mergeable when its parts are); a filter-context function (`calc`, `share_of`, `parent_of`, `as_of`) the engine's sub-plan (`bicompute.context.FilterContext`, with `Periods` for balances); a window, a window over the answer. A differential test runs every operator, every scalar function, the aggregates with and without `where =` and every shipped expression through the Java and the SQL forms on a table of edge cases (nulls, empty lists, masks, zero divisors, mixed currencies) and compares value for value. The pivot's custom formulas are read by the same parser (`bicompute.formula.FormulaLowering`). How BI uses it, the scopes and the filter-context algebra: HOW_IT_FITS.md §A28 section 18.

### A23.3 The function catalogue

Each function lands **once**: scalars in Rachana, aggregates, windows and context functions in the engine. The catalogue page (Help → Functions, RACHANA_REFERENCE.md) is generated from the two registries (`FunctionCataloguePage`) and checked against ../guides/RACHANA_REFERENCE.md, the list function by function with its Excel, SAS and MATLAB equivalents. Every function has typed arguments and result, a null rule, a mergeable form or the word *holistic* (so totals and the cluster stay right), currency and unit rules (a sum of two currencies is refused unless converted), masks respected, a worked example the docs test runs, and its Excel equivalent where one exists, so the pivot's Excel export writes a live formula. Excel and SAS names are documentation, never aliases.

| Family | Examples | Runs in |
|---|---|---|
| Conditional aggregates | any aggregate with `where =`; paths with filters `sum($.LEGS[?@.LEG_TYPE == 'FIXED'][*].NOTIONAL)` (`SUMIFS`, `COUNTIFS`) | engine |
| Descriptive and two-variable statistics | `nmiss`, `range`, `mode`, `stderr`, `cv`, `skewness`, `kurtosis`, `iqr`, `geomean`, `trimmean`, `corr`, `covar`, `slope`, `rsq` | engine (moments and sums of products are mergeable) |
| Shares, ranks, windows, time | `pct_of_total`, `pct_of_row`, `pct_of_parent` (of the visible totals, Excel's "Show values as"), `rank`, `ntile`, `top_n`, `lag`, `lead`, `moving_*`, `running`, `ytd`, `prior_period`, `diff`, `pct_change` | engine |
| Filter context and balances | `calc`, `share_of`, `parent_of` (they drop filters, DAX's `ALL`), `as_of`, `opening`, `closing`, `latest`, `earliest` | engine |
| Text aggregate | `string_agg(x, sep, distinct?)` (`TEXTJOIN`) | engine |
| Math, logic, text, lists, dates | `round`, `power`, `if`, `ifs`, `switch`, `let`, `left`, `split`, `regex_extract`, `sort`, `unique`, `map`, `reduce`, `edate`, `workday(d, n, calendar)`, `yearfrac(a, b, basis)`, `intck`, `intnx` | Rachana (no aggregation: `reduce` with `+` is refused in favour of `sum`) |
| Finance on given flows, distributions | `npv`, `xnpv`, `irr`, `xirr`, `pmt`, `accrued`, `modified_duration`, `norm_dist`, `t_dist` | Rachana; `discount_factor` and `forward_rate` wait for a curve reader agreed with the quant engine |
| MATLAB-style list analytics | `cumsum`, moving windows, `zscore`, `isoutlier`, `polyfit` | registered into Rachana, computed with the engine's `AggFunction`s |
| Matrix functions | `mmult`, `solve`, `inverse`, `cov_matrix`, `eigen`, `pca(k)`, `cholesky` | planned, in the engine on drishti-native's linear algebra (HIGH_PERFORMANCE_NATIVE.md section 6); small matrices stay in plain Java |

**The boundary.** Pricing a trade from market data is the quant engine's job, and VaR, ES, exposure, FRTB-SA and SIMM come only from their own engines through `measure(...)`: the catalogue never re-implements them (no `var(vector, p)`). Python custom functions (NumPy in the sandboxed pool) serve a person's own code and are not a second implementation of a catalogue function. Candidates decided one by one: frequency tables with chi-square, regression with several variables, `forecast` and `trend`, winsorising, histogram bins as a dimension.

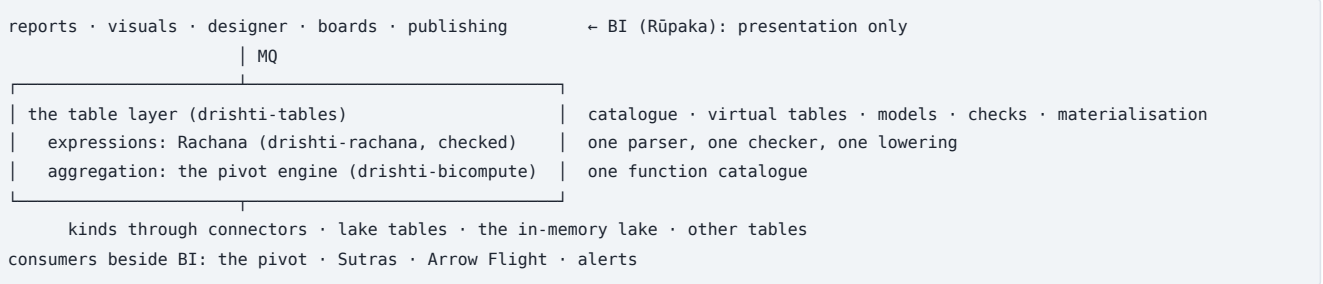
### A23.4 Guards, so it stays one

- `OneGrammarRulesTest` (`drishti-it`, ArchUnit): no class outside `drishti-bicompute` implements `AggFunction` or `FunctionProvider`; Rachana never depends on the engine; only `drishti-rachana` parses expressions. The listed exceptions are the VaR scope selector (`ScopeExpr`) and the migration tool's reader of BI's old expression language (it reads the texts it rewrites, never evaluates them).
- **The browser computes nothing.** `test_console_does_not_aggregate.py` finds every aggregation idiom in console Python and JavaScript (strings and comments left out) and lets through plain counting, sizes and layout arithmetic only. A figure the console needs over rows it read (a landing page's MtM and DV01 by book, a market median, a column filter's average) is asked of the

pivot engine through `POST /api/v1/pivot/inline` (`drishti-console/core/engine_figures.py`) and shown as the engine answers; with the engine switched off the figure is left empty with the reason. `ConsoleFiguresTest` (`drishti-bicompute`) proves each on the console tests' fixtures.

## A24. Virtual tables: the central table layer

A **virtual table** is a named, typed table defined by configuration in Rachana over any kind any connector serves (a lake table, a database table, an in-memory table), over a SQL query, a Python transform, a model's answer or other virtual tables. A **model** relates virtual tables (keys, relationships, cardinality, direction) and defines **measures** over them once. Entity documents (a trade with its `LEGS`, `CASH_FLOWS` and `EVENTS` in one JSON `doc`) become rows and typed columns, joined where they relate, and every consumer reads them the same way. BI keeps what is BI's (reports, visuals, the designer, boards); its datasets and models are the layer's tables and models. Every key, check and command: BI_GUIDE.md; the walk-through: the virtual tables tutorial (docs/guides/BI_GUIDE.md#tutorial-virtual-tables-and-models).




**The module.** `drishti-tables` is plain Java on `drishti-rachana` and `drishti-bicompute`; `drishti-server` and BI depend on it, never the reverse (`ArchitectureRulesTest` holds both directions). Its packages: `def` and `yaml` (definitions read by key and written in one canonical form, `rachana: 3`), `lang` and `check` (Rachana's tree and checked mode with the fields a table's source paths or a model's `alias.COLUMN` give; `LateBound` turns `fx(...)` and `related(...)` into columns worked out before the expression runs), `catalog` (files under `config/tables/<domain>/` and the stored items of BI's item store, `was:` names, tables over tables, lineage; `PackTables` generates a pack's kinds and the lists it names as tables, `trading.trade` and `trading.trade_legs`), `read` (a read for one viewer on one business date), `answer` (`ModelAnswers`: a model query answered in memory, each table as the reader may see it, joined many-to-one from each measure's fact, every measure compiled by the one compiler of section A23.2), `materialize` (`LiveStore`: live by key from pushed entities; `ParquetOnLoad`: a date's rows as Parquet, each build its own file moved in and swept), `preview` and `migrate` (`tables migrate`). The server's `server.tables.TablesService` reads the files and BI's item store as one catalogue, so the pivot, Sutras and BI see the tables and models the editors publish.

**How it is read.** Each column goes to the fastest route that holds it (a promoted column of a lake table, the materialised Parquet, a connector's column scan, else DuckDB's JSON over `doc`); a model's query compiles to the engine's grouping sets; the answer is Arrow batches. MQ (`mq: 1`, `mq: 2`) stays the one way to ask a model; a table's implicit model is `table:<name>`. The viewer travels with every call as one `ViewerContext` (user, roles, attributes, raw power, masks, kinds), never in a global, a shared cache or a thread-local. Materialised files are a cache: rebuilt at will, never the truth, never written into a lake's own tables.

**Decided** (the numbers are cited in code comments):

| # | Decided |
|---|---|
| V1 | A central layer of its own (`drishti-tables`, definitions under `config/tables/<domain>/`), not inside BI and not in the lake file: its sources are any kind any connector serves |
| V2 | BI's datasets are flat virtual tables and BI's models the layer's models, migrated by one tool (`drishti.py tables migrate`, dry run and report); stored items migrate at the server's first start, every version rewritten in place with the text before kept in its history |
| V3 | A flat table's columns are scalar Rachana paths or row expressions; aggregates exist only as a model's measures; a table that is not flat (SQL, Python, `keep: { latest }`) is allowed as a source, and the one way to aggregate into a table is a **summary table**, a model's answer (`from: { model, query }`) |
| V4 | Materialisation is one mechanism: `none`, `on-load` (Parquet per business date), `{ live: { rows, age } }` (in memory by key); a model's `rollups:` keep partial states; a model's `history:` samples measures of a live table |
| V5 | Every consumer reads the layer: BI, the pivot, Sutras, alerts and Arrow Flight; Flight's `table` form (a virtual table read as Arrow with its rules and masks, scope `data:read` or `data:read:<domain>`) and Flight SQL's `tables` and `models` catalogs are designed, as are Export data and the AI reading tables |
| V6 | One migration and one parity proof for the move of BI's datasets and models and of BI's expressions into Rachana |
| V7 | Many tables over one source (`trading.trade_summary` and `trading.trade_legs` both `from: { kind: trade }`); within one request the reads of one source for one business date are shared |
| V8 | A table a data load creates is **offered** a flat virtual table in one click (`drishti.py tables offer`), never made automatically |
| V9 | Administrators and BI authors define tables and models; reading is always as the source (the reader's row rules and masks on the source kind), then the table's own rules and masks, which apply to every reader (BI, the pivot, Flight), then a model's; nothing adds rights. Definitions are audited and versioned like BI items |
| V10 | A model may relate tables from different connectors; the engine joins them, and the scan estimate warns when a side is larger than the model's `scan-warn-rows`, else `drishti.tables.scan-warn-rows` (1,000,000), and unfiltered |

| # | Decided |
|---|---|
| V10a | Tables use look-ups (`related(…)`, many to one) and never multiply their rows; combining tables row by row is a model's relationships |
| V11 | Dates as the source has them: every read names a business date (none: the source's latest); known-at time travel where the source keeps history; an undated source gives its current rows, dated today |
| V12 | Names are `<domain>.<name>`, unique across virtual tables and stored tables |
| V13 | A Sutra panel may name a virtual table as its source (`source: { table, where, limit, as-of }`), read for the page's viewer; a page built for nobody in particular reads no table |
| V14 | DuckDB is the one SQL engine, required; the layer adds no new place that calls it |
| V15 | Rachana is the layer's only expression language; whatever moved into it was moved, never copied |

**Screens.** The editors (Build › Tables & models, BI_GUIDE.md section T11) follow the chosen mock-ups: tables as a visual view over a sample document with the columns and the rows a chosen role reads, switched with a YAML view (BC (docs/lenses/mockups/virtual-tables/BC.html)); models as a diagram with a live answer and a measures-first sheet behind one switch (model-BC (docs/lenses/mockups/virtual-tables/model-BC.html)). Every view edits one draft; a live YAML panel below every visual view follows each edit; invalid YAML is marked at its line and the visual side keeps the last valid version.

## A25. drishti-native: native maths behind abstractions, with Java fallbacks

*Product owner:* "*Key is we maintain abstractions and have fallbacks so we can run on any environment.*" Every native integration of the server sits behind a Drishti interface with a pure-Java implementation that always works, so Drishti runs anywhere. It realises the seam of HIGH_PERFORMANCE_NATIVE.md section 2.8 on Java 21 (JNI through JavaCPP) and keeps room for the Java 25 paths of that design (FFM, the Vector API) as further implementations of the same interfaces.

**What runs today.** Linear algebra: OpenBLAS (bundled through bytedeco for Linux and Windows x86-64) behind `LinearAlgebra` in `drishti-compute` (`compute.linalg`: `JavaBlas`, `NativeBlas`, `GuardedBlas`, `Canary`) and the server's `server.nativelib` (`NativeLibraries`, `OpenBlasEngine`), with the Java kernels as the fallback, `drishti.native.blas.mode` (`auto` or `java`), Health DEGRADED and an alert when it falls back (HIGH_PERFORMANCE_NATIVE.md section 6). The module `drishti-native` below is the design that gathers every native capability into one home.

**The rule.**

1. Every native capability sits behind a Drishti interface; only `drishti-native` imports a native binding (an architecture rule).
2. Every interface has a pure-Java implementation that always works; native code is an optimisation chosen at run time.
3. A native library that is missing, built for another platform, fails its self-test or misbehaves later is replaced by the Java implementation: a loud warning in the log, **DEGRADED** in Health, a banner in Admin; the server always starts.
4. `drishti.native.mode` (planned: `auto`, `off`, `on`) for everything, and a mode per capability (`auto`, `java`, `native`; today `drishti.native.blas.mode`), in configuration and live in Admin → Engines.
5. Both paths are proven equal by tests and a canary; one drill lane runs with every native capability off.
6. One version of everything: Maven Enforcer `dependencyConvergence`, `requireUpperBoundDeps` and `banDuplicateClasses`; only the `linux-x86_64` and `windows-x86_64` classifiers, never the `-platform` artifacts.

| Capability | Interface | Native | Java | Parity |
|---|---|---|---|---|
| Linear algebra, level 1–3 | `LinearAlgebra` (`dgemm`, `dgemv`, `ddot`, `daxpy`) | OpenBLAS (bytedeco) | Drishti's kernels (`JavaBlas`, `MultiSums`) | the near-tie rule makes VaR's order identical; sums within 1e-12 relative |
| Decompositions and solvers | `Lapack` (Cholesky, LU, QR, symmetric eigen, SVD, least squares, solve) | LAPACK from the same OpenBLAS jar (no new dependency) | Hipparchus | within 1e-10 relative on reconstruction; eigenvalues sorted and compared |
| Statistics and optimisation | `Stats`, `Optimise`, `Interpolate`, `Distributions` | none needed | Hipparchus, the one maths library of the server; Rachana's statistical functions call it through `drishti-native` | |
| Compression | `Codecs` (zstd, Snappy, LZ4, gzip) | zstd-jni, snappy-java, lz4-java | aircompressor (the same formats) | byte-identical after a round trip |
| Key-value disk store | `DiskStore` (get, put, remove, scan in key order) | RocksDB | H2's MVStore (on disk, keys in order; each keeps its own folder, so a switch starts the other's cache empty) | the same answers for every key |
| Embedded SQL engine | `SqlEngine` (run a statement with memory, thread and time limits, answer in Arrow) | DuckDB | **none, by decision**: DuckDB is required like the JVM; if it cannot load the server says so in plain words and does not start | |

SQLite, the default database of a single server, is required the same way (its driver registered for status and version; no third schema). LAPACK's first uses: Cholesky for correlated Monte Carlo paths and positive-definiteness checks, symmetric eigen for PCA of the scenario P&L matrix and repairing a non-PSD correlation matrix, SVD for low-rank views, least squares for hedge ratios and curve fitting.

**The module.** `com.ash.drishti.nativ.core` (`Capability`, `Registry`, `Guard`, `Canary`, `NativeMode`, the status record), `.linalg`, `.maths`, `.codec`, `.store`, `.sql`; plain Java on `drishti-common` only, wired by the server from its settings. Its users: `drishti-compute` (VaR and exposure kernels), the lake modules and loaders (codecs), `drishti-diskcache` (the store), the server (Health, Admin, settings). The Java 25 sources (FFM, Vector API) compile only under a Maven profile active on JDK 25 and are found by `ServiceLoader`: one home for every native and accelerated implementation, no second seam.

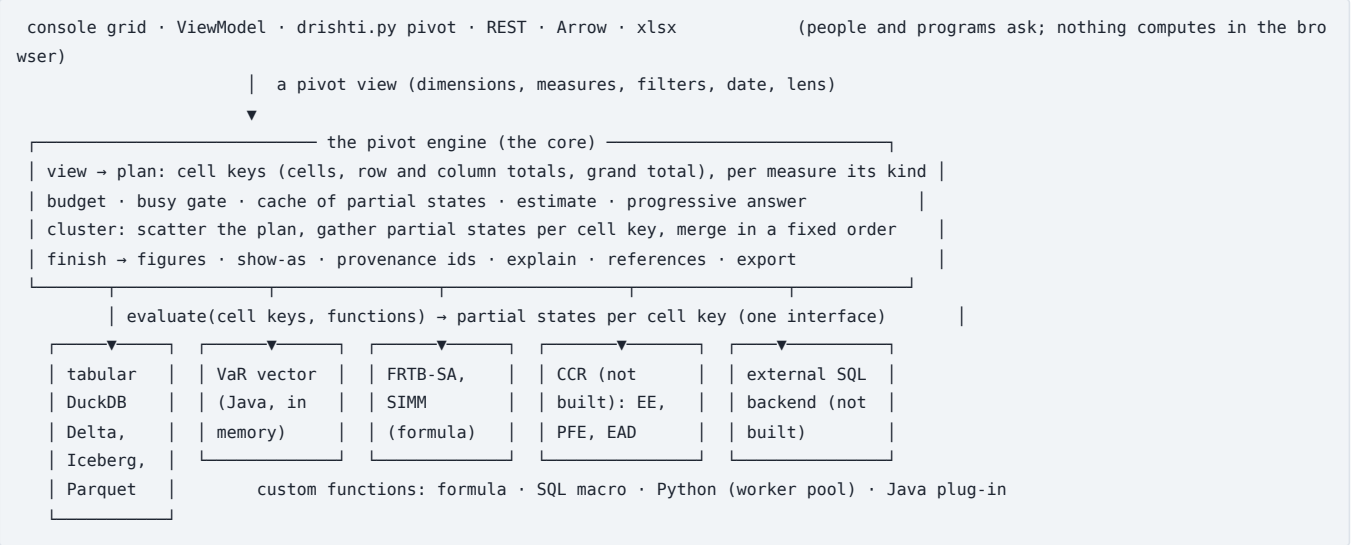
**The capability contract.** Every capability has the same parts: the interface in Drishti's types (Java arrays and buffers, never a native type); the Java implementation, always present and the reference for parity; the native implementation, loaded lazily once, declaring its OS, architecture, library file and Java version; a self-test at start (a mismatch keeps Java); a guard on each call's inputs (sizes, NaN and Inf, bounds), so a bad input is a Java exception, never a crash; a breaker (a native call that throws or answers a non-finite result where Java's is finite trips the capability to Java for the rest of the run); a canary every few hours; a switch; and a status in Health (`native.<capability>`), Admin → Engines, the start log and metrics (planned: `drishti_native{capability, impl}` and an admin endpoint). Native libraries unpack under `drishti.native.cache-dir` (`<data>/native-cache`).

**Beyond: the Python worker and other capabilities.** What exists only in Python (statsmodels, scikit-learn, the quant engine's models) is reached through a Python worker beside the server over Arrow Flight (`DoExchange`), with its own switch, timeout, health and canary; designed, not built. QuantLib is parked: when it is revisited, it runs in single-threaded worker processes (its Java binding, `io.github.ralfkonrad.quantlib_for_maven:quantlib`, ships Linux, macOS and Windows natives and has no sessions, so its evaluation date is process-wide), with a pure-Java core as its fallback. Further capabilities are chosen by capability, not by industry, each on the same contract: adopted are ONNX Runtime for machine-learning inference (Tribuo as the Java side) and Google OR-Tools for optimisation (Hipparchus simplex and Timefold); pure Java only are JTransforms (FFT) and simdjson-java; candidates when a client asks are GDAL and PROJ (GeoTools, proj4j), H3, and libdeflate if a benchmark shows gzip as the bottleneck. Not taken: FAISS, CUDA, a second SQL engine.

## A26. The pivot engine

One generalised pivot for all of Drishti: "these measures, broken down by these dimensions on rows and columns, under these filters, as of this date", for any amount of data and every measure Drishti has (a sum of notional, a distinct count, a 99% VaR, an FRTB charge, a function a quant wrote this morning), with every total right. How people use it is the Pivot guide (docs/guides/BI_GUIDE.md#part-1--pivot-cubes-and-explain); this section is how it is built. The mock-ups the screen was chosen from are in mockups/pivot/ (docs/architecture/mockups/pivot/A.html) (A, the Excel-familiar layout, was built; B and C are kept for reference).

### A26.1 The parts




**Terms.** A **view** is what the person asked. A **cell key** is a cell's coordinates (dimension → value, or "all" for a total) plus the view's filters; it names a cell without listing its members. A **function** is an aggregation with four steps (26.5). A **partial state** is what `add` and `merge` produce for one cell key on one node. A **provider** turns cell keys and functions into partial states over the data it owns. The **core** merges, finishes and shows; it never computes a figure itself.

**Why one engine.** The MR cube, the CCR cube, BI's matrix and the Pivot tab each draw a pivot; four pivots would mean four sets of totals, four budgets, four exports and four ways of being wrong. One engine means one rule for a total, one provenance, one Excel export, one test suite and one cluster merge. The MR and CCR cubes are **presets** of the pivot screen: a provider set and a landing, not engines of their own. Rachana's aggregates are the engine's functions too (section A23).

### A26.2 The pivot model

**The view** (what a saved view stores; the API takes the same as JSON): `source` (a table of a pack, a lake table, a panel's rows, or a cube's source such as `var`), `date` (or two for a comparison), `as-of` (latest, a Delta version, an Iceberg snapshot), `rows`, `columns` (each a field, optionally grouped: `{ field: TRADE_DATE, group: month }`), `values` (a measure, its label, `exact`, `show-as`), `filters`, `slicers`, `totals`, `currency` (report currency and FX source), `sort` and `top` (`{ field, n, others: true }`: never silent, "top 10 of 41, the other 31 in Others"), and the `expanded` nodes.

**Dimensions are typed and ordered**, published by each provider's catalogue: string (manual groups, top N with Others; a configured order or by value), date and time (day to year, the bank's calendar and fiscal year, buckets from today), number (buckets by width, by edges, quantile buckets), hierarchy level (MGMT, LEGAL, limit letters, risk-factor trees; ragged), scenario day (the VaR provider). A masked dimension cannot be used as rows or columns, and says why.

**Cell keys, not member lists.** A view of R rows and C columns has R × C cells, R row totals, C column totals and one grand total, plus every open subtotal level; the core turns them into cell keys (`{DESK: RATES-NY, month: 2026-10}`, `{DESK: RATES-NY, month: *}`, `{DESK: *, month: *}`) and each provider decides how to evaluate them: the tabular provider compiles the whole set into **one** `GROUP BY GROUPING SETS (...)` query; the VaR provider maps each key to the `OrdinalSet` of its positions and sums their vectors; the formula providers sum the members' sensitivities to the formula's grain. The **axis** (which values appear) is asked first, cheaply, so the cell count is known before any measure runs and a view over the budget is refused with the count, never truncated.

**The answer** carries per cell its coordinates and values, per total its kind and level, the diversification of non-additive measures (Σ children − node), the provenance id, the exactness per measure, the layer that answered (a rollup or the lake), progress and timing. Every value is a number, a number with `approx: true` and its error, or `null` with the reason in `notes` (masked, refused, a shard down, a currency that cannot be converted); nothing is blank without a reason.

### A26.3 Measure kinds and how each total is made

**A cell or a total is made from its members, never from other cells' figures.** What differs by kind is how cheaply:

| Kind | State | Merge | A total is | Marked |
|---|---|---|---|---|
| **Mergeable** | small and fixed (a sum; a sum and a count; Welford's count, mean and M2) | exact (associative and commutative) | the merge of its cells' states: no second scan | exact |
| **Sketch** | bounded (HyperLogLog registers, a KLL summary, a Space-Saving list) | approximate, within a stated error | the merge of its cells' sketches | ≈ with the error; exact on request |
| **Holistic** | unbounded (every value, or a vector per member) | none that is small | recomputed from its own members | exact (or refused over the budget) |

VaR and ES are **holistic statistics over mergeable inputs**: a scope's state is its summed scenario P&L vector, which merges exactly by element-wise addition, and the statistic (a rank, a tail mean) is taken per cell. Each cell and total is summed from its own members' vectors in the engine's fixed order, so it equals the VaR engine's answer for that scope to the bit (recomputing from members was chosen over merging the cells' vectors, which is faster but equal only to the cent).

**Built-in states.** `sum` (Kahan-compensated float64, or decimal for declared decimal columns), `count`, `min`/`max`, `avg` as (Σx, n), `wavg` as (Σwx, Σw), `var`/`stddev` as (n, mean, M2) merged by Chan's formula, a formula of mergeable parts as its parts' states, `first`/`last` by a time column. `distinct`, `percentile` and `median` are holistic and exact by default, with a sketch **form** (not a second name): HyperLogLog with 2^p registers (p = 12: about 1.6% relative standard error, 1.04 / √m), KLL with k = 200 (about 1.65% normalised rank error), Space-Saving with 10 × n counters for a level's top N (counts over by at most the smallest counter). Which form answers is the request's (`"exact"`) or, by default, the size's (`sketches.approximate-above-rows`).

**Exact on request without giving up scale.** A distinct count where the table is partitioned or clustered by the counted column adds per-node exact counts; otherwise each node sends its distinct values per cell up to a budget (5 million values a view), above which exact is refused with the size and ≈ kept. Exact percentiles use **iterative selection**: the KLL sketch bounds the answer to a narrow window, one more mergeable pass counts the values below it and collects those inside it, and the rank is found among them (two passes on most data, a third when the window holds too many values). VaR and ES are never sketched.

**Correctness the engine enforces.** Non-additive totals are right by the rule above; a sum across currencies is converted at a named FX source and time (in the provenance) or refused as mixed currencies; percentages, rates and day counts are not summed (the catalogue marks units, and the editor offers only the functions that make sense); row rules and masks apply in the compiled query before grouping, a masked column is never a dimension, and an aggregate-only role may require a minimum number of members per cell; the cell budget refuses with the count, top N always says "of how many"; every value carries its exactness; merges happen in a fixed order (shard order, then cell-key order), so the same question gives the same digits on every run and every node count.

### A26.4 The providers

**The interface** (`PivotProvider`, one for all): `catalogue` (typed, ordered dimensions, the measures and the functions it evaluates natively), `capabilities` (grouping sets, sketches, scope text, time travel, live, what-if, partials), `axis`, `estimate` (bytes, rows, files, seconds per node), `evaluate` (an aggregation plan → partial states per cell key, for this node's part), `drill` (a cell's member rows), an optional `scopeOf` (the cell as the provider's own scope text) and `version` (the lake versions read, part of every cache key). A plan is the cell keys, the functions with their kinds and state types, the filters, the date, the as-of version and the lens. A partial is, per cell key and function, a serialised state with its member count and notes; a provider may answer **final** figures for functions only it can finish (the regulatory formulas), and the core treats those as holistic.

**The tabular provider (DuckDB)** reads any table a pack declares by three routes chosen per table: the BI layers (the same DuckDB connection and Parquet files BI reads), a direct lake scan of large Delta or Iceberg tables (DuckDB's `delta` and `iceberg` extensions over the lake's files with the table's version pinned), and memory-only tables (`mem://`, registered as Arrow views with DuckDB's temporary directory off). A view compiles to **one** parameterised statement (values bound, never spliced), row rules and masks first:

```
SELECT DESK, date_trunc('month', TRADE_DATE) AS month,
       GROUPING(DESK, month)                    AS g,          -- which grouping set a row belongs to
       sum(NOTIONAL_USD)                        AS s0,         -- mergeable state of sum
       sum(PNL_1D), count(PNL_1D)               AS s1a, s1b,   -- avg as (sum, count)
       hll_registers(hash(COUNTERPARTY_ID), 12) AS s2          -- a sketch state
FROM   lake_scan('trading.trades', version => ?) t
WHERE  <row rules> AND PRODUCT_GROUP IN (?, ?) AND TRADE_STATUS <> ?
GROUP BY GROUPING SETS ((DESK, month), (DESK), (month), ())
```

It returns **states**, not figures, so answers merge across nodes, cache and finish in the core. DuckDB's native aggregates are used on a single node when the answer is final there; their mergeable state forms whenever partials must be merged or cached. A `direct` table (a PostgreSQL or DuckDB connector) gets the same SQL pushed to its source when every function is native to it. An in-memory provider (`RowsProvider`) computes the same states by the functions' own `add` over rows held in memory (a panel's rows, Export data's staged tables).

**The VaR vector provider** (`VarPivotProvider`): members are the date's positions or trades as the person sees them; a cell key maps to an `OrdinalSet` by dimension partitions (desk, book, limit letter, any hierarchy level, any column of the vectors); the state is the summed scenario vector (and the stressed vector when asked), summed in the engine's fixed order, on a sharded date one call per shard owner for all of a view's cells (`VarBook.vectors`, `ShardedVectors.sumSets`); the finish is VaR, ES and stressed VaR at the lens's confidence and horizon, component VaR as a cell's share of the grand total's tail, and the diversification beside each total. A cell's scope in VaR's grammar (`DESK:RATES-NY & CCY:USD`) lets any cell be asked of the VaR engine itself. The MR cube's figures, endpoints and landing are this provider plus the MR lens (`MrCubeExtras`, `MrSensitivities`); nothing in the console computes a figure.

**The FRTB-SA and SIMM providers**: state, the members' net weighted sensitivities summed per risk factor to the formula's grain (a sparse map, added in shard order); finish, the regulatory formula (bucket K_b, the three correlation scenarios; SIMM per CSA version), on each cell's own sums. They need those engines on.

**Designed, not built.** The **CCR provider**: state per netting set, the path matrix of MtMs netted and collateralised per netting set first (non-linear), then summed across netting sets per cell; finish EE, EPE, PFE quantiles, EEPE; EAD from SA-CCR's formula provider; a netting set never split across shards (with the counterparty engines; the CCR cube preset waits for it). An **external distributed SQL backend** (Trino, StarRocks or ClickHouse) as a provider, for interactive views over tens of terabytes by many people at once: the tabular compiler would emit its dialect and its answers come back as mergeable states; parked unless the benchmark shows common views over five minutes on three nodes.

**Documents: paths, lists and grains.** An entity document (a trade, a counterparty) is stored whole in a JSON column `doc` beside a few promoted typed columns (`layout.<kind>.columns`). Every pivot field is a **Rachana path** (`$.LEGS[0].CURRENCY`, `$.LEGS[?@.LEG_TYPE == 'FLOAT'][0].INDEX.NAME`, `size($.LEGS)`, `$.LEGS[*].NOTIONAL`), parsed by Rachana's own parser and compiled: a path a promoted or dataset column holds reads that column; anything else becomes DuckDB JSON over `doc` (`json_extract`, `unnest` for `[*]`, `list_filter` for `[?...]`); a Rachana function with no SQL form is refused for a pivot, never evaluated a second way (`DocPath`, `DocFields`, `DocView`). Three routes, fastest first, chosen by the engine: the promoted column, a BI dataset column extracted once per business date into the `cached` layer, an ad-hoc path at query time (shown by the scan estimate first; a large one refused with the cheaper route named; administrators see the most used ad-hoc paths with **Promote** and **Add to dataset**). **The grain comes from the path**: no `[*]` is the entity's grain, each `[*]` steps into a list, and the deepest list a view uses is its grain; a coarser figure in a finer view is counted **once per parent** (an aggregate over the distinct parents in each cell, `OncePerParent`); two lists not one inside the other (`LEGS` and `EVENTS`) are never crossed (refused with the reason); a filter on a list field means "any element" at the entity's grain and "this element" at the list's grain; masks and row rules apply per path.

### A26.5 Custom aggregation functions

**The contract**, the same for built-ins and custom functions in every language:

| Step | Called | Receives | Returns | Rule |
|---|---|---|---|---|
| `start(params)` | once per cell key per worker | the parameters | an empty state | pure; no data |
| `add(state, batch)` | once per columnar batch (Arrow, 64K rows by default) of the cell's rows | the declared input columns only, masked as the reader sees them | the state | vectorised over the batch |
| `merge(a, b)` | when two states of the same cell meet (batches, threads, nodes, the cache) | two states | one state | associative and commutative, within the declared tolerance |
| `finish(state)` | once per cell, on the coordinator | the merged state | a number or a small record | pure |

A state is serialisable with a declared size bound; it crosses processes and nodes and lives in the cache. A function declares its kind: mergeable, sketch (with the error its author states) or holistic (no `merge`: it gets all of a cell's rows in one call, bounded by `functions.holistic-max-rows` and `holistic-max-mb`, refused per cell over them; holistic cells are gathered to one node within the budget, and a view that would gather more is refused before any data moves).

**Four languages, one contract.** A **formula of built-ins** (the first route offered: compiled into the provider's plan, SQL on DuckDB and Java on the VaR provider, of its parts' kind); a **DuckDB SQL macro** (mergeable when the engine can rewrite it into states of mergeable aggregates, else holistic); **Python** in the node's sandboxed `pivot-fn` pool, vectorised over Arrow batches; a **Java plug-in**

loaded from an approved, sha256-pinned jar. A function is named `fn:<owner>/<name>@<version>`. User Python never runs inside the server's DuckDB (DuckDB's Python API has scalar functions only, and the server's process is not user code's place): the provider feeds the pool Arrow batches.

**The harness** (`FunctionHarness`): a function is not saved, shared or used until its split-and-merge answer equals its one-pass answer. It runs one pass; batches of 1, 7, 1,000 and 65,536 rows; splits into 2, 3 and 16 parts merged in random orders and tree shapes over five seeds; empty, null, single-row, all-equal, infinite and NaN inputs; and every state serialised and read back before merging. It fails a mergeable function beyond the tolerance (1e-12 relative; exact for integers and decimals), a sketch beyond its declared bound, and any function that is non-deterministic, too slow per million rows or whose state outgrows its bound. The **playground** (Pivot → Functions → New) shows the code, sample rows (pasted or a seeded, masked draw), the states after each part and merge, the check on every edit, the measured cost and its projection on a real table.

**Python's sandbox** (`drishti_compute.pivot_fn`, the compute SDK's audit-hook guard and Arrow codec): warm workers, `python -I`, an empty environment, one call at a time, a worker serving one function version only; no network, no file reads outside the installation and no writes at all, no processes; the pool's package allow-list; per-call time and per-worker memory limits; only the declared columns of rows already ruled and masked; a state within its declared size, validated before it is merged or cached. Batches travel as Arrow IPC stream frames over the worker's pipe, never through scratch files; Python functions never read memory-only stores (the view is refused in words).

**Versions, approval and labels.** Every save is a new version and a view names the version it uses; a function is private until an approver who is not its author approves it for a group or everyone (audited, revocable: views then say "function withdrawn"); every figure from a custom function is labelled with its name, author and version; the official risk measures stay Java engines, and a custom function can never take an official measure's name or appear in an official run.

### A26.6 Scale

**How a large pivot runs.** (1) Plan: the axes first, so the cell count is known; then the cell keys, functions and kinds. (2) Estimate: bytes, files and seconds from the lake's metadata after pruning, before any data is read; over the confirm level the person chooses (run, narrow, use a rollup, run in background). (3) Route: aggregate awareness picks the smallest layer that answers every function exactly (a rollup, the BI columns layer, the lake); a plan partly answerable from a rollup is split. (4) Scatter: files or shards are assigned to nodes; each node's executor compiles its part to one DuckDB query (or one vector batch) and returns partial states per cell key. (5) Gather and merge in node order, then cell-key order. (6) Finish, show, and cache the **states** by (provider data version, cell key, function).

**One executor per node**: a bounded pool of DuckDB connections (one per four cores by default), each with its threads and memory limit; a slow node's files are re-assigned (hedged) to an idle node holding the same data after `cluster.hedge-after-seconds`, and the first answer wins (both equal by construction). On shared storage files are assigned by hashing the file path over the nodes that run the `pivot` engine, so caches stay warm; node-owned data (VaR shards, a pack bound to a node, memory-only tables) is computed by its owner and only partial states move; a small table runs on one node. One request per node carries all of a view's cell keys. The ordered combine is the one VaR sharding uses (Part III · Data architecture, VaR sharding), lifted into the core for every provider (`OrderedMerge`, `PartialCodec`, `PartialGather`).

**Read less.** Only the plan's columns are read (Parquet is columnar); partition pruning and statistics skipping use Delta's per-file minimum and maximum (the first 32 columns by default), Iceberg's manifests and Parquet's row-group footers; tables are partitioned by business date and clustered (Z-order on Delta, sort orders on Iceberg) by the two or three dimensions most used as filters; the lake readers' disk cache keeps hot files local.

**Sketches.** HyperLogLog is computed in SQL in the same `GROUPING SETS` statement (`hash(x)`: the first p bits pick the register, the rank of the first 1-bit in the rest is its value; the maximum per register taken in Java; merges are register-wise maxima, exact to the byte; two hash families never merge). KLL is built in Java from each cell's quantile points (`quantile_disc` at 2k + 1 fractions) and its count, each point weighted by the values it stands for; its merge's coin is a hash of the sketch's own count, so a fixed order repeats to the byte. Space-Saving is built from a cell's value histogram; a level's top N by a count of rows may be chosen by one summary of the parent's cell when ≈ is asked (`TopFirst`), with exact figures and membership within the floor.

**Rollups of partial states, refreshed on load.** A rollup stores the states of mergeable and sketch functions at its grain; an appending load adds only the new files' states, a restatement recomputes only the partitions it touched; a plan is answered from the smallest rollup whose grain covers its dimensions, filters and row-rule columns (`AggregateAwareness`, with date coarsening), and the answer names it; the engine counts views that scanned the lake for want of a rollup and proposes one. Holistic functions are never rollup material.

**Estimates and progressive answers.** Seconds come from the bytes after pruning divided by each node's measured scan rate per storage kind plus a per-file cost, always as a range (`ScanRates`). A view estimated over `progressive.after-ms` answers twice: first ≈ within about a second from honest sources only (sketch states in a covering rollup, or a random sample of files within each partition with each mergeable figure scaled up and its 95% interval; never a coarser figure split by guessed proportions), then exact cell by cell (`Progressive`). Holistic measures are never sampled.

**Budgets, busy protection, the cache.** `max-cells` and `max-holistic-cells` refuse with the count and the dimension that makes a view large; views over `busy-cells` pass the admission gate first; the partial-state cache is weighed in bytes in the one memory budget, and a new data version invalidates only the cells of the tables it changed.

**DuckDB's limits, and what the design does.** One process on one machine: Drishti scatters and gathers states, DuckDB is the per-node executor. High-cardinality grouping can run out of memory despite spilling: the cell budget bounds the groups, states are kept small, holistic functions are size-bounded, the per-connection memory limit is set, and an out-of-memory is a refusal in words. Memory-only data may never spill: such views must fit in memory or are refused. Object-store bandwidth bounds S3 scans: prune first, cache on disk, add nodes. Many small concurrent queries: a bounded pool per node, queuing, the cache, rollups, coalescing of identical plans. The `delta` and `iceberg` extensions ship with the distribution and load from it (no download at run time), their versions pinned with DuckDB's.

### A26.7 Beyond Excel: explain, time travel, what-if, provenance, live, references

- **Explain a difference** between two cells or one cell over two dates (`Explain`, `PriceVolume`): for mergeable measures the difference splits exactly into contributions by member along any dimension, and by cause where the provider knows causes (new members, members gone, value changes; for price × quantity measures, the price and quantity effects in a fixed order with the cross term shown); for VaR, the VaR engine's exact steps per cell. The steps always add up, and a residual is named.
- **Time travel and compare**: any view as of a business date and a data version; compare two dates or versions cell by cell, new and gone members marked, each side's provenance naming its versions.
- **What-if overlays** (`Overlay`, `WhatIf`, `WhatIfResolver`): exclude members, scale or override an input column, add hypothetical rows; the tabular provider joins the overlay into the plan; only affected cells recompute; the original stays beside the result; an overlay is private, saved with the view and never written to the lake.
- **Provenance** per answer and cell: data versions, files or shards and nodes, rows counted, filters and row rules in words, functions with their versions, the layer, exactness and error, time taken; one click to the rows.
- **Cell references** by coordinates (`pivot://owner/view@version/measure[DIM=value, ...]`): they survive re-sorts, new columns and new dates; a gone coordinate says "not in the view", never 0; Rachana's `cell(...)` reads them.
- **Live pivots** (`LivePivots`, `AnswerPatch`): on a signal (a data load of a kind for a date, `LoadService.onReady`, or a live panel's rows through the `TopicHub`) the touched views are computed again through `PivotCore.query`; the cache keyed by each date's data version means only what moved is read, and the source's rollups are refreshed first. `AnswerPatch` sends `status` (nothing moved), `patch` (the moved cells whole) or `reshape` (rows, columns or measures changed); the patch applied to the last answer equals a fresh query, cell by cell. Recomputes are coalesced and ordered per view and exact unless the view asked ≈. Not built: the batch-as-merge form that updates per-cell states straight from a stream's rows.
- **The screen's switches** (Exact / Auto / ≈, Progressive, Use rollups, Live) are `ScaleOptions` beside the view, never in it; every answer carries `computedAs` (what was asked, which measures sketches answered with their errors, which levels a summary chose, the routes, the sample, one line of words).
- **Cell comments** (`PivotCellComments`) are kept beside a saved view's numbered versions (one document changed atomically), each naming its cell by coordinates and the view version it was written on; audited without the text.

### A26.8 Expand and collapse

Rows and columns are nested levels with **+** and **−**, compact, outline or tabular layout and subtotals above or below. Only expanded nodes are computed: opening a node is a new plan of that node's children only (one `GROUPING SETS` query, or one vector batch for VaR), children already computed come from the cache, and the expansion state is part of the saved view and of a link. Any member may be expanded by a **different** dimension (each node is its own cell key, so totals stay right); hierarchies may be ragged or parent-child (a cycle is refused at load with the members named); a level's top N has one **Others** computed from its own members (for VaR, from their vectors), never total minus the top N; each level sorts independently and never moves a child out of its parent; the last level expands into the member rows, virtualised and paged; the export keeps the outline as Excel's native grouping levels.

### A26.9 BI on the pivot engine

BI keeps its own design (datasets and models, one model per report, MQ as the one way to ask a model, time modes; HOW_IT_FITS.md §A28 section A14) and its compiler of a model's rows (row rules, masks, joins, layers, dates). What moved is **who aggregates**: a matrix's totals and subtotals, a table's total row and a model query's `totals` are every level in one `GROUPING SETS` statement of the engine's function states over the model's rows as the reader may see them; a measure's additivity is the engine's answer for its function (`plainSum`); the console adds nothing up. BI's compiler writes the model's **detail** query and does not run it: it becomes the FROM item of the engine's statement (`QueryService.Outer`), run in BI's engine where the layers are, so no row leaves it; measures of several facts get one statement per fact, the levels joined (`BiMeasures`, `BiAggregate`). A model's measure that is an expression of aggregates of one fact's rows is a pivot function; time intelligence, running and analytic functions, filter contexts, balances and Python measures read beyond their group and have no total (a dash, with the reason in the warnings). A dataset read directly from its source (`mode: direct`) gets no engine totals. The move had one path: parity tests against figures recorded from the code it replaced (`BiParityTest`, `TrueTotalsTest`), then the old aggregation code deleted in the same change. Not built: models as sources of the pivot screen, and BI's rollups as rollups of partial states.

### A26.10 The module: drishti-bicompute

A plain Java 21 library, no Spring, holding everything that aggregates; its dependencies are `drishti-api`, `drishti-common`, DuckDB (`duckdb_jdbc`) and `arrow-format`. **No other module re-implements aggregation**: a build test (`AggregationOwnerTest`) fails any class outside it that implements the function SPI's steps or emits `GROUP BY` for a pivot.

| Package | Holds |
|---|---|
| `core` | the pivot model, plans, merge and finish: `PivotView`, `CellKey`, `CellSet`, `Axis`, `AggregationPlan`, `PivotCore`, `PivotAnswer`, `ShowAs`, `ScaleOptions`, `TopFirst` |
| `spi` | `PivotProvider`, `AggFunction` (start, add, merge, finish), `Kind` (mergeable, sketch, holistic), `Batch`, `Viewer` |

| Package | Holds |
|---|---|
| `functions` | the built-ins and sketches: `Builtins`, `Functions`, `Hll`, `Kll`, `SpaceSaving`, `Sketches`, `OncePerParent`, `Windows`, `Final` |
| `sql` | `GroupingSetsCompiler`, `DuckDbExecutor`, `TabularProvider`, `TabularSource`, `SampledResolver`, `RowFed` |
| `rows` | `RowsProvider` (rows held in memory) |
| `formula`, `custom`, `python`, `harness` | formula functions and SQL macros, custom-function storage, versions, approval and official-run guards, the Python worker client, the harness and playground |
| `cluster`, `cache`, `rollup`, `scale`, `live`, `explain`, `whatif`, `export` | `PartialCodec`, `OrderedMerge`, `PartialGather`, `ClusterRoute`; `StateCache`; `Rollups`, `AggregateAwareness`; `ScalePolicy`, `ScanRates`, `Progressive`; `LivePivots`, `AnswerPatch`; `Explain`; `WhatIf`; `PivotExport` |

The server wires it (`server.pivot`: `PivotController`, `PivotViews`, `PivotViewVersions`, `PivotCellComments`, `KindSources`, `PanelSources`, `DatasetLayers`, the document fields, the Python process client, the scale and live configuration): Export data's tables as sources, the VaR, FRTB-SA and SIMM providers (which implement the SPI; the vector arithmetic stays where it is), REST, busy protection, the memory budget for the cache, saved views and audit. The console's `drishti_pivot-engine.js` asks and lays the answer out; `drishti_pivot-grid.js` is the one grid; the presets are `drishti-console/config/pivot-presets.yaml`.

### A26.11 The screen, charts and export

The screen is Excel's PivotTable as people know it (field list and four zones, show-as, calculated fields over states so they are right at every level, typed grouping, slicers, conditional formats, drill-through) with what Excel lacks where Excel's equivalents would be (the switches, provenance, comments). On a phone the grid's top-left and the headline come first, charts at a glance, expand, drill, explain and provenance under More, and the field list and the function playground on the desk only (not built beyond the narrow layout). **Charts of a selection** are suggested from its shape, linked both ways to the grid, and **never stack a non-additive measure**: VaR by desk is drawn beside the bank's VaR with the diversification as its own segment, from the engine's own gap; drawn with drishti-chart over the vendored ECharts. **Export** asks what (the pivot as shown with its outline, a cell's underlying rows, a chart's data, the view definition) and how: xlsx with Excel's native outline levels, a `SUM` formula only where a total is a plain sum of its parts and the server's value otherwise, a Provenance sheet (data versions, functions with versions and authors, filters and rules in words, exactness, who and when); CSV (one row per cell and measure, or the grid; zip and gzip for several or big files); JSON Lines; Arrow. Exports stream from the server, memory-only data is never written to a temporary file, and every export is audited without its values. Not built: a native PivotTable with its pivot cache (only measures Excel re-aggregates correctly would be data fields, the rest on a Values sheet), charts bound to it, references as named ranges, Parquet and Power Query outputs, pinned charts with PNG and SVG export.

### A26.12 The switch, the API, the command line, rights and audit

`pivot` is a compute engine with its own file, `config/engines/pivot.yaml` (the engine catalogue: `mr-cube` needs it, `ccr-cube` will); off, it does not exist on that server (`PIVOT`, the menu, Help pages, `drishti.py pivot` and the playground disappear, `/api/v1/pivot` answers `404 DRS-1020`, the cubes cannot be on). Its settings are in CONFIGURATION.md. The API (`/api/v1/pivot`: query, estimate, expand, cells and their rows, provenance, compare, views, catalogue, functions and their test, live, export; explain under `/api/v1/explain` kind `pivot`; function approval in the one Approvals queue) and `drishti.py pivot` use the same words, and a refused view prints the same sentence as the screen. **Rights**: the provider applies the person's row rules and masks in the plan before grouping (BI's security rewriter for models, the VaR engine's scope entitlements for VaR); a shared view opens with the reader's entitlements and never carries the author's numbers; custom functions receive only declared columns of rows the reader may see. **Audit**: every view run (who, source, data versions, rows read, functions with versions, time), every export, every function save, test, approval and revocation.

### A26.13 Tests that prove it

`GroupingSetsCompilerTest` (one query for cells and every total level, parameters bound, row rules before grouping), `MergeOrderTest` and `ClusterMergeTest` (the same digits on 1, 2 and 3 nodes and any arrival order), `TrueTotalsTest` (every non-additive total equals the measure over the union of its members), `SketchErrorTest` (HLL, KLL and Space-Saving within their stated errors on seeded data; exact on request equals exact), `ExactQuantileTest`, `FunctionHarnessTest`, `CustomFunctionsTest`, `FormulaFunctionTest` and `PivotFormulaParityTest` (formulas equal in Java and SQL), `test_pivot_fn.py` (Python's four steps and sandbox), `VarPivotApiTest` (every cell and total equal to `/api/v1/risk/var/scope` to the bit, sharded and whole), `CurrencyAndUnitsTest`, `ShowAsTest`, `ExplainAddsUpTest`, `WhatIfOverlayTest`, `RollupTest`, `ProgressiveTest`, `ScaleSwitchesTest`, `LivePivotTest`, `ParquetRouteTest`, `DocumentPivotTest` (paths, grains, once per parent, lists never crossed), `PivotExportTest`, `BiParityTest`, `AggregationOwnerTest`, and the browser tests of the screen. The benchmark harness is `tools/bench/pivot_bench.py`; its runs (synthetic trades of 5, 25 and 75 GB on one and three nodes, cold and warm, with and without a rollup; larger sizes only extrapolated and labelled so) and results are in BENCHMARK.md.

### A26.14 Not built yet

Beyond the items named above: the per-person limit on running large views; the batch-as-merge form of live pivots on streams; layout advice (a proposed clustering from the dimensions views filter by); asking for a view in words through the AI layer (it would propose a definition the person sees before it runs, never computing a figure); automatic insights (outliers, sudden changes, concentration, flagged with the reason); references in formulas and Excel; the native PivotTable export; tabular reads of the BI layers and of object stores through the pivot (local Parquet files are read today).

## A27. Compute engines

A compute engine (VaR, exposure, sensitivities, FRTB-SA, SIMM, the MR cube, the pivot engine, the in-memory lake, the data inbox, Arrow Flight, OpenBLAS) is switched on or off per node, each with its own configuration file, and off means the engine does not exist on that node. How an administrator uses it (the files, what off means for people, dependencies, Admin → Engines, the cluster, the command line) is ADMIN_GUIDE.md section 21; this section is how it is built.

### A27.1 The catalogue

The engine catalogue is part of the product: `drishti-server/src/main/resources/engines/catalogue.yaml`, with the console's byte-identical copy `drishti-console/config/engines.yaml` (`EngineCatalogueTest`). Each entry holds the engine's id, title, file, the settings prefixes it owns, what it **needs** (on the same node: the needing engine calls the needed one in process) and **uses** (works without it, with the parts that read it hidden), and everything it brings to the screen: `words`, `menus`, `help`, `landings`, `tiles`, `lenses`, `bi`, `cli`, `playgrounds`, `connectors`, `kinds`, `ai-tools`, and the console's paths (`console:`; `=/play` is that path only). The console's menus, Help, tiles and lens links, and Admin → Engines' "what turning it off hides", all read these entries; nothing is coded per engine. The longest API prefix wins (`/api/v1/risk/sensitivities` is the sensitivities', not FRTB-SA's). A new engine (SA-CCR, FVA, the CCR cube, liquidity, climate) adds one entry, its files and a `ComputeEngine` bean; `installed: false` marks an engine listed ahead of its build. The catalogue's build test refuses a cycle in `needs`. A dependency read over Drishti*net (`needs-anywhere`) is reserved; no engine declares one.

### A27.2 The files and their order

Each engine's file exists twice, byte for byte: `drishti-server/src/main/resources/engines/<id>.yaml` in the jar and `config/engines/<id>.yaml` for the site. `application.yaml` imports both, jar copy first, **before** the local file and the packs overlay:

```
spring:
  config:
    import:
      - classpath:engines/var.yaml
      - optional:file:${DRISHTI_ENGINES_DIR:./config/engines}/var.yaml
      # ... one pair per engine, in the catalogue's order
      - optional:file:./application.local.yaml
      - optional:file:${DRISHTI_PACKS_OVERLAY:${drishti.data.dir:./data}/packs/added.yaml}
```

Spring Boot's order gives, strongest first: the command line and environment, the packs overlay (no engine keys), `application.local.yaml`, the site's engine file, the jar copy, `application.yaml` (no engine keys). `drishti.engines.{dir, drain, watch-seconds}` stay in `application.yaml`, and so do `drishti.admission.*`, `drishti.memory.*` and `drishti.disk.*`, which guard the whole server. A lens's presets (`EditionEnvironmentPostProcessor`) sit above the engine files.

**Switch and source** (`EngineSwitches`) reads Spring's own property sources in order and reports which one decided each switch. The keys `drishti.var.enabled`, `drishti.ccr.exposure.enabled`, `drishti.frtb.enabled`, `drishti.simm.enabled` and `drishti.pivot.enabled`, set above the shipped files, decide their engine's switch (`DRS-1024` logged once); the in-memory lake is on when its stores are configured in the local file and nothing sets its switch; `drishti.frtb.sensitivity-kind`, `factor-kind` and `cache-minutes` read `drishti.sensitivities.*` through a lowest-precedence alias layer (`EngineKeyAliases`). Each lens's `config/lenses/features.yaml` entry names the engine switch (`var: { server: [drishti.engines.var.enabled] }`).

### A27.3 The lifecycle and what off means on the server

Every engine implements one lifecycle, `ComputeEngine` (`start`, `stop(drain)`, `state`, `memory`, `health`); each engine's configuration makes one `BasicEngine` bean, and `EngineRegistry` finds every `ComputeEngine` bean, so an engine still to come registers by itself. The services keep their Spring beans, but they are thin: their open dates, caches, pools, schedulers, watchers and answers are created on first use and released by `stop` (the same release as Admin → Caches' purge of the engine). Each service asks the registry before any work and refuses with `EngineOffException` (`404 DRS-1020`). An engine off at start creates no thread, opens no date, registers no connector of its own and no computed kind (`var-result`, `exposure-result` ...), and reserves no memory in the safety nets (the automatic `open-heap-mb` shares go to the engines that run); `/actuator/health` shows it `off`, never `down`. The in-memory lake off at start is built with no store.

**Connectors and kinds.** An engine's catalogue entry splits what it touches: **its own** connectors and computed kinds (not registered while off) and **what it reads**. An input kind marked `engine-input: true` (large, used only to feed engines: `pnl-vector`, `scenario-pv`, `exposure-path`, `simulation-run`, `position-sensitivity`) is not read ahead, warmed or pinned on a node where no running engine reads it; its connector stays registered for the pack's own pages (`XPATH` browses a path). Reference kinds are never affected.

**Live on and off.** Off: the gate refuses new requests, the requests in flight drain (`drishti.engines.drain`, then cancelled with a message), then `stop` releases everything and unregisters connectors and kinds; the live channel tells open pages. On: `start` reads and checks parameter files, registers connectors, creates threads. `memory-lake` and `flight` are written to their file and wait for a restart (direct-memory caps and the Flight port are set at start; the launchers' `tools/lake/memory_caps.py` reads the lake's file).

**Writing the file.** Admin → Engines writes only `enabled` (keeping its `${VARIABLE:default}` form) and `changed`, with every comment and other key kept; atomically (a temporary file in the same folder, then a rename); refused with `409 DRS-1023` when the file changed on disk since it was read or the folder is not writable. The folder is watched (`drishti.engines.watch-seconds`), so a hand edit applies as a page change does and is recorded in History with no actor.

### A27.4 The gate, routing and the digest

`EngineGateFilter` (filter order 4, after the caller is known) maps a request's path to its engine by the catalogue and counts the requests in flight per engine (the drain). Not running here: `EngineCluster` forwards it to the least loaded peer whose digest says it runs the engine (`RunCluster.call`, signed with the cluster key, on behalf of the caller; the answer carries `X-Drishti-Engine-Node`), else `404 DRS-1020`. An engine `starting` or `stopping` on a node is not routed to. The capability digest carries the engines in its `services` map (entry `engines: var:running,frtb-sa:off,...`), so the digest's shape is unchanged and an older peer reads it; a change announces a new `digestHash` at once. A node with `var` off never owns a VaR shard (it leaves the shard map as a draining owner does). A peer's change from another node's Admin → Engines arrives as `POST /api/v1/net/engines`, signed, and is applied as a local change; both nodes audit it.

### A27.5 The console

`/public/about` (what the console polls) and `/api/v1/about` carry `engines: [{id, title, here, cluster}]`; the console learns them with the edition (`core/engines.py`, `Editions.learn`) and again at once after a change made on Admin → Engines. `core/engines.py` answers every question: `engine_available` (running here or on any node), `word_hidden`, `typed_engine`, `path_engine`, `menu_hidden`, `help_hidden`, `tile_hidden`, `lens_hidden`. What runs is each app's own (`engines.State` on its editions, read per request through `AppEngines` in `core/app.py`), never shared between apps of one process. It is wired into `Edition.word_on`, `command_shown` and `typed_shown` (so FUNC, type-ahead, Home cards and chips follow), the route gate (`core/edition_gate.py`), the menus (`requires: engine:<id>` and the catalogue's `menus:`; an Engines-menu column names its engine with `engine: <id>` and its readers with `opens:`, checked by `core/menus.py` `engine_refused` for the column and the typed word alike), Help (`help.yaml engine:` and the catalogue's `help:`), the lens links and `/go`. BI: `EditionBi.engines(...)` drops a dataset or model whose name starts with an engine's `bi:` prefix, and every item on it, while the engine runs nowhere. Admin → Engines is `routes/admin_engines_routes.py`, `templates/admin/engines.html`, `static/js/drishti_admin-engines.js` and `static/css/drishti_admin-engines.css`; the command line is `tools/enginescli.py` (`server engines` and `admin engines`; `problem_error` maps `DRS-1020` to exit 3 for every command).

### A27.6 Configuration document and checks

`tools/settings_catalogue.py` has the area **Compute engines:** `config/engines/*.yaml`, one table per file, read with the same comment reader as `application.yaml`; CONFIGURATION.md and Help → Configuration list every engine key under its file. `--check` fails when a shipped engine file and its jar copy differ, when a catalogue entry has no file or a file no entry, and when a key appears in both `application.yaml` and an engine file.

### A27.7 Decisions

| # | Question | Decided |
|---|---|---|
| 1 | Where the command line lives | `drishti.py server engines` beside `server commands` and `server packs`, with `admin engines` accepted |
| 2 | Off answers `404` or `403` | **404** (`DRS-1020`): nothing is forbidden, the engine is not on this server; the words say so |
| 3 | `memory-lake` turned on from the page | written to the file with **restart needed** |
| 4 | Changing another node's engine from any node | yes, signed with the cluster key and audited on both nodes |
| 5 | Regulatory VaR (backtesting, reports, sign-off) | part of `var`: it reads only VaR's official records; split when a node needs one without the other |

### A27.8 Mock-ups and build checklist

Three options of Admin → Engines, self-contained HTML (dark by default, light with `#light`); **A was chosen**, with C as its Cluster tab:

| Option | What it is | Screenshot |
|---|---|---|
| A: the engine list with a review panel (docs/architecture/mockups/engines/A.html) (**chosen**) | one table of engines on this node (switch, state, memory bar, needs, used by, where the switch comes from); a side panel opened by a change: the reason, the dependency check, what will be hidden, live or restart; History below; the cluster as a second tab | A.png |
| B: engine cards and a dependency map (docs/architecture/mockups/engines/B.html) | a card per engine arranged by family, with a small dependency map; a change opens a confirm dialog | B.png |
| C: the cluster matrix (docs/architecture/mockups/engines/C.html) | nodes × engines in one grid, each cell a switch with its state and memory; an "available to people" row | C.png |

A is the shape administrators know from Functions (a list, staged changes, a review panel with the reason, History) and shows what depends on an engine and what disappears at the moment of the change; C's matrix is the better view for a large cluster and is A's Cluster tab.

| # | Part | Status |
|---|---|---|
| 1 | The catalogue and its build test | built |
| 2 | The engine files, their jar copies, the imports, the other key names and the start warning, `features.yaml` naming the switches | built |

| # | Part | Status |
|---|---|---|
| 3 | `ComputeEngine`, `BasicEngine`, `EngineRegistry`; every built engine behind `start` and `stop`; `404 DRS-1020` | built |
| 4 | Connectors and kinds registered only while the engine runs; `engine-input` kinds not read ahead | built |
| 5 | Live on and off: drain, release, unregister, the live-channel notice | built; the admission check on an engine coming on (`503 DRS-9502`) waits for its first consumer |
| 6 | Admin → Engines (mock-up A, Cluster tab C), the admin API, writing the file, History, audit | built; the impact answer lists the engines that need or use it and the nodes that still run it, not yet the saved BI reports and monitors that read its kinds |
| 7 | The console everywhere: words, menus, Help, landings, tiles, lenses, FUNC, type-ahead, playgrounds, BI | built; AI tools per engine wait for their consumers |
| 8 | Drishti*net: `engines` in the digest, routing by engine, the cluster matrix and a peer's change | built |
| 9 | `drishti.py server engines` (and `admin engines`) | built |
| 10 | CONFIGURATION.md's engines area and its checks | built |
| 11 | SA-CCR, FVA and the CCR cube as engines | each adds its entry, files and bean when its engine is built |

### A27.9 Tests that prove it

| Test | Proves |
|---|---|
| `EngineCatalogueTest` | the catalogue and the console's copy are identical; no cycle; every word known |
| `EngineFilesTest` | the import order (environment > local file > engine file > shipped copy), each other key name with its warning, the jar copies identical to the site files |
| `EngineRegistryTest` | an engine off at start creates no thread, opens no date, registers no connector or computed kind, reserves no memory; on, it does |
| `EngineApiTest` | a whole server: an engine turned off answers `404 DRS-1020` in words and on again answers; a dependency refusal (`DRS-1021`); the file written with its switch and `changed`; History; VaR off with its dependents hides its BI datasets and API |
| `EngineClusterTest` (two members) | `frtb-sa` on one member only: the other's console shows FRSA and its answer is the first member's, routed; off there: `404` within the gossip time; the digest carries `engines` |
| `EditionWorldServerTest` | a lens's `var: off` preset switches the engine off |
| `tests/test_engine_off_everywhere.py` (console) | for every engine of the catalogue (generated from it): its words unknown on the command line, in FUNC, type-ahead and search; no menu item, Help entry, tile, chip, BI dataset or playground link on any route; the CLI exits 3 with the words |
| `tests/test_engines_browser.py` | the page: switch, reason, review, refusal words, restart pill, history (with the guide's screenshots) |
| `tools/test_enginescli.py` | the command line and its exit codes |

## A28. Business intelligence on the table layer

BI owns **presentation**: reports, visuals, the designers (BI Studio), boards, statements, publishing, subscriptions and the presentation of alerts. What it computes or defines underneath belongs to the platform, once: expressions are Rachana (section A23), aggregation and totals are the pivot engine's, and tables and models are the table layer's (section A24). The user's view is BI_GUIDE.md; every key of tables and models is BI_GUIDE.md.

Principles that shape every part below:

| # | Principle | What it forces |
|---|---|---|
| P1 | The servers compute every number; the browser draws | no WebAssembly, Pyodide or DuckDB-Wasm on a BI page; no client totals, sorts or pivots; every device shows the same answer |
| P2 | Security before arithmetic, on the server, at every tier | row rules and masks bind to tables and models, are injected into the plan before optimisation, and no language construct removes them |
| P3 | A model is a mini database rooted in packs | every table traces to pack kinds; the model is the unit of query, security, contract, cache and audit |
| P4 | Consumers reference one model; they never own it | reports, boards, alerts, notebooks and subscriptions hold layout and local measures only; *Used by* is an index the server keeps |
| P5 | Data is not copied | the only stored data is disposable (refresh layers, rollups, result caches, run results, standing-query state); deleting it changes latency, never an answer |
| P6 | Streaming by nature | time (live, as-of, window) is a parameter of every plan; every query can be subscribed |
| P7 | Every query is bounded and visible | admission, budgets, estimates and background runs instead of unbounded work in the request path |
| P8 | AI only through the query layer | AI writes model queries, SQL or YAML drafts that are checked and run as the asking user; it never reads rows |

### A28.1 The table layer and models in Rachana

| Concern | Owner |
|---|---|
| Aggregation, totals, rollups | the pivot engine (`drishti-bicompute`) |
| Expressions (measures, computed columns, filters, row rules, the pivot's formulas) | Rachana (`drishti-rachana`) in checked mode; one compiler in the engine (`bicompute.lang`) |
| Tables (BI's "datasets") | virtual tables of the table layer (`drishti-tables`; `config/tables/<domain>/` and stored items of kind `table`) |
| Models (relationships, measures, hierarchies, parameters, model security, contract) | the layer's models (`.model.yaml`, items of kind `model`) |
| Cached, rollup and live layers | the layer's materialisation: `on-load`, a model's `rollups:`, `{ live: { rows, age } }`, a model's `history:` |
| Who reads tables | BI, the pivot, Sutras, alerts, Arrow Flight |

How BI reads the layer:

| Part | How | Where |
|---|---|---|
| BI's folder | reads `drishti.tables.root`; a table is an item of kind `table`, a model of kind `model`; a file in BI's older dataset or model form is refused with the migration's command (`DRS-9196`); a pack carries `config/bi/tables/` and `models/` in the layer's form | `bi.BiFolder`, `bi.mq.ModelCatalog`, `bi.PackBi` |
| A table in BI's compiler | the layer's reader (`TableYaml`) and checks, then the table in BI's terms: paths as columns, the grain as BI's `each`, `keep: { latest }` as one row per key, rules and masked columns as BI's security, a table over a table as a narrowing, `materialize: { live }` as live mode | `bi.layer.LayerTables` |
| A model | read with `tables:` (aliases to tables and roles); `history:` samples the model's live facts | `bi.layer.ModelDefParser`, `Flattener.withHistory` |
| One parser | Rachana's parser; its tree lowered to BI's compiler tree (`NodeLowering`); what BI composes written back as Rachana (`NodeText`); calculation-group items read without a parser of their own | `bi.dataset`, `bi.mq.ExpressionCheck`, `CalculationGroups` |
| The pivot's measures and formulas | BI hands the engine its measures from Rachana's tree; the pivot's formulas are lowered from the same parser | `bi.pivot.BiMeasures`, `bicompute.formula.FormulaLowering` |
| Pack-generated tables | the layer generates a pack's kinds as tables with the same columns, roles, meanings, references, grain, time mode and entitlements, and the lists it names as grain tables; regenerated, never migrated | `tables.catalog.PackTables` |

**Three layers.** A **table** is one governed table over a root (a pack kind, a SQL source, a Python transform, a model's answer, another table); it owns grain and keys, column types and meanings, additivity, time mode, refresh, masks and row rules. A **model** puts tables in a star (facts and shared dimensions, never physically denormalised) with relationships, measures, calculation groups, hierarchies, parameters, perspectives, field properties, tighter security, a semver contract and stable ids. **Consumers** bind to exactly one model at a major version; a report may add local measures, never joins. A consumer naming one table gets an **implicit model** (the table as the fact, its pack dimensions one hop away, default measures), which "Make it a model" writes out as one shared model per table. Correctness by construction: each measure aggregates at its own grain; facts meet through shared dimensions; a join that repeats rows is refused; additivity is declared; amounts in different currencies are never summed without a conversion.

### A28.2 MQ: the one query object

The model query (`mq: 1`, `mq: 2`) is the contract for the UI, the API, AI and MCP, and the **semantic leaf** of the logical plan: Drishti SQL compiles to MQ (or to MQ leaves under relational nodes), Rachana is carried inside it (`define`, `having`, row filters), and Python nodes take MQ answers in and give typed tables out. `mq: 2` adds only what later features need, all reserved in v1:

```
mq: 2
model: counterparty-cockpit@1
perspective: credit-officer          # a field list restricted to a perspective
params: { shock: -0.1 }              # what-if and field parameters (param.<name> also in select and sort)
calc: { Time: YTD }                  # calculation-group items applied to the selected measures
select: { fields: [counterparties.sector], measures: [MTM, MTM % of desk] }
time: { as-of: 2026-10-09 }          # or live: true | window: { last: 1h }
options: { explain: true, budget: { millis: 2000, rows: 1000000 } }
```

The answer is `columns`, `rows`, `totals` and `meta` (the generation or data-version vector, tiers, `layer`, `rollup`, cache hit, warnings, `dataAsOf`, `noAccess`, the canonical query, the plan hash, the security signature id, never the predicates). **Canonicalisation** (sorted keys and filter values, ids instead of names, defaults filled, relative dates resolved, visual-only properties dropped) makes the same question in any language produce the same plan hash, so caches, runs and standing queries are shared across SQL, MQ and the designer. A report visual becomes one MQ (`VisualMq`); a report's own measures travel as `define`. `model:` resolves in the layer's catalogue; `POST /datasets/{id}/query` answers `410 DRS-9150` with the equivalent MQ.

### A28.3 Drishti SQL

Drishti SQL (DSQL) is read-only SQL whose **tables are the aliases of one model** and whose **measures are** `MEASURE('name')`. It is never sent to a connector as text; a connector receives only the fragments the planner pushes down, with bound parameters. One SELECT (or WITH … SELECT, UNION of compatible selects); `JOIN` without `ON` (or `ON` equal to a model relationship) follows the model; inline aggregates over alias columns become local measures; window functions run over the **result**; a time clause (`AS OF DATE`, `FOR LIVE`, `FOR WINDOW`) sets one time per query. Refused (`DRS-9170` to `9179`): DDL and DML, multiple statements, table functions and file or URL access, joins on arbitrary predicates, a measure in `WHERE`, another model, `SELECT DISTINCT` on a fact without grouping.

The binder resolves every name against the model's `describe` **as the caller** (hidden columns do not exist, masked ones are typed masked). An **MQ-shaped** statement becomes exactly one MQ (`meta.query` shows it); a **composite** statement (CTEs, UNION, subqueries, result windows) becomes a plan whose leaves are secured semantic aggregates and whose inner nodes are relational operators over those answers, executed in the Java tier, so it can never see more than its leaves; a join or filter on a masked value is refused (`DRS-9161`). Inline expressions are read as Rachana (`bi.sql.SqlToRachana`); SQL keeps SQL's rule for `!=`. DSQL is allowed in `POST /api/v1/bi/sql`, `drishti.py bi sql`, the designer's Query panel, Notebook SQL cells, tables whose source is a DSQL statement, and tests; never in measures, row rules or computed columns.

### A28.4 The expression language (DEL) and its checks

BI's expression language is Rachana; DEL, the earlier Python-like spelling, is folded into it with one spelling and no aliases. The migration writes Rachana; the editors complete it.

| DEL | Rachana |
|---|---|
| `a and b`, `a or b`, `not a` | `a && b`, `a \|\| b`, `!a` (booleans only in checked mode) |
| `a if c else b` | `c ? a : b` (or `if(c, a, b)`) |
| `0 < x <= 10` | `0 < x && x <= 10` (a chained comparison is refused) |
| `x is None`, `True`, `False`, `None` | `x == null`, `true`, `false`, `null` |
| `x ** y`, `x // y` | `power(x, y)`, `floor(x / y)` |
| bare `column` in a table's row scope | `$.PATH` (or `@.PATH`, `^.PATH`): a table's expressions read the source |
| `alias.column.month` in an expression | `month(alias.COLUMN)` (in MQ field lists and hierarchies `trades.TRADE_DATE.month` stays a field name) |
| `last`, `first`, `cumulative`, `today()` | `latest`, `earliest`, `running`, `asof()` |

**The null rule** is Rachana's (`null != 'X'` is true); the migration rewrites each DEL comparison whose meaning would change (`a != b` becomes `a != b && a != null`), so every figure stays the same.

**Scopes.** *Row* (a table's column `expr`, `filter`, `keep`: `$`, `@`, `^`, `param.x`; aggregates refused, `DRS-9127`); *rule* (a table's `rules[].where`, a model's `security.rows.<alias>[].where`, with `user.x`; never removable by any function); *aggregate* (a model's measures, MQ `define` and `having`, a report's local measures: `alias.COLUMN` inside an aggregate, `measure('name')`, `param.x`; a bare column outside an aggregate is refused with the fix); *context* (the first argument of `calc`, `share_of`, `parent_of`, `as_of`). `ctl.x`, `link(...)` and `fmt(...)` are a Sutra's display concerns and are refused in tables and models.

**Filter context.** A measure is evaluated per group in four layers: (1) **security**, row rules and masks for this reader, fixed; (2) the **query** (filters, slicers, drill, cross-filter); (3) the **group** (the current group's values); (4) the **measure**'s own `where` and `calc` edits. `calc` edits layers 2 and 3 only (`where`, `ignore = [fields] | ALL`, `keep`, `via` for an inactive relationship), so `ignore = ALL` means "all *you* may see"; nesting depth at most three; no context transition (iteration is explicit). It lives in the engine (`bicompute.context.FilterContext`, `context.Periods`); BI's `Contexts` and `ContextFunctions` bind it to BI's SQL compiler. `share_of` and `parent_of` drop filters (of everything the reader may see); the pivot's `pct_of_*` are of the visible totals, so the two families keep different names.

**Checks.** Checked mode is Rachana's type checker (`rachana.el.check.Checker`) with BI's codes kept: `DRS-9127` type or scope, `DRS-9129` not yet, `DRS-9152` unknown name, `DRS-9161` a masked field filtered. The same text means the same thing checked or not; checked mode only refuses more. One function catalogue feeds validation, completion, `describe` for AI and the documentation. The console never parses an expression for meaning: it asks the server's validate endpoints as you type.

#### Filter-context control: the design beside DAX

The principle stays: **every edit of the context is written, and one function does each job.** Rachana gets no second `CALCULATE`: `calc` grows just enough, and the things DAX does implicitly become named arguments.

**Filter edits.**

| DAX | Rachana | Status |
|---|---|---|
| `CALCULATE([MTM], KEEPFILTERS(trades[STATUS] = "LIVE"))` | `calc(measure('MTM'), where = trades.STATUS == 'LIVE')` | built |
| `CALCULATE([MTM], trades[STATUS] = "LIVE")` | `calc(measure('MTM'), ignore = [trades.STATUS], where = trades.STATUS == 'LIVE')` | built |
| `CALCULATE([MTM], REMOVEFILTERS(desks[NAME]))`, or `ALL(desks[NAME])` as a modifier | `calc(measure('MTM'), ignore = [desks.NAME])` | built |

| DAX | Rachana | Status |
|---|---|---|
| `CALCULATE([MTM], REMOVEFILTERS())`, or `ALL()` | `calc(measure('MTM'), ignore = ALL)`: everything *you* may see | built |
| `CALCULATE([MTM], ALL(desks))` | `calc(measure('MTM'), ignore = [desks])`: a table name means all its fields | built |
| `CALCULATE([MTM], ALLEXCEPT(desks, desks[REGION]))` | `calc(measure('MTM'), ignore = [desks], keep = [desks.REGION])` | built |
| `CALCULATE([MTM], ALLSELECTED())` | `calc(measure('MTM'), ignore = GROUP)`: the group layer only, slicers and page filters stay | built |

The second row is the one difference a DAX author must learn. A DAX filter argument **replaces** the filters on its column; Rachana's `where` **adds** to them, which is DAX's `KEEPFILTERS`. To replace, say so with `ignore`; the converter writes both arguments. We keep `where` as it is: it can only narrow an answer, so a reader sees what it does.

`keep` alone stays as it is today: every layer 2 and 3 filter goes except those named. With `ignore`, `keep` narrows what `ignore` drops, which is what `ALLEXCEPT` means: filters on other tables are untouched. `ALL` as a table function, returning rows, has no equivalent in an expression: rows are a calculated table (below).

**Context transition.** DAX turns a row into a filter silently, whenever `CALCULATE` or a measure runs inside an iterator. Rachana writes the iteration as `*_by`: per value of the `by` fields the inner aggregate is evaluated with that value as a filter, and the outer function combines the results. That *is* the transition, with its grain visible.

| DAX | Rachana | Status |
|---|---|---|
| `AVERAGEX(VALUES(dates[DAY]), [MTM])` | `avg_by(measure('MTM'), by = [dates.DAY])` | built |
| `SUMX(VALUES(desks[NAME]), CALCULATE(MAX(trades[MTM])))` | `sum_by(max(trades.MTM), by = [desks.NAME])` | built, with `min_by`, `max_by` |
| `CALCULATE([MTM], FILTER(ALL(desks[NAME]), [VaR] > 1000000))` | `calc(measure('MTM'), ignore = [desks.NAME], by = [desks.NAME], having = measure('VAR') > 1000000)` | built: a semi-join on the groups that pass |

An engine measure can be iterated but not added: `max_by(measure('VAR'), by = [desks.NAME])` is the largest desk VaR; `sum_by(measure('VAR'), ...)` is refused with `DRS-9127`, as any stacking of VaR is. An aggregate in a table column stays refused too: where DAX writes a calculated column with `CALCULATE`, Rachana writes a calculated table and relates it.

**Calculated tables** are not a new thing: a virtual table can already take a model's answer as its source (`from: { model: ..., query }`, see BI_GUIDE.md), so `SUMMARIZECOLUMNS` and `ADDCOLUMNS` become that query, and `FILTER` its `where`. `UNION` and `CROSSJOIN` are a SQL table. Such a table may be related in the same model that produces it, as long as its query does not read itself (that cycle is refused). The table is computed per business date, like any answer, never held as a stale copy.

```
# DAX: Desk Usage = SUMMARIZECOLUMNS(desks[NAME], "Notional", SUM(trades[NOTIONAL]))
name: risk.desk_usage
from: { model: risk-model@1, query: "an ordinary MQ: sum(trades.NOTIONAL) by desks.NAME" }
```

**Calculation groups** are built: a group's items are templates over `SELECTED` with a precedence (higher applies first) and a format per item (`ModelDef.CalcGroup`). DAX's `CALCULATE(SELECTEDMEASURE(), DATESYTD(dates[DAY]))` is the item `ytd(SELECTED, dates.DAY)`, and `SELECTEDMEASURE() / CALCULATE(SELECTEDMEASURE(), ALL(desks))` is `share_of(SELECTED, over = [desks])`. When a template cannot apply to a measure (a year-to-date of VaR), the cell reads *not applicable, and why* rather than failing the whole visual.

**`via:` per measure.** Inactive relationships are followed with `calc(agg, via = 'name')`, by name or generated id. Relationships have names, and a measure may say so once for all of its expression. It is the same mechanism: `via:` compiles to the `calc`.

```
measures:
  Notional by settle date:              # DAX: CALCULATE(SUM(trades[NOTIONAL]), USERELATIONSHIP(trades[SETTLE_DATE], dates[DAY]))
    expr: "sum(trades.NOTIONAL)"
    via: settle_date
  Notional, groups one way:             # DAX: CALCULATE(SUM(trades[NOTIONAL]), CROSSFILTER(bridge[BOOK_ID], books[BOOK_ID], ONEWA
Y))
    expr: "calc(sum(trades.NOTIONAL), via = 'bridge_book', filter = 'one_way')"
```

The `filter =` argument is the per-measure override of a relationship's direction. Two active paths between the same tables are refused when the model is saved, naming both paths; `via:` is how a measure chooses the other one. Screens: bi-depth mock-ups (docs/lenses/mockups/bi-depth/README.md).

### A28.5 The query engine and the plan

Every request, whatever the language, becomes one logical plan:

```
request (MQ | DSQL | visual | alert | AI | MCP)
  |
  ▼
1 Parse ─► 2 Resolve ─► 3 Bind ─► 4 Secure ─► 5 Guard ─► 6 Optimise ─► 7 Place ─► 8 Run ─► Answer
MQ/DSQL    model@v,     names,    rewriter    fan-out,    pushdown,     tiers,     admission,
Rachana    contract,    types,    per table   currency,   rollups,      fragments  deadline,
           envelope     scopes                reach       cache                  budgets
```

| Node | Meaning |
|---|---|
| `SecureScan(table, reader, time)` | one table's rows for this reader at this time: row rules ANDed, masks projected, hidden columns removed; produced by the security rewriter for every table touched |
| `Derive`, `Dedup`, `Regroup` | row-scope columns; explicit grain; grouping |
| `SemiJoin(fact, dim, rel)` | a dimension's filter (security or query) propagated to a fact |
| `Aggregate(fact, by, measures)` | a measure's leaves at the query's fields, along many-to-one paths only |
| `Meet(parts, on)` | full outer join of per-fact aggregates on shared fields (several facts) |
| `Context(sub-plan, edits)` | a sub-plan in a modified filter context |
| `Window`, `Having`, `Sort`, `Limit`, `Totals(levels)` | post-aggregation |
| `Rel(op)` | relational operators over secured answers (composite DSQL) |
| `PyTransform`, `PyAggregate`, `PyRender`, `PyCell` | Python nodes, run on the compute pool `python` (section A29) |
| `Answer(columns, meta)` | typed, masked rows and meta |

**Guards** refuse fan-out, an unreachable field or rule (`DRS-9156`, `DRS-9020`), summing across currencies (`DRS-9158`) and semi-additive misuse. **The optimiser** is rule-based and answer-preserving: predicate pushdown into `SecureScan` and the source fragment; projection pruning (row-rule columns always kept); the grain split into per-fact aggregates; **rollup matching** (only when the rollup's `by` covers the fields, filters and the reader's row-rule columns, and holds the measures' components; security first); cache lookup; common subplans across a page's visuals and standing queries; context lowering; top-N as its own sub-plan; aggregation pushed into a SQL-capable source when every scan of the fragment is in one store.

**Tiers.** The *source* (via the pack's connector: its SQL or fetch, with bound parameters only), the *Java server* (layer and rollup reads, joins, aggregates, totals, masks, live tables, standing-query state; an embedded DuckDB with bounded memory and spill), the *compute pool* (`python`), and the *browser* (none: it draws). **Layers** are a physical choice for `SecureScan`: the columns layer (Parquet per table and business date, rebuilt on load signals or a schedule, disposable), a rollup, the live table in memory, or the source (`direct`). `meta.layer` and explain say which. **Totals** of a matrix or a table are the pivot engine's: every level, in one grouping-sets statement over the reader's rows.

**Execution.** Fragments run concurrently on virtual threads under admission (section A28.7) with deadlines; cancelling a request cancels its fragments; Arrow flows between tiers; JSON is served up to `drishti.bi.query.json-max-rows`. **Explain** (`options.explain`, `EXPLAIN SELECT`) returns the annotated plan, showing *that* a rule applied and its text only to stewards.

**Caches, all disposable.** L0 parse and plan (per model version); L1 results, keyed by canonical plan hash, model version, security signature, resolved time and generation vector; L2 refresh layers; L3 rollups (live rollups maintained by group); L4 Python results; L5 standing-query state; plus the durable run result store (section A28.7).

### A28.6 Time: live, as-of and windows in one plan

Time is a plan parameter bound **per `SecureScan`**, resolved before the cache key: `as-of D` reads each table at its latest business date on or before D (`per-table`, the default; `aligned` reads the newest date every table of the visual holds; a pinned or relative date is resolved first); a live table at `as-of` reads its sampled history at the end of D (`DRS-9157` without one); `live: true` reads live tables' current state and snapshot tables' current business date; a window reads the partitions in range, or a history. So one plan answers *now*, *as of a date* and *over a window*, and a model mixes a live fact with snapshot dimensions. An as-of answer is immutable until a back-dated load bumps that date's generation. Knowledge time ("as known at 10:00 yesterday") is out of scope; the version vector makes an answer reproducible while its layers hold.

### A28.7 BI runs: every BI query as a bounded, visible job

Every BI question (report visuals, board tiles, model queries, SQL, statements, subscription sends) enters one asynchronous pipeline, `RunService` (`com.ash.drishti.server.bi.runs`):

```
submit ──► plan ──► admit ──► run ──► store ──► notify
           |        |         |        |         └─ inbox row (type bi-run) and the bell, for a visible run only
           |        |         |        └─ Parquet under <data>/bi-runs/results, keyed; never for live
           |        |         └─ a RunEngine: local DuckDB (interactive or run instance); split and external are interfaces
           |        └─ RunQueue (per user, per server), then Admission (class caps, slots, person and model shares)
           └─ RunPlanner: fingerprint, data version, reuse, estimate, tier, engine, confirm?
```

- **The inline wait.** The console asks with `Prefer: respond-async, wait=2`: done within the wait is `200` with the ordinary body and the run stays invisible; otherwise `202 {run}` and the run becomes visible; a run that needs confirmation is `409 DRS-9082` with the estimate. A caller without the header waits as before.
- **The planner** reads metadata only, never data: a partition's `meta.json` (`Layers.peek`), `SourcePlugin.rowEstimate(kind, date)` (Delta add-action statistics, Iceberg manifest record counts, store statistics), a learned rows-per-day history, else *unknown*, which is never zero. It estimates read and build rows, bytes and seconds as a range with its basis (`layers`, `delta-stats`, `iceberg-manifests`, `store-stats`, `history`, `unknown`); the factors narrow to measured percentiles. Progress is measured (dates built, then DuckDB's query progress) or absent.
- **Tiers.** *Interactive* (bounded work: one date, a covered question); *known heavy* (a rollup or product covers every date asked); *ad-hoc heavy* (a background job on the run engine, `BACKGROUND` class, a separate DuckDB instance with its own memory and threads). A live table in the plan is always interactive and never stored. An **interaction** (slicer, cross-filter, drill) never becomes a job: ad-hoc heavy fails at once with `DRS-9089` and the estimate, and the console offers to run it; a newer interaction on the same visual cancels the older.
- **Confirmation** above `confirm.above-seconds` (60) or `confirm.above-rows` (500 M) or an unknown basis; refusal past the largest engine's limit (`DRS-9086`).
- **The result store** writes `result.parquet` and `answer.json` directly from DuckDB (`COPY ... TO`, never through the heap), under `tmp/` then renamed. The **key** is the plan hash, the resolved dates, the **data version vector** (per connector, kind and date), the security signature and an engine-semantics tag; a source without a data version makes the result non-reusable. **Reuse**: a matching `done` result answers at once; a submit matching a running twin joins it (single-flight); report visuals share by **access profile** unless the visual depends on the viewer. Retention is 30 days after the last open, a total and per-person budget, a per-result cap.
- **Masks and sharing.** A run is computed as its runner. Sharing re-decides per recipient on every read: *sees the same* (same signature), *re-masked* (the stored rows with the recipient's extra masks), or *recomputed as them* (a run of their own); someone who may not read the model gets nothing. Administrators see state, size and node, never values.
- **Promote to a snapshot.** A model query's or SQL run's result is copied to `snapshots/<name>/v<N>/` and a draft table `from: { run }` is created (optionally a report on it), pinned by a token so only that table reads it; masked columns stay masked, `provenance` records whose rights computed it, and review applies.
- **Schedules.** A run's question is asked again on the subscriptions' schedule grammar, as its owner at firing time, each firing a visible run.
- **Rollup suggestions.** The run log counts finished heavy model queries per model, grain and filter shape (fields and operators, no values); past `suggest.min-runs` (5 in 7 days) and `suggest.min-seconds` (60) the rollup is drafted, and **Accept** writes it into the model's draft under `rollups:` for review.
- **Fairness.** Admission has a cap per work class (`drishti.bi.exec.background-share`, `standing-share`), so background work never holds the slots interactive questions need; a person's share and a model's share bound the rest; background fragments take at most a share of a connector's pool. Refresh enters admission as `BACKGROUND`.
- **Cancellation** frees everything at once: queue place, admission waiter, building threads (interruptible connector slots), the DuckDB statement, the `tmp/` folder. Work whose caller is gone is cancelled unless the run is visible.
- **Restart.** `RunRecovery` requeues queued and running runs (reads are safe to restart; attempts bounded) or fails them with `DRS-9085` and a reason, and tells the runner; nothing is lost without a word.
- **The cluster.** Run records live in the content store, so every member sees the same runs. An ad-hoc heavy run executes on the model's **owner** (rendezvous hashing over alive Drishti*net members, `ShardDirectory`), which holds its warm layers; a submit elsewhere is forwarded with the caller's on-behalf-of token. A run's own calls go to the node holding it; the list is scattered and merged. Leases (`lease.renew`, `lease.ttl`) are compare-and-sets on the run's document, so exactly one node adopts the run of a dead member.
- Settings: `config/runs/runs.yaml` and `drishti.bi.exec.*`; codes `DRS-9080` to `9089`; Admin → BI → Runs shows the gate, the queues, the store, estimate accuracy and the repeated heavy questions.

### A28.8 Live answers and standing queries

Every MQ or DSQL question can be **asked** or **subscribed** (`POST /api/v1/bi/mq/subscribe`, `GET /api/v1/bi/mq/stream`). When a model's data moves (a live batch, a load, a refresh) the **generation bus** names the model with its change set, and only the subscriptions on it are evaluated. The `StandingQueryRegistry` groups subscriptions by (plan hash, security signature): **one evaluation per group**, however many subscribers share it; each subscriber has its own throttle (at least 100 ms, default 250 ms) and a latest-wins mailbox, so nothing queues.

| Mode | When | On a change |
|---|---|---|
| `incremental` | `sum`, `count`, `avg` of the live table's own columns grouped by its columns | each changed row's old contribution out, the new in; no query |
| `incremental` | `min`, `max`, `distinct`, `percentile` | the touched groups are asked again |
| `recompute` | filter contexts, fields of another table, `top` filters, loads and refreshes | the question is asked again |
| `tick` | windows, time intelligence, Python, a model over its maintenance budget | asked again every `drishti.bi.mq.tick` |

An evaluation slower than its throttle three times doubles it (`throttled`); a model over its maintenance budget degrades to `tick`; caps per node and per user refuse with `DRS-9181/9182`. A change of a reader's roles or `user.*` attributes changes their signature: their subscriptions are re-planned, never patched. A subscribed answer equals the asked answer at the same cursor. In the console, report visuals and board tiles subscribe over the browser's one live channel (`mq:<subscription>`), a page holding at most 12 (a board 16), and

closing a tab ends them. A live table is maintained by key from the connector's pushes in one transaction per batch, with retention by rows and age, rollups kept by group, and a sampled `history`. The stream that tells a page a live report changed carries no data: the page asks again as the reader.

### A28.9 Reports, boards and controls

A **report** is pages of visuals on a 12-column grid, bound to one model, parsed, checked and written by the server (`DRS-9030` to `9038`); nothing in it executes. Its `format` is typed per visual family by JSON Schemas in the server (`drishti-server/src/main/resources/bi/report/format/`), from which the designer's Format pane is generated, so the pane offers nothing the server refuses. A **board** is a page with `layout: board` whose tiles name another report's visual and are answered by that report as the reader; tile colours are evaluated in the browser over the reader's answer, notifications on the server per reader. A **statement** is a page with `layout: statement`, laid out by the server (`StatementLayout`) into page-exact PDF (written by the server, Helvetica, no library) and Excel with print setup, burst per recipient by subscriptions. **Controls** (slicers, field parameters, what-if parameters, calculation groups, bookmarks, drill and cross-filter) are view state: the console sends the state and the server builds each visual's MQ from it, in a fixed order of authority; a field parameter is a name substituted into the wells and sort on the server. **BI Studio** is one shell over the report and theme designers; designers edit a YAML concrete syntax tree (comments and key order kept), the server validates with the same parser as publish, and saves are guarded by the item's etag (`DRS-9007`). Exports, subscriptions, alerts and the embedded `<drishti-report>` ask each visual through the same server code as the viewer, as their reader. **Alerts** on numbers are evaluated per person, server-side, with hysteresis and cooldown; one alert engine serves report visuals, model measures, Sutra fields and board tiles.

### A28.10 Governance: versions, review, certification, lineage, impact, environments

| Mechanism | Design |
|---|---|
| Items | table, model, report (boards and statements are report layouts), theme, notebook, python; stable ids; an owner, workspace, audience and lifecycle (`draft`, `proposed`, `published`) |
| Versions | a saved state for every body change and a published version per publish; restore makes a new saved state, so nothing is lost |
| Contract | models carry semver; consumers pin a major; a breaking change (removed or redefined measure, hidden field, removed relationship) needs a major (`DRS-9126`) |
| Review | propose freezes the item; stewards and members with the `approve` power decide in the one Approvals queue; four eyes |
| Tests | expected values (and data tests) gate publish (`DRS-9043`) |
| Certification | only stewards; revoked automatically when a model changes incompatibly; never travels in a bundle |
| Lineage | report → model → table → … → pack kind → connector, past the connector to the place its file names (masked for non-administrators) |
| Impact | by name, recorded per place a report names a field; a publish that breaks published reports is refused (`DRS-9040`) unless forced; renames recognised by path or expression and applied to stored reports on request; changed types and meanings listed |
| Environments | bundles with every published version and checksums; import is a dry run first, all or nothing, with workspace and owner mapping; every import kept for roll back; a pack's BI content is imported as a bundle into the `packs` workspace |
| Audit | every act, and every query (`bi-query`: model, version, consumer, reader, signature id, plan hash, tiers, rows, time, cache hit, refusals) |

### A28.11 Security: row rules and masks

| Layer | Declared in | Enforced as |
|---|---|---|
| Pack entitlement | the pack | `SecureScan` returns no rows for a kind the reader may not open |
| Table row rules | the table's `rules` | ANDed into `SecureScan`; a reader matches any of their roles' rules; no rule, no rows |
| Masks | the table's masked columns, `drishti.security.redact` | projection to `•••` before any aggregation; masked dimensions not grouped; measures over masked columns null; filter, sort or top on them refused |
| Hidden | `hidden-for` | the field or measure does not exist for the role (absent from `describe`, SQL and AI) |
| Model rules and masks | `security.rows.<alias>`, `masked` | only tighter; a rule on one fact limits another through `related_rows('alias')` |
| The safeguard | `may-see-all` | publish refused without a rule or an audited note (`DRS-9130`) |
| Item access | workspace roles, audience | the catalogue |
| Model read and the envelope | workspace membership or the model's audience | needed to author on a model or send free MQ/SQL to it; a report reader without it may run only the questions the published report can generate |

Security is applied at every tier and in every language: the rewriter runs before optimisation and the optimiser has no rule that removes a `SecureScan` predicate; DSQL binds to the same semantic query; context functions cannot reach layer 1; rollups match only when their `by` holds the reader's row-rule columns; Python receives secured answers only; standing queries group only identical signatures; exports, subscriptions and statements render per recipient; embedded calls mask whatever the user's roles; caches key on the **security signature**, the hash of a reader's effective rules (with `user.*` resolved), masks, hidden set and entitlements, an opaque id that lets readers with identical security share answers and never reveals the predicates.

### A28.12 AI hooks

```
 question → policy (on? provider? who? mode) → grounding: describe(model, as the user) + ai: metadata + perspective + golden exam
ples
           → provider → MQ, DSQL or a YAML draft → validate as the user → one repair on refusal → run as the user → answer (e
ngine numbers)
           → optional wording call (only under `aggregates`, secured aggregates only)
```

The hooks (`bi-draft`, `bi-ask`, `bi-explain`, `bi-suggest`, `bi-explain-checks`, `bi-helper`, `subscription-narrative`) pass two gates, the BI power and the AI layer's policy, and are audited without prompt or answer. The provider never receives rows, connector names or paths, masked or hidden field names, or a measure's definition; in `labels-only` (the default) it never sees a figure, in `aggregates` only the asking user's secured aggregates. A draft is shown only after the report checker passes it; nothing is saved or published by the model. Explain's lineage and breakdown are computed by the engine; the model only words them.

### A28.13 Migrations of old BI files

BI reads only the table layer's form (`rachana: 3`) and Rachana. `drishti.py tables migrate` (dry run, `--check` for CI, a report, one audit line per file) maps every older dataset and model key to the layer's, translates each DEL expression through DEL's parser into Rachana's tree (never by text substitution), keeps the null semantics, checks the result, and rewrites the consumers in `config/bi` in place with comments kept. The server migrates stored items at its first start: every version rewritten under its number, the text before kept as a saved state, an item that cannot be migrated left as it was with the reason. A grouped dataset becomes a rows table and a summary table over its implicit model. Refused (`DRS-9195`): an expression whose meaning cannot be kept, two items that give one name and differ, a pack-generated table (regenerated, never migrated). A file in the older form is refused at read with `DRS-9196` (the tables layer's `DRS-9614`). Stored boards become reports with board pages at start (`drishti.bi.migrate.on-start`), keeping their ids; board files are converted by `drishti.py bi migrate --boards`. The pivot's saved formulas are migrated in place at start, idempotent and audited (`pivot-formula-migrate`). Every migration is idempotent.

## A29. Python compute: one Python runtime

Drishti runs computation that is not Java, and code people write, **out of the server's process**, in worker processes the server supervises: the module `drishti-compute` (supervisor, pools, transport, limits, scheduling, fan-out, environments, approvals) and the pure-Python worker SDK `drishti_compute`. Nothing in the module knows about BI or VaR: an engine is a Python class a pool loads. How to use Python, and the administrator's part, is BI_GUIDE.md; the operator's guide to the pools is OPERATIONS_GUIDE.md › Compute workers; every setting is in CONFIGURATION.md.

The fixed decisions: Python runs in **out-of-process workers** with data passed as **Arrow**, **as the user**, from a **pinned environment**; **no FFI and no embedded CPython** in the server (no Panama/FFM, no JEP or GraalPy; Java stays on 21); a native library is never loaded by the server; libraries a connector needs are **approved by an administrator** before use.

### A29.1 One Python runtime

Every Python run in Drishti is the server's, on the compute pool **`python`**: python visuals and Python measures of reports, boards and alerts, Transforms behind Python datasets, Python-driven tables and models (their sample run and their rows when read), the Notebook's Python cells, the Transform editor's step-by-step run, the profile of a frame and an item's tests. The console runs none and imports no compute library (a static test holds it: `drishti-console/tests/test_console_runs_no_compute.py`); the server never calls the console to run Python, and no BI page loads Pyodide or WebAssembly (Calc, the user's own browser scratch pad, is the one exception and never publishes). One way per job: one runtime, one sandbox, one set of pinned libraries.

```
console page ──► console route (identity only) ──► server: PythonRuns (the one door)
                                                     │  PythonFrames: rows read AS THE PERSON (row rules, masks)
                                                     ▼
                                          drishti-compute: queue · fairness · limits · audit
                                                     │  JSON-lines control + Arrow IPC files in the scratch folder
                                                     ▼
                        pool python: worker (pandas, NumPy, pyarrow, SciPy imported once) ──fork──► child per request
                                                     │  strict sandbox: netns without interface, Landlock, audit hook
                                                     ▼
                                  answer: JSON (a number per group, a chart spec) or out/result.parquet (a Transform)
```

**The server's side.** `PythonRuns` (drishti-server, `bi/python`) is the one door: it refuses in words while code cannot run safely, checks the item's packages against the pinned allow-list and the input size, caps the item's time and memory (`drishti.bi.report.python-max-seconds`, `python-max-memory-mb`), submits to the pool **as the person** (their viewer, for fairness and audit), maps the answer to `DRS-9070` to `9074` (the traceback for authors only) and writes one audit line per run (`python-run`: who, what, the pool, the outcome, the time, the bytes). `PythonFrames` reads the rows a run needs **as the person running it** (a model query, a dataset's fields, a Transform input, a visual's rows): row rules and masks apply before anything reaches Python. A Transform's result is checked against its **output contract** (the columns and types the dataset or table declares; a Python-driven table's sample run also checks its keys and never-empty columns). Dates reach Python as ISO text, so a text column the contract declares `date` or `timestamp` is read as one on the pool before the check; a value that does not read leaves the column as text and the check refuses it.

**The engine.** The pool loads `drishti_compute.items:ItemsEngine` (`drishti-compute/src/main/python/drishti_compute/items.py`) with kinds `run` (an item: manifest, source, parameters, input files, limits), `check` (the source parsed, its entry and imports checked; nothing runs) and `about` (the interpreter's versions, for the pin check). A failure of the author's code is an **answer** (`ok: false`, its kind and the author's own traceback), never a crash of the worker. The plan's Python nodes (`PyAggregate`, `PyColumn`) are placed on the tier `python`.

### A29.2 Pools and workers

A **pool** is a set of worker processes of one type and environment, configured per site in `config/compute/compute.yaml` (`compute.pools.<p>`: `type`, `engines`, `environment`, `isolation`, `workers.min/max`, `mode`, `memory`, `cpu-seconds`, `recycle-after`, `batch-share`, `idle-stop`, `timeout`, `timeout-batch`). `compute.enabled` is off as shipped: no worker starts. The worker count is configuration only, per pool and per node (`compute.nodes.<node>.pools.<p>.workers`); the code has no upper bound and no part of it assumes one worker.

- **One request at a time per worker.** Concurrency comes from the number of workers; a worker never holds two users' data at once.
- **Linux: fork-server.** Each worker is a parent that imports the environment's heavy modules once and **forks one child per request**; the child runs it and exits, so isolation between requests is a fresh address space, and a child can be killed alone at a timeout or cancel while the worker stays. The pool `python` is always `mode: fork`.
- **Windows: warm workers.** No `fork`: workers start with `spawn`, are reset between requests (caches registered with `register_cache` cleared, temporary files removed) and recycled after `recycle-after` requests (50 by default in warm mode) or on any error.
- **Supervisor.** Starts, watches, limits, recycles and kills workers; a pool whose workers keep crashing is `DEGRADED` in Health.
- **Fair scheduling.** One queue per pool and class (`interactive`, then `event`, then `batch`); `batch` holds at most `batch-share` of the workers; within a class users take turns and none runs more than `compute.scheduling.max-inflight-per-user` at once; a full queue refuses at once (`DRS-9203`; `DRS-9074` for a Python run). Waiting counts against the request's budget.
- **Fan-out and scale-out.** A request may be split into chunks of `compute.fan-out.chunk-rows` rows of one named input (other inputs go whole with each chunk); each chunk is an ordinary request (`<parent>-c<i>`) with the parent's viewer, class and budget, sent to the target with the lowest *(outstanding + 1) / workers* among this node and every Drishti*net member that serves the pool (signed, on behalf of the viewer, one hop). Answers are gathered in chunk order; an unreachable node's chunk runs here; any other failure fails the whole request. At most twice the total worker count of chunks is outstanding.
- **Java workers** (`type: java`, separate JVMs, one request at a time, recycled like Python workers) are designed for native pricing libraries and are refused until built (29.8).

### A29.3 Data hand-off

| Channel | Carries | How |
|---|---|---|
| control | request, progress, result metadata, errors | JSON lines on the worker's standard input and output |
| data in | the inputs | Arrow IPC files in the request's scratch folder `<data dir>/compute/scratch/<request id>/in/` (memory-mapped by the worker); small inputs inline |
| data out | large results | Arrow IPC or Parquet files in `…/out/` (a Transform: `out/result.parquet`), read by the server, validated, then the folder is deleted |

Files rather than sockets: they work the same on Linux and Windows, need no port, and let the worker memory-map large inputs. The scratch folder is created per request, readable only by the server's OS user, and deleted when the request ends (also after a crash, by the supervisor's sweep at start). For the pool `python` the rows go over as Arrow IPC streams exactly as read (a BI frame holds dates and decimals the SDK's own codec does not read); `ComputeRequest.files` and `ComputeResult.files` carry them, also between Drishti*net members. Results are untrusted: the server validates each against its schema and size limits (`compute.limits.max-result-bytes`) before it is cached or shown.

**"As the user".** Workers never read data themselves. The server reads inputs as the requesting user and masks them; the request carries the user's identity for audit (never credentials); results go only to that user's request id; limits count against the user's quota. A worker does not switch operating-system user.

### A29.4 The strict sandbox

Approved engines run in the **standard sandbox**: an empty environment (only `PYTHONHASHSEED`, the engine's temporary folder variable and thread limits; no tokens, no proxies), `RLIMIT_CPU`, `RLIMIT_AS` (Windows: the working set watched every 250 ms), `RLIMIT_FSIZE`, the wall time enforced by killing the process group, writes only under the scratch folder, and an audit hook refusing sockets, `subprocess`, `os.fork` and `os.exec*`. A request carries parameters, never code.

The pool `python` is the one exception: its engine runs **code people write**, so it runs only with `isolation: strict`:

| Part | How | Where it cannot be had |
|---|---|---|
| No network | the worker moves itself into a user and network namespace of its own at start, with no interface; Landlock also refuses TCP (ABI 4 and later) | the worker does not start and says why |
| Files | each request's fork child is held by **Landlock** to reading the Python installation, the module path, the system libraries and its scratch folder, and to writing that folder only, before the request's code runs (so C and C++ code is held too) | as above |

| Part | How | Where it cannot be had |
|---|---|---|
| Processes, loader | the engine's audit hook refuses `subprocess`, `os.system`, `exec*`, `spawn*`, `fork`, `ctypes`; `exec`, `eval`, `compile` are not given to the code | |
| Imports | an import guard allows only the manifest's packages (each on the pinned allow-list) and safe standard-library modules | |
| Time, memory, output | `RLIMIT_CPU` and `RLIMIT_AS` of the item's own (lowered from the pool's), the pool's wall-time budget; an answer at most 4 MB, the input at most 256 MB | |
| Pinned libraries | 29.5 | every run refused |
| Audit | one `python-run` line per run | |

**Where it cannot be safe, it does not run.** The strict sandbox needs Linux (fork mode, user namespaces, Landlock, kernel 5.13+). There is no weaker fallback: elsewhere every run answers `DRS-9074` with the reason. Approved network sources are read by the server and handed in as inputs; the code itself never reaches a network. Tests: the SDK's pytest suite (`drishti-compute/src/test/python`, among them the no-network, Landlock and refusal tests), `ComputePoolTest` (crash, loop, limits, isolation, cancel) and `PythonRunsTest` in the server.

### A29.5 The locked environment and the lock file

A pool's `environment` is either `system` (the site's interpreter, `compute.python`, and the SDK) or a **locked environment** `{ python, lock, index }`.

- **The lock file** pins every package of the environment, transitive ones included, to one version with the sha256 of each of its distribution files. It is written once by `tools/python_lock.py` from a `name==version` list (for the pool `python`: `config/compute/locks/python.in` → `python.lock`), reading the hashes from the package index's JSON API, reviewed and committed; it is never resolved again at install time.
- **One environment per lock.** Its id is the sha256 of (Python version, platform, lock file); pools and connectors with the same lock share it. It is built under `<data dir>/compute/envs/<env id>/` (wheels cache `<data dir>/compute/wheels/`), never in a system location or `/tmp`: `python -m venv`, then `pip install --require-hashes --no-deps --only-binary=:all:` from approved sources only (`shipped` wheels, a named index in `compute.indexes`, or a local path; public PyPI only when the site names it). No source builds, no resolution.
- **Read-only and verified.** The built folder is made read-only and a manifest of hashes is checked at every worker start; a changed file stops the worker (`DOWN` in Health, the file named). A new lock is a new environment; the old one stays until nothing uses it. A failed build leaves no environment and names the wheel.
- **The pin check for the pool `python`.** The allow-list `drishti.bi.report.python-packages` offers only what `python.lock` pins (a test holds the two together). With `environment: system`, the server asks the pool (`about`) for its interpreter's versions before any run and refuses every run, naming each package, while one differs from its pin.

### A29.6 Requirements and approval

A connector file may declare `requires:`: `python` (`version`, a `lock` beside the connector file, an `index`), `java` (Maven coordinates with `sha256`, `classifier`, `source: shipped | bank-maven`, `native`) and `plugins` (connector plugins it depends on, checked, not installed). Nothing in it is fetched, built or run until an **administrator approves** it in Admin → Compute, which shows the diff against what is approved per package (name, version, hash, source, size, native code). A new package, version, hash, source or Java library needs approval; removing one does not. The approver is any administrator or one holding `compute.approval.roles`, never the person who changed the file (four eyes; `requested-by:` names them). Each decision is an audit entry, approvals are kept in the Drishti database so every node sees them, and a revocation refuses new requests at once. A connector whose requirements are not all approved stays off (state `NOT_APPROVED`, *waiting for approval of N requirements*). Java libraries are checked by sha256, scanned for native code (a jar with a `.so`, `.dll` or `.dylib`, or `native: true`, may run only in a Java worker) and staged under `<data dir>/compute/jars/<sha256>/`. Environments are built manually or on approval (`compute.build`). A signed pack's wheels and jars still need approval.

### A29.7 Windows

Python workers, locked environments (from `win_amd64` wheels, the lock listing both platforms) and the standard sandbox work on Windows (warm workers, limits watched by the supervisor). There is no network namespace and no Landlock, so the pool `python` does not run there: code people write is off with the reason. Native libraries are never placed on the Windows QA machine; their tests skip with the reason. Every path is under the data folder (no `%TEMP%`).

### A29.8 Pricing engines (planned)

Designed, not built: a pricer SPI in the SDK (`price`, `scenario_pnl` returning a P&L vector per trade over the scenario days) behind which **QLRisk** is the default provider once its API is declared stable (until then Drishti does not import it), and **QuantLib-SWIG Java** in Java workers, its native library extracted under `<data dir>/compute/native/` and checked by sha256 before loading, with a parity test between the two. Base VaR needs neither: it adds precomputed vectors.

## A30. The client renderer and Drishti Elements

How other web applications show Drishti screens without an iframe, and how the code that draws them is built. The how-to for host developers (the elements, tokens, the sign-in window, the render API, theming, versions) is EMBED_GUIDE.md; the screen contract both renderers draw is Part IV · The ViewModel contract; the JavaScript component libraries are ELEMENTS.md. Decisions: ADR-021

(embedding, the token, served versions) and ADR-022 (one renderer in the browser).

### A30.1 One page renderer, one embed renderer

- **The console's pages are drawn by the server**: Jinja and the view macros (30.3). That is the one page renderer.
- **Drishti Render** (`drishti-renderer`, Elements library 7, renderer 2.0.0) draws a screen in the browser from the ViewModel JSON. It draws **embeds** (Elements 2.x, the embed API v2) and `<drishti-report>` only. Renderer 1.2.0 stays served, frozen, for pages that pinned it.
- **Drishti Elements 1.x** (`web/embed/`, version 1.4.5, frozen) is HTML over the wire: the console renders panels with the same macros and the element swaps sanitised fragments into a shadow root. It carries `<drishti-report>`, `<drishti-about>` and `<drishti-search>`.
- **Elements 2.x** is `<drishti-view>`, `<drishti-panel>` and `<drishti-screen>` on Drishti Render, with 1.x's element API.

The rule that holds it together: **the server is the only judge of what a user may see** (roles, masks, row rules, per viewer, before the JSON or HTML leaves); the browser code decides nothing about rights and cannot unmask. A parity suite (30.8) keeps the browser renderer and the macros drawing the same thing.

```
host backend ──POST /api/v1/embed/token (RFC 8693)──► drishti-server  (EmbedTokenService, TokenFilter, AccessRecorder embed:<app>)
     │ token to its own page                                ▲  bearer passed through
host page ──module + sheet (immutable, SRI)──► drishti-console /embed/v{1,2}/**  (EmbedAuth, CORS per app, EmbedLimits)
  <drishti-view> ── GET /embed/v2/views/{kind}/{id} ──────────┤  ViewModel JSON (v2) / macro HTML (v1)
  one Connection per page ── GET /embed/v2/channel?s=vm:… ──┘  frames over ChannelSession, token renewed in place
```

### A30.2 The renderer as an Elements library

The renderer has no grid, chart, format, live or input code of its own: it draws panels with the same components the console uses, so one component test covers both (`test_render_library.py` fails an engine call, an `EventSource`, `toFixed`, `Intl.NumberFormat` or a table built by the renderer).

| Part of a screen | Drawn by |
|---|---|
| tables, trees, drills, the pivot kind, every chart's Data table | drishti-grid's document table (`docPivotable`, `docDrill`, `docPivot`, `docPlainTable`, `enhanceDocTables`) with the renderer's cells as hooks |
| the nested pivot in `behaviours: 'console'` | drishti-grid's grid with tree data (every figure the server's text, `clientAggregation` off) |
| every chart | drishti-chart's panel charts (`PanelCharts`, `panelOption`): the console's options series for series; the only place an engine is called |
| the element maker, untrusted-data rules, formats, the API client | drishti-core (`h`, `attrAllowed`, `safeUrl`, `roundHalfUp`, `formatNumber`, `formatPythonFloat`, `createClient`) |
| streams and the one connection per page | drishti-data's `LiveClient` (SSE over fetch so the bearer travels; back-off; resume) |
| About this page | the renderer, from JSON; its source a drishti-insight hook (`options.insight.about`) |
| funnels, Chart any table, zoom, controls (`behaviours: 'console'`) | still the console's root-scoped scripts, loaded beside the library |

The renderer's own modules (`static/elements/drishti-renderer/src/`, each under 530 lines) hold screen logic only: `drishti_render.js` (entry, `render`, the `Handle`, `<drishti-screen>`), `drishti_screen.js` (paint, patch, stream messages, in-place navigation, events), `drishti_view.js`, `drishti_layout.js` (a port of `core/screen_grid.py`'s `arrange`), `drishti_kinds.js` (the panel frame, its states, each kind mapped onto the libraries), `drishti_cells.js`, `drishti_behave.js`, `drishti_net.js`, `drishti_assets.js`, `drishti_console.js`, `drishti_nested.js`, `drishti_about.js`, `drishti_signin.js`, `drishti_embed.js`, `drishti_elements.js`, `drishti_dom.js`. `tools/elements/build.py` bundles them as `drishti_renderer.js` for classic pages; hosts import the public entry `static/render/2.0.0/drishti-render.js`, which finds the libraries, the console's static files and the sheet from its own address, so the same files work under `/embed/v2/render/2.0.0/`. Size: 168 KB, 63 KB gzip of its own; ECharts (330 KB gzip) only when a screen has a chart, once per page.

**One paint and one patch path for three sources.** A static ViewModel, a live one and a streamed one go through the same code: a streamed screen is a skeleton painted like any ViewModel, then `panel-replace` messages applied exactly like live `panel` patches. A streamed screen therefore ends equal to a one-shot render (tested on the showcase, a Sutra grid with areas, tabs and a split, and every panel state). GenUI (screens composed by an AI agent and streamed this way) is a design, not built.

### A30.3 The view macros shared with the console

The console's view parts are macros in `templates/_macros/view.html` (`vtitle`, `strip`, `vhead`, `panels(items, embed)`, `provenance_banners(...)`) and `_macros/panels.html` (`panel(p, embed=false)`, one macro per kind). `terminal/view.html` and the embed API v1 (`routes/embed_routes.py` `_render`) call the same macros, so there is no second template; `embed=true` drops only what means nothing in a host page (the help link and the CSV download), and `tests/test_view_macros.py` proves a panel's HTML is the same on the console page and in the embed payload. Live frames reuse the same macros (`_panel_html`), so live and static views look identical.

The server formats every value a person reads; both renderers compute only bar widths, gauge levels, donut shares, heat buckets and progress, with **half-up rounding on the decimal value** (the macros' `half_up` filter, `core/half_up.py`, and drishti-core's `roundHalfUp`), so the two print the same characters.

### A30.4 Embedded views and sign-in

**The embed payload.** v1 (`GET /embed/v1/views/{kind}/{id}`) answers the head, strip cells (as data, because live strip patches update cells by index), banners and each panel as macro HTML, plus provenance, `masked` (count and panel ids, counted on the built view by the server), the channel key and the controls. v2 (`GET /embed/v2/views/{kind}/{id}`) answers the ViewModel itself (`"contract": "1.0"`, `Drishti-Subscribe` naming its live key); `…/panels`, `…/about` (`{explain, index}`) and the live patches are JSON too. A control change asks only for the panels that read it (`…/panels?panels=&ctl.…`).

**The embed token.** The host's backend exchanges its user for a token at the server (`EmbedTokenService`, RFC 8693): client authentication by a hashed secret or `private_key_jwt` (RSA or ECDSA JWKS); the subject is an OIDC ID token or a host-signed assertion (`exp` ≤ 60 s, `jti` remembered), never a bare username, so a stolen secret alone names nobody. The token is an ES256 JWT signed with the server's own key (`EmbedKeys`, `GET /api/v1/embed/jwks`, rotated by `kid`), not the console's HS256 secret, so the console can verify it but not mint it. Claims: `sub`, `azp` (the application), `aud`, `scope`, `origins`, `typ: drishti-embed+jwt`, five minutes by default.

**On every call.** The console's `EmbedAuth` (`core/embed_auth.py`) verifies signature, `typ`, `exp`, audience and the request's `Origin` against the token's `origins` and the registered origins (one refetch of the key set for an unknown `kid`, throttled), then passes the token on untouched. The server's `TokenFilter` verifies it again, checks the application and user are enabled and the kind or `report:<id>` allowed, loads the user's current roles **minus** `raw` (embed calls always mask, because host scripts can read the open shadow root), applies the scope's `METHOD path` allow-list (`drishti.embed.scopes`; nothing writes), and records the access as `embed:<app>`. Rates are per application and per user in one-minute windows (`RateWindows`, `EmbedLimits`); the console's `embed.max_streams` bounds upstream streams so embedding cannot starve the console's own users. Host applications live in the identity database (`EmbedAppStore`, table `drishti_embed_app`), or in configuration (`drishti.embed.apps`, read-only); usage counters are in-memory `LongAdder`s (`EmbedUsage`) and Micrometer meters.

**CORS.** Only the embed paths answer CORS, echoing a registered origin (`Vary: Origin`), never with credentials; the allow-list comes from the server (`GET /api/v1/embed/apps/origins`, cached), so it has one source. Static library files are `Access-Control-Allow-Origin: *`.

**The sign-in window** (`auth: 'popup'`, for pages with no backend). The console's `GET /embed/v2/sign-in` runs single sign-on with its OIDC client (PKCE; state, nonce and verifier sealed in a ten-minute cookie), the callback has the server verify the ID token twice (as a sign-in, and for the window by the console's service identity, `POST /api/v1/embed/sign-in-window`), and the server mints the embed token for the registered application whose origin opened the window. The token reaches the page by `postMessage` to the exact origin, accepted only from the console's window with the page's random state; it lives in module memory only. Renewal uses a handle sealed by the console (AES-GCM: refresh token, origin, user) at `POST /embed/v2/sign-in/refresh`; `/check` lets an unregistered page show a disabled button with no window; `mode=redirect` returns a one-time code in the fragment, redeemed within two minutes.

### A30.5 The live channel

`core/channel.py` `ChannelSession` is the console's live channel as a class: subscriptions and their upstream responses, the queue, the detached tasks, the watchdog and the SSE text, with no request or cookie in it. The identity, the `render(event, data)` that turns an upstream event into what the caller wants, the subscription limit, the owner and the registry are constructor arguments. `/api/channel` (the console's tabs) and `/embed/v{1,2}/channel` are thin routes over it; the owner of a channel is **the user plus the host application**, so a token of another application for the same user cannot touch it. With several console workers the channel id names the worker holding the stream and changes are relayed to it (`core/channel_relay.py`).

- **One connection per page.** `DrishtiElements` keeps one connection per `server`, the console's hub built with an injected transport (an `EventSource`-shaped class over `fetch` and `ReadableStream`, because `EventSource` cannot send `Authorization` and a token never goes in a URL); keys from all elements are deduplicated. v1 subscribes `view:` keys with HTML panel patches, v2 `vm:` keys with panels as JSON.
- **Tokens outlive nothing.** `StreamGuard` (`core/embed_stream.py`): at the token's `exp` the console sends `event: token`; the client, already renewing before expiry, posts a fresh token for the same `sub` and `azp` to `POST /embed/v{n}/channel/{cid}/token`; without one within `embed.token_grace_seconds` the stream ends `gone DRS-8001`. Every `embed.recheck_seconds` the console asks the server (`GET /api/v1/embed/check`) and ends streams of disabled users or applications.
- **Repaint only for a reason.** Ticks patch in place (strip cells, whole panels, chart data); a reconnect or a subscription's first `view` resyncs by fetching the view and applying only differences; only an attribute change, `refresh()`/`reload()`, a restore or a Retry repaints. Elements pause when hidden or off-screen for 10 s and give their keys back. The stream is never compressed.

### A30.6 The renderer's sheet and integrity record, and the element stylesheet

**The renderer's sheet and integrity record.** `tools/render_sheet.py` (library `core/render_dist.py`) generates `static/render/<version>/drishti-render.css` from the console's own sheets in the order the view page loads them, with drishti-grid's sheet, rewritten for a shadow root: `:root`, `html` and `body` become `:host`, width media queries become container queries on the host (a screen answers to its container's width), plus the glyphs the macros and modules use and a short addendum (box sizing, `[hidden]`, sort buttons, pager, back bar, 44 px targets). It is adopted as a constructable sheet, so a host with `style-src 'self'` needs nothing; the icon font is registered with the `FontFace` API because `@font-face` inside a shadow root is not used. The same tool writes the version's `integrity.json` (SHA-384 of every distributed file and of the renderer's modules).

**The element stylesheet.** `tools/elements_sheet.py` (library `core/element_sheet.py`) writes Elements 1.x's `web/elements/drishti-view.css` and its manifest (tokens, theme, view parts, layout, pivot, about, controls; `:root[data-theme=x]` becomes `:host([theme=x])`; of Bootstrap Icons only the glyphs the macros name) and `drishti-report.css` from the BI sheets. Its guard (`tests/test_element_sheet.py`) fails when the committed files differ from a fresh build, on a missing or unused glyph, or over the gzip target.

### A30.7 The element package distribution

- **Immutable versions.** One folder per renderer version (`web/static/render/<version>/`); Elements 1.x at `core/element_dist.py VERSION` with the committed record `web/elements/integrity.json`. `test_render_library.py`, `test_render_phase2.py` and `test_element_package.py` fail when a byte of a released version changes without a new version: a version names the same bytes for ever.
- **Paths.** `/static/render/<v>/` on the console's origin; `/embed/v2/render/<v>/` for other origins (immutable, one year; a partial version `302`s to the exact one; an unserved one is `410 DRS-8006`); `/embed/v2/elements/<v>/drishti-elements.js` `302`s to the renderer that carries that Elements version; `/embed/v2/elements/<library>/…`, `/embed/v2/js/<name>.js` and `/embed/v2/vendor/…` are allow-listed files the library loads from beside itself; `/embed/v1/elements/<v>/…` serves 1.x with `ETag` = the file's SRI value.
- **Lifecycle.** `embed.lifecycle` (one entry per API major) adds `Deprecation` (RFC 9745), `Sunset` (RFC 8594) and a `Link` to every answer from the announcement, and `410 DRS-8006` from the sunset day; a `Drishti-Embed-Api` header naming an unserved major is `410`.
- **Packages.** `@drishti/render` is the version folder laid out as an npm package (documented, not published); the Elements libraries are distributed by `drishti.py elements pack|dist` as a zip, npm tarballs and a WebJar (ELEMENTS.md section 8.7). `core/element_api.py` holds the element API as data: it generates the reference tables in EMBED_GUIDE.md and the TypeScript declarations, and a test checks it against the elements.

### A30.8 The parity suite

`tests/test_render_parity_browser.py` (with `render_harness.py` and `render_cases.py`; Playwright and Chromium) draws each ViewModel two ways: the console's macros on a page with its sheets and first-paint scripts, and the library into a blank page, both served from one made-up origin with the console's CSP. It compares the accessible tree (`aria_snapshot`), the visible text, every cell, every link and its target, every tone class, every chart's ECharts series and points (not pixels), and the page (no sideways scroll at 390 and 1280 px, no CSP violation, no request off the console's origin, no script error). The cases: the captured views, the all-panels showcase in all 11 themes, seven edge-case ViewModels (every state, link, tone and odd shape), and every Sutra of every shipped pack bound on its sample document by a real server: 476 pairs, no difference beyond the intentional ones (an unknown kind says so; outside URLs are events, not new tabs; console-only affordances such as the download menu, Customize and the column resize handles are absent). The same method runs against every renderer version before it ships, and a hostile ViewModel (markup and script in every text field) must draw as text only.

### A30.9 Security model

| Concern | Answer |
|---|---|
| Who the user is | the host's backend, by client authentication plus a verifiable subject token; or single sign-on in the sign-in window |
| Token theft | five-minute, audience- and origin-bound, read-only, always masked, never in a URL or storage; `jti` and `azp` in the access log |
| Markup from data | Drishti Render never parses markup: `createElement`, text nodes and an attribute allow-list (no `on*`, `style`, `src`, `srcdoc`, `formaction`); safe URLs only (`javascript:`, `data:`, `vbscript:`, `//host` dropped); no `eval`, `new Function`, `innerHTML` (a test reads the sources). Elements 1.x parses fragments inertly in a `<template>` and rebuilds them from the macros' allow-list |
| Trusted Types | one policy each, `drishti-render` (script URLs of the console only) and `drishti-elements` |
| Host page policy | constructable sheets, `FontFace`, no inline script or style, no frames; the console keeps `frame-ancestors 'self'` |
| Host scripts in the shadow root | accepted (open roots); the reason embed calls always mask and never write |
| A malicious host | sees only its own users' data within its registered kinds; every view logged as `embed:<app>`; disabling the application revokes at once |
| Supply chain | immutable versions with published SRI, vendored engines only, nothing from a CDN |
| User switch on one page | one connection per server and token subject; a token for another user closes it and every element reloads |

### A30.10 Drishti Elements

Drishti Elements is two things with one name: the **embedding elements** above, and the **component libraries** (drishti-core, -inputs, -grid, -chart, -data, -insight, -renderer, -risk) that the console, the renderer and outside applications all draw with. The libraries' design (their APIs, the grid and its pivot, the AG Grid-style adapter, the chart spec and maps, packaging and licensing, the guards) is ELEMENTS.md; using them in another application is EMBED_GUIDE.md chapters 13 and 14. The renderer is library 7, so "no bespoke UI" holds for embeds too: a fix in a component reaches the console, the renderer and every host at once.

## A31. Collaboration, write-back and approvals

Three features let people act on the data together without leaving Drishti's controls: **share with a note** and **discussion threads** anchored to the data, **write-back** (change requests that correct, own or amend records) and the **one Approvals queue** for every four-eyes item. The user's side is in USER_GUIDE.md (share, inbox, discussion) and APPROVALS_GUIDE.md (corrections and approvals); form design in BUILD_GUIDE.md.

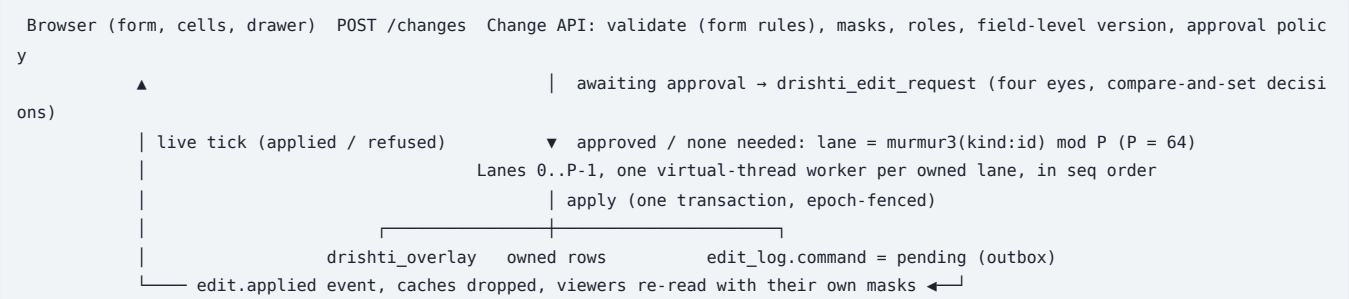
### A31.1 Why, and the rules that follow

People otherwise talk about a number elsewhere: a screenshot in chat shows what the *sender* may see, records no business date or version, and leaves no trail. Collaboration keeps the conversation inside the controls: **what a person receives is computed for that person, when they look, from what they may see now.** A share or comment stores the author's words and a *pin*, never a copy of the data. Non-goals: no general chat (every message hangs off a share or a data anchor), no social graph (recipients are users and roles from the directory), no rich text, no external identities; Drishti keeps a tamper-evident record and exports it, the firm's archive is where it is retained (no WORM claim).

Write-back follows the product owner's model: a dedicated edit queue whose worker applies updates **sequentially within a queue and concurrently across queues**, so at one moment only one edit applies to a given record. Its principles:

1. Nothing is editable unless a Sutra's form says so (`save:`), field by field; masks win.
2. Drishti never silently changes a source system: an edit is a change request whose destination is declared.
3. One change at a time per record, many records at once; no lost updates (a stale change is conflicted, never applied blindly).
4. Acknowledged means durable: a change is acknowledged only once its edit-log row is committed.
5. The UI, the CLI and the API make the same requests through the same handler; every change is audited and replayable.
6. Off by default at three levels (site switch, the form's `save:`, the connector's `writes:`); everything re-checked at apply time.
7. The connector is the one place that knows a source system, for reading and for writing.

### A31.2 Write-back: the parts and the flow




The change request carries `id` (a time-ordered ULID), `key` (`kind:id`), `kind`, `entity`, `businessDate`, `items` (`path`, `from`, `to`), `basedOn`, `target`, `connector`, `reason`, `by`, `at` and `state`: `submitted → (awaiting approval) → queued → applying → applied | refused | conflicted | rejected | withdrawn`. A form's Submit (`POST /changes {draft}`) turns its draft into one request per record and keeps the draft id; the draft is closed with the change id when applied, and marked when refused or conflicted. `POST /changes/preview` fills the review and confirm sheet (the difference, the destination, the approvers, the triggers' `affects:` scopes) without computing. Code: `drishti-server` `com.ash.drishti.server.edit` (`EditLog`, `Lanes`, `EditHandler`), the router's overlay layer in `drishti-engine` (`DocumentLayer`), `form.save` in `drishti-rachana`, `tools/changecli.py`.

### A31.3 Lanes, the edit log and the transaction handler

The key is the **record**, not the Sutra: per Sutra, two traders editing different trades would wait, and two Sutras showing one trade would not serialise its edits. Records share a **fixed number of lanes** (a partitioned log) rather than a queue each, so cost is bounded whatever the number of entities. The trade-off (unrelated keys in one lane wait) stays small: an apply is short (a source command is only *recorded*, its send and confirmation happen off the lane), consecutive changes to one key apply as a batch, and lane depth is a metric with a warning. Changing `P` moves keys, so it is done with the lanes drained.

`drishti_edit_lane` (one row per lane: `owner_node`, `epoch`, `lease_until` in epoch milliseconds on the database's clock, `next_seq`, `applied_seq`) and `drishti_edit_log` (`lane`, `seq`, unique `change_id`, `epoch`, `state`, `change_key`, the record's fields, `body` as JSON, `command` state and attempts, `signature`) live in the Drishti database (today the identity database's pool) in both schema files; beside them `drishti_edit_request`, `drishti_overlay`, `drishti_owned`, `drishti_edit_version` and `drishti_edit_cursor`. A lane *is* its rows, so it survives any restart; a new owner replays from `applied_seq`, and a row found `applying` is re-applied safely (overlay and owned writes skip an applied change id; commands are idempotent).

The handler, per change in seq order: re-check the switch, edit power, kind, masks, the form's roles and read-only fields, restate and override roles; compare each item's `from` with the field's current value (overlays included); run the form's rules against the record as it is now (lookups re-read); then **in one transaction** check the lane's epoch, write the overlay or owned rows (or mark the command `pending`), set the state and advance `applied_seq`; then bump the record's version and publish. A lane deeper than `max-lane-depth` refuses with *busy*. No lock is held across I/O.

### A31.4 Concurrency: why a queue alone is not enough

A queue serialises writes, not decisions: two people at version 7 would each overwrite the other in turn. Every change therefore carries what the person saw, and the check is **field-level by value**: unchanged fields apply, a moved field is *conflicted* with the three values; *Apply mine anyway* needs the override role. Changes made at the source (a load, a ticket) move values too, so they are detected the same way. A form over another kind's rows (the book desk's trades) writes the overlay for the record and for the page row it was made on.

### A31.5 Ownership and fencing across a cluster

Gossip and the ring cannot guarantee one owner after a network split, so ownership is **taken in the store** and every write is **fenced**:

```
UPDATE drishti_edit_lane SET owner_node = :me, epoch = epoch + 1, lease_until = <now> + :lease
 WHERE lane = :lane AND (owner_node IS NULL OR lease_until < <now>);           -- take: 1 row mine, 0 someone else's
UPDATE drishti_edit_lane SET applied_seq = :seq
 WHERE lane = :lane AND owner_node = :me AND epoch = :myEpoch;                  -- every append/apply: 0 rows → roll back
```

The ring (rendezvous hashing over alive members) only suggests who tries; renewal runs every lease/3 and a failed renewal stops the lane at once; a clean leave releases the lease. A zombie owner (split off, paused by GC) can never write: its epoch is stale. Any node accepts a change and forwards it, signed and on behalf of the person, to the lane's owner (`POST /net/edits/append`); applied changes are broadcast so caches drop (each node caches overlays per record, also dropped every 30 s). `edit.log: auto` picks: PostgreSQL Drishti database → `postgres` (full failover); a node alone → `sqlite`; `edit.primary` set → the primary's SQLite, the others forward the whole write-back API (`EditForwardFilter`) and follow its overlays, with no automatic failover because two sides of a split could both elect themselves; otherwise edits are refused in the cluster. An S3 edit log (lease by ETag, fence entries) is designed, not built; `edit.log: s3` is refused.

### A31.6 Where a write lands: overlays, owned rows and the outbox

The lane owner's handler does every write. Overlays and owned rows share the transaction of the edit-log row and the epoch check, and readers lay overlays over the source document at read time (one indexed lookup per entity, cached by generation, invalidated by the applied tick); *How this view was built* names each. The lake is never written per edit (each edit would be a tiny file and commit, and the daily rebuild rewrites the date anyway); a live plan folds approved overlays into the next lake build with `_drishti_overlay` columns.

Source targets use the **transactional outbox**: the apply transaction marks `command: pending`; a sender sends in seq order, one in flight per record, carrying the change id; the confirmation (`reply` or `next-load`) marks it `applied` or `refused` (which withdraws the pending overlay and fires triggers again); a new owner resends pending and unconfirmed commands with the same id. Three layers each say one thing: the form says **what** (fields, rules, roles, target, command), the connector's `writes:` says **how** (templates with a required guard and idempotency clause, validated at load, parameters always bound), the lane owner says **when** and in what order. `SourcePlugin.writeKey/checkWrite/write` is implemented once per plugin; JDBC is built, guards for REST (`If-Match`, `Idempotency-Key`), Kafka, Aerospike and MongoDB are designed. A connector's writes are switched on by an administrator (audited, the switch ledger).

### A31.7 The applied event, triggers and viewers

After the commit the owner publishes `edit.applied` (`changeId`, `lane`, `seq`, `kind`, `entity`, `businessDate`, `target`, `paths`, `by`, `appliedAt`) to every reactive connector (`ReactiveHub.publish`, a `write-back` event; `edit.confirmed` / `edit.refused-at-source` for source targets, chosen by the trigger's `when:`). It is durable because it *is* the edit log: a per-lane cursor (`drishti_edit_cursor`) is replayed when a lane is gained. It carries paths, never values; a connector reads the record through the router, pinned to the change's seq. Recomputes are stamped with lane watermarks (`L17:3412`), coalesced, never conflict (a late result older than a newer one is dropped), and a failed recompute leaves the change applied and the result marked stale. Viewers of the record re-read it with their own masks; the submitter sees the recompute status. A conflicted, refused or withdrawn change fires nothing. Live plans: the submitter's own incremental impact (with VaR's incremental step) and a second approver driven by a recompute threshold.

### A31.8 The one Approvals queue

Every four-eyes kind (Sutra proposals, corrections, BI items, pivot functions, stress windows, report sign-offs, compute packages) keeps its own records and rules; the queue (`/api/v1/approvals`, `tools/approvalscli.py`) reads each kind's items into one list with the same shape (what changes, effect if approved, rules, questions, the actions the caller may take), names any source it could not read, pins items due soon (`drishti.approvals`), and sends each decision back to its kind. Decisions are compare-and-set, the maker never decides their own, and questions are a thread on the item whose notice to the maker carries no text. A correction awaiting approval is a row of `drishti_edit_request` and enters a lane only when approved, so the version check runs against the data at apply time.

### A31.9 Collaboration: the pin

Every share and comment records a **pin**: `businessDate`, `live`, `knownAt` (the as-of instant, else the write time), `generation` (from the page's provenance, accepted only if not above what the server holds, else `422 DRS-7011`), `source` and the Sutra. Opening a pin passes `asOf` and `knownAt` as **request parameters**, never the recipient's sticky cookie, and the sign-in redirect keeps the full query. A versioned store (Delta, Iceberg) returns the data as it was; a dated store without versions answers `DRS-1007` and the console retries without `knownAt` (*the latest for the date*); an undated or live source returns today's data and links to *What changed*. Generation is evidence, never an address.

### A31.10 Collaboration data model and API

Records live in the content store by default (`drishti.collab.store: content`, which every cluster node reaches), or in the identity database (`jpa`, one node) or files (`file`, one server; refused with a PostgreSQL identity database); every store passes one contract. Ids are prefixed ULIDs (`sh_`, `th_`, `cm_`): time-ordered, unguessable, no sequence needed. Tables: `drishti_share` (sender, kind, entity, panel, advisory `gate_kind`, pin, body, masked spans, channels, view state, thread id), `drishti_share_recipient` (addressed vs expanded, state `notified | no-access | no-pack | disabled | over-limit`, `opened_at`), `drishti_thread` (anchor `entity | panel | field | share`, state `open | resolved | locked`), `drishti_comment` (current state) and the insert-only `drishti_comment_revision` (SHA-

256 chain per thread), `drishti_mention`, `drishti_follow`, `drishti_inbox` (pointers, not content), `drishti_outbox`, `drishti_collab_hold`, `drishti_collab_seal`, `drishti_note_link` (old notes imported once as one-comment threads). The API (`/directory`, `/shares`, `/threads`, `/comments`, `/me/inbox`, `/admin/collab/*`, problem codes `DRS-7001-7016`) is in API_REFERENCE.md; a share id the caller may not read is `404 DRS-7001` whether it exists or not, so ids are no oracle. Personal API tokens only read.

### A31.11 Share and discussion, end to end

1. The people picker (`drishti_people.js`, `DirectoryService`) offers users who share a pack with the caller, from two letters, rate limited; a role shows its size, never its members.
2. `ShareService.share` requires the `collaborate` power, the pack's sharing switch and the sender's own right to open the kind; it resolves recipients (roles expand, `admin` and `service` never) and checks each **at delivery** with `Entitlements.mayReach` (the role opens the kind and its pack is assigned, so a pack switched off still delivers and the page offers *Switch on*), through `Principals` (any user's principal, cached a minute).
3. The note is scanned against the stored document at the pin: `Entitlements.maskedValues` lists masked values and `NoteText.spans` records where they occur; readers without `raw` see `•••` there. `{$.path}` quotes are stored as paths and rendered per reader.
4. One transaction (`CollabTx`) writes the share (with its own hash), recipients, inbox rows, outbox rows and the access-log `share` row (`AccessLog.recordNow`, never the droppable queue). After commit, `InboxHub` pushes a `notice` event on the existing `/alerts/stream` (no new connection).
5. `/share/{id}` resolves as the reader: no access gives a clean page with no entity facts; otherwise a `303` to the view with the pin as parameters and `X-Drishti-Share`, so the access log writes `view` with `share:sh_…` and `opened_at` is set.
6. Comments follow the same path (`ThreadService`, `Audience` for mentions with the same reach rule; skipped people get nothing). `CommentRenderer` returns structured parts set with `textContent`; quotes are evaluated at the comment's pin as the reader (`PinnedDocs`, cached a minute).

### A31.12 Security and privacy

| Threat | Defence |
|---|---|
| a recipient without rights, or rights removed later | delivery checks `mayReach`; every read (inbox, share, thread, email at send time) renders for current rights; no-access page shows nothing of the entity |
| a masked value typed into text | write-time spans, normalised (case, spacing, zero-width, NFKC, look-alikes); `•••` for readers without `raw`; an author without `raw` gets no warning (no oracle), one with `raw` is warned or refused (`on-masked-copy`); each detection is audited |
| values known from elsewhere | not detectable; `deny-patterns` refuse formats (an MRN) |
| a panel or field anchor on something denied | the thread records the gate kind; readers must open it too; markers never carry text |
| cross-pack leakage through `@mention` or the directory | mentions reach only people who `mayReach`; the directory is scoped to shared packs and rate limited; `reach` shown only under `undeliverable: tell` |
| email and chat | rendered per recipient (quotes filled, masks `•••`), `link-only` per pack; links to the console only; no tracking; addresses only from the directory; bridges render as the `render-as` role and never fill quotes; URLs and secrets from environment variables, an allow-list, no redirects |
| injection | plain text only, structured parts, `textContent`; email templates are fixed files with one-pass `${name}` substitution over a closed map; subjects stripped of CR/LF |
| abuse | per-user rate limits, recipient caps, per-recipient hourly mail cap; the console's CSRF guard |

Tamper evidence (the revision chain, seals over live rows, thread heads, holds and the audit trail) is evidence, not prevention; exports whose manifests the firm archives make a rewrite detectable. Write-back's edit-log rows carry a SHA-256 digest over their place and body.

### A31.13 Notifications pipeline

The domain row and its deliveries commit together (a transactional outbox), so a crash never loses a notice or sends one for a rolled-back share. `Notifier` beans (`InAppNotifier`, `EmailNotifier`, `BridgeNotifier`) receive the audience, so a new channel adds a bean and touches no service. `OutboxDispatcher` (a virtual thread per server, `outbox.enabled`) claims due rows under a lease, renders **at send time** for the recipient's current rights (or cancels), sends, and retries with doubling backoff to a dead letter (permanent failures at once). A `Message-ID` of `<seq.ref@host>` collapses the one possible duplicate. Notices to one person within `email.coalesce-window` become one digest. Across servers, `InboxHub` polls the inbox table above its high-water mark only while it has open streams. Preferences (`notify.email.share|mention|reply`) live in user settings; addressed shares and mentions cannot be muted in the app.

### A31.14 Retention, legal hold, export and the snapshot picture

Retention per kind, per pack, then global (`retention.keep-days`, default keep forever); a daily `CollabPurge` removes whole threads (never the middle of a chain) and old shares, audited with the final hash, and skips anything held. Retract and hide change what readers see, never the record. A legal hold (`compliance` power) has a scope (`entity`, `kind`, `user`, `thread`, `all`), a date range and a reason; it blocks purge and permanent removal (`423 DRS-7010`). An export is a job (ask, poll, download once, streamed through the console): `shares.jsonl`, `threads.jsonl` (unscrubbed, every revision), `chains.jsonl`, `holds.jsonl`, `inbox.jsonl`, `outbox.jsonl`, `README.txt` and a `manifest.json` with each file's SHA-256; `verify` recomputes chains. A share's optional picture is drawn on the server by Java2D (`SnapshotLayout`, `SnapshotPainter`; no browser) for the **most restrictive recipient**, with a tiled watermark; it is not stored but redrawn from the pin and cached by view, generation and rights profile, and every picture is an access-log `export` row.

### A31.15 Console design and performance budget

Routes `collab_routes.py`, `thread_routes.py`, `collab_admin_routes.py`; clients `core/collab.py`, `core/threads.py`; one side drawer with About and Discussion tabs (`drishti_about.js` hosts, `drishti_discussion*.js` draw from JSON); `drishti_share.js`, `drishti_people.js` (one ARIA combobox), `drishti_inbox.js`, `drishti_alerts.js`; `drishti_collab.css` with theme tokens only. Keys `Alt+S`, `Alt+N`, `Alt+I`; on a phone the dialog and drawer are sheets. Times use one formatter (`local_when`). Budget: the view page does no new work; thread counts p95 ≤ 10 ms (fetched after paint, cached 30 s); a 50-comment thread ≤ 40 ms; a 25-recipient share ≤ 80 ms in one transaction; a notice reaches the bell within 1 s on one server and `inbox.poll` + 1 s across servers; the collaboration scripts ≤ 40 KB. Rendered comments cache well because masks are binary (two mask classes per entity).

### A31.16 Key decisions and constraints

- Collaboration records are durable per-user state with a transactional outbox and links instead of data (ADR-020); notes grew into threads; roles are the groups; error codes use the first digit 7.
- Comments: a 15-minute edit window keeping every revision, then retract only; administrators hide with a reason; nothing erased before retention.
- Write-back: one Drishti database for the edit log, overlays and owned data; a durable shared log with ownership by conditional update and epoch fencing; the handler does every write; the lake is never written per edit; source writes through the outbox, JDBC first, no shipped connector writes; what is editable is declared by a form's `save:`; a new record cannot be an overlay; a person not in the directory is re-checked at apply with the roles they submitted with.
- Live plans: folding overlays into the next lake build; archiving edit-log entries past the hot window to signed files; the S3 edit log; REST and Kafka writes.

## A32. Clusters inside: membership, placement and routing

How Drishti*net and the shard machinery work inside: the member table and gossip, the capability digest, routing, the copying of identity and user state, the shard map, rendezvous placement, handover and the scatter-gather of partials, and the resource bounds of one server. Running a cluster (files, Admin → Cluster, roles, sizing, AWS, tuning) is the operators' CLUSTER_GUIDE.md. Section A13a is the short form.

### A32.1 Membership by gossip (SWIM over HTTP)

`drishti-net` (no Spring, no engine; an ArchUnit rule keeps it so) holds `MemberTable`, `Gossiper`, `PeersFile`, `NodeIdentity`, `NetFolder` and `NetSchema` (the `config/net/` folder with line numbers, history and `If-Match`), `ClusterKey`, `OnBehalfOf` and `PeerHttp`. In the server, `NetService` runs the node, `NetCallFilter` checks the cluster key before the token filter, `NetController` and `NetAdminController` serve the membership and management endpoints.

**The member entry** (`Member`): `nodeId` (generated once, kept in `<data dir>/net/node-id`), `url` (a real address; loopback, wildcard, link-local and container-bridge addresses in any spelling are refused at the intake by `MemberTable.merge` and `HostAddress.callableUrl`), `gossipPort`, `incarnation` (raised only by the member itself), `heartbeat` (raised every round), `state` (`alive`, `suspect`, `dead`, `left`), `digestHash`, `switchesHash`, `version`, `protocol` (`Member.PROTOCOL` = 1; another protocol is listed, never routed), `tags`, `since` and `drained`. Beside each entry a member keeps local times only (`beatAt`, `heardAt`, a smoothed latency, `stateAt`, `MemberTable.Known`), so clock skew never makes a member suspect.

**A round** (`Gossiper.round`, every `interval`, on the platform thread `drishti-net-gossip`):

1. raise its own heartbeat;
2. push-pull with `fanout` random members that are alive or suspect (a suspect one hears of its suspicion and can refute), in parallel on the eight platform threads `drishti-net-gossip-io-*`, each cut at `max(200 ms, 2 × interval)`: `POST /api/v1/net/gossip` with the whole table; the answer is the peer's whole table; both merge;
3. reaching no one: try the join list (seeds, then `peers.json`, alive first), three at a time; and every 30th round exchange with one seed or saved member outside the reachable table, which heals a split;
4. detect: an alive member whose heartbeat has not risen here for `suspect-after` is pinged directly on its gossip port, then through `indirect-probes` others (`GET /api/v1/net/ping?target=`); still silent, `suspect`, keeping its incarnation;
5. suspect for `dead-after`: `dead`; dead or left entries are forgotten after `forget-after`;
6. tell the listeners (save `peers.json` on a join or leave, pull new digests); save `peers.json` every `persist-every` anyway.

**The merge rule** (`Member.supersededBy`): the higher incarnation wins; at equal incarnation the worse state (`left` > `dead` > `suspect` > `alive`); at equal incarnation and state the higher heartbeat (it carries the newest url, tags, digest hash and drain flag). A member's own entry is changed only by itself: news of itself worse than `alive`, or at a higher incarnation, makes it raise its incarnation and stay `alive`. **Leaving** (`Gossiper.leave(3)`): `left` with a raised heartbeat, pushed to every reachable member three times. **Rejoining** raises the incarnation. **Joining** happens on Spring's `ApplicationReadyEvent`; the node is *joining* (`DigestService.joining()`) until it has the digest of every alive peer, at most `join-grace`.

**Two lanes.** `PeerHttp` has a **control lane** (gossip, probes, digest pulls: HTTP/1.1, its own connections, two platform threads, a connect timeout of half the gossip timeout, to the member's gossip port) and a **data lane** (views, documents, history, searches, state, the live relay: HTTP/2 when routing.yaml's `peer-protocol` is `http2`). The receiver's **gossip port** (`GossipConnector`) is a second Tomcat connector with its own acceptor, 4,096 connections and eight platform threads, serving only gossip, ping and digest; on both ports those three are answered by the first filter (`GossipPath`), from memory, after the cluster key check. A per-path executor is not possible

in Tomcat (an executor belongs to a connector) and a priority filter alone still needs a request thread, which is what a busy server lacks (`NetGossipBusyTest`: with `gossip-port: off` every node suspected both peers under load; with the port, none). HTTP/2 is primed per peer by one signed ping made alone (the first call, and the first after 30 s of quiet), so the upgrade happens once; streams per connection are raised to 1,000 (`NetConfiguration.peerHttp2Limits`); a peer without HTTP/2 answers in HTTP/1.1 on the same call.

**Signatures.** `METHOD\npath?query\ntime\nnode\nsha256(body)`, HMAC-SHA256 with the cluster key, the node in `X-Drishti-Net-Node`, a 60 s window (`ClusterKey.WINDOW_MS`). The on-behalf-of token names the user, roles, embed app, the forwarding node, the request id and its audience node (`aud`), valid 30 s. Mutual TLS is the transport on top, never in place of the HMAC (which binds the call to its body, path, time and node).

### A32.2 Message sizes and convergence

A member entry is about 350 to 450 bytes of JSON. Each round a member sends and receives `fanout` whole tables (estimates):

| Members | Table | Bytes a member a second | Rounds for news to reach all |
|---|---|---|---|
| 8 | ~3.5 KB | ~21 KB | 2 |
| 20 | ~9 KB | ~54 KB | 2–3 |
| 50 | ~22 KB | ~130 KB | 3–4 |
| 100 | ~45 KB | ~270 KB | 4–5 |

With push-pull the informed share grows about fourfold a round: $\log_4 N$ rounds plus one or two. A heartbeat seen second-hand is up to that many rounds old, so at 100 members a healthy member's heartbeat can look 4 to 5 s old, just under `suspect-after`: probes answer, but their traffic rises, which is why larger clusters raise `suspect-after`.

### A32.3 Failure detection and its timing

With the defaults (1 s, 5 s, 30 s), a member that crashes at t = 0: at 5–6 s peers find its heartbeat stale, the direct ping fails (refused at once, or cut at 2 s on a black hole), two indirect pings fail; at 5–8 s it is `suspect` everywhere within 1–3 rounds, and `ShardPeers.peers` and `Member.routable` leave it out; at 35–38 s `dead`; at 24 h forgotten. A member that was only slow answers a probe or refutes with a higher incarnation, and nothing moved unless it stayed out longer than `stable-seconds`.

### A32.4 The capability digest

`Digest` and `CapabilityTable` live in `drishti-net`; the server's `DigestService` rebuilds this node's digest every round from cheap reads, announces a new hash at once (a round out of turn), compares peers' hashes every 250 ms, pulls a changed digest (`GET /api/v1/net/digest`) and swaps the capability table whole. The digest holds `nodeId`, `url`, `version`, `protocol`; `packs` (name, code, version, content hash, `peerOnly`); `kinds` with their sources (connector, type, table identity, `earliestDate`, `latestDate`, `dataVersion`, columnar, dated, live); `load` (the average latency of the last full minute, from `RequestActivity`); `calendar`; masked fields, nodes per role and BI report ids; and `services` (name to compact text: `engines`, the VaR engine's `var-shards`). The hash leaves out the load and the build time, so it moves only when what the node serves moves; a held digest is refreshed once a minute for the load.

**Table identity** is `SourcePlugin.tableIdentity(kind)`: the Delta connector gives `delta:<id>` from the first commit, else its location (a local path qualified by the machine's name); the `file` connector its folder and a SHA-256 of the kind's files (`file:<folder>#<hex>`); any other source nothing, and is a shard of its own node. `SourcePlugin.holdsNone(kind)` leaves out a source that surely holds nothing of the kind. Two nodes naming the same identity are replicas for the kind; different identities are shards.

**Switches** (commands and packs on or off) carry a hybrid-logical-clock stamp; their hash rides the member entry (`switchesHash`), and a node whose hash differs pulls the peer's switches and keeps the newer of each.

### A32.5 Routing a request

`NetRouting` answers three questions from the capability table cut to the caller's roles and to the member table's newest word: answer here (the pack is on, the newest version among reachable nodes, not peer-only here, a local source holds the entity); **view forwarding** (`ViewForwardFilter`, after the token filter: the whole request, with `X-Drishti-Net-Hop: 1` and an on-behalf-of token, the answer streamed back unchanged); **data forwarding** (`PeerSources`, the source router's `Fallback`: after the local sources, `PeerSource` fetches `GET /api/v1/net/fetch/{kind}/{id}` and this node renders and masks again). A `PeerSource` is not in the source registry, so nothing that lists sources (BI, caches, Health) sees a peer. A forwarded read answered `404` goes to the next renderer (at most three), renderers whose sources hold data first. Peers are chosen: alive → holds the date → freshest data version → lowest latency → a stable hash of the entity id. A digest of a member that left or died is kept (never routed to) until it is forgotten, so an answer can name the holder that is down; a down node counts only for the dates between its sources' `earliestDate` and `latestDate` (`Fallback.downHolders`).

- **History** goes to peers in one call for every date this node lacks (`GET /api/v1/net/history/{kind}/~`), merged by date, the higher generation winning an overlap; never cached when failed, partial or empty.
- **Scatter-gather** (`SearchScatter`): the targets are `CapabilityTable.scatterTargets` (shards, never a replica of a node already asked; the fewest nodes that cover every table, fastest first), asked in parallel through `GET /api/v1/net/search` within `scatter-budget`; rows merged by kind and id (this node's first), re-sorted, cut; `matched` and `scanned` deduplicated through each share's `matchedIds` (up to 20,000, removed from the client's answer); a down holder named in `failed`.
- **Grouped pages** (`GroupGather`, `GET /api/v1/net/group/{kind}/{id}`, `DerivedSourcePlugin.share`): exact (members united by id) when every node lists all its members up to `max-members`, otherwise combined from each node's totals; partial pages carry `partial` and `missing`.

- **Live relay** (`PeerSource.subscribe`, `GET /api/v1/net/subscribe/{kind}/{id}`): the topic hub is partitioned by the subscriber's roles while the cluster runs (an upstream is masked at the peer for the user it was opened for); a broken relay reopens with a fresh token.
- **BI reports** route by `NetRouting.report` to a node whose digest lists the report and holds its datasets' kinds; **BI runs** go to the model's owner by rendezvous (`RunCluster`), their actions to the run's node (`RunForwardFilter`), adoption after a lapsed lease by one conditional update in the shared database.
- **Selections** of a Sutra Next screen are tokens kept on the node that issued them (`tok_<node>~…`); `SelectionForwardFilter` forwards a request that reaches another node to the issuing one.
- **Masks across nodes**: a cache of what a peer sent masked is keyed by the caller's security signature (`Principal.securitySignature()`: sorted roles and embedding app), through `SourceRouter.partition()`.
- **The node that answered**: `NetServedBy` adds the *Node* line for administrators; `NetCaller.Served` records what a request asked of other nodes.

### A32.6 Identity and user state across nodes

`StateSync` (signed `POST /api/v1/net/state`, batches of 200 every quarter second, up to 10,000 waiting per peer) copies the namespaces `settings`, `layouts`, `pivots` and `workspaces` (`ReplicatedPreferences`), console sessions (the SHA-256 of the id) and, through `IdentitySync` (a `StateSync.Part`), `identity-users` (with the password hash), `identity-roles` (`RoleStore.adopt`) and `identity-tokens` (the secret's SHA-256). A change is queued for every member not gone (alive or suspect, drained included) and sent once it is alive; a member that joins gets a catch-up applied only where absent; a member that goes dead drops its queue and gets the catch-up when back. Applied changes are written under a thread-local flag so listeners do not echo them. The newest write wins (each node's wall clock, within the key's 60 s); one user name made apart on two nodes is settled by the older creation (`UserStore.putIfEmpty` seeds only into an empty store); deletions are tombstones (`<data dir>/net/identity-deleted.json`, 90 days). The last-enabled-administrator rule is checked where a change is made and again where it is applied, under a lock, one change at a time: the disabled administrator with the newest `updatedAt` (then the name) is enabled again, so every node picks the same; audited `last-admin-kept`, listed in `NetHealth.summary().warnings`. Alerts, monitors and scheduled reports are not copied: they act.

### A32.7 The shard map

`ShardMap` (`drishti-compute/.../shard/`): per business date, the rows counted per value of the shard key (VaR: `LEGAL_ENTITY`, then `DESK`), read as columns without the vectors; walked in key order, a shard closes when the next key would take it over `target-shard-mb` (2048 by default; a row is 4 × scenarios bytes, the float32 hot row). A key value (a desk) is never split; the ranges cover the whole key space, so every row has exactly one shard. The map is a pure function of the counts, so every member computes the same map; its digest names the cut, not the data, so an intraday batch that moves no cut keeps the map. At 1-day width (509 scenarios) 2.3M positions are 4.7 GB of rows, 3 shards at the default; at 1,518 values 14 GB, 7 shards (estimates).

### A32.8 Rendezvous placement

`ShardOwners.owners`:

```
weight(node, key) = first 60 bits of SHA-256( node + "|" + key )      key = the shard's range "LE-A/DESK-X..LE-B/DESK-Y"
ranked            = candidates sorted by weight descending, ties by node id
owners            = the pinned member first (if a candidate), then repeatedly the heaviest candidate not chosen whose zone is unuse
d,
                    else the heaviest candidate not chosen, until `replicas` owners
```

**The candidates** (`ShardMembership`, `ShardPeers.Peer.eligible`): this server and every member that is alive, speaks the protocol, is not drained, has sharding on, owns shards (not `coordinator-only`) and is not draining. The view moves only once it has been the same for `stable-seconds`; a draining member leaves at once; it is looked at again at most every 250 ms.

**Why rendezvous and not a ring.** A consistent-hashing ring needs hundreds of virtual nodes per member to balance and a shared ring state; rendezvous needs neither: each shard ranks the members independently (even in expectation, no tuning), gives the top-k replica set directly, moves only what a joiner wins or a leaver held, makes the zone rule "the next heaviest in an unused zone", and is a pure function with nothing to agree on. Its cost, O(members) hashes a shard, is microseconds. **Rebalancing:** with N members and R × S owner slots, a joiner wins each slot with probability 1/(N + 1); a leaver's slots go each to the next heaviest member (`ShardOwnersTest`: a fourth member takes 20 to 30% of first owners and replica slots over 2,000 shards). The same rendezvous choice places a BI model's runs (`RunCluster`).

### A32.9 Handover, readiness and the gather of partials

**Handover** (`ShardHandover.decide`, `VarShards` every `rebalance-seconds`): a new owner builds its shard from the lake (its key range only, the files skipped by statistics), writes `<hot>/<date>-shard-<i>/`, announces it ready in the digest service `var-shards`; the old owner keeps serving and releases (unmaps, deletes) only `grace-seconds` after every owner announced it ready. So a shard in use always has a ready holder. A date put together again is swapped in whole (`VarService.swap`) only when complete.

**Readiness in the digest** (`ShardPeers.State`): sharding on, owner, draining, per date the maps held and per shard the first 16 characters of the data digest it serves, and the dates open (a member of an older release has its held dates counted as open, `State.openOrHeld`). A change rebuilds the digest at once (`ShardPeers.announce`): readiness needs no protocol of its own. A USD shard book (and its currencies) of a date open nowhere for `release-after-seconds`, not among the newest `hot-dates` and not read here meanwhile is released (unmapped; its file left to the disk budget).

**The gather** (`ShardGather`): each shard's part of a scope goes to one member, in this order: owners ready at the date's data version (first owner first), any other member ready at it (an old owner mid-handover), owners not yet ready (they build). `POST /api/v1/risk/var/partial`, signed, one hop, on behalf of the caller; the answer is labelled with shard, map digest, data digest and row count and refused when any differs. Exactly one answer per shard, added in shard order; a shard nobody answers fails the scope (`503 DRS-5215`); a member at another data version answers `409 DRS-5216` and the next is asked. Never a partial VaR. The skeleton (ids, columns, links, aggregates) is fetched from owners (`GET /api/v1/risk/var/shards/skeleton`, `ShardSkeleton`) and copied to every member; node figures are summed bottom-up from the shards' book aggregates, equal to one server to the bit (`VarShardClusterTest`, `ShardAssemblyTest`). The machinery (`com.ash.drishti.compute.shard`, `com.ash.drishti.server.shard`) is measure-agnostic; VaR is its first `ShardedMeasure`.

### A32.10 Why no consensus protocol

Ownership is a pure function of shared inputs (the shard map from the same lake, the membership gossip converges on). Two members that briefly disagree cannot produce a wrong number: a partial is served only by a member that holds the shard at the data version asked, so disagreement costs a refused call. A Raft or ZooKeeper quorum would add a service and a leader for no gain in read correctness. It would help singleton writers under a partition (one official capture, one PV-stream reader); Drishti uses its database for that, by lease, as BI runs do. A second site is a separate cluster, never one cluster over a WAN.

### A32.11 Exposure sharding (design, not built)

Exposure runs on one server today. The design reuses `ShardMap`, `ShardOwners`, `ShardHandover`, `ShardMembership` and `ShardPeers` under a digest service of its own (`exp-shards`), behind `drishti.ccr.exposure.sharding.enabled`:

- **Key** `[COUNTERPARTY, NETTING_SET]`, no group prefix: a netting set is never split, a counterparty's sets sit together.
- **Sharded:** per set the V, E and N matrices (float32, `VMAT`, `EMAT`, `NMAT`) and the load itself, each owner streaming only its sets' `exposure-path` rows one set at a time (the heap holds one set's float64 matrix, not the date's). **Copied:** the set index (about 200 B a set), the run's grid, memberships, collateral, CSAs, limits, credit curves.
- **The owner** nets paths, applies collateral with the margin period of risk, computes E and N per path and step. **The partial** is the summed E matrix of the scope's sets (steps × paths float64: 800 KB at 50 × 2,000), plus per-counterparty EE rows for CVA. **The coordinator** adds partials in shard order, then EE, ENE, EEE, EPE, EEPE, PFE quantiles, peaks and CVA: quantiles only after adding. Contributions take a second round for the paths around the quantile.
- **Sizes:** 800 KB a set at P = 2,000, G = 50 (80 GB a date for 100,000 sets, 40 GB an owner of 4 with two replicas); 4 MB a set at P = 5,000, G = 100.
- **Before switching it on:** bit equality against one server for every scope kind on Delta and Iceberg, with an owner joining, one down and both down; official captures of 20 dates identical; a week side by side with the single server as the reference.

### A32.12 Bounded resources inside one server

- **One cache budget** (`MemoryBudget`, `drishti.memory.cache-share` 0.5 of `-Xmx`): weighted pools (`connector.idmaps` 20, `connector.columns` 20, `connector.documents` 6, `connector.parts` 4, `engine.answers` 20, `engine.layouts` 10, `small` 20), each a set of Caffeine caches weighed in bytes. At `squeeze-at` (0.85 of the old generation after a collection) the connectors' and small pools are halved for a minute. `CachesWeighedInBytesTest` fails the build on a count-only cache not listed in `count-only-caches.txt`.
- **One disk budget** (`DiskBudget`): folders register name, path, cap, size and how to free their least recently used items; a writer calls `reserve(folder, bytes, what)` before a build (VaR reserves three times its largest file, exposure twice), which trims, checks the guard and refuses with `503 DRS-5218` before anything is written; every folder is checked each `drishti.disk.check-interval`.
- **Mapped id maps** (`MappedIdFile`, `IdMapFolder` in `drishti-api`; `FileMaps` for unmapping): one mapping a file, reads inside a guard, `close` unmaps once no reader is inside; release by idleness, the folder's cap, an optional TTL, never a map being read or of a date open in an engine (`DatesInUse`).
- **Load pools** (`LoadPools`): VaR dates on `drishti-var-load-*`, shard books on `drishti-var-shard-load-*`, exposure on `drishti-exposure-load-*`, bounded `ForkJoinPool`s that compensate a nested join; never the common pool.
- **A fixed reduction order** (`ParallelSums.sum`): a set under 1,024 rows in row order, a larger one in blocks aligned to the absolute ordinal (about 64 blocks, a power of two), each from zero, partials added in block order: any scope's bits depend on the set only, never on the thread count.
- **The skeleton** of a VaR date (ids, columns, links, aggregates; about 0.72 KB a position measured) is on the heap and copied to every member; its id indexes are sorted `int` permutations with a binary search (`IdIndex`). **Designed, not built:** memory-mapping it from hot-file format 4 (`TIDX`, `PIDX`, mapped `TCOL`, `PCOL`, `AGGS`), which takes an owner from `-Xmx36g` to about `-Xmx20g` at 2.3M and makes 10M practical.
- **Busy protection** (`AdmissionGate`, `com.ash.drishti.server.admission`) gates new sign-ins, date loads, exports, BI runs and bulk loads on the heap after collection, loads in flight, open dates' heap, requests in flight and free disk; reads of open dates are never gated.

### A32.13 Super scale (design, not built)

The target: 500 GB of JSON a day (about 50 million documents) kept seven years (about 88 billion rows, 115 TB of Parquet) in one Delta lake in S3, read by many servers and consoles as one Drishti, each server's heap 8 GB. What breaks today at that volume: per-day id maps and column sets on the heap (3 GB and 11.5 GB a day), grouped pages computed by every server, one table per source holding every year (350,000 files in one log), one loader per source (about 3 hours for 500 GB), servers on one lake as replicas that each cache everything. The design:

- **Products computed when loading, not when asked:** link indexes, group totals, rollups and id lists written with the data by the slice that has the rows sorted in hand.
- **Find by ranges, not maps:** files of a day hold sorted, disjoint id ranges; the log's per-file min and max id (and Parquet Bloom filters on `id`) give a range map, nothing per entity on the heap.
- **Tables by year** (`root: …/{year}`), closed when the year ends.
- **A loader fleet:** a planner cuts the day into slices; loader tasks parse, promote, sort, write and compute products; safe concurrent commits (conditional PUT); a publisher and a day register, so a day appears whole or not at all.
- **Each server owns a share:** (table, day, group) slices placed on the Drishti*net ring; a request is forwarded to the slice's owner (one hop), searches and aggregates split over owners, so the cluster's memory adds up instead of repeating; anyone can still read anything from S3.
- **Warm tier:** memory-mapped column files of owned days; type-ahead from the day's id-list product.
- **Questions over all of history** go to an external engine (Athena, Trino, Spark) on the same tables.

Steps S1 (range maps, Bloom filters), S2 (tables by year), S3 (the loader fleet), S4 (products), S5 and S6 (slice ownership, split queries), S7 (the warm tier) and S8 (a benchmark at scale in AWS) are not scheduled.

---

*Copyright (c) 2026 Ashutosh Sinha ajsinha@gmail.com (mailto:ajsinha@gmail.com). All rights reserved. Proprietary and confidential. Unauthorised copying, use or distribution is prohibited.*

---

## Part III · Data architecture

How Drishti's data layer is built, for developers: **live updates** (section D1: streams, frames, coalescing, reconnects, live BI), **history queries** (section D2: an entity over time), **drishti-iceberg** (section D3: Drishti's own Apache Iceberg engine), **reactive connectors** (section D4: R1 to R4 built; R5 and R6 planned), **the content store** (section D5: user content every node reaches), **the one Drishti database** (section D6: design, not built; the write-back tables are built), **loading** (section D7: the data inbox), **the lake file** (section D8), **the in-memory lake** (section D9), **the parallel lake loader** (section D10) and **Arrow Flight** (section D11: data out to any client).

| Section | Status |
|---|---|
| 1. Live updates | built |
| 2. History queries | built and built |
| 3. drishti-iceberg | built (`native` is the default engine) |
| 4. Reactive connectors | R1 to R4 built and built; R5 and R6 planned |
| 5. The content store | built, built |
| 6. The one Drishti database | design, not built (the write-back tables are built) |
| 7. Loading: the data inbox and the load rules | built (internal notes; the guide is DATA_LAKE_GUIDE.md) |
| 8. The lake file | built |
| 9. The in-memory lake | built (what is not yet: 9.10) |
| 10. The parallel lake loader | built |
| 11. Arrow Flight | built |

The people who load data read DATA_LAKE_GUIDE.md and DATA_GUIDE.md; the connectors are ../guides/DATA_GUIDE.md section 8 and the per-store documents; VaR's intake is the parallel lake loader (section D10). Each section keeps its own numbering.

### Contents

1. Live updates
    - What you see in the browser
    - Try it yourself
    - Event reference
    - How a tick reaches the screen
    - Guarantees and limits
    - Monitors and alerts
    - Live BI: a report's live tables
    - Latency
    - Settings
    - Streaming sources
    - The demo source
    - Implementation note
2. History queries: an entity over time
    - 1. The ask
    - 2. What exists already
    - 3. Principles
    - 4. The command

    - 5. The history view
    - 6. The `history:` block in pack.yaml
    - 7. The server endpoint
    - 8. The JSON contract
    - 9. The connector SPI
    - 10. Fast paths, store by store
    - 11. Limits, cache and settings
    - 12. Drishti*net and super scale
    - 13. Security
    - 14. Performance targets
    - 15. Extras: DIFF, export, CLI, BI, plain words
    - 16. Errors
    - 17. The data flow
    - 18. Build steps H1..H14
    - 19. Tests
    - 20. Docs to update when it is built
    - 21. Where the code forced a change from the agreed design
    - 22. Open questions
3. drishti-iceberg: Drishti's own Apache Iceberg engine
    - 15. Two connectors: Drishti tables and external tables
    - 16. Vector columns and orphan files
4. Reactive connectors
    - 1. The ask, in one paragraph
    - 2. Decisions this design rests on
    - 3. Four kinds of connector
    - 4. What a reactive connector is
    - 5. The connector file
    - 6. The request and its lifecycle
    - 7. Triggers: computing on events
    - 8. Point-in-time stamping, ordering and idempotence
    - 9. Isolation: two users never see each other's answers
    - 10. Coalescing, incremental recompute and back-pressure
    - 11. Caching
    - 12. One screen, many connectors
    - 13. Failure
    - 14. Several servers (Drishti*net)
    - 15. Observability and audit
    - 16. The SPI and the APIs
    - 17. Configuration reference
    - 18. Build plan
    - 19. Tests
    - 20. Open questions for the product owner
    - 21. Built: R1 to R4
    - 22. How the VaR engine plugs in
5. The content store: user content every node reaches
    - 1. The problem
    - 2. The interface
    - 3. Backends
    - 4. Configuration
    - 5. Designs: the first user
    - 6. Several servers
    - 7. Tests
    - 8. Next steps: what is still kept on one node
    - 9. BI content and BI runs
    - 10. Collaboration, notes and the admin data
    - 11. UI-written configuration: shared overlays (built for load overrides)
6. The one Drishti database (design, not built)
    - 1. The decision
    - 2. What lives in it, and what does not
    - 3. SQLite or PostgreSQL
    - 4. No migrations: two complete schema files
    - 5. The schema version stamp
    - 6. Switching and upgrading: `db copy`
    - 7. Configuration: config/db/database.yaml (planned)
    - 8. Several servers
    - 9. Today and the change
    - 10. Build plan
    - 11. Tests

    - 12. Decisions (product owner, 2026-10-20)
7. Loading: the data inbox and the load rules (internal)
8. The lake file: one description of every lake
9. The in-memory lake: Delta tables that never touch a disk
10. The parallel lake loader (drishti-lakeload)
11. Arrow Flight: Drishti's data to any client
12. Computed and derived attributes
13. VaR sharding: how it is built
14. VaR sharding for operators


## D1. Live updates

Some entities change while you look at them: a trade's mark-to-market, a curve, an FX spot. When the source that holds such an entity can push changes, Drishti keeps the open view up to date. Only the cells that changed are repainted, and each one flashes briefly. This document explains how that works, how to watch it happen with `curl`, what the events look like, and which settings control it.

If you only want to *use* live views, read the first section and stop. Operators should read Try it yourself and Settings. The full path is described in How a tick reaches the screen.

### What you see in the browser

1. Open a live entity, for example type `TRD END-1000008 <GO>` (a commodity forward in the `trading` pack's samples).
2. The top bar shows **`Live, p99 N ms`**. N is the 99th percentile, over the last 30 seconds and across the whole server, of the time from a source tick to a finished frame. Single-digit milliseconds is normal.
3. The `MTM (USD)` figure in the header strip changes every few hundred milliseconds and flashes when it does. The *How this view was built* panel shows the source generation going up (`endur-comm, gen 29`, `gen 30`, ...).
4. Other states of the same indicator:

| Top bar | Meaning |
|---|---|
| `Live` | The stream is open; no frame has arrived yet. |
| `Live, p99 2 ms` | Frames are arriving. |
| `Reconnecting…` | The connection dropped, or could not be opened within 10 s; the browser keeps trying by itself. The page itself works meanwhile. |
| `Paused while hidden` | The tab has been hidden for 10 seconds and gave its subscriptions back. It takes them again and repaints when you return. |
| `Static` | This view does not tick: the source is not live, or you picked a past business date. |
| `Deleted` (red dot) | The source deleted the entity. A banner over the view says when; see Deleted entities. |

A view is live only when **both** are true: the source holding the entity can push changes, and you are looking at today's business date. Pick a past date in the top bar and every view becomes a static snapshot.

### Try it yourself

The server streams over Server-Sent Events (SSE): a plain HTTP response that never ends, made of text events. `curl -N` (no buffering) shows them as they arrive. The examples run against a local server with security off (`drishti.security.enabled: false`); with security on, add an `Authorization: Bearer …` header (see API_REFERENCE.md).

#### D1. Stream one view from the server

```
curl -N http://localhost:18480/api/v1/views/trade/END-1000008/stream
```

You should see one `view` event, then a `frame` event about every 400 ms (the demo source's tick rate). Abbreviated:

```
event:view
id:0
data:{"ref":{"kind":"trade","id":"END-1000008"},"mnemonic":"TRD","title":{"pill":"Commodity · Commodity forward","id":"END-100000
8","with":{"text":"Harbor Point Bank",…}},"strip":[{"label":"Notional","text":"USD 231,000,000","path":"$.currency"},…,{"label":"MT
M (USD)","text":"+2,148,146","tone":"pos","emphasis":true,"path":"$.mtm"}],"panels":[…],"provenance":{"layout":"Sutra cmd-forward v
1 + inference","fingerprint":"205d…ad0d","source":"endur-comm","generation":29,"fetchedAt":"2026-10-01T01:01:40.877Z","live":tru
e,"businessDate":null},"timings":{"fetch":0.09,"layout":0.13,"links":0.21,"bind":0.15,"total":0.57}}

event:frame
id:1
data:{"seq":1,"generation":30,"patches":[{"op":"strip","index":4,"cell":{"label":"MTM (USD)","text":"+2,101,143","tone":"pos","emph
asis":true,"path":"$.mtm"}},{"op":"panel","panel":{"id":"built","kind":"provenance",…}},{"op":"provenance","provenance":{…,"generat
ion":30,"live":true}}],"latencyMs":1.197765,"p99Ms":1.198}

event:frame
id:2
data:{"seq":2,"generation":31,"patches":[{"op":"strip","index":4,"cell":{"label":"MTM (USD)","text":"+2,095,229",…}},…],…}
```

Press Ctrl+C to stop. The server notices the closed connection, releases the stream's slot and, if you were the last viewer of END-1000008, closes the source subscription.

When nothing changes for 15 seconds you see a heartbeat line instead. It is an SSE comment, which clients ignore:

```
:hb
```

#### D2. Ask for a past business date

```
curl -N "http://localhost:18480/api/v1/views/trade/END-1000008/stream?asOf=2026-09-28"
```

You should see exactly one `view` event whose provenance says `"live":false,"businessDate":"2026-09-28"`, after which the server ends the response and `curl` returns. A past date never ticks and holds no stream slot. The header `X-Drishti-As-Of: 2026-09-28` does the same as the `asOf` parameter.

#### D3. Check the live counters

```
curl -s http://localhost:18480/api/v1/health/live
```

You should see something like this (while one stream from step 1 is open):

```
{"streams":1,"topics":1,"frames":11652,"p50Ms":0.886,"p99Ms":1.917}
```

| Field | Meaning |
|---|---|
| `streams` | Open live streams on this server (view streams and monitor streams together). |
| `topics` | Entities with an open source subscription. Several viewers of one entity share one topic. |
| `frames` | Frames built since the server started. |
| `p50Ms`, `p99Ms` | Tick-to-frame latency percentiles over the rolling window (`drishti.live.window`, 30 s). Both are `0.0` when no frame was built in the window. |

The same figures appear under `live` in `GET /api/v1/admin/health` (Admin → Health), which also reports `droppedFrames`. Frames for a slow client are merged rather than dropped, so it stays `0`.

#### D4. Watch what the browser receives

The web console does not let each panel, or each tab, open its own stream. Each **browser** opens **one** channel to the console, shared by all its tabs and workspace panes, which relays the server's streams and turns panel patches into ready HTML:

```
curl -N "http://localhost:17480/api/channel?s=view:trade/END-1000008&s=alerts"
```

(With console sign-in on, add your session cookie: `-b "drishti_session=…"`.) You should see:

```
event: channel
data: {"ch": "", "d": {"id": "NavddrgbWaTX_LlT", "subs": ["view:trade/END-1000008", "alerts"], "who": "5f0c9e2a7d1b4c3e8a6f2d10"}}

event: hello
data: {"ch": "alerts", "d": {"user": "ash"}}

event: view
data: {"ch": "view:trade/END-1000008", "d": {"generation": 18}}

event: frame
data: {"ch": "view:trade/END-1000008", "d": {"seq": 1, "generation": 19, "patches": [{"op": "strip", "index": 4, "cell": {"label":
"MTM (USD)", "text": "+2,110,140", …}}, {"op": "panel", "panel": {"id": "built", "kind": "provenance"}, "html": "<section class
=…"}, …]}}
```

Every event says which subscription it belongs to (`ch`). The first `channel` event carries the channel id and `who`, a fingerprint of the server, user and sign-in session the channel runs as (pages carry the same value in `<meta name="drishti-live">`). The relayed `view` event carries only the generation, because the page was already painted by the console.

### Event reference

#### Server: one view (`GET /api/v1/views/{kind}/{id}/stream`)

| Event | `id:` | Data |
|---|---|---|
| `view` | `0` | The full ViewModel, the same JSON as `GET /api/v1/views/{kind}/{id}`. Always the first event, on every connect and reconnect. |
| `frame` | the frame's `seq` | `{"seq", "generation", "patches": [...], "latencyMs", "p99Ms"}`. |
| comment `:hb` | none | Heartbeat every `drishti.live.heartbeat` (15 s) when no frame was sent. |

The ViewModel and the frame are defined field by field in Part IV · The ViewModel contract, with a JSON Schema (`$defs.Frame`) at `GET /api/v1/viewmodel/schema`. A frame's `patches` are a list of these:

| `op` | Fields | Apply it by |
|---|---|---|
| `strip` | `index`, `cell` | Replacing header strip cell number `index` (0-based) with `cell`. |
| `panel` | `panel` | Replacing the panel with the same `panel.id`. |
| `provenance` | `provenance` | Replacing the view's provenance (generation, source, fetched-at). |
| `deleted` | `at` (ISO-8601 instant) | Saying the entity was deleted at `at`; keeping what is shown; applying no later patches. |
| `restored` | none | Repainting the whole view: a deleted entity came back (the patches after it bring it up to date). |

#### Server: a monitor (`GET /api/v1/me/monitors/{name}/stream`)

A monitor is a saved list of 1 to 50 entities shown as rows. Its stream is **one** connection for all rows:

| Event | Data |
|---|---|
| `hello` | `{"rows": 3}`, the number of rows. |
| `row` | `{"kind", "id", "patches", "p99Ms"}`. Only `strip` patches are sent, because a row shows only the strip. |
| comment `:hb` | Every 15 s when quiet. |

Rows whose entity is missing, not live, or not visible to you simply never tick.

#### Server: alerts (`GET /api/v1/alerts/stream`)

```
curl -N http://localhost:18480/api/v1/alerts/stream
```

```
event:hello
data:{"user":"anonymous"}
```

With security off and no `X-Drishti-User` header the user is `anonymous`; the console sends `X-Drishti-User: ash` (or the signed-in user). When one of your rules raises an alert you receive it as an inbox row:

```
event:notice
id:12
data:{"seq":12,"at":"2026-09-30T14:02:11.120Z","type":"alert","kind":"alert","id":"/alerts/measure/mtm-limit","title":"mtm-limit —
holds (trade/END-1000008 · $.mtm > 2100000)", ...}
```

The alerts stream does not count against `drishti.live.max-streams`.

#### Console: the browser's channel (`GET /api/channel?s=...`)

| Subscription `s=` | Relays |
|---|---|
| `view:<kind>/<id>` | The view stream above. `frame` patches for panels other than line and area charts gain an `html` field with the rendered panel. |
| `alerts` | The alerts stream. |
| `monitor:<name>` | A monitor's stream. |
| `compute:<request id>` | A computed panel's request (the reactive connector (docs/connectors/types/reactive/README.md)): `hello`, `progress` (`{done, of, estimate}`), `state` (stale, with words), `done`, `failed`, and `updated` while the panel stays open and a trigger recomputes what it shows. Its owner's only. The page's `drishti_compute.js` subscribes for every panel with `data-compute-req` and fetches the panel again on `done`, `failed` or `updated`. |
| `bilive:<report id>` | A BI report's live tables (below): `hello`, `tick` and `status` events with no data. |

Up to `live.max_subscriptions` (32) subscriptions per channel, that is per browser: a view open in several tabs counts once. One more is answered at once with `gone`, `DRS-5003` "this browser already follows 32 live subscriptions (live.max_subscriptions): close some tabs or panes", and the view shows `Static`. Events: `channel` (first; carries the id, the subscriptions it opened with and `who`), the relayed events wrapped as `{"ch": "<subscription>", "d": <data>}`, `gone` (`{"code", "detail"}` when one subscription fails; the others carry on), `end` (every subscription has finished; the browser then closes instead of reconnecting) and a `: hb` comment about every 14 s when nothing else is sent.

`GET /api/channel/who` answers `{"who": "…"}`: the fingerprint of the session the browser's cookies carry now. The browser's hub asks it when a page of another session joins (see One connection per browser).

`gone` does not pass the server's own error code through: a view the server refuses (an entity that does not exist, `DRS-1001` on the server; a kind you may not open, `DRS-5002`) arrives as `DRS-5003 stream refused`, and a console that cannot reach the server as `DRS-5003 backend unreachable: …`. For example:

```
curl -sN --max-time 3 "http://localhost:17480/api/channel?s=view:trade/END-1000008&s=view:trade/NOPE-1"
```

You should see, among the frames of `END-1000008`:

```
event: gone
data: {"ch": "view:trade/NOPE-1", "d": {"code": "DRS-5003", "detail": "stream refused"}}
```

To see the real reason, ask the server directly: `curl -s http://localhost:18480/api/v1/views/trade/NOPE-1/stream` answers `404` with `"code":"DRS-1001"` and `no source holds trade/NOPE-1`.

`POST /api/channel/{id}` with `{"add": ["view:trade/MX-20000001"], "remove": ["view:trade/END-1000008"]}` changes the subscriptions of an open channel and answers `{"ok": true}`. A workspace uses it as its panes load one by one. It answers `404` with `DRS-5001` "no such channel: open a new one" when the channel has closed or belongs to another user. Removing a subscription closes its upstream stream on the server; adding one that is already there does nothing, and one beyond the limit is answered with `gone` as above.

The console also keeps `GET /api/stream/{kind}/{id}` (one view, same relaying) for API clients. Pages do not use it.

### How a tick reaches the screen

```
SourcePlugin.subscribe ─► TopicHub topic (one per entity, one source subscription, latest-wins slot)
                              │ leading-edge frame throttle (drishti.live.frame, 50 ms)
                              ▼
                          ViewStream (one per client view; also listens to its charts' source entities)
                              │ rebuild with the cached layout → PatchDiffer (old vs new ViewModel)
                              ▼
                          FrameMailbox (latest-wins; merges frames for a slow client)
                              │ one virtual-thread writer per client
                              ▼
            SSE  /api/v1/views/{kind}/{id}/stream   event: view (full ViewModel), then event: frame (patches)
                              │
            console /api/channel: ONE stream per BROWSER carrying the views of all its tabs (every workspace
                              │ pane), the alerts bell and monitors; panels re-rendered to HTML with the same
                              │ Jinja macros; charts sent as data
                              ▼
            drishti_live-hub.js (a SharedWorker, or the leader tab): routes each event to the tabs that subscribed
                              ▼
            drishti_channel.js → drishti_live.js: strip cells updated in place, panels swapped, charts moved; changes flash
```

Step by step, for one tick of END-1000008:

1. The demo source moves the trade's price one random-walk step and raises its generation (29 → 30).
2. The entity's **topic** in `TopicHub` receives the new document. If no frame went out in the last 50 ms it is delivered at once; otherwise it waits in a one-slot "latest wins" holder until the frame is due.
3. Each **ViewStream** watching END-1000008 rebuilds the view from the new document and diffs it against the previous ViewModel. The rebuild re-evaluates the Sutra's `match` (cheap) and takes the effective layout from the layout cache, so inference does not run again unless the document's shape changed or a Sutra was reloaded. Here only the MTM cell, the provenance panel and the provenance changed, so the frame has three patches. If a rebuild fails (the source is down, the entity is gone), the stream retries after 0.5 s, doubling to 30 s, and any new tick retries at once.
4. The frame goes into that client's **FrameMailbox**. The client's writer (a virtual thread) takes it and writes `event:frame`.
5. The console's channel relays it to the browser, rendering the `built` panel to HTML.
6. In the browser, the hub (`drishti_live-hub.js`) passes it to every tab that subscribed to `ch`, and each tab's `drishti_channel.js` hands it to `drishti_live.js`, which swaps the strip cell and the panel and flashes the change.

#### One connection per browser

Browsers open at most six connections to one site over HTTP/1.1, shared by all the site's tabs. A stream per view and one for the alerts bell used them up with three tabs open; a channel per tab used them up with six tabs on live views (several screens on a desk), and after that **no** page of the console loaded at all, with no message (QA finding UX-01). So the number of live connections no longer grows with tabs or panes: each browser holds **one** channel (`/api/channel?s=view:trade/MX-20000001&s=alerts`), shared by all its tabs, and a workspace's panes share their page's subscriptions. Subscriptions that arrive later are added to the open channel (`POST /api/channel/{id}`) instead of reconnecting. Five of the six connections are always left for pages, searches and suggestions, however many tabs are open.

The channel is held by a **hub**, `drishti-console/web/static/js/drishti_live-hub.js`:

1. In a **SharedWorker** that can hold an `EventSource` (Chrome, Edge and Firefox on the desktop): one worker per console origin and browser profile, started by the first tab and ended by the browser when the last tab closes.
2. Where there is no SharedWorker, or it cannot hold an `EventSource` (Chrome on Android, for example): the tabs elect a **leader** with the Web Lock `drishti-live|<version>`. The leader holds the channel and relays to the other tabs over a `BroadcastChannel` of the same name. When the leader closes, the browser hands the lock to another tab, which opens a new channel; the other tabs register with it and their views repaint.
3. In a browser with neither (very old ones): a hub inside each tab, so one channel per tab.

A view open in several tabs is subscribed **once**; a tab that subscribes later is first given what a stream of its own would have opened with (the view's generation, the alerts `hello`, a deleted entity's `deleted` frame, a `gone`), then the frames as they come. A tab hidden for 10 s gives its subscriptions back (and the hub closes the channel when no tab holds any); shown again, it takes them back and its views repaint from fresh data.

**Sessions.** The hub serves only the tabs of one browser (one origin, one browser profile), which share the cookies the channel was opened with. Every frame is still built by the console for the signed-in user of that channel, with that user's permissions and field masks. Pages carry the fingerprint of their session in `<meta name="drishti-live" content="…">` (server, user and session, hashed; never the session id), and the channel's first event says which session it runs as (`who`). The hub sends a channel's events only to tabs with the same fingerprint. When a page of another session joins (someone signed in again, or picked another server, in another tab), the hub asks `GET /api/channel/who` which session the browser has now: if it is the new page's, the channel is reopened as it; pages of any other session are told they are stale and get nothing more (their Live pill says `Reconnecting…` until reloaded).

**A page never waits for live updates.** Pages load without a live connection; the hub is started after the page and connects in the background. When the channel cannot be opened within 10 s, or a tab cannot reach the hub within 5 s, each view says `Reconnecting…` rather than waiting silently, and the hub keeps trying (after a refusal, again after 2 s, doubling up to 30 s).

Upstream, the console reads the server's streams through its pooled HTTP client and closes each one within seconds of the browser going away, busy or quiet: it closes the HTTP response itself from a task outside the request's cancel scope, because Starlette's scope cancels every clean-up await of a finished request. A test guards that no page opens its own `EventSource` (only `drishti_live-hub.js` does), and `drishti-console/tests/test_live_tabs_browser.py` opens eight live tabs and a four-pane workspace in Chromium and checks that the next page loads at once and every tab and pane keeps ticking.

#### Hub protocol

Tabs talk to the hub with small messages (over the worker's port, or the `BroadcastChannel`):

| From a tab | Meaning |
|---|---|
| `{t: "hello", tab, who}` | register (again), with the page's session fingerprint; followed by a `sub` per key |
| `{t: "sub", tab, key}` / `{t: "unsub", tab, key}` | take or give back one subscription |
| `{t: "ping", tab}` | every 10 s; a tab silent for 45 s (closed without a word) is dropped |
| `{t: "bye", tab}` | the page is going away |

| From the hub | Meaning |
|---|---|
| `{t: "hi"}` | registered |
| `{t: "ev", key, type, d}` | one event of one subscription (`view`, `frame`, `gone`, `alert`, `row`, `hello`) |
| `{t: "error"}` | the channel dropped or cannot be opened; it keeps trying |
| `{t: "stale"}` | the page belongs to another session than the browser's current one |
| `{t: "who"}` | the hub does not know this tab (a new leader, or it was dropped): say hello again |

#### In the browser: `drishti_channel.js`

`drishti-console/web/static/js/drishti_channel.js` gives every script on the page one object, `window.Drishti.Channel`, with a single call, `subscribe(key, handlers)`, which returns a function that unsubscribes:

```
var off = window.Drishti.Channel.subscribe('view:trade/END-1000008', {
  view:   function (d) { /* first event: d.generation */ },
  frame:  function (f) { /* f.patches, f.p99Ms */ },
  gone:   function (d) { /* d.code, d.detail: this subscription ended */ },
  error:  function () { /* the connection dropped, or cannot be had; it keeps trying by itself */ },
  paused: function () { /* the tab was hidden for 10 s and gave its subscriptions back */ }
});
```

Handlers for `alert`, `row` and `hello` work the same way. `drishti_live.js` (entity views), `drishti_alerts.js` (the bell) and `drishti_monitor.js` (monitor pages) are its only users. `Drishti.Channel.transport()` says how the tab reaches the hub: `shared-worker`, `leader`, `follower`, `tab` (a hub of its own) or `none`. How it behaves:

| Situation | What happens |
|---|---|
| the first subscriptions of the browser | the hub waits 60 ms so that subscriptions made together open **one** `EventSource` on `/api/channel?s=…&s=…` (keys sorted) |
| a subscription added later (another tab, a workspace pane loading) | `POST /api/channel/{id}` with `add`, on the open connection; no reconnect |
| a key another tab already holds | nothing on the server; the tab gets the key's opening events from the hub at once |
| the last tab holding a key gives it back | `POST … {"remove": [key]}` after 60 ms; when no keys are left, the connection is closed |
| two handlers for one key (two panes of one entity) | one subscription; both handlers receive every event |
| a `POST` answered `404` (the console restarted, or another process holds the channel) | closes and opens a new channel with every current key |
| `end` | closes, so the browser does not reconnect to a channel with nothing left; a new subscription opens a new one |
| a dropped connection | every tab's handlers get `error`; the browser reconnects; the new `channel` event carries a new id, and the hub re-sends anything subscribed meanwhile; the new stream's `view` makes each view reload |
| a refused connection (the browser gave up) | `error`, then the hub opens it again after 2 s, doubling up to 30 s |
| no `channel` event within 10 s | `error` (the Live pill says `Reconnecting…`); still trying |

| Situation | What happens |
|---|---|
| tab hidden for 10 s | the tab gives its keys back and calls `paused`; when shown again it takes them back, and each view reloads from fresh data |
| inside a workspace pane (an iframe of the same site) | uses the parent window's `Drishti.Channel`, so a workspace of four panes is one tab to the hub |
| a page of another session | `GET /api/channel/who`; the channel follows the browser's current session, and other pages get `stale` |

#### In the console: lifetime of a channel

Each `GET /api/channel` gets a random id and a task per subscription that reads the server's stream through the console's pooled HTTP client. The channel ends, and every upstream stream is closed within about a second, when the browser disconnects, when nothing has read the channel for 5 s (a browser that vanished without closing it), or after `end`. A channel opened with no subscriptions stays open, waiting for `POST … add`.

Open channels live in the memory of the console process that served them, and the channel's id names that process (its host and relay port). A `POST /api/channel/{id}` that reaches another worker, or another console behind the load balancer, is relayed to the holder (`core/channel_relay.py`, signed with the relay key; across hosts with `live.relay.across_hosts`, CLUSTER_GUIDE.md section 9.4), so sessions need not be sticky. A channel whose holder has stopped gets `404` and the browser's hub reopens its channel.

### Guarantees and limits

- **One source subscription per entity**, however many people watch it. It closes when the last viewer leaves.
- **Frames.**
- A tick after a quiet period goes out immediately.
- Ticks within 50 ms of the last frame are coalesced: the latest wins, so a fast source costs one delivery per frame.
- Rebuilds for one view never overlap; a change that arrives during a rebuild causes exactly one more.
- **Minimal patches.** Only changed strip cells, changed panels, and the provenance when the generation moves. A Sutra edit changes the layout label, and the affected panels are re-sent.
- **Slow clients** never slow down others. Their pending frames merge (newest value per cell or panel) in a one-slot mailbox, so memory is bounded.
- **Reconnects.** The browser's EventSource reconnects by itself. The server opens every stream with a fresh `view` event, so a client that missed frames repaints from it; nothing is replayed. In the console, a `view` event that arrives after the first one (a reconnect) makes the page reload itself, so it shows the current state rather than patching a stale one.
- **Sutra edits.** A Sutra saved while views are open (hot reload) clears the layout cache; open live views pick up the new layout on their next tick, as a `panel` patch for every panel that changed.
- **Heartbeats.** An SSE comment every 15 s keeps proxies from closing idle streams.
- **Capacity.** `drishti.live.max-streams` (20,000) caps the streams per server, view and monitor streams together. A slot is taken atomically before any work, so concurrent requests never overshoot it, and released exactly once however the stream ends. A request over the cap is refused with `400`:

```
json {"type":"about:blank","title":"bad request","status":400,"detail":"DRS-5001 too many live streams on this server","code":"DRS-5001"}
```

- **Entitlements.** Opening a stream checks that your roles may open the entity's kind, exactly like a plain view request (`403`, `DRS-5002`, otherwise). A missing entity is `404`, `DRS-1001`, before any stream starts.

#### Deleted entities

A source that learns an entity was deleted (a Kafka tombstone; an ActiveMQ or RabbitMQ delete message) tells the views that show it, through the same subscription that carries its updates:

1. The connector drops the entity from its state, its caches and type-ahead (a cheap removal from the `HitIndex`), and hands every subscriber `EntityDocument.deleted(ref, provenance)`: a document whose `deleted()` is true, whose data is missing, and whose provenance's `fetchedAt` is when the source learnt of the delete (a tombstone's timestamp). Updates and deletes travel through one listener, so a delete followed by a re-creation stays in order in the topic's latest-wins slot.
2. Each `ViewStream` of the entity sends one frame with a single `deleted` patch, `{"op": "deleted", "at": "2026-10-01T09:30:05Z"}`, and rebuilds nothing while the entity stays deleted (ticks of its charts' sources included). The server also forgets the entity in every user's "recent" list, so the command line stops offering it. Alerts on the entity do not evaluate a deletion.
3. The console relays the patch unchanged. `drishti_live.js` keeps the view's last state, greys its panels and strip, puts a banner over it, "*MX-29000001 was deleted at 1 Oct 2026, 09:30:05 by its source. What you see is its last state; it no longer updates.*", and sets the top bar to `Deleted`. It is not an error page: links still work, and the last values can still be read and copied.
4. If the entity comes back (a new message for it), the next frame starts with a `restored` patch and the page repaints from the server. A page reloaded while the entity is deleted gets the usual "not held" answer (`DRS-1001`), since no source holds it.

A source plugin that deletes (see §D5.2 of the developer guide (docs/guides/DEVELOPER_GUIDE.md#52-add-a-source-plugin)) pushes `EntityDocument.deleted(ref, provenance)` to its listeners; one that never deletes needs no change. A listener that does not care about deletes skips documents whose `deleted()` is true. Eviction from a full state store is not a deletion and is not pushed.

### Monitors and alerts

- A **monitor** stream holds one `ViewStream` per row and multiplexes their strip patches as `row` events over one connection, with a latest-wins mailbox per row.
- The **alert engine** subscribes to the topics of every entity that some enabled rule watches, whether or not a browser is open. It evaluates each rule's Rachana-EL condition on every new document of that entity, fires on the false → true edge, and pushes `alert` events to that user's open streams. A rule's subscription keeps the source subscription open too, so an entity watched by a rule ticks even with no viewer.

#### Worked example: a monitor

A monitor is a named list of entities, saved per user. The examples use `curl` against the server with security off; `X-Drishti-User` says whose monitor it is (with security on, your token does).

1. Save a monitor of three live entities:

```
bash curl -s -X PUT http://localhost:18480/api/v1/me/monitors/rates-desk \ -H 'Content-Type: application/json' -H 'X-Drishti-User: ash' \ -d '{"entities": [{"kind": "trade", "id": "MX-20000001"}, {"kind": "trade", "id": "END-1000008"}, {"kind": "netting-set", "id": "NS-MERIDIAN-RE-NY"}]}'
```

You should see the saved list echoed back: `{"entities":[{"kind":"trade","id":"MX-20000001"},…]}`. Between 1 and 50 entities are allowed (`DRS-5001 a monitor has 1 to 50 entities` otherwise), and each must be a kind you may open. 2. Read its rows once: `curl -s -H 'X-Drishti-User: ash' http://localhost:18480/api/v1/me/monitors/rates-desk`. You should see one object per entity with `ref`, `mnemonic`, `title`, `strip` and `live`; an entity that cannot be shown has `error` instead (`DRS-1001 no source holds …`). 3. Stream it:

```
bash curl -N -H 'X-Drishti-User: ash' http://localhost:18480/api/v1/me/monitors/rates-desk/stream
```

You should see `event:hello` with `{"rows":3}`, then `event:row` events such as `{"kind":"trade","id":"END-1000008","patches":[{"op":"strip","index":4,"cell":{"label":"MTM (USD)","text":"+2,104,880",…}}],"p99Ms":1.2}`. Only strip patches are sent. The stream counts as one live stream against `drishti.live.max-streams`, however many rows it has. 4. In the console, the monitor page subscribes with `monitor:rates-desk` on the tab's channel.

#### Worked example: an alert that fires

1. Look at the value you want to watch. `END-1000008`'s MTM moves around 2.1 million:

```
bash curl -s http://localhost:18480/api/v1/views/trade/END-1000008 | python3 -c " import json, sys; print([c['text'] for c in json.load(sys.stdin)['strip'] if 'MTM' in c['label']])"
```

You should see something like `['+2,102,537']`. 2. Keep a stream open in a second terminal, so you see the alert arrive:

```
bash curl -N -H 'X-Drishti-User: ash' http://localhost:18480/api/v1/alerts/stream
```

You should see `event:hello` with `{"user":"ash"}`. 3. Save a rule (`PUT /api/v1/alerts/rules/{name}`): a field source whose field is a Rachana-EL condition over the entity's document, and the condition `rule`, looked at live (the demo source pushes its ticks):

```
bash curl -s -X PUT http://localhost:18480/api/v1/alerts/rules/mtm-above-2-1m \ -H 'Content-Type: application/json' -H 'X-Drishti-User: ash' \ -d '{"title": "END-1000008 MTM above 2.1m", "severity": "warn", "source": {"type": "field", "kind": "trade", "id": "END-1000008", "field": "$.mtm > 2100000"}, "condition": {"type": "rule"}, "trigger": {"mode": "live"}}'
```

You should see the rule echoed back with `"describe": "holds: $.mtm > 2100000"`. Saving checks the condition (`DRS-2101` for a typo, `DRS-9062` for anything else wrong, such as a severity other than `info`, `warn` or `critical`), then looks **at once** at the current document. 4. If MTM is already above 2.1 m, an alert is raised immediately; otherwise on the first tick that crosses the line (after the rule's quiet time, `trigger.debounce`). In the second terminal you should see an `event:notice` of type `alert`. It is not raised again while MTM stays above 2.1 m; when MTM falls below and rises above again, it is raised again. 5. The alerts raised for you are listed by `curl -s -H 'X-Drishti-User: ash' 'http://localhost:18480/api/v1/alerts'` (stage `raised`, then `acknowledged` or `resolved`: `POST /api/v1/alerts/acknowledge {"id": ...}`); the bell in the console shows the same. Delete the rule with `curl -s -X DELETE -H 'X-Drishti-User: ash' http://localhost:18480/api/v1/alerts/rules/mtm-above-2-1m`.

Packs can suggest rules for a kind (`alerts:` in `pack.yaml`), which the console offers when you create one: `GET /api/v1/alerts/suggestions/netting-set` returns, with the retired example pack `finance` enabled, `{"kind": "netting-set", "name": "PFE near limit", "when": "$.utilisation > 0.8", "severity": "warn", …}`. With the banking packs enabled, no pack suggests rules for `trade`, so `GET /api/v1/alerts/suggestions/trade` returns `[]`.

Because a rule raises on the false → true edge, you get one alert when MTM crosses 2.1 m, not one per tick while it stays above. Users normally create rules from the console's Alerts page; see USER_GUIDE.md.

### Live BI: a report's live tables

Business intelligence uses this machinery too (Rūpaka phase 5, HOW_IT_FITS.md §A28 section 12; the pages are BI_GUIDE.md section 18). A dataset table with `mode: live` (BI_GUIDE.md) is **fed by the same topics**: for each entity of its kind the engine subscribes through the source layer (the connector's `subscribe`, the same call a live view makes), applies each change to an in-memory table by key (batched every 100 ms, so ticks coalesce as they do for a view), and keeps a counter that grows with every change. It is **not** a

view stream: it carries no panel patches and costs the entities' topics one more listener (`TopicHub`), not one more source subscription; a dataset that names a connector (`source: { connector: ... }`) subscribes through that connector directly instead, for a stream that keeps no state but ticks what a store serves.

What reaches a browser is small. A report over such a table has one stream, `GET /api/v1/bi/reports/{id}/live` (API_REFERENCE.md), relayed by the console's one channel as the subscription `bilive:<report id>`:

| Event | Meaning |
|---|---|
| `hello` | the state of the report's live tables when the stream opens (the generation, the last change, `live` or `stale` or `failed`, counters per table) |
| `tick` | the tables changed: the **latest** state, at most one per `drishti.bi.live.frame` (250 ms); a slow browser skips ticks, it never queues them (the same latest-wins rule as a view's frame mailbox) |
| `status` | how the tables are doing when nothing changed, every `drishti.bi.live.status` (5 s): the heartbeat of this stream |

A tick carries **no data and no mask**. The page asks its visuals again, as the person looking, through the BI query path: **row rules and masks are applied to every answer, never to the stream**, so a stream shared by a hundred readers is the same bytes and each reader's picture is their own. The hub (`drishti_live-hub.js`) forwards the two event names `tick` and `status`; a tab hidden for ten seconds gives the subscription back like any other and the report redraws everything when it takes it again.

| Limit | Setting | |
|---|---|---|
| streams open at once | `drishti.bi.live.max-streams` (100), `max-streams-per-user` (10) | `429 DRS-9051`; these are BI streams and do not count against `drishti.live.max-streams` |
| one stream per distinct report | | a board of ten tiles over two reports holds two |
| the browser's subscriptions | `live.max_subscriptions` (32) | a `bilive:` subscription counts like a view |

Pause, the "last update" time and the stale and disconnected states of the indicator are the page's (`drishti_bi-live.js`): *stale* is "nothing heard for `bi.live.stale_seconds`" (the 5-second status is what keeps a quiet healthy stream from looking stale) or the table's own `stale` state (a feed that pushes and then goes silent for `drishti.bi.live.stale-after`).

#### Results computed by a streaming producer

A live table keeps the current rows of a kind and recomputes measures over them; it does not maintain heavy aggregates over a large stream incrementally (only the plain rollups a dataset declares, HOW_IT_FITS.md §A28 section 20.11). That belongs in a **producer**: Pravaha, Flink, ksqlDB or a custom BI server computes keyed result rows (a total per desk, per desk and currency) and owns its arithmetic; Drishti **serves** them: a screen per result, live delivery to many browsers, field masks, alerts, sharing and embedding, as for any kind.

This needs no Drishti code. The producer writes each result row to a **compacted Kafka topic** as an envelope keyed `<kind>/<id>` (KAFKA_CONNECTOR.md (plugins/drishti-plugin-kafka/README.md)), and a pack declares the result kind, its command and a Kafka connector:

```
kinds:
- desk-exposure                          # a result kind, declared like any other
mnemonics:
  DEXP:
    kind: desk-exposure
    label: Desk exposure (live)
connectors:
  risk-results:
    plugin: kafka
    kinds:
    - desk-exposure
    settings:
      bootstrap-servers: ${DRISHTI_KAFKA_BOOTSTRAP:localhost:9092}
      topics: ${DRISHTI_RISK_RESULTS_TOPIC:risk.results}
      stale-after: 2m                    # amber in Health if the producer goes quiet
      disk-cache.enabled: true
```

`DEXP RATES <GO>` then opens a live view of the desk's exposure that ticks as the producer upserts, and a dataset table with `mode: live` over `desk-exposure` puts the rows in a report. The rows follow a few rules:

- **Stable ids from the group-by key**: `RATES` for a desk, `RATES|USD` for desk and currency (`|` between key parts, distinct from other kinds' ids); a grand total is a row like any other with the id `ALL`.
- **State, not deltas**: each message is the whole current row (an upsert); a row that leaves the result is deleted with a tombstone, not sent as zeros. A producer that thinks in weighted deltas (+1 / −1) turns them into state before it writes.
- **Per key, in order**: one row's messages go to one partition; a new Drishti server reads the compacted topic from the start and has the current state, so a producer never replays for Drishti.

- **Small rows**: under 64 KB, a list inside a row under 500 items; detail belongs in the rows of the source kind.

Not built (proposed 2026-10-06): a `bi` source plugin that takes an HTTP stream and turns deltas into state, metadata read from a `_bi` field (generation, as-of, input watermark: today it shows as an ordinary field), drill-down from a total to its rows by predicate, and protection against aggregates that leak (a result field computed from a masked input declared `masked-from: [field]`, and a minimum group size below which a row's figures show as `•••` for readers without `raw`).

### Latency

The top bar shows `Live, p99 N ms`: the rolling (30 s) p99 of source-tick → frame-built across the server, recorded lock-free in an HdrHistogram. `GET /api/v1/health/live` reports streams, topics, frames, p50 and p99. Prometheus exposes the same as `drishti_live_latency_p99_milliseconds`, `drishti_live_streams`, `drishti_live_topics` and `drishti_live_frames_total` at `/actuator/prometheus`.

```
curl -s http://localhost:18480/actuator/prometheus | grep ^drishti_live
```

```
drishti_live_frames_total 11680.0
drishti_live_latency_p99_milliseconds 1.917
drishti_live_streams 0.0
drishti_live_topics 0.0
```

If p99 stays above 40 ms, follow runbooks/live-latency-high.md (docs/guides/OPERATIONS_GUIDE.md#203-live-latency-high-or-live-views-stop-ticking). Measured numbers are in CLUSTER_GUIDE.md.

### Settings

All under `drishti.live` in the server's configuration (CONFIGURATION.md):

Every setting of this part, with its type, default, environment variable, meaning and whether a change needs a restart, is in the one configuration document: CONFIGURATION.md, `drishti.live` (also at Help → Configuration in the console).

Example: coalesce more aggressively and protect a smaller server.

```
# application.local.yaml next to the jar
drishti:
  live:
    frame: 100ms
    max-streams: 5000
```

Or for one run: `java -jar drishti-server-2.9.11.g-exec.jar --drishti.live.frame=100ms`.

The console has one live setting, in `drishti-console/config/application.yaml`:

| Key | Default | Effect |
|---|---|---|
| `live.max_subscriptions` (`DRISHTI_LIVE_MAX_SUBSCRIPTIONS`) | `32` | Subscriptions one browser's channel carries at most: views across all its tabs and workspace panes, the alerts bell, monitors (a view open in several tabs counts once). Each holds one stream to the server from the console's pool (`backend.pool_size`, 64), so keep it well below that. One more is answered with `gone`, `DRS-5003`, and the view shows `Static`. |

Behind a reverse proxy, turn off response buffering for SSE (`proxy_buffering off;` in nginx; the console already sends `X-Accel-Buffering: no`) and set the read timeout above the heartbeat interval. See OPERATIONS_GUIDE.md.

### Streaming sources

A plugin that declares `live` pushes each new generation of an entity through `subscribe`. The `kafka` plugin does this from a topic: it rebuilds the latest document per entity from the start of the topic, then forwards every new message to the views that are open on that entity, through the same TopicHub, frames and patches as any live source. For Live, the router asks live sources first, and a real stream before the demo samples; a picked business date never streams.

`GET /api/v1/sources` lists every source with `"live": true` or `false`. In the sample configuration only `demo` is live; the `trading` pack's Kafka connector `trading-stream` becomes live when you set `DRISHTI_STREAM_TRADING=true` and point `DRISHTI_KAFKA_BOOTSTRAP` at your brokers.

#### Worked example: making trades live from Kafka

The trading pack declares the connector in `config/packs/banking/trading/pack.yaml`:

```
connectors:
  trading-stream:
    plugin: kafka
    enabled: ${DRISHTI_STREAM_TRADING:false}
    kinds: [trade]
    settings:
      bootstrap-servers: ${DRISHTI_KAFKA_BOOTSTRAP:localhost:9092}
      topics: ${DRISHTI_TRADING_TOPIC:drishti.trading.trades}
      kind: trade                 # "mapped" messages: the whole value is the trade document …
      id-field: tradeId           # … and its tradeId is the entity id
      disk-cache.enabled: ${DRISHTI_STREAM_DISK_CACHE:true}
```

1. Create the topic (compacted, so the latest message per trade is kept):

`bash kafka-topics.sh --bootstrap-server localhost:9092 --create --topic drishti.trading.trades \ --partitions 3 --config cleanup.policy=compact` 2. Start the server with the connector on:

`bash DRISHTI_STREAM_TRADING=true DRISHTI_KAFKA_BOOTSTRAP=localhost:9092 java -jar drishti-server/target/drishti-server-*-exec.jar`

`curl -s http://localhost:18480/api/v1/sources` should now list `trading-stream` with `"live":true` and `"health":"UP"` (`"UP (catching up)"` while it reads the topic from the beginning). 3. Publish a trade. In mapped mode the message **value** is the document; the key should be the trade id (it is used for deletes):

`bash echo 'MX-29000001|{"tradeId":"MX-29000001","productType":"IRS_FIXFLOAT","productName":"Interest rate swap (fixed/float)","assetClass":"Rates","status":"Live","currency":"USD","notional":50000000,"mtm":12500,"counterparty":{"id":"CP-MERIDIAN-RE","name":"Meridian Reinsurance Ltd"}}' \ | kafka-console-producer.sh --bootstrap-server localhost:9092 --topic drishti.trading.trades \ --property parse.key=true --property key.separator='|'` 4. Open `TRD MX-29000001 <GO>` (or `curl -N http://localhost:18480/api/v1/views/trade/MX-29000001/stream`). The view uses the `irs-fixfloat` Sutra (its `where` matches), `How this view was built` says `trading-stream, gen <offset>` (the generation of a Kafka document is its offset), and the top bar says `Live`. This small document lacks the legs, schedule and risk the Sutra reads, so those panels say *No data available*; a real feed carries them. All live sources that serve `trade` are asked, live ones first, so a trade held both in Kafka and in the Delta Lake store is read (and ticks) from Kafka. 5. Publish the same trade again with `"mtm":13750`. Within a frame (50 ms) the stream sends `event:frame` with a `strip` patch for `MTM (USD)` (`+13,750`) and the provenance's new generation, and the cell flashes in the browser. 6. Publish a *tombstone*: a **null** value, not an empty one (an empty value is not a delete). With `kafka-console-producer` use `--property null.marker=NULL` and send `MX-29000001|NULL`. The trade is deleted from the connector, and a new view of it is `DRS-1001`. The delete is pushed: a view already open gets a `deleted` patch and says "*MX-29000001 was deleted at …*" over its last state, greyed (Deleted entities), and the id leaves type-ahead at once.

Other shapes: without `kind`/`id-field` the connector expects **envelopes**, `{"kind": "trade", "id": "MX-20000001", "doc": {…}}`, and a tombstone's key is `kind/id`. `mode: ticks` keeps nothing in memory (although it still replays the topic from the beginning at start, `UP (catching up)`) and only drives the ticks of open views while a store (Delta Lake, a database) serves the documents: such a view is live although its document came from the store, because a live connector declares that it pushes the kind (`SourcePlugin.pushes`), and it subscribes there. See DEVELOPER_GUIDE.md for every Kafka setting.

### The demo source

The `demo` plugin ticks entities whose fixture says `live` (for example the `trading` pack's sample trades, curves, spot, netting sets) every `tick-ms` (400 ms), but only while someone watches. It takes a random-walk step and keeps dependent values consistent: for the future, MTM follows the last price, and the intraday settlement row and variation margin follow too. Its settings:

```
drishti:
  sources:
    plugins:
      demo:
        settings:
          tick-ms: 1000     # one step a second instead of 400 ms
          ticking: false    # or freeze every demo entity
```

`DRISHTI_DEMO_ENABLED=false` turns the demo source off altogether, so no sample entity ticks.

### Implementation note

The architecture first proposed re-binding only the panels whose expressions read the changed paths. Every expression can report its paths (`Expr.paths`), and every cell and row carries its `path`. Measurement showed that rebuilding the whole view with the cached layout and diffing takes a few hundred microseconds, well inside the budget. The simpler rebuild-and-diff design was therefore kept. The path index remains available if a very large view ever needs it.

## D2. History queries: an entity over time

Status: **built**; the implementation notes say where the build differs from the design. Written 2026-10-20 from the design agreed with the product owner, checked against the code on `develop` and against the two connector branches that are not merged yet (drishti-iceberg and iceberg-external; Cassandra). Related: HOW_IT_FITS.md section A5 (a command becomes a screen), CLUSTER_GUIDE.md (forwarding), CLUSTER_GUIDE.md section 10 (slices and owners), CLUSTER_GUIDE.md (the history row of its table), section D3, CASSANDRA_CONNECTOR.md (plugins/drishti-plugin-cassandra/README.md).

### D1. The ask

The product owner (2026-10-20): "*if someone says `trd dffgg 30` then we should show top level attributes for that trade for last 30 days; if someone says `eq` and `30` then we should show latest 30 days of data for that equity and so on.*" It is called a killer feature. Traders and risk managers compare Drishti with Bloomberg, so the bar is a terminal: typed, keyboard-first, dense, fast and beautiful in every theme.

One sentence: **add a range after the id, and the screen becomes that entity over time.**

```
TRD 1000015 30 <GO>          the trade, one row per business day, the last 30 days it has data
EQ EQ-CSCA 3M <GO>           the equity: a price chart, the high, low and change, and the same table
FX FX-AUDUSD YTD <GO>        the FX spot since 1 January
CRV CRV-USD-SOFR FROM 2026-09-01 TO 2026-09-30 <GO>
TRD 1000015 30 DIFF <GO>     only the fields that changed, day by day
```

### D2. What exists already

History queries are built from parts that are in the code now. Nothing below is new.

| Part | Where | What it gives the design |
|---|---|---|
| One field over time | `HistoryController.series`, `GET /api/v1/history/{kind}/{id}/series?path=&days=` (drishti-server `server/api/HistoryController.java`) | One Rachana-EL path over up to `MAX_DAYS` = 260 business days, one `router.fetch(ref, AsOf.of(d))` per day, all at once, redacted per day with `Entitlements.redact`. Used by the field dialog `web/static/js/drishti_history.js` (click a number). |
| What changed | `HistoryController.diff`, `GET /api/v1/history/{kind}/{id}/diff?from=&to=&fromKnownAt=&toKnownAt=`; `engine/history/DocumentDiff` (limit 2000 changes) | The diff between two days, or two "known at" instants. Console page `/compare/{kind}/{id}`, export `/export/compare/{kind}/{id}.csv`. |
| Dated reads | `SourcePlugin.fetch(EntityRef, AsOf)`, `AsOf(businessDate, knownAt, live)`, `Provenance(source, generation, fetchedAt, live, businessDate)` (drishti-api) | Every dated connector answers one entity on one date; the provenance says which date the data is for and its generation. |
| Time travel | `SourcePlugin.timeTravel()`; `DRS-1007 NO_TIME_TRAVEL` | Delta and Iceberg answer `knownAt`; others refuse it rather than show today's data. |
| A range of days as columns | `SourcePlugin.scan(ScanRequest)` with `ScanRequest(kind, paths, filters, from, to, batchRows)` and `ColumnScans.over` (at most `MAX_DAYS` = 800 calendar days) | Promoted columns over a date range with an `EQ` filter, no document parsed. Delta implements it; the others fall back. |
| Business days | `engine/time/BusinessDates` (`calendar().previous(d)`, `earliest()`, `current()`, `parse(date, knownAt)`) | The calendar the series already walks. |
| Routing | `engine/source/SourceRouter` (`fetch`, `fetchAll`, `readOrder`, `Fallback` for Drishti*net peers), `SourceRegistry` | Several connectors per kind in read order (Redis for recent days, Delta behind it); peers when no local source holds the entity. |
| Forwarding | `server/net/ViewForwardFilter` (already forwards `/api/v1/history/{kind}/{id}...`), `PeerSource`, `NetDataController` (`/api/v1/net/fetch/{kind}/{id}`) | A history request reaches the node that serves the kind with no new routing code. |
| The command line | `engine/command/CommandParser`, `server/api/CommandController` (`POST /api/v1/command`, `/suggest`, `/history`, `/aliases`), `CommandMemory.expand` (aliases), `engine/search/SearchQuery.pick/looksLikePick`, `engine/search/PhraseParser`; console `routes/terminal_routes.py` `/go`, `web/static/js/drishti_command.js` | Where the range is parsed and the page chosen (section D4). |
| Pinned links | console `core/asof.py` (`?asOf=&knownAt=`), `share_url()` and `view()` in `routes/terminal_routes.py` (`?gen=`, `pin = asit / changed / retry`) | A history row opens "the normal view as of that day" with the machinery that exists. |
| Export | console `routes/export_routes.py`, `core/export.py` (`to_csv` with BOM and CRLF, `filename()`, `to_xlsx`, the formula guard) | The conventions history export follows. |

CLUSTER_GUIDE.md measured the series at **0.02–0.12 s for 30 days** and diffs at 6–14 ms (2026-10-18, Delta). History queries generalise the series from one field to a whole row and give it its own page, command and API.

### D3. Principles

1. **One command, any kind.** The range follows the id; the kind decides the shape (a document table, or a market chart and table).
2. **Days with data, not calendar days.** `30` is the last 30 business days on which that entity has data. The header names the dates.
3. **Per-kind behaviour is YAML.** A pack's `history:` block picks columns, chart fields and limits. No code per kind.
4. **The SPI stays optional.** A new default method loops over today's dated reads; a connector adds a fast path only when its layout allows one. Every connector works on day one.
5. **Same masks, same rights.** Every day's row is redacted exactly as the live view is. Rights are checked per kind.
6. **Bounded.** A maximum range, a row cap, a timeout, bounded parallelism and a cache keyed by the data's version.
7. **Terminal-grade.** Keyboard-first, dense, every theme, every contrast rule, under a second on a laptop.

### D4. The command

#### D4.1 Grammar

```
<MNEMONIC> <ID> <RANGE> [DIFF] <GO>

RANGE  :=  N                    N business days on which the entity has data (1 ≤ N ≤ max-days)
       |  N D | N W | N M | N Y calendar days, weeks, months, years back from the date box (30D, 2W, 3M, 1Y)
       |  YTD | MTD | QTD       since the first day of the year, month or quarter
       |  FROM yyyy-mm-dd [TO yyyy-mm-dd]
```

Case never matters (`30d`, `ytd`, `from`). `<GO>` anywhere is ignored, as today. The end of every range is the date box (the console's `drishti_asof`, live means the current business date) unless `TO` says otherwise. `N D` and the other calendar forms take every business day in the span that has data, so `30D` gives about 21 rows and `30` gives 30.

#### D4.2 Where it is parsed, and every clash checked

Today `CommandParser.parse` splits the text in two (`split("\\s+", 2)`): the mnemonic and **everything else as the id**. So `TRD 1000015 30` is read now as the id `1000015 30`; `CommandController.resolve` then finds no such trade and, because `SearchQuery.looksLikePick` is true for any id containing a space, shows a pick list. The range must therefore be taken off **before** that, in a new class `engine/command/HistoryRange` (parse and render, about 150 lines), called from `CommandController.resolve` after `CommandMemory.expand`.

The rule: **a range is the last one to four words, never the first word after the mnemonic, and only after an id.** Then:

| Typed | Today | With history | Why it cannot clash |
|---|---|---|---|
| `TRD 1000015` | opens the trade | the same | no range word |
| `TRD 1000015 30` | pick list (id `1000015 30` not found) | history, 30 days | the range is the third word |
| `TRD 30` | opens trade `30` | the same | the range is never the first word after the mnemonic, so a numeric id is always an id |
| `TRD MX-20000001 30` | pick list | history | as above |
| `TRD MX-2000 30` (a prefix) | pick list | the id is resolved as today first: one match opens its history; several give the pick list `/s?q=TRD MX-2000&days=30`, whose rows open history | the range is kept on the pick list |
| `BOND UST 2030` where `UST 2030` **is** an id | opens it | opens it | the whole rest is tried as an id first (the existing `exists()` read); only when it is not an entity is a trailing range taken off |
| `TRD MX-2* 30`, `TRD desk=rates 30` | pick list or search | the same, unchanged | a `*`, an operator (`SearchQuery.OPERATOR`) or `where / order by / limit` (`looksLikeSearch`) means a search; ranges apply to one entity only |
| `TRD limit 30` | search (`SUFFIX`) | the same | `looksLikeSearch` is checked first |
| `MYBOOK 30` (alias for `BOOK BOOK-RATES-1`) | `BOOK BOOK-RATES-1 30`, a pick list | history of the book | `CommandMemory.expand` appends trailing words (`MYBOOK F3` becomes `BOOK BOOK-RATES-1 F3`), so the range survives the alias |
| `MYPICK 30` (alias for `TRD desk=rates`) | search | the same | an alias that expands to a search stays a search |
| `MX-20000001 30` (bare id) | not read (two words), plain words | history | allowed only when `ReferenceCatalog.kindOf(id)` names the kind (the pack's `graph.id-patterns`), as for a bare id today |
| `1000015 30` (bare, no id pattern) | plain words | plain words, which now understand it (section D15.5): an id that names one entity of one kind, then a range or a time phrase | no id pattern for `1000015` |

| Typed | Today | With history | Why it cannot clash |
|---|---|---|---|
| `MKT`, `RUPAKA …`, `FORM …`, `FUNC …`, `WHAT …`, `DATAMAP …` | pack overview, BI, form, functions, Knowledge, data map | the same | single words and reserved first words are handled before the command in `/go` |
| `TRD 1000015 FROM 2026-09-01` | pick list | history from that date to the date box | `FROM` and `TO` are keywords only in the last four words and only followed by a date |
| `TRD YTD` | opens trade `YTD` | the same | first word after the mnemonic |

`HistoryRange.trailing(String rest)` returns `Optional<Split(String id, HistoryRange range, boolean diff)>`; it never throws. A range that is read but not allowed (too large, `TO` before `FROM`, a future `FROM`) fails with its own code (section D16), not with a pick list. `HistoryRange.malformed` names what reads as a range but is none (`0`, `-5`, `99999`, `1.5M`, `3X`, `FROM yesterday`, `FROM 2026-02-30`, `DIFF DIFF`, a bare `DIFF`, `30 30`, and since QA1-203 `30.`, `1e3`, a dangling `FROM` or `TO` (`FROM 2026-10-01 TO`: *TO needs a date*), `TO` without `FROM` (*TO follows FROM*), `DIFF 30` (*DIFF follows a range: TRD 1000001 30 DIFF*)); `CommandController` answers it with `DRS-4022` (`DRS-4021` for a number of days past `max-days`) and what to type when what is before it is one entity the caller may open and the whole text is not itself an entity. `TO` before `FROM` and a date that does not exist are `DRS-4022` too, as at the endpoint. The console's `/go` shows the message on the terminal page. Digits of another script are read as digits first (`TRD 1000001 ３0` is `TRD 1000001 30`, `０` is *0 days is empty*), as the endpoint reads `days=０`. A bare id with such a range (`1000015 -5`, `1000015 since yesterday`) goes to plain words, which answer the same `DRS-4022` (section D15.5); the console shows it on the terminal page too.

#### D4.3 The answer and the redirect

`ApiDtos.CommandResponse` gains one field, `history` (null when there is no range):

```
{ "ref": {"kind": "trade", "id": "1000015"}, "mnemonic": "TRD", "list": null, "matched": null, "pack": null,
  "history": {"range": "30", "days": 30, "from": null, "to": null, "diff": false} }
```

The console's `/go` (`routes/terminal_routes.py`) redirects to **`/v/{kind}/{id}/history?days=30`** (or `?from=&to=`, plus `&diff=1`). `/v/{kind}/{id}?days=30` is accepted too and redirects there (303), so the short link the product owner wrote works. The route is declared before `/v/{kind}/{id_:path}`, as `/about` and `/panels` are. A pick list carries `days` through `/s?q=…&days=30`.

#### D4.4 Type-ahead, history and aliases

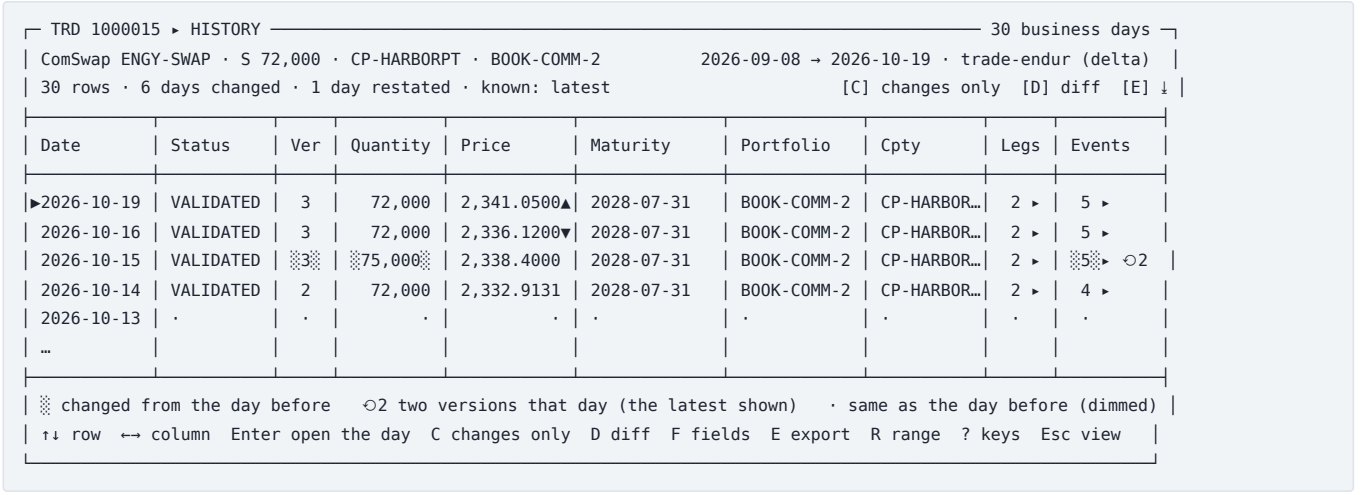
- `SuggestionService.suggest` (engine `command/SuggestionService`): when the text is `<MNEMONIC> <ID>` and the id names one entity, the first suggestions are ranges: `30 — last 30 days`, `3M — last 3 months`, `YTD — since 1 January`, `30 DIFF — what changed`. `Tab` completes, as today. The kind's `history.default-days` is the first one offered. Across Drishti*net the peers' suggestions are merged with one row per range (each completes differently) and the ranges first, then the entity whose id was typed, from whichever node holds it (on a node that does not hold `1000002`, `TRD 1000002` lists the peer's `1000002` before this node's `10000020`…; QA1-204), then the rest; for a caller with field masks the range rows keep their labels (only entity rows are described again from the document as the caller sees it). A partial range (`TRD 1000015 3`) offers the ranges that start with it to every caller.
- `CommandMemory.remember` keeps the typed text, so `↑` recalls `TRD 1000015 30` as typed.
- Aliases store any command text, so `MYTRD = TRD 1000015 30` works without change.
- Share links carry the range and pin the end: `/v/trade/1000015/history?days=30&to=2026-10-19` (and `&knownAt=` when pinned), so the recipient sees the same rows a week later. A link without `to` is live: it moves with the business date.

### D5. The history view

#### D5.1 Shapes

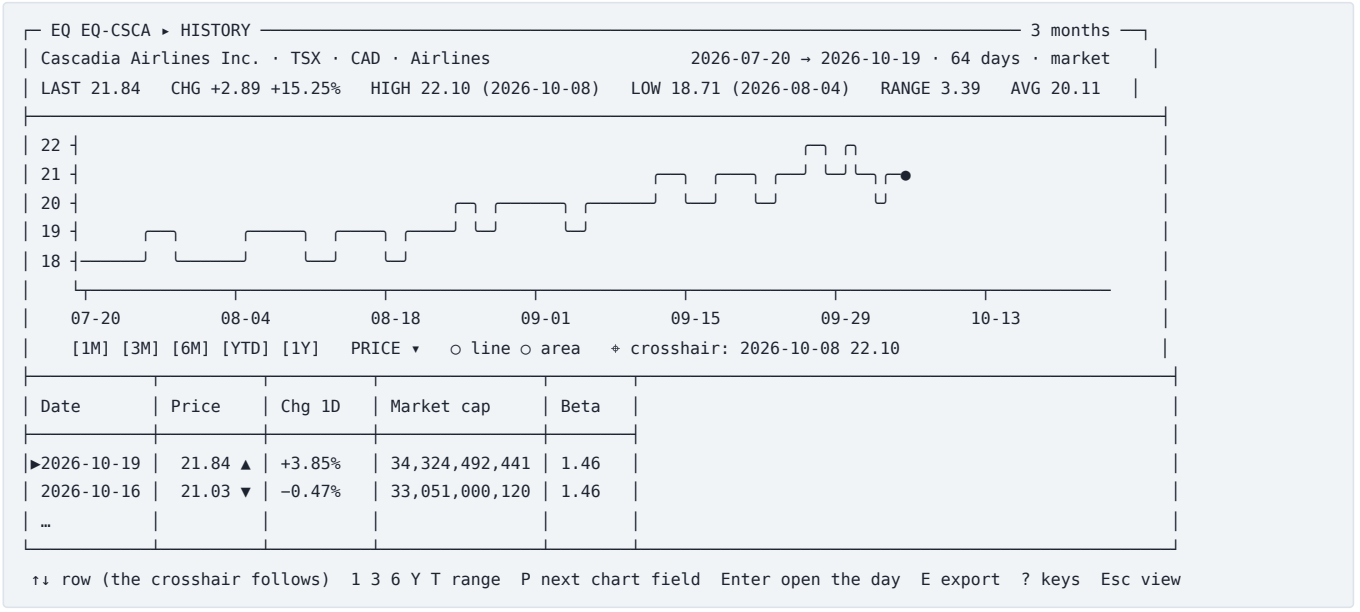
| Shape | When | Screen |
|---|---|---|
| `document` | default (trades, counterparties, agreements) | header, the table, the drawer of the day opened |
| `market` | `history.<kind>.chart` is set (equity, curves, FX), or `shape: market` | header, headline numbers, a chart, the same table |
| none | no source of the kind keeps daily history and no embedded series (section D6) | today's view with a banner: "*FX-AUDUSD keeps no daily history here: its connector holds today's data only.*" (`DRS-1012`) |

#### D5.2 Trade history (document shape)




- **One row per business day**, newest first on screen (the JSON is oldest first, section D8). A day with no data for this entity is left out; it is listed in the header's tooltip ("no data on 2026-09-14").
- **Columns** are the kind's top-level scalars: the pack's `history.<kind>.columns` when given, otherwise the kind's search key columns (`drishti.search.columns`, from `columns:` in pack.yaml) first and then every other top-level scalar, labelled as the view labels them. A column empty on every row is hidden (`F` shows it). Values use the pack's formats (`fmt($.PRICE, 'price2')`).
- **Nested parts** (top-level lists and objects: `LEGS`, `EVENTS`) show as a count with `▸`. Clicking, or `Enter` on the cell, opens that day's part in the drawer. A count that changed is highlighted like any changed cell.
- **Changed cells** (different from the previous day, the day before the range included, section D7.3) use the `--d-warn` tint and a `▲`/`▼` for numbers; unchanged cells after the first change are dimmed with `--d-muted`, so the eye finds the moves.
- **Changes only** (`C`) hides days on which nothing moved; the header says "6 of 30 days".
- **Several versions** in one day (a restatement, a second load) show `⟲2` beside the date; its tooltip names the versions or generations. The row is the latest version of that day.
- **A row opens the normal view as of that day**: `Enter` (or a double click) goes to `/v/{kind}/{id}?asOf=2026-10-15&gen=<generation>&via=history` (and `&knownAt=` when the history is "known at"), so the banner says "as it was, from its history" or "changed since the history read that day" (never "shared"), with *Back to the history*.

#### D5.3 Equity history (market shape)




- The **headline numbers** are computed by the server on the headline field (`history.<kind>.headline`, default the first chart field): last, change and change % over the range (first to last row), high and low with their dates, range (high − low) and average.
- The **chart** is ECharts, as the field dialog in `drishti_history.js` already uses, coloured from the theme's tokens (`--d-ink`, `--d-muted`, `--d-link`, `--d-border`) read at draw time, so every theme works. Several chart fields (a curve's `TEN_Y` and `SLOPE_2S_10S`) are one axis each, `P` steps between them. The table's selected row and the chart's crosshair move together.
- Range buttons re-ask the server; the page URL follows (`?range=3M`), so `Back` works and a link reproduces it.
- Curves and surfaces: the table shows the scalars (`TEN_Y`, `SLOPE_2S_10S`, `AS_OF`); `POINTS` is a part (`12 ▸`), and the drawer draws that day's curve with the previous day's as a faint line.

#### D5.4 Keys and accessibility

| Key | Does | Key | Does |
|---|---|---|---|
| `↑` `↓`, `j` `k` | previous, next row | `Home` `End`, `PgUp` `PgDn` | first, last, page |
| `←` `→` | previous, next column (the header cell is announced) | `Enter` | open the selected day's view |
| `Space` | open the selected cell's part in the drawer | `C` | changes only, on and off |
| `D` | diff mode (section D15.1) | `F` | choose fields (a checklist; the choice is kept per kind in the user's settings) |
| `R` | type a new range in place | `1` `3` `6` `Y` `T` | 1M, 3M, 6M, 1Y, YTD (market shape) |
| `P` | next chart field | `E` | export menu (CSV, JSON, Excel) |
| `?` | the keys | `Esc` | close the drawer, else back to today's view |

The table is a real `<table>` (`.tbl`, `data-plain`) with a roving `tabindex` on rows, `aria-selected` on the selected row and `aria-sort` on the date column. A changed cell carries `data-changed` and visible text in its title ("changed from 75,000") so colour is never the only signal. The chart has an `aria-label` with the headline numbers and a "table" fallback (the table below it). Keys are ignored while focus is in an input, as the command line's `/` is. Reduced motion turns off the chart's animation.

**Themes:** all ten (`terminal`, `light`, `wallstreet`, `blue`, `green`, `crimson`, `crimson-dark`, `terminal-green`, `terminal-crimson`, `terminal-blue`). The new `web/static/css/drishti_history.css` uses only tokens: text in `TEXT_TOKENS` (`--d-ink`, `--d-muted`, `--d-link`, `--d-warn`, `--d-pos`, `--d-neg`…), fills with `--d-on-fill` text, so `drishti-console/tests/test_contrast.py` (every `color: var(--d-X)` in `web/static/css/*.css` must be a text token or `on-accent` / `on-fill`; 4.5:1 on every ground) covers it with no new rule.

### D6. The `history:` block in pack.yaml

Kinds are declared in `pack.yaml` as a list (`kinds: [trade, …]`), and per-kind settings live in top-level maps keyed by kind: `ingest: {trade: …}`, `form: {ir-curve: […]}`, `mnemonics:`. `history:` follows the `form:` pattern exactly: read through `Pack.manifest()` by a new server component `server/design/HistoryDefinitions` (as `FormDefinitions` reads `form:`), parsed by a new `engine/history/HistorySpec.parse(kind, raw, problems)` (as `engine/search/form/FormSpec.parse`).

```
history:
  trade:
    columns: [TRADE_STATUS, TRADE_VERSION, QUANTITY, PRICE, MATURITY_DATE, PORTFOLIO, COUNTERPARTY]   # default: top-level scalars
    exclude: [SOURCE_UPDATE_TIME, _meta]          # never shown
    ignore-changes: [SOURCE_UPDATE_TIME]          # shown, but a change here is not "a change" (changes only, DIFF)
    parts: [LEGS, EVENTS]                         # default: every top-level list or object
    default-days: 30
    max-days: 250                                  # at most drishti.history.max-days
  equity:
    shape: market                                  # implied by chart:
    chart: [PRICE]
    headline: PRICE                                # high, low, change; default: the first chart field
    columns: [PRICE, CHANGE_1D, MARKET_CAP, BETA]
    series: {path: HISTORY, date: DATE, value: CLOSE}   # used when no source keeps daily documents
  ir-curve:
    chart: [TEN_Y, SLOPE_2S_10S]
    columns: [TEN_Y, SLOPE_2S_10S, AS_OF]
  news-topic:
    enabled: false                                 # no history for this kind (the command says so, DRS-1012)
```

#### D6.1 Schema

| Key | Type | Default | Rule |
|---|---|---|---|
| `enabled` | boolean | `true` | `false` turns history off for the kind |
| `shape` | `document` \| `market` | `market` when `chart` is set, else `document` | |
| `columns` | list of a path, or `{path \| expr, label?, format?}` | key columns first, then top-level scalars | a path is a top-level field; `expr` is Rachana-EL (as the series' `path` is today, e.g. `first($.LEGS).NOTIONAL`); at most `drishti.history.max-columns` |
| `exclude` | list of paths | `[]` | |
| `ignore-changes` | list of paths | `[]` | |
| `parts` | list of paths | every top-level list or object | each must be a list or object in the sample documents (a warning otherwise) |

| Key | Type | Default | Rule |
|---|---|---|---|
| `chart` | list of paths or `{path \| expr, label?, format?}` | `[]` | numeric; at most 4 |
| `headline` | path | first of `chart` | must be in `chart` |
| `series` | `{path, date, value \| values}` | none | a list inside the document with a date per item (the market samples carry `HISTORY: [{DATE, CLOSE}]`) |
| `default-days` | 1..`max-days` | `drishti.history.default-days` (30) | |
| `max-days` | 1..`drishti.history.max-days` | the server's | |

Unknown keys are warnings, bad values errors, with a new code **`DRS-2061` `HISTORY_INVALID`** (422) beside `DRS-2060` `FORM_INVALID`. The checks run where pack manifests are checked: the deploy stage (`manifestCheck` in `server/deploy/PackDeployService.java`), `sutra lint <pack-dir>` (`server/cli/SutraCli`) and pack reload (a warning in the log and on Admin → Packs, as `FormDefinitions` logs a bad form). The Build workbench gets a History tab later (open question 3). `docs/guides/BUILD_GUIDE.md` gets a step for it.

#### D6.2 Embedded series

The market samples (`config/packs/banking/market-data/samples/equity.jsonl`, `fx-spot.jsonl`) carry their own history inside today's document (`HISTORY: [{DATE, CLOSE}], [{DATE, MID}]`). When `series:` is declared and the kind's sources hold no daily documents in the range, the rows come from that list in the newest document (`"basis": "embedded"` in the answer): a chart and a one-column table, no versions. When daily documents exist they win, because they carry every field. *Implementation:* when the stored days start inside the window (a store that keeps a week, asked for 3 months), the days before the oldest stored one are filled from the newest document's series (`basis: "documents+series"`, those rows `source: "series"`), so the chart and the headline numbers cover the whole range; beside a series row only the series' own fields count as changes.

#### D6.3 Shipped packs

H13 adds `history:` to `trading` (trade), `market-data` (equity, equity-index, fx-spot, ir-curve, credit-curve, commodity, bond, inflation-index), `counterparty-risk` and `market-risk` (their headline measures), with `enabled: false` on derived kinds whose pages are rebuilt per day at a cost (`counterparty-trades`, `portfolio-trades`: open question 2).

### D7. The server endpoint

#### D7.1 Path

```
GET /api/v1/history/{kind}/{id}?days=30 | range=3M | from=yyyy-mm-dd&to=yyyy-mm-dd
    [&fields=PRICE,TRADE_STATUS] [&diff=true] [&knownAt=<ISO-8601>] [&order=asc|desc]
GET /api/v1/history/{kind}/~?id=<id>&…          an id a path cannot carry ("/", "\", ".."), as entity_path() does today
```

It sits in the existing `HistoryController` (`@RequestMapping("/api/v1/history")`) beside `/series` and `/diff`, not under `/api/v1/entities` (section D21). The date box's `AsOf` (resolved by `AsOfResolver` from `X-Drishti-As-Of`) is the default end; `to` overrides it. The access log gets `"/api/v1/history/{kind}/{id}" → "history"` in `AccessRecorder.ACTIONS`. A mnemonic or a plural in place of the kind is `DRS-4004` saying which kind to use (*TRD is a mnemonic: use the kind trade*, *unknown kind trades: did you mean trade?*). Kinds are matched exactly, as everywhere in the API: one in another case is `DRS-4004` too (*unknown kind Trade: kinds are written in lower case, did you mean trade?*), and one that no pack, source or mnemonic knows names the kinds there are (*unknown kind nokind: the kinds here are …*), unless a source that serves any kind holds that entity. A field in `fields` that is no column but a top-level field of the documents is shown as a column; one that no day's document has is `DRS-4004` (*no trade has a field NOPE on these days*), never dropped silently.

#### D7.2 What it does

```
HistoryController.rows(kind, id, params, AsOf, Principal)
  1. entitlements.requireOpen(principal, kind)                      DRS-6xxx as today
  2. spec   = historyDefinitions.of(kind)                            enabled? shape, columns, limits
  3. window = HistoryWindow.resolve(range, AsOf, BusinessDates, spec) dates; DRS-4021 / DRS-4022 on a bad range
  4. key    = (kind, id, window, fields, knownAt, router.dataVersion(kind, LATEST))
     cache hit → step 7
  5. days   = router.history(ref, window + one day before, fields, knownAt, budget)   section 9
  6. rows   = HistoryRows.build(days, spec): latest version per day, baseline compare, changed[], parts counts, summary
  7. redact every row's document for the principal, then project the columns   section 13
  8. answer (section 8); partial when some days failed or timed out (DRS-1014 in problems)
```

The service class is `engine/history/HistoryService` (engine, so the CLI and BI reuse it), with `HistoryWindow`, `HistoryRows` and `HistoryCache` beside `DocumentDiff`. The controller stays thin. `series` is re-pointed at the same service (one field is a one-column history), so the field dialog gets the fast paths and the 260-day constant becomes the setting.

#### D7.3 Days, versions and "known at"

- **Days on which the entity has data.** For `days=N` the window ends at the date box and walks back over business days (`BusinessDates.calendar().previous`) until N days with data are found or `max-days` business days are passed. A connector that can list the entity's days says them directly (section D9: `days(…)`), so no empty day is read. A **snapshot** connector that answers a date with an earlier date's document (its `lookback-days`) does not count that date: a row is kept only when `provenance.businessDate()` equals the date asked, as `ColumnScans` already skips such days.
- **Effective kinds** (a row only when the entity changes, `mode.<kind>: effective`) have data on every day by definition. Each business day is a row; days carried forward from an earlier change are marked `carried: true` and dimmed, so `C` (changes only) shows exactly the change days.
- **The baseline.** One more day before the window is read, so the first row's changes are real. It is reported as `baseline`, not as a row.
- **Several versions in a day.** A row is the latest version of its day. `versions` is how many versions of that day the store keeps (Delta: commits in the retained log that added files for the date; Iceberg: snapshots that added files to the partition; others: `null`, unknown) and `restated` is true when there is more than one. Only connectors that can say it cheaply fill it (section D10).
- **Known at.** Without `knownAt` each day is its latest version (what we know now). With `knownAt=K` the whole history is read as the store stood at instant K: each day is the version of that day that was known at K, and days loaded after K are absent. It is Delta's and Iceberg's time travel (`timeTravel()`); a connector without it that may hold one of the dates refuses with `DRS-1007` as reads do today. One that holds none of them (`holdsNone`, or `NOT_HELD` for every date: the shipped `file` connector before any file of the kind) is not asked, so it never refuses for the Delta or Iceberg sources behind it. A `knownAt` that is not an ISO-8601 instant fails with `DRS-4003` naming it (*knownAt '2026-10-20' is not an instant*). A window with no data as known at K is `DRS-1013` saying so, and the days it has now are named as known now, since they may have been loaded after K (*… has no data in the 5 business days to 2026-10-20 as known at 2026-10-20T13:00:00Z: none of its data in that window was known then; as known now its days with data run from 2026-07-27 to 2026-10-20, which may have been loaded after 2026-10-20T13:00:00Z: ask without knownAt, or with a later one*). "Each day as it was known at the end of that day" (as first reported) is a different question, kept as open question 1.

### D8. The JSON contract

This is the contract between the two builders (section D18). Builder B builds the console against a fixture of exactly this shape (`drishti-testkit/src/main/resources/history/trade-1000015-30.json`, added in H1) and the server's tests assert it.

```
{
  "ref": {"kind": "trade", "id": "1000015"},
  "mnemonic": "TRD",
  "title": "ComSwap ENGY-SWAP · CP-HARBORPT",
  "shape": "document",
  "basis": "documents",
  "window": {
    "asked": "30",
    "from": "2026-09-08",
    "to": "2026-10-19",
    "days": 30,
    "mode": "data-days",
    "baseline": "2026-09-07",
    "missing": ["2026-09-14"],
    "knownAt": null,
    "loadedTo": null,
    "loadedFrom": null
  },
  "columns": [
    {"path": "TRADE_STATUS", "label": "Trade status", "type": "text", "format": null, "masked": false, "key": true},
    {"path": "QUANTITY", "label": "Quantity", "type": "number", "format": "compact", "masked": false, "key": true},
    {"path": "PRICE", "label": "Price", "type": "number", "format": null, "masked": false, "key": false},
    {"path": "TRADER_ID", "label": "Trader", "type": "text", "format": null, "masked": true, "key": false}
  ],
  "parts": [
    {"path": "LEGS", "label": "Legs"},
    {"path": "EVENTS", "label": "Events"}
  ],
  "chart": [],
  "rows": [
    {
      "date": "2026-09-08",
      "dataDate": "2026-09-08",
      "carried": false,
      "values": {"TRADE_STATUS": "VALIDATED", "QUANTITY": 72000.0, "PRICE": 2332.9131, "TRADER_ID": "•••"},
      "display": {"TRADE_STATUS": "VALIDATED", "QUANTITY": "72K", "PRICE": "2,332.9131", "TRADER_ID": "•••"},
      "parts": {"LEGS": 2, "EVENTS": 4},
      "changed": [],
      "changes": null,
      "versions": 1,
      "restated": false,
      "generation": 41,
      "source": "trade-endur",
      "deleted": false
    },
    {
      "date": "2026-10-15",
      "dataDate": "2026-10-15",
      "carried": false,
      "values": {"TRADE_STATUS": "VALIDATED", "QUANTITY": 75000.0, "PRICE": 2338.4, "TRADER_ID": "•••"},
      "display": {"TRADE_STATUS": "VALIDATED", "QUANTITY": "75K", "PRICE": "2,338.4000", "TRADER_ID": "•••"},
      "parts": {"LEGS": 2, "EVENTS": 5},
      "changed": ["QUANTITY", "PRICE", "parts.EVENTS"],
      "changes": null,
      "versions": 2,
      "restated": true,
      "generation": 58,
      "source": "trade-endur",
      "deleted": false
    }
  ],
  "summary": {
    "field": null,
    "last": null, "first": null, "change": null, "changePct": null,
    "high": null, "highDate": null, "low": null, "lowDate": null, "range": null, "average": null
  },
  "changedDays": 6,
  "partial": false,
```

```
  "problems": [],
  "truncated": false,
  "dataVersion": "7@1760700000000",
  "cache": "miss",
  "tookMs": 184
}
```

| Field | Meaning |
|---|---|
| `shape` | `document` or `market` (section D5.1) |
| `basis` | `documents` (daily documents), `embedded` (the kind's `series:` list), `documents+series` (daily documents, and the days before the oldest stored one from the newest document's `series:` list, those rows with `source: "series"`), or `none` (never sent: `DRS-1012` instead) |
| `window.mode` | `data-days` (a bare number), `calendar` (`30D`, `3M`, `YTD`…), `between` (`FROM … TO`) |
| `window.to` | the last day shown: for `data-days` the entity's newest day with data (not the date box), else the end of the span |
| `window.missing` | business days in the window with no data for this entity (every mode; capped at 400), up to the newest date the kind's sources hold |
| `window.loadedFrom` | set only when a `calendar` or `between` span starts before the entity's first day with data: that day (the days before it are not missing: not loaded here, or the entity did not exist yet) |
| `window.loadedTo` | set only when a `calendar` or `between` span ends after the newest date the kind's sources hold: that date (the days after it are not loaded yet, not missing) |
| `columns[].masked` | true when the caller's masks hide the field: its values are the mask on every row |
| `columns[].key` | one of the kind's key columns (shown first, frozen when scrolling) |
| `rows` | oldest first; at most `drishti.history.max-rows`; `order=desc` reverses |
| `rows[].values` | raw values after masks (numbers stay numbers); `display` is the same values formatted for the screen |
| `rows[].dataDate` | the date the data is for; equal to `date` except for `carried` rows of effective kinds |
| `rows[].changed` | paths that differ from the previous row (or the baseline); `parts.<path>` for a part whose count or content changed; fields in `ignore-changes` never appear |
| `rows[].changes` | with `diff=true`: the `HistoryController.Change` list (`path`, `label`, `kind`, `before`, `after`, `delta`), as `/diff` returns |
| `rows[].versions`, `restated` | section D7.3; `null` when the connector cannot say |
| `rows[].deleted` | the entity was deleted that day (a tombstone); its values are empty |
| `summary` | market shape: the headline field's numbers; all null for the document shape |
| `partial`, `problems` | some days could not be read: `[{"code": "DRS-1014", "source": "trade-endur", "days": ["2026-09-21"], "message": "…"}]` |
| `truncated` | the row cap cut the answer (the oldest rows are dropped and the header says so) |
| `dataVersion` | the kind's `dataVersion` at the time of the read (the cache key's last part) |

A market answer differs only in `shape`, `chart` (`[{"path": "PRICE", "label": "Price", "format": "price2"}]`) and a filled `summary`. Errors use the existing problem shape of `ApiExceptionHandler` (`code`, `message`, `fix`).

## D9. The connector SPI

#### D9.1 New types in drishti-api

```
/** One entity over business dates. fields empty means the whole document. */
public record HistoryRequest(EntityRef ref, LocalDate from, LocalDate to, Set<String> fields, Instant knownAt,
        int lastDays, Duration budget) {}

/** One business date of one entity: the document (or only the asked fields as a document) and where it came from. */
public record HistoryDay(LocalDate date, EntityDocument document, Integer versions) {}

/** What a source answered: the days it holds in the request, the dates it is authoritative for, the days it failed. */
public record HistoryRead(List<HistoryDay> days, Set<LocalDate> covered, Map<LocalDate, String> failed) {}
```

`lastDays > 0` means "the last N days with data on or before `to`" (then `from` is the earliest date the walk may reach).

#### D9.2 New methods on `SourcePlugin`

```
/**
 * The entity on each business date of the request that this source holds, newest last. The default reads day by day
 * through fetch(ref, asOf), at most drishti.history.parallel-days at once (HistoryReads.perDay).
 */
default HistoryRead history(HistoryRequest request) throws Exception {
    return HistoryReads.perDay(this, request);
}

/** The business dates on which this source holds the entity, from..to; empty when it cannot list them cheaply. */
default Optional<NavigableSet<LocalDate>> days(EntityRef ref, LocalDate from, LocalDate to) {
    return Optional.empty();
}
```

`HistoryReads` (drishti-api, beside `ColumnScans`) is the default:

1. The dates: `days(ref, from, to)` when present, else the business days of the request (the caller passes them; a plugin has no calendar).
2. Read each date with `fetch(ref, new AsOf(d, knownAt))` on virtual threads under a `Semaphore` of `parallel-days` (8), newest first, so `lastDays` stops early once N days with `provenance.businessDate() == d` are found.
3. A date whose read throws goes to `failed`; a date answered by an earlier date (snapshot lookback) is skipped; `covered` is the dates for which `coverage(kind, AsOf.of(d))` is `HELD`.
4. `fields` are applied after the read (the default cannot read less than a document).

`ConnectorInstance` (engine, the wrapper of every configured connector) delegates both methods and counts them in `SourceStats`. `DerivedSourcePlugin` keeps the default. `PeerSource` implements `history` with one call to the peer (section D12).

#### D9.3 The router

`SourceRouter.history(EntityRef, HistoryRequest)` merges the kind's connectors in their read order (`readOrder`, as `fetch` uses): the first connector answers the dates it holds; the dates it does not cover (`covered`) go to the next, so Redis (today and the last days) and Delta (years) behind it give one history. Each connector gets the remaining budget. Dates no local connector holds go to `Fallback.peers` (Drishti*net). The router never asks a connector for a date an earlier authoritative one covered, as `fetch` does not.

### D10. Fast paths, store by store

What each layout allows, from the code (and the two unmerged branches, read with `git show`).

| Connector | Layout (real) | History fast path | Versions |
|---|---|---|---|
| **delta** (`DeltaSourcePlugin`, `DeltaTable`) | one table per kind, partitioned `business_date=`; `(id, doc)` plus promoted columns (`layout.<kind>.columns`); `Layout.files()` is a `NavigableMap<LocalDate, List<Row>>` of the snapshot; a date's **id map** (`MappedIdMap`, memory-mapped) gives the file; one row group read by id statistics; heap and disk document caches | `days() = layout.files().subMap(from, true, to, true).keySet()`: the dates from the snapshot already in memory, no read. Per date: `idMap(kind, l, d).file(id)` (absent = no data, nothing read), then the one row group. When every asked field is promoted, read **only those columns** of that row group (a new `DeltaTable.row(l, d, file, id, columns)`; no `doc` decode, no JSON parse); otherwise the document through the existing caches. Reads go under `docReads` (`max-concurrent-reads`, 16). `knownAt` uses `layout(kind, knownAt)` once for the whole range. Effective tables: the entity's change dates are the dates whose id map holds it. | commits that added files for the date in the retained log (read once per table version, cached); `filesStamp(l, d)` already tells a restated date |
| **drishti-iceberg** (branch `worktree-agent-aea970bf1efcd2886`: `IcebergSourcePlugin`, `KindTable`, `NativeKindTable`, `drishti-iceberg scan/Planner`, `scan/Pruning`) | one table per kind; `id`, `doc`, `business_date`, promoted columns (`LEGS.CURRENCY` as `LEGS__CURRENCY`); partitioned `identity(business_date)`, **sorted by id**, files of about 250k rows with disjoint id ranges, 1 MB row groups; manifests keep id bounds | **One planned scan**: `id = X AND business_date BETWEEN from AND to`, pruned by partition summaries and the manifests' id bounds, gives about one file per day with no per-day id map (an id map reads the whole day's id column, about 75 MB per 1M ids). Then each file's row group by its id statistics, projecting `(business_date, id, <promoted fields>)` or `doc`. Files read in parallel (8). `days()` from the planned layout's dates. `knownAt` plans the snapshot `snapshotAt(K)`. | snapshots that added files to the partition, from the snapshot log (`meta/Snapshot`) |
| **iceberg-external** (same branch: `IcebergExternalPlugin`, `ExternalTable`, `KindConfig`) | any layout: id from identifier fields or named columns, date from a column, `day(ts)` or the partition spec; a table with no date is current-only | the same planned scan when the date is a partition or has column bounds; otherwise the per-day **id index** it already builds (`id-index-mb`); a table with no date has no history (`DRS-1012`) | as drishti-iceberg |
| **iceberg** (develop: `IcebergTable`, `IcebergLake`; kept as `engine: library` on the branch) | as drishti-iceberg | the default (it re-reads the Parquet footer on every read) | none |

| Connector | Layout (real) | History fast path | Versions |
|---|---|---|---|
| **cassandra** (branch `worktree-agent-ad01b2d6c9c414a21`: `CassandraSourcePlugin`, `CassandraLayout`, `DayReads`) | data table `PRIMARY KEY ((business_date, generation, bucket), id)`; **`entity_days`** `PRIMARY KEY ((kind, id), business_date)` DESC, written for every row; `kind_days` per date with its `generation` and `buckets` | `days()` = one single-partition range read: `SELECT business_date FROM entity_days WHERE kind=? AND id=? AND business_date >= ? AND business_date <= ?`, filtered by `kind_days` (an `entity_days` row may outlive its data). Then one token-aware point read per date (generation and bucket computed locally), in parallel. The documents cannot come from one query: `business_date` is in the data table's partition key. No `knownAt` (one generation kept, `DRS-1007`). | 1 (one generation kept) |
| **jdbc** table mode (`PostgresLayout`, `TableCatalog`; `(kind, id, business_date, doc jsonb)`, partitioned by month) | one row per entity per date | **one query**: `SELECT business_date, doc FROM <table> WHERE kind = ? AND id = ? AND business_date BETWEEN ? AND ? ORDER BY business_date` (only the asked fields with `doc -> 'F'` when `fields` is set), partition pruning by month; `days()` the same without `doc` | `null` |
| **jdbc** query mode (`query.<kind>` with `:id` and `:asOf`) | a site's own SQL | an optional `history.<kind>` query with `:id`, `:from`, `:to` (documented in `JDBC_QUERIES.md`); without it the default per-day loop of the existing query | from a `generation` column when the query returns one |
| **duckdb** (`DuckDbLayout`, `<domain>.entities`, promoted columns) | a row per entity per date | one query with `BETWEEN` as table-mode JDBC, promoted columns read directly | `null` |
| **mongodb** (`MongoLayout`, a document per entity per date, `day_ids` index) | `_id` per entity and date | one indexed range read over the `_id` range of the entity between the two dates (the `effective` read already finds "the last `_id` of the entity's range"), with a projection when `fields` is set | `null` |
| **aerospike** (`AerospikeLayout`) | a record per entity per date, **an index record per entity listing its dates** | `days()` = the entity's index record (one get), then one batch get of the dates' keys | `null` |
| **redis** (`RedisLayout`) | a compressed document per entity per date, **each entity's days**, a day's current generation | `days()` from the entity's days, then one pipelined `MGET` for the dates; it holds the recent days only, so the router asks Delta for the rest (section D9.3) | 1 |
| **file** (`FileSourcePlugin`, `JsonlDay`: `<root>/<yyyy-MM-dd>/<kind>.jsonl` indexed with byte offsets) | a dated folder per day | `days()` = dated folders in range whose day index holds the id (indexes are cached by `index-cache-mb`); a positioned read per day | `null` (generation = file time) |
| **s3** (`S3SourcePlugin`), **rest**, **feeds**, **news**, **kafka**, **rabbitmq**, **activemq**, **yfinance**, **demo** | current data or a stream | the default; most are undated (`capabilities().dated()` false), so the kind has no daily history unless the pack declares an embedded `series:` (yfinance's documents carry recent prices) | none |

A connector that cannot do better simply does not override `history`. The contract test (section D19.2) holds every one of them to the same answers.

### D11. Limits, cache and settings

New `@ConfigurationProperties("drishti.history")` class `engine/history/HistoryProperties`, a `LiveSettings` (changes apply without a restart, as `SearchProperties`), with its defaults in drishti-server's `application.yaml`:

Every setting of this part, with its type, default, environment variable, meaning and whether a change needs a restart, is in the one configuration document: CONFIGURATION.md, `drishti.history` (also at Help → Configuration in the console).

**Cache.** `HistoryCache` keeps unmasked, unformatted rows keyed by `(kind, id, from, to, fields, knownAt, dataVersion)`, where `dataVersion` is `SourceRouter.dataVersion(kind, AsOf.LATEST)` (Delta: the table's version and commit time; a connector that holds nothing of the kind counts as `none`) followed, in a cluster, by the data versions of the peers the read would ask, as their digests carry them (`SourceRouter.peerVersion`, `net:<node>{<connector>=<version>,…}`). A new load, here or on a peer, changes the version, so the cache never serves stale data and needs no expiry. A connector or peer that gives no version (`Optional.empty()`) is not cached, nor is an answer that failed, is partial or has no day. Masks and formats are applied on every answer, so an entry of days this node's own sources read serves every user.

**Days a peer answered are the caller's alone**. A peer masks the days it sends for the user it acts for (section D13), so they are not unmasked rows. When a peer answered any day of a read (`SourceHistory.Routed.fromPeer`), the entry is kept under the key above followed by `|viewer=<signature>`, the caller's security signature (`Principal.securitySignature()`: the sorted roles and the embedding app, which decide every node's masks and the nodes the caller reaches; the live relay's topic partition is the same signature). A caller with another signature misses it and reads the peer again for themselves, so an administrator never gets a viewer's `•••` and a viewer never gets an administrator's raw values. A read whose days were all this node's own is kept under the shared key, and a lookup tries the shared key first, then the caller's. A caller without a signature (`Viewer.of(redact, masked)`) is never cached when a peer answered.

Before, the first caller's masked days were served to everyone for the life of the entry: in soak 2, 77 administrator histories of peer-held trades showed `PRICE` as `•••` in a column marked `masked: false`, with the price's changes missing. `purgeCaches` (Admin → Cache) empties it.

**One budget, in real heap**. The cache is one `HistoryCache` per server: every key, shared or signed, counts against the same `cache-mb`, and the least recently used entries are dropped to stay under it (`HistoryCacheBoundTest`: forty signatures of a hundred peer-held trades each stay under the budget). An answer over a quarter of the budget is never kept. The size of an entry is the heap its days take on a 64-bit JVM (`HistoryCache.estimate`: 56 bytes a field of an object, its value's box and a string's characters, the day's records; within a few per cent of the measured heap). Before, it counted 48 bytes a node, 2.3 to 3.2 times under the heap, so `cache-mb: 64` held 150 to 210 MB. What a steady server looks like: the cache grows while new entities and ranges are asked, then stays at `cache-mb` (Admin → Cache: `bytes` near the budget, `entries` flat); a new load on any node changes the data version and the old entries leave as new ones come. The other caches a cluster keeps per caller signature are bounded by count: a view's shape (`drishti.engine.fingerprint-cache-size`, 100,000 entries of about 300 bytes, about 30 MB at most), the effective layouts (`layout-cache-size`, 10,000, keyed by shape and not by caller), and the map panels' listings and documents (32 and 8 entries, kept `drishti.panels.map-cache-seconds`, 60 s).

**Docs and test.** Every setting goes into `docs/admin/CONFIGURATION.md`; `drishti-console/tests/test_docs_settings.py` already fails when a key of `application.yaml` or a `@ConfigurationProperties` binding is not in that document.

### D12. Drishti*net and super scale

**Today's cluster.** No new routing is needed for the page:

- `ViewForwardFilter.target()` already matches `/api/v1/history/{kind}/{id}` (area `history`, three segments or more) and forwards the whole request to a node that renders the kind (`NetRouting.view(kind, principal, dated, entityId)`), retrying the next node once (`DRS-5052`, `DRS-5053`).
- When the pack is here but the data is elsewhere, `SourceRouter.history` sends the dates no local connector holds to `Fallback.peers` (`PeerSources`, ordered by `NetRouting.dataPeers`, replicas left out). `PeerSource.history` makes **one** call per peer, not one per day: a new `GET /api/v1/net/history/{kind}/~?id=&from=&to=&fields=&knownAt=` on `NetDataController`, beside `/net/fetch`. It carries the same headers (`X-Drishti-Net-Hop: 1`, `X-Drishti-Net-On-Behalf-Of`, `X-Drishti-As-Of`, `X-Drishti-Known-At`), checks `entitlements.requireOpen`, applies the caller's masks at the peer, and asks no further peer (one hop).
- The answers are merged by date; a date two peers both answer (shards that overlap) takes the higher generation.
- A node that holds the kind but is down, suspect or drained cannot be asked. Dates no node that answered holds are then failures of that node (`Fallback.downHolders`): with no day read the answer is `502 DRS-5052` naming it, as the view says; with some days read it is partial, a `problems` entry with code `DRS-5052` naming it. Never `DRS-1013`, and never cached.
- **A down node counts only for the dates it may hold**. Each source of a node's digest carries `earliestDate` and `latestDate` for the kind (the oldest and newest business dates its table holds), and the last digest heard before the node went down is kept. A date outside every one of its sources' ranges cannot be there, so it is not named (`Fallback.downHolders(kind, date)`, `NetRouting.downHolders(kind, date)`). A `range=3M` history of a CALYPSO trade asked while the MUREX holder is down, whose lake starts on 2026-10-02, is complete: the 54 days before 10-02 are before any lake, as the window's `loadedFrom` says. A source that keeps no dates, or a digest without the two dates (a node of an older release), may hold any date.
- **A peer that refuses the connection** (it just died and gossip has not said so yet, or it is restarting and its port is not open: `ConnectException`, a reset, no route) is down as far as the read goes (SOAK-02, SOAK-03): the next holder is asked for the same dates, and the dates no one answered are `DRS-5052` naming it, `… cannot be reached now: drishti-c (not answering: the connection failed)`, for the dates its digest says it may hold. Never `DRS-1003`.
- A peer that is asked but does not answer in time (frozen, overloaded) is a time-out of those days: `504 DRS-1014` *the node qa1-b did not answer the history of trade/10000901 within 10 s* when no day was read, else a `problems` entry with `DRS-1014`. Any other failure of the peer (it answered with its own error) is about that node too: `DRS-5052` *the node qa1-b that holds trade failed reading the history of trade/10000901: …*, as a view says, never `DRS-1003`, which names a source of the server that answered. A date that one peer failed and another answered is not failed.

**Super scale (CLUSTER_GUIDE.md › Super scale: 500 GB of JSON a day for seven years (design, not built), section D10, not built).** A slice is `(table, business day, group)`; each slice has owners on the ring (`hash(table, day, group)`, 64 virtual nodes, 2 owners for the `hot-days`). A history of one entity spans one slice per day, so the days spread over owners. `HistorySlices` (built with that design) groups the window's days by `owners[0]` of each day's slice (the group comes from the day's range map, section D9.1 there), sends one `/net/history` per owner with its days, merges, and reads any owner that does not answer within `forward-timeout` from S3 itself (section D10.3 there: anyone can read anything). 30 days on a ring of 10 servers is at most 10 calls in parallel.

### D13. Security

- **Rights per kind.** `entitlements.requireOpen(principal, kind)` first, as `series` and `diff` do. A part opened in the drawer is the same entity; a linked entity opened from it goes through its own view and its own check.
- **Masks on every row.** Each day's document goes through `Entitlements.redact(principal, data)` **before** any column is projected, any change is computed or any summary is taken, so a masked field reads as the mask on every row, never shows as "changed", and cannot be probed by the headline numbers. `columns[].masked` comes from `Entitlements.masked(principal, field)`. Hidden fields (no role may see them) are dropped from the columns entirely.
- **The cache holds unmasked rows** of this node's own days and never leaves the server; masks are applied per answer. Days a peer answered arrive masked for the caller and are kept under the caller's security signature only (section D11, SOAK2-01).
- **Peers** apply masks themselves (`NetDataController`) and the asking node applies them again, as type-ahead does ("masks twice"). A field that reads as the mask on any day, whoever masked it, is `masked: true` in `columns[]` and never in `changed` or the diff's changes, even where this node's own rules do not mask it (a peer's `redact` stricter than this node's). A field not masked for

the caller shows every real change.
- **Export** (section D15.2) goes through the same endpoint, so it is masked the same way; the console never masks.
- **Audit.** Each request is one access-log row (`history`), with the range in the detail, as `series` is today.

### D14. Performance targets

| Case | Target | Reasoning |
|---|---|---|
| 30 days of one trade, Delta, 10K trades a day, laptop, cold (first read since start) | **< 1 s** at the browser; < 400 ms at the server | `days()` reads nothing; 30 id maps of 10K ids are small; 30 row groups at about 20 ms cold, 8 at a time ≈ 80 ms; formatting and masks < 20 ms |
| the same, warm (documents in the doc cache) | < 150 ms at the server | Delta's warm single read is 4.3 ms median (DELTA_CONNECTOR.md); 30 / 8 × 4.3 ≈ 20 ms |
| the same, cached answer | < 30 ms at the server | the cache key needs only `dataVersion`, which is cheap by contract |
| 250 days, Delta, promoted fields only | < 1.5 s cold | column reads of one row group per day, no `doc` |
| 30 days, drishti-iceberg | < 400 ms cold | see below |
| 30 days, Cassandra | < 100 ms (estimate) | one `entity_days` read and 30 point reads of about 5 ms in parallel |

**Iceberg.** The Iceberg builder measured one trade at **54 ms** median (native engine, doc cache emptied; 94 ms for a past date not yet read) against Delta's **4.3 ms** warm (ICEBERG_CONNECTOR.md section 17 on the branch). Thirty single reads would cost 1.6 s one after another, or about 300–600 ms eight at a time, and each new day also reads its whole id column for the id map. History must not go that way on Iceberg: the planned scan (section D10) prunes to one file per day by the manifests' id bounds, reads no id map, and reads only the needed columns, so the per-day cost is a footer (cached by the native engine) and one row group.

The benchmark (section D19.4) measures these at 10K trades × 30 days and records them in CLUSTER_GUIDE.md's history row.

**Measured (2026-10-20, H14).** `HistoryBench` (`drishti-benchmarks`), 10,000 synthetic trades × 30 business days (2026-08-19 .. 2026-09-30, about 6% amended a day) written with Drishti's writer under `target/`, the native Delta engine, 200 random trades, the 30-day history at the service (`HistoryService.read`, columns, changes and all), laptop, one process, `-Xmx4g`:

| Case | Median | p95 | Target |
|---|---|---|---|
| cold (each trade's documents read for the first time, the table's id maps loaded) | **13.2 ms** | 19.1 ms | < 400 ms |
| warm (documents in the caches) | **0.83 ms** | 1.6 ms | < 150 ms |
| cached answer | **0.29 ms** | 0.49 ms | < 30 ms |
| default per-day loop (the connector's `days()` hidden), cold | 11.1 ms | 18.5 ms | |
| the Delta fast path again, cold, after the default loop | 8.5 ms | | |

The write took 7.6 s. **The fast path is 1.3× the default loop here, not the 3× of section D19.4**: on Delta the default loop already costs nothing for a date without the entity (its id map, cached, answers "not here" without a read), and in this table every date holds every trade, so both read the same 30 row groups. The fast path's gain is in the reads it never starts: dates the entity was not loaded, and stores where a per-date call is a network round trip. The first fast-path run, before the default loop, measured 13.2 ms against the default's 11.1 ms (an order effect: the file pages were cold for the first run), so the table reports both. Not measured yet: 250 days of promoted fields only (that read is not built, section D21), and the other stores (H6 and the rest of H5).

### D15. Extras: DIFF, export, CLI, BI, plain words

#### D15.1 `TRD X 30 DIFF`

`diff=true` adds `rows[].changes`, each row against the one before, with `DocumentDiff` (the "What changed" engine, nested paths included, `LIMIT` 2000 per row). The screen (`D`) becomes a change log: one block per changed day, each change on its own line (`LEGS[FIXED].NOTIONAL 10,000,000 → 12,500,000 +2,500,000`), days with no change left out. A day's block links to `/compare/{kind}/{id}?from=<previous>&to=<date>` for the full compare page.

#### D15.2 Export

| Route (console, `routes/export_routes.py`) | Gives |
|---|---|
| `GET /export/history/{kind}/{id}.csv?days=…&fields=…` | the table as shown: date, then the columns (display values), `changed` as a column; `to_csv` (BOM, CRLF, the formula guard) |
| `GET /export/history/{kind}/{id}.json?…` | the server's answer as it is (section D8) |
| `GET /export/history/{kind}/{id}.xlsx?…` | through `to_xlsx` (one sheet, bold frozen header, numbers stay numbers) |

Names follow `export.filename()`: `1000015-history-2026-09-08-2026-10-19.csv`. The specific routes are declared before `/export/{kind}/{id}/{panel}.csv`. Row caps are the server's `max-rows`.

#### D15.3 CLI

In `tools/drishti.py`, a new group `history` (argparse, `cmd_history_get(a, extra)`, the shared `server_parent()` options):

```
drishti.py history trade 1000015 --days 30
drishti.py history equity EQ-CSCA --range 3M --fields PRICE,CHANGE_1D
drishti.py history trade 1000015 --from 2026-09-01 --to 2026-09-30 --diff
drishti.py history trade 1000015 --days 30 --changes-only --json
drishti.py history trade 1000015 --days 30 --csv > trade-1000015.csv
```

Text output is a table, newest first, changed cells marked `*`; `--json` prints the answer (`say_json`); `--csv` writes the CSV of 15.2. Exit codes: 0, 1 (problems), 2 (usage), as the other groups. `tools/test_drishti_cli.py` gets its cases. The Python client (`clients/python/drishti_client.py`) gets `history(kind, id, days=…)`.

#### D15.4 A BI data source "history"

A dataset could chart a field over time: `source: {history: {kind: trade, ids: [1000015], fields: [PRICE], range: 3M}}`, giving the table `(business_date, id, PRICE, …)`. It needs a new `DatasetDef.Source` type `HISTORY`, `DatasetDefParser` (its own `DRS-91xx` code), `bi/mq/DatasetSources.answer()` and `fingerprint()` (the fingerprint is the history cache key), and `core/model_layer.py` in the console. The BI time machinery (`bi/dataset/Histories.java`, live-table history with a `ts` column) is the precedent. **It is specified here and built as step H12**, which is not built yet; nothing else depends on it.

#### D15.5 Plain words

`PhraseParser` (deterministic, no model) learns one more form before searches: *kind word, id, time phrase*. Time phrases: `last N days` → `N`, `last N weeks|months|years` → `NW|NM|NY`, `last month` → `FROM <first of the previous month> TO <its last day>`, `this month` → `MTD`, `this year`, `ytd` → `YTD`, `since <date>` → `FROM <date>`, `between <date> and <date>`, `from|since <date> to|until <date>` (two dates are put in order), `past 30 days`. The id is the word that names one entity of the kind (`Vocabulary.entityId`: its id in any case, or the one id ending `-<word>`, so `audusd` is `FX-AUDUSD`), even when it is also a field's value (`EQ-CSCA`, the ticker); else a word with a digit or a dash that is no field's value. Without a kind word, an id that names one entity of one kind (`Vocabulary.kindOfEntity`) followed by a range or a time phrase is that entity's history (`1000015 30`, `BOOK-RATES-3 last week`); followed by what reads as a range but is none it is `DRS-4022` with what to type (`1000015 -5`, `1000015 since yesterday`). A kind word and one id with no time phrase open that entity (`fx audusd` is `FX FX-AUDUSD`). A search reads one business date, so a time phrase in one (`trades in BOOK-RATES-3 last week`) is never dropped silently: it is listed as not understood, with the step *left out: a search reads one business date (the date box); for one entity over time name it with the time phrase alone (BOOK-RATES-3 last week)*, and the console does not run the search on its own. `since <date>` in a search stays a condition on the kind's date (`trades in BOOK-RATES-3 since 2026-01-01`).

```
trade 1000015 last month                 TRD 1000015 FROM 2026-09-01 TO 2026-09-30
equity EQ-CSCA last 3 months             EQ EQ-CSCA 3M
fx audusd since 2026-10-01               FX FX-AUDUSD FROM 2026-10-01
trade 1000015 since 2026-10-01 until 2026-10-10   TRD 1000015 FROM 2026-10-01 TO 2026-10-10
fx audusd                                FX FX-AUDUSD
1000015 30                               TRD 1000015 30
what changed on trade 1000015 this week  TRD 1000015 FROM 2026-10-19 DIFF
```

The console's `/go` already sends a failed command of two or more words to `/s?words=`; the search page shows "*Read as `TRD 1000015 FROM 2026-09-01 TO 2026-09-30` — Enter to open*", with the steps, as phrases are explained today, and opens it directly when the phrase was read fully (no ignored words).

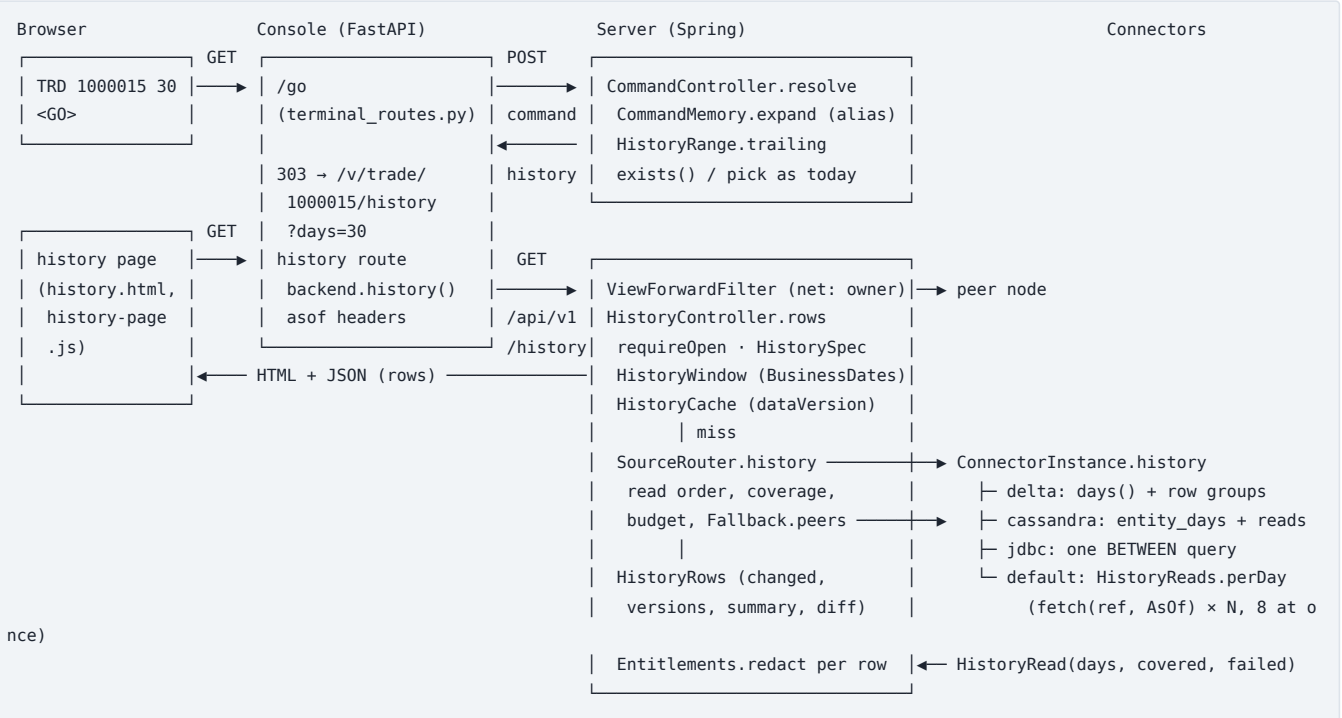
### D16. Errors

New codes in `drishti-common` `ErrorCode` (none of them used before):

| Code | Name | HTTP | When | Message (example) |
|---|---|---|---|---|
| `DRS-1012` | `HISTORY_NOT_KEPT` | 404 | the kind has no daily history: history off for it (`enabled: false`), or every connector of the kind is undated and no `series:` is declared | "equity keeps no daily history here: market-store holds today's data only" |
| `DRS-1013` | `HISTORY_NO_DATA` | 404 | the entity exists but has no data in the window | "TRD 1000015 has no data from 2026-01-02 to 2026-01-30; its days with data run from 2026-06-20 to 2026-10-19" |
| `DRS-1014` | `HISTORY_TIMEOUT` | 504 (or 200 with `partial`) | a connector, or a Drishti*net peer (an HTTP time-out of its one call), did not answer within the budget: the whole request when no day was read, otherwise a `problems` entry | "trade-endur did not answer 4 of 30 days within 10 s" |
| `DRS-2061` | `HISTORY_INVALID` | 422 | the pack's `history:` block is wrong | "history.equity.headline: PRICE2 is not one of chart" |
| `DRS-4021` | `HISTORY_RANGE_TOO_LARGE` | 400 | more than `max-days` (the kind's or the server's) | "366 business days at most (asked 500): narrow it with FROM … TO" |
| `DRS-4022` | `HISTORY_BAD_RANGE` | 400 | `TO` before `FROM`, a future `FROM`, an unreadable date | "FROM 2026-10-15 is after TO 2026-10-01"; "FROM 2026-10-30 is in the future (the business date is 2026-10-20)" |

A `knownAt` on a store without time travel stays `DRS-1007`; an unknown entity stays `DRS-1001`; a Drishti*net node that holds the kind and cannot be reached, refuses the connection or fails its read is `DRS-5052` naming it, only for the dates it may hold (section D12); a peer's time-out is `DRS-1014`. `DRS-1003` names only a source of the server that answered, so the console never sets a healthy server aside for another node's failure (CLUSTER_GUIDE.md 11.1). The server logs `DRS-5052`, `DRS-5053` and `DRS-1014` as one short WARN line per change of what they say about a node, with a count once a minute, never with a stack trace. Each new code goes into API_GUIDE's "Error code table" and TROUBLESHOOTING's "Error codes" table in the same push; `drishti-console/tests/test_docs_error_codes.py` fails otherwise (and checks `DRS-2061` in TROUBLESHOOTING). The console shows each on the history page as the view's `missing.html` does, with the fix: the largest allowed range as a link, the first day with data as a link.

### D17. The data flow




The same service answers `drishti.py history` (REST), the export routes, the field dialog (`/series`) and, later, the BI source.

### D18. Build steps H1..H14

Two builders in parallel. **Builder A**: server, SPI, connectors (Java). **Builder B**: command, console, CLI (Java in `engine/command` and `engine/search/PhraseParser` only, Python, JavaScript, CSS). The contract between them is section D8, fixed in H1 as a fixture file; neither touches the other's files. Each step is one push with its tests and docs; the drill runs once, at the end.

| Step | Builder | What | Done when (its own tests) | Docs in the same push |
|---|---|---|---|---|
| **H1** | A | The contract: `HistoryRequest`, `HistoryDay`, `HistoryRead`, `SourcePlugin.history` and `days` defaults, `HistoryReads.perDay`; `ConnectorInstance` delegation; the fixture `drishti-testkit/src/main/resources/history/trade-1000015-30.json` (and an equity one); `DatedSourceContract` history cases | every plugin that extends `DatedSourceContract` passes the new cases with the default | `docs/guides/DEVELOPER_GUIDE.md` (the SPI) |
| **H2** | A | `engine/history`: `HistoryProperties`, `HistoryWindow` (ranges, business days, baseline), `HistoryRows` (latest per day, `changed`, parts, summary, diff), `HistoryCache`, `HistorySpec` (+ `DRS-2061`), `HistoryService`; `SourceRouter.history` (read order, coverage, budget, partial) | unit tests (section D19.1) | CONFIGURATION (settings), API_GUIDE and TROUBLESHOOTING (codes) |
| **H3** | A | `HistoryController.rows` (`/{kind}/{id}` and `/~`), the new `ErrorCode`s, masks, `AccessRecorder.ACTIONS`; `series` re-pointed at the service (260 becomes `max-days`); `HistoryDefinitions` reading `history:`; deploy and `sutra lint` checks | controller tests with a fake router: masks per row, rights, every error, the JSON equals the fixture's shape | API_GUIDE endpoint, PACK_DEVELOPER_GUIDE `history:` |
| **H4** | A | Drishti*net: `PeerSource.history`, `NetDataController` `/net/history`, merge by date | in-process cluster test (two nodes, a kind's dates split across them) | OPERATIONS_GUIDE.md routing table |

| Step | Builder | What | Done when (its own tests) | Docs in the same push |
|---|---|---|---|---|
| **H5** | A | Fast paths: delta (with promoted-column row read and versions), jdbc table and query mode, duckdb, mongodb, aerospike, redis, file | the contract test per plugin with the fast path, plus "reads no document when the fields are promoted" for delta | each connector's document in `docs/connectors/` |
| **H6** | A | Fast paths on the merged branches: drishti-iceberg / iceberg-external (planned scan), cassandra (`entity_days`) | their contract tests; starts only after those branches are on develop | ICEBERG_CONNECTOR, CASSANDRA_CONNECTOR |
| **H7** | B | The command: `HistoryRange`, `CommandController.resolve` (range after alias expansion, before pick), `CommandResponse.history`, `SuggestionService` range suggestions | `HistoryRangeTest` (every row of section D4.2's table), `CommandControllerTest` cases | USER_GUIDE command line |
| **H8** | B | Console: `/v/{kind}/{id}/history` route, `/v/…?days=` redirect, `/go` redirect, `backend.history()`, `terminal/history.html`, `web/static/js/drishti_history-page.js`, `web/static/css/drishti_history.css`; document shape, keys, changes only, drawer, open the day | `FakeBackend.history` from the fixture; route and template tests; `test_contrast.py` passes | USER_GUIDE history section |
| **H9** | B | Market shape: chart, headline numbers, range buttons, crosshair, embedded series | template tests with the equity fixture | USER_GUIDE |
| **H10** | B | DIFF mode, export routes (csv, json, xlsx), share links and pick lists carrying the range | export tests (names, BOM, formula guard, masked values as served) | USER_GUIDE, API_GUIDE "console's own JSON routes" |
| **H11** | B | `drishti.py history`, the Python client's `history()`, `PhraseParser` time phrases | `test_drishti_cli.py` cases, `PhraseParserTest` cases | CLI_GUIDE, USER_GUIDE plain words |
| **H12** | A | BI source `history` (not built, section D15.4) | dataset parser and `DatasetSources` tests | BI_GUIDE.md |
| **H13** | A + B | The shipped packs' `history:` blocks (section D6.3); help card; tutorial 21; CHANGELOG | `sutra lint` of every shipped pack clean | PACKS, tutorial 21, help |
| **H14** | B, then both | Browser test and the benchmark (section D19.3, D19.4); then the release drill, once | both green under the 10K-trade cap | SCALE_STRATEGY history row, RELEASE_NOTES |

Order: H1 first (a day; it fixes the contract). Then A runs H2 → H3 → H4 → H5 (→ H6 when the branches land) and B runs H7 → H8 → H9 → H10 → H11 at the same time against the fixture. They meet at H13 and H14, when B's console points at A's real endpoint. At most two builders, one heavy test process each, as the house rule says.

## D19. Tests

#### D19.1 Unit

| Test | Checks |
|---|---|
| `HistoryRangeTest` (engine) | every row of section D4.2: bare number, `30D`, `2W`, `3M`, `1Y`, `YTD`, `MTD`, `QTD`, `FROM … [TO …]`, `DIFF`, case, `<GO>`; never the first word; ids with spaces; operators and `*` stay searches |
| `HistoryWindowTest` | data days vs calendar days over the business calendar (weekends, holidays), the baseline day, `max-days` from pack and server, `DRS-4021`, `DRS-4022` |
| `HistoryRowsTest` | latest version per day, snapshot lookback days skipped, effective days carried and marked, `changed` against the baseline, `ignore-changes`, parts counts, summary (high/low with dates, change %), diff rows equal `DocumentDiff`'s |
| `HistorySpecTest` | the schema of section D6.1, defaults, every `DRS-2061` problem, unknown keys as warnings |
| `HistoryCacheTest` | a new `dataVersion` misses; no version, no caching; size bound |
| `SourceRouterHistoryTest` | two connectors in read order split a range by coverage; peers asked only for the rest; a slow connector gives `partial` with `DRS-1014`; the budget is shared |
| `HistoryControllerTest` (server) | masks on every row (a masked field is never `changed` and not in the summary), `requireOpen`, every error code, the answer's shape equals the fixture's (field by field), `~` ids |
| console `test_history_routes.py` | `/go` redirects, `/v/…?days=` redirect, pick list with `days`, the template renders both shapes from the fixtures, missing data and `DRS-1012` pages |
| `test_history_export.py` | names, BOM and CRLF, the formula guard, the JSON as served |
| `tools/test_drishti_cli.py` | `history` text, `--json`, `--csv`, `--changes-only`, exit codes |

#### D19.2 Contract across plugins

`drishti-testkit` `DatedSourceContract` (extended today by the delta, jdbc (two), redis, aerospike, file, duckdb, iceberg (two), mongodb tests and the `DayFolderContractTest` example) gets history cases over its fixture `ROWS` (D1, D2, D3; `T-1` restated on D3):

- `historyOfASnapshotKindHasOneRowPerDateItHolds`: `T-1` from D1 to D3 gives 3 days, mtm 100, 110, 125; `T-3` gives D1 only.
- `historySkipsDatesAnsweredByAnEarlierDate`: a snapshot date with no partition is not a row.
- `historyOfAnEffectiveKindListsTheChangeDates`: `CP-X` gives D1 and D3 (`A`, `A-`).
- `historyAsksOnlyTheFields`: `fields={mtm}` gives documents with `mtm` (and nothing a mask would need is lost: the server masks before projecting).
- `historyDaysAgreeWithHistory`: when `days()` is present it lists exactly the dates `history` returns.
- `historyMatchesFetchDayByDay`: for every date, the day's document equals `fetch(ref, AsOf.of(d))`. This one keeps every fast path honest.
- `historyWithKnownAtFollowsTimeTravel`: when `timeTravel()` is true, a `knownAt` before D3's load gives `T-1` without `restated`.

Cassandra's and the new Iceberg plugins' tests extend the class when their branches land.

#### D19.3 Browser

`drishti-console/tests/test_history_browser.py` (Playwright, as the other `*_browser.py`), on the demo stack with the sample packs: type `TRD 1000015 30` in the command line, Enter; the page shows the dates in the header and 30 rows (or the days the sample holds); `↓ ↓` moves the selection (`aria-selected`); `C` hides unchanged days and the count in the header changes; `Enter` opens the view of that day with the pinned banner; `Esc` returns. `EQ EQ-CSCA 3M` draws the chart and the headline numbers; `1` switches to 1M and the URL follows. Repeated in `light` and `terminal` themes. It waits for the page's own "loaded" marker (`data-history-ready`), never fixed sleeps, so the test is deterministic.

#### D19.4 Performance

`drishti-benchmarks` `HistoryBench`: **10K trades × 30 business days** in a Delta table under the build's temporary folder (never `data/`, never more than 10K trades locally), cleaned at the end. It measures, for 200 random trades: cold and warm 30-day history at the service, the cached answer, 250 days with promoted fields only, and the default loop against the fast path (the fast path must be at least 3× faster cold). It fails when the warm median passes 150 ms or the cold p95 passes 400 ms. One heavy job at a time.

*Implementation:* a plain `main` (not JMH), run as `java -Xmx4g -cp <the module's class path> com.ash.drishti.benchmarks.HistoryBench [trades=10000] [days=30] [sample=200]`; it refuses more than 10,000 trades, deletes its folder at the end and exits 1 on a missed target. The numbers are in section D14.

### D20. Docs to update when it is built

| Document | What |
|---|---|
| `docs/guides/USER_GUIDE.md` › The command line | the range after the id (section 4), the history page, keys, changes only, DIFF, plain words |
| `docs/guides/API_REFERENCE.md` | `GET /api/v1/history/{kind}/{id}` with the JSON of section 8; `/net/history`; the codes in "Error code table"; the console's export routes |
| `docs/guides/CLI_REFERENCE.md` | `drishti.py history` |
| `docs/guides/BUILD_GUIDE.md`, `docs/guides/BUILD_GUIDE.md` | the `history:` block (section 6), with the shipped packs' blocks as examples |
| `docs/admin/CONFIGURATION.md` | `drishti.history.*` (test_docs_settings.py) |
| `docs/guides/OPERATIONS_GUIDE.md` | the six codes, each with its fix (test_docs_error_codes.py) |
| `docs/guides/OPERATIONS_GUIDE.md` | `/net/history` in the routing section |
| `docs/guides/DEVELOPER_GUIDE.md` and each connector's document | the SPI methods and each connector's fast path |
| help cards | `drishti-console/config/help.yaml` (a "History" guide under the command line category) and the trading and market-data packs' `config/help.yaml` |
| `CHANGELOG.md`, `RELEASE_NOTES.md` | each step's line |
| `docs/guides/CLUSTER_GUIDE.md` | the history row with the new measurements |
| this document | implementation notes, as APPROVALS_GUIDE.md and CLUSTER_GUIDE.md keep them |
| **tutorial 21** `docs/tutorials/01-first-15-minutes.md` | hands-on and complete: (1) `TRD 1000015 30` in the UI, the same with `drishti.py history trade 1000015 --days 30`, and the `history:` YAML that shapes it; (2) changes only and DIFF (UI, `--changes-only`, `--diff`); (3) open a day (UI; `drishti.py view get trade/1000015 --date …`); (4) `EQ EQ-CSCA 3M`, the chart and the numbers (UI, CLI, YAML `chart:`, `headline:`); (5) add `history:` to your own pack and `sutra lint` it (YAML, CLI, Admin → Packs); (6) export (UI, `--csv`); (7) plain words; (8) limits and errors (`max-days`, `DRS-4021`). Every step shows the UI, the CLI where there is one, and the YAML, with screenshots in `img/` |

### D21. Where the code forced a change from the agreed design

| Agreed | In this design | Why |
|---|---|---|
| `GET /api/v1/entities/{kind}/{id}/history` | `GET /api/v1/history/{kind}/{id}` | `HistoryController` already owns `/api/v1/history/{kind}/{id}/series` and `/diff`; `ViewForwardFilter`, `AccessRecorder` and `NetRouting` already treat the `history` area; `/entities/{kind}/{id}/raw` is the raw document. One area for history keeps forwarding, audit and rights in one place. |
| link `/v/trade/1000015?days=30` | canonical `/v/trade/1000015/history?days=30`; the short form redirects to it | the view route `/v/{kind}/{id_:path}` renders panels; history is its own page (as `/about` and `/panels` are sub-routes); the short link still works |
| the parser takes a third word | the range is taken off **after** the alias is expanded and **after** the whole rest is tried as an id | `CommandParser` reads everything after the mnemonic as the id, and ids may hold spaces; `CommandMemory.expand` appends trailing words |
| `history(kind, id, from, to, fields)` | `history(HistoryRequest)` returning `HistoryRead`, plus `days(ref, from, to)` | the read also needs `knownAt`, "the last N days with data", a budget, and must say which dates it covers so the router can ask the next connector (Redis in front of Delta); `days()` lets Delta, Cassandra, Aerospike, Redis and files list dates without reading |
| a row cap and a timeout | also `parallel-days` and `max-concurrent` | the existing series fires every day's read at once with no bound |
| market kinds from daily documents | also an embedded `series:` | the shipped market samples keep their history inside today's document (`HISTORY: [{DATE, CLOSE}]`) |
| Cassandra: `entity_days` gives the days | it gives the days; the documents still need one point read per day | `business_date` is part of the data table's partition key |
| default columns: top-level scalars | key columns first, empty columns hidden | a trade has about 45 top-level scalars (`config/packs/banking/trading/samples/trade.jsonl`) |
| *Implementation (H1):* `HistoryRequest(ref, from, to, fields, knownAt, lastDays, budget)` | `HistoryRequest(ref, from, to, dates, fields, knownAt, lastDays, budget, parallelDays)` | a plugin has no calendar, so the business dates travel in the request (`dates`, empty means every calendar day); the default loop's parallelism is the server's `parallel-days`, so it travels too |
| *Implementation (H5):* Delta: `days()`, a promoted-column row read, and `versions` from the log | `days()` only | the demo came first; with `days()` a history reads only the dates that hold the entity. The column-only read (no `doc` decode) and the per-day version count are still to build, so Delta's `versions` is `null` |
| *Implementation (H3):* effective kinds: every business day a row, carried days marked | the change days only | a source's history leaves out a date answered by an earlier date's document, so the rows of an effective kind are its change dates; `carried` is always false. Filling the days between is a later step |
| *Implementation (H14):* the fast path at least 3× the default loop | 1.3× on Delta | the default loop is already cheap on Delta (section D14) |
| *Implementation (H3):* `series:` only when no source keeps daily documents | also to fill the days before the oldest stored day; `window.loadedFrom` | the demo cluster's market lake keeps 7 days: `EQ EQ-CSCA 3M` drew 7 points and said "no data on 55 days" |
| *Implementation (H3):* `window.to` is the date box | `data-days` ends at the newest day with data; `loadedTo` added | found in the first integration run: a lake loaded up to Friday, read on Tuesday, said "no data on 2 days" for Monday and Tuesday, which are only not loaded yet |
| *Implementation (H1):* `window.missing` only for `calendar` and `between` | filled in every mode | the document screen's header lists the days with no data for a bare number too (section D5.2: "no data on 2026-09-14"); the fixture's 30 rows span 31 business days |

### D22. Open questions

All five were decided by the product owner on 2026-10-20, before the build started.

1. **"As first reported."** Should a mode show each day as it was known at the end of that day (bitemporal, for audit), beside "latest" and "known at K"? It needs one time-travel layout per day on Delta and Iceberg: affordable for 30 days, not for 366. **Decided: later.** Today there is "latest" and "known at K" only.
2. **Derived kinds** (`counterparty-trades`, `portfolio-trades`): their pages are rebuilt from every member trade per day. History is off for them in the shipped packs; turn it on later with a per-day page cache? **Decided: off in the shipped packs** (`enabled: false`); a per-day page cache is not planned yet.
3. **The Build workbench:** a History tab to edit `history:` with a preview, now or with write-back's forms? **Decided: with write-back's forms.** The block is edited as YAML and checked by `sutra lint` and the deploy check.
4. **Ranges over a pick list** (`TRD desk=rates 30`: many trades over time) are BI's job; keep them out of the command for good? **Decided: left to BI** (the `history` data source, section D15.4). The command takes a range for one entity only.
5. **Tutorial numbering:** the ask names tutorial 1, Part E; 20 is not used on develop today. Keep 21? **Decided: 21** (`docs/tutorials/01-first-15-minutes.md`).

## D3. drishti-iceberg: Apache Iceberg on iceberg-core, with no Hadoop

Status: **built.** Modules `drishti-lakeio`, `drishti-iceberg`, `plugins/drishti-plugin-iceberg`. The Iceberg protocol comes from Apache Iceberg's own `iceberg-core`; Drishti adds thin wrappers for file access and speed. There is one engine: nothing chooses between engines, and Hadoop is not on the run-time class path, so the connector, loader and maintenance run on Windows with no `winutils.exe`.

| Layer | From where | What it does |
|---|---|---|
| Table metadata, snapshots, schema and partition evolution | `iceberg-core` (`TableMetadata`, `TableMetadataParser`) | format versions 1, 2 and 3 |
| Manifests, planning, delete-file matching | `iceberg-core` (`ManifestFiles`, `TableScan`) | Iceberg's own pruning and sequence-number rules; parsed manifests kept by Iceberg's manifest cache |
| Commits: append, overwrite, rewrite, expire, schema and properties | `iceberg-core` snapshot producers | conflicts retried by `commit.retry.*` |
| Catalogs | `RESTCatalog` from `iceberg-core`, SigV4 from `iceberg-aws`; `PathTableOps` for path-based tables | REST (OAuth2, vended credentials, Glue, Unity, Nessie, Polaris), S3 Tables, static metadata files |
| File access | Drishti: `DrishtiFileIO` over `java.nio` and the AWS SDK | local disk (Windows included), S3, `mem://`; `relocate` for a table copied elsewhere |
| Rows | Drishti's Parquet reader and writer, cached footers | read by field id, with position and equality deletes |

`drishti-iceberg`'s `Schema`, `Type`, `DataFile` and `FileTask` are plain values the Parquet reader and writer work in, and `Bridge` converts them from Iceberg's objects. `Transform` and `Pruning` remain as the reader's own partition pruning for `iceberg-external`.

**Path-based tables** keep the `HadoopTables` layout (`metadata/vN.metadata.json`, `version-hint.text`), so existing tables and tables other engines wrote work unchanged. A commit writes `vN+1` as one exclusive create (a hard link on local disk, a conditional PUT on S3), so one of two writers wins and Iceberg retries the other. Stores that do not honour conditional writes are single-writer, detected by a probe at the first commit. A table a catalog wrote (`NNNNN-<uuid>.metadata.json`) is read at its newest file.

**Format version 3, in plain words.** A v3 table is read when it uses nothing v3 adds. Deletion vectors (Puffin), `variant`, `timestamp_ns` and geospatial columns are refused by name, never read wrongly. Drishti's writers create v2 tables.

**Stores.** Local disk, `s3://`, `s3a://`, `mem://`. Azure (`abfs://`), Google Cloud Storage (`gs://`) and HDFS are refused by name; reach them through a REST catalog vending S3-compatible access, or add a `FileIO` for them.

**The `engine` setting and `DRISHTI_ICEBERG_ENGINE` are ignored.** A connector file that sets them keeps working, and the loader and maintenance still accept `--engine`.

**Proof.** `IcebergSpecInteropTest` runs both ways against the Apache Iceberg library (kept in test scope only, with Hadoop): the library writes every transform with every special partition character, position and equality deletes, and Drishti reads every row; Drishti writes every special partition character and the library reads every row. `PyicebergReadsNativeTest` and `NativeReadInteropTest` do the same against pyiceberg.

### D15. Two connectors: Drishti tables and external tables

The product owner (2026-10-20): Drishti needs **a connector for Iceberg tables Drishti writes and one for Iceberg tables it does not write**. Both run on `drishti-iceberg` (the `iceberg` connector also on the `library` engine; `iceberg-external` refuses it); they differ in what they expect of a table.

| | `iceberg` (Drishti tables) | `iceberg-external` (anyone's tables) |
|---|---|---|
| Tables | written in Drishti's layout by `IcebergLoader` (ICEBERG_CONNECTOR.md section 3): `id`, `doc`, `business_date`, promoted columns, sorted by id | written by Spark, Flink, Trino, Snowflake, Glue, dbt, pyiceberg, in any layout |
| The document | the `doc` column, exactly as loaded | **the row**, as JSON (every column; structs as objects, lists as arrays, maps as objects), or a JSON column named by `document: column:<name>` |
| The id | the `id` column | one column or several (`id: [SOURCE_SYSTEM, TRADE_ID]`, joined by `id-separator`, `:`); by default the table's **identifier fields** (Iceberg v2's `identifier-field-ids`) |
| The business date | `business_date` | a date column, or a timestamp column with `date-from: day` (in `date-zone`); by default the partition spec's date (`identity` of a date, or `day(ts)`); none: the table is **current only** (every read is the current snapshot) |
| Searchable fields | the promoted columns the pack declares | **every primitive column** (nested ones flattened to `depth`, 2): the table is already columnar, nothing to promote |
| One entity | id map or range map: one row group | pushed-down filter on the id columns with partition, metric and row-group pruning; for a table **not sorted by id**, an **id index** built by Drishti on first use per snapshot and day (id → file and row group, bounded by `id-index-mb`), so later reads are one row group |

| | `iceberg` (Drishti tables) | `iceberg-external` (anyone's tables) |
|---|---|---|
| Writes | the loader and maintenance | **never**: the connector only reads; the owners' engines maintain the tables |
| Deletes, time travel | as section D7 | the same: merge-on-read position and equality deletes from Spark and Flink are applied; *known at* uses the snapshot log |

The external connector's reference, which this section once held in full (configuration, rows as documents, reading at scale, settings, tests), is one document: ICEBERG_EXTERNAL_CONNECTOR.md (plugins/drishti-plugin-iceberg-external/README.md). What was 15.1 *Configuration* is its section D2 (plugins/drishti-plugin-iceberg-external/README.md#examples) and section D8 (plugins/drishti-plugin-iceberg-external/README.md#discovery-tables-auto) (`tables: auto`); 15.2 *Rows as documents* is its section D3 (plugins/drishti-plugin-iceberg-external/README.md#data-structure) and *Make a pack* its section D9 (plugins/drishti-plugin-iceberg-external/README.md#make-a-pack); 15.3 *Reading at scale* is its sections D5 and D6 (plugins/drishti-plugin-iceberg-external/README.md#one-entity-at-scale); 15.4 *Settings* is its section D10 (plugins/drishti-plugin-iceberg-external/README.md#settings) (generated settings: the plugin's README (plugins/drishti-plugin-iceberg-external/README.md)); 15.5 *Tests* is its section D12 (plugins/drishti-plugin-iceberg-external/README.md#how-it-is-tested).

### D16. Vector columns and orphan files

**`list<double>` columns.** The layout's vector column (`layout.<kind>.vectors`, a P&L vector of 509 values) is an optional `list<double>` whose elements are optional: field ids are assigned after the promoted columns, the column's then its element's. The writer (`ParquetDataWriter`) writes a three-level list; a null element is written as an empty element group, so it reads back as null (never 0). The reader's list converter (`Converters.ListConverter`) gives a `List` with the nulls in place; the connector turns it into a `double[]` with NaN for null (ICEBERG_CONNECTOR.md section 18 (plugins/drishti-plugin-iceberg/README.md#parity-with-delta-vectors-and-disk-caches)). Adding a vector column to an existing table (`Operations.addColumns`) gives the column the next id and its element the one after, and records the higher as `last-column-id`. Delta stays the default lake format; this is the Iceberg side of the same column.

**Orphan files.** A load that dies after writing data files but before its commit leaves files no snapshot references, and Iceberg's expiry only removes the files of the snapshots it expires. `OrphanFiles` (in the plugin, run by `IcebergMaintenance --remove-orphans`) lists every file under the table's location first, then reads the references: the current metadata file and every one its `metadata-log` lists, `version-hint.text`, each snapshot's manifest list, every manifest and every data and delete file they name (any entry status, so a file a kept snapshot may read is never taken) and the statistics files. A file not referenced and older than `--older-than-hours` (72 at least: longer than the longest load) is deleted; `--dry-run` lists them. Reading the references after the listing means a commit landing in between only adds references; the age keeps a load in flight safe. The sweep reads the table with drishti-iceberg whichever engine wrote it, on local disk and in S3 alike (`IcebergOrphanFilesTest`).

### Connector and data-source review (standards)

Other connectors, reviewed against the rule that each format or protocol comes from its official library. Open items, most serious first. **L** needs the product owner's decision.

| # | Where | Finding | Size / recommendation |
|---|---|---|---|
| 1 | every connector with promoted columns; `QueryMode`, `JsonlFormat` | Promoted columns are doubles: numeric ids above 2^53 and decimals lose precision | **L**: doubles for analytics columns only; exact values in documents; warn at load when a value does not round-trip |
| 2 | `redis` `ColumnCodec`, `DocCodec`; `cassandra` `CassandraLayout` | Bespoke binary column format and hand-written Murmur3 | **L**: decide whether the standards rule covers internal storage; if so Arrow IPC with zstd and a library Murmur3 |
| 3 | `kafka` `SchemaRegistryClient`, `ConfluentValueDeserializer` | Hand-written registry client; Protobuf unsupported | **L**: Confluent's client if its licence suits, else keep this thin one |
| 4 | `jdbc` `QueryMode.plain` | `TIMESTAMP WITHOUT TIME ZONE` depends on the JVM zone | small: `getObject(i, LocalDateTime.class)` |
| 5 | `jdbc` `QueryMode`, `JdbcWrites` | Two hand-written named-parameter parsers miss comments and `$$` bodies | small: one tokenizer with tests |
| 6 | `file` `CsvReader`, `feeds` `Feeds` | Hand-written CSV parsers (embedded newlines, `""` escapes, leading zeros) | small: Apache Commons CSV, or remove the file CSV path (already deprecated) |
| 7 | `news` `RssParser` | Regex HTML stripping; RFC 822 zones such as `EST` yield a null date | small: jsoup and a zone-tolerant parser |
| 8 | `rest` `RestSourcePlugin` | 3xx and 204 reach the JSON parser | small: treat as errors or follow |
| 9 | `duckdb`, `file` `JsonlDay` | JSON needle not encoded: ids with quotes or backslashes miss | small: Jackson-encoded needle |
| 10 | `jdbc` `EntityTable`, `PostgresLoader` | Unquoted identifiers; COPY escape without NUL handling | small: `enquoteIdentifier` and the NUL case |
| 11 | `feeds` `Feeds` | ECB CNY relabelled CNH; par yield published as `zeroRate`; unencoded URL parameters | small: per item |

## D4. Reactive connectors

Status: **R1 to R4 built and built (2026-10-26); R5 and R6 planned, in no release yet.** Designed with the product owner (2026-10-24). What is built, and where it differs from the design below, is in section D21; the reference for people who write and run reactive connectors is the reactive connector's README (docs/connectors/types/reactive/README.md) and tutorial 19, Part C (docs/tutorials/19-build-a-pack-and-a-screen.md#part-c--computed-panels-a-reactive-connector). Everything named below that section D21 does not list as built is **planned**. Related designs: BI_GUIDE.md (the workers that run the computations, `requires:` and approval), VAR_GUIDE.md Part III (the first reactive connector: a generic VaR engine), APPROVALS_GUIDE.md (a saved change can trigger a computation), RACHANA_REFERENCE.md (screens that read many sources), section D1 (how results reach the screen), DEVELOPER_GUIDE.md (the connector SPI).

### D1. The ask, in one paragraph

Today a connector brings data that already exists somewhere: a lake, a database, a queue, a web service. The product owner wants connectors that **compute** their answer: a VaR for any group of trades, a what-if price for a trade that is not booked, the effect of an amendment on a desk's risk. The computation runs **when someone asks**, and **again when something changes** (a saved edit, a message on Kafka, a schedule). Packs stay exactly as they are (kinds, routes, screens); the connector carries the execution and the libraries it needs. A screen can combine as many connectors as it likes: stored data, live data and computed data side by side, all for the same business date, and each panel says on its own if its source failed.

### D2. Decisions this design rests on

| # | Decision (product owner, 2026-10-24) | What it means here |
|---|---|---|
| 1 | **Packs stay as they are.** "Connector can be reactive, on demand, static, whatever it is." | No computation and no library is declared in a pack. A pack names kinds, routes each kind to a connector and draws screens. |
| 2 | **A connector may be static, live, on-demand or reactive.** | One `mode:` per connector file (section D3). |
| 3 | **A pack combines results from as many connectors as it wishes.** | Per-panel `source:` already exists (RACHANA_REFERENCE.md › Sutra Next: fragments, events, children, sources, layout and forms, section D6); this design adds parameters, one pinned as-of per screen, and per-panel failure (section D12). |
| 4 | **Reactive connectors compute on request and on events** (a write-back, a Kafka change, a schedule). | `triggers:` (section D7) with coalescing, incremental recompute, point-in-time stamps, ordering, idempotence and back-pressure. |
| 5 | **Two users' write-backs or requests never mix their responses.** | A correlation id on every request and event, per-user sandboxes, cache keys that include the viewer's rights, masks applied per recipient (section D9). |
| 6 | **The connector declares `requires:`** (Python, Java, plugins); an administrator approves before use. | The block, the locked environments and the approval are in BI_GUIDE.md. |
| 7 | **Python runs out of process, Java stays on 21, no FFI, no embedded CPython.** | A reactive connector's code runs in a worker process, never inside the server (BI_GUIDE.md). |

### D3. Four kinds of connector

| Mode | What `fetch` means | Data lives | History (a picked business date) | Pushes changes | Examples today |
|---|---|---|---|---|---|
| **static** | read stored records | in the store, written by a loader | yes, from the store's dated partitions | only when a new load is published (the data version moves) | `delta`, `iceberg`, `jdbc`, `duckdb`, `cassandra`, `file` |
| **live** | read the latest value and subscribe to its changes | in a stream or a cache fed by one | through a lake behind it, if any | yes, every message | `kafka`, `rabbitmq`, `activemq`, `redis` in subscribe mode |
| **on-demand** | call a service; the id is the question | nowhere: the answer is fetched or worked out per call, then cached | only if the service can answer for a date | no | `news`, `yfinance`, `rest` |
| **reactive** (new) | **run a computation** parameterised by the request, over data read from other connectors | the result is computed, stamped and cached; nothing is stored unless the connector says so | yes: the computation is run on the inputs **as of** the picked date | **yes: the result is recomputed and pushed when a declared trigger fires** | planned: `var-engine` (VAR_GUIDE.md Part III) |

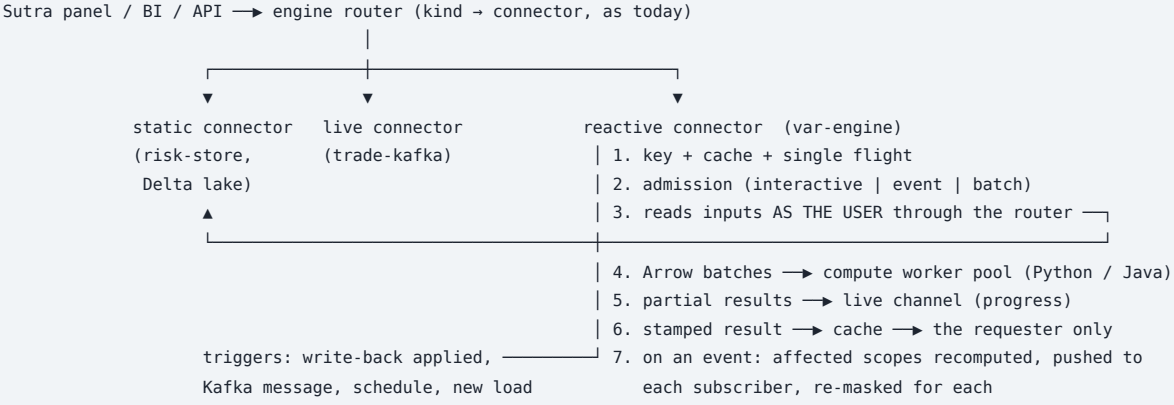
- The first three exist today and do not change. Writing `mode:` in their connector files is optional (the plugin's manifest already says what it can do: `live`, `search`, `dated`); a planned `mode` field in the manifest makes it visible in Admin → Connectors and in the connector type's README front matter (`capabilities:` gains `compute`).
- An **on-demand** connector is a reactive connector without triggers, without parameters beyond its id, and without workers: it stays the light pattern of the developer guide (docs/guides/DEVELOPER_GUIDE.md#1011-connectors-that-compute-data-on-demand). A reactive connector is for computations that are heavy, parameterised, need their own libraries, or must react to events.

### D4. What a reactive connector is

A reactive connector is **a connector whose answer is a computation**. It has five parts:

| Part | Says | Section |
|---|---|---|
| **kinds** | the kinds it answers (`var-result`, `var-contribution`, `what-if` ...), each with its parameters and the shape of its result | 5 |
| **inputs** (`reads:`) | which kinds of which connectors it reads, always **as the requesting user** | 6 |
| **engine** | where the code runs: a Python worker, a Java worker, or (trusted, light code only) in the server's own process | BI_GUIDE.md |
| **requires** | the libraries the engine needs, pinned, approved by an administrator | BI_GUIDE.md |
| **triggers** | the events on which results are recomputed and pushed | 7 |

Everything that already consumes connectors works on it without change: a Sutra panel's `source:`, BI datasets, history queries, entitlements and masks, Drishti*net routing, Admin → Connectors and Health, the README standard. That is why the execution unit is a connector and not a new concept.




### D5. The connector file

A reactive connector is a connector file in `config/connectors/` like any other (the file name is its name). The plugin is the generic planned `reactive` plugin; what is specific to the computation is configuration plus the code the `engine:` names.

```
# config/connectors/var-engine.yaml                          (planned)
plugin: reactive
mode: reactive
enabled: true
description: Generic VaR, ES and what-if over precomputed 509-day P&L vectors
kinds:
  var-result:                          # one answer: VaR, ES and the headline numbers for one scope
    params:
      scope:      { type: scope, required: true }          # a hierarchy node, a saved basket, or a scope expression (VAR_GUIDE.md
section 5)
      confidence: { type: decimal, default: 0.99, allowed: [0.95, 0.975, 0.99] }
      horizon:    { type: int, default: 1, allowed: [1, 10] }
      method:     { type: enum, default: historical, options: [historical, age-weighted, filtered] }
      whatif:     { type: sandbox-ref, required: false }   # a private what-if sandbox of the requester (section 9)
    result: { schema: schemas/var-result.v1.json }
  var-contribution: { params: { scope: { type: scope, required: true }, by: { type: enum, options: [child, position, risk-class] }
} }
  var-path:         { params: { scope: { type: scope, required: true } } }     # the 509 scenario P&Ls
  var-history:      { params: { scope: { type: scope, required: true }, days: { type: int, default: 250, max: 750 } } }
  what-if:          { params: { sandbox: { type: sandbox-ref, required: true } } }
reads:                                 # the inputs, always read as the requesting user, through the normal routing
  - { kind: pnl-vector }               # served by the static connector risk-store (Delta)
  - { kind: scenario-set }
  - { kind: trade }                    # only for what-if: the trade to amend
  - { kind: pnl-hypothetical }         # backtesting
engine:
  type: python                         # python | java | in-process (trusted Java only)
  entry: drishti_var.connector:VarEngine
  pool: risk                           # a worker pool of config/compute/compute.yaml (BI_GUIDE.md)
requires:                              # BI_GUIDE.md section N4; nothing runs until an administrator approves it
  python:
    version: "3.12"
    lock: requires/var-engine.lock     # pinned wheels with sha256
  java: []
triggers:                              # section 7
  - on: write-back
    kinds: [trade]
    affects: "scopes_of(trade.TT_BOOK)"          # the book and every node above it
    recompute: [var-result, var-contribution]
  - on: data-version
    kinds: [pnl-vector]
    affects: all
    recompute: [var-result, var-contribution, var-history]
  - on: schedule
    cron: "0 */15 7-19 * * MON-FRI"
    zone: America/New_York
    affects: "subscribed"                        # only scopes someone is watching
    recompute: [var-result]
coalesce: { quiet: 250ms, max-wait: 2s }         # section 10
cache:   { results: 20000, bytes: 512MB, ttl: 8h, store: disk }  # section 11
limits:  { timeout: 30s, timeout-batch: 10m, max-inflight-per-user: 4, max-scope-positions: 5000000 }
history: { dated: true }                         # a picked business date computes on that date's inputs
```

- **The pack does not change** except for routes: `var-result: var-engine` beside `pnl-vector: risk-store`, as for any kind.
- **`reads:` is checked at load:** each kind must be routed to a connector; a reactive connector may read another reactive connector, but a cycle is refused at load with the cycle named.
- **`params:` is checked twice:** at load (types, defaults, allowed values) and per request (a value outside `allowed` is refused in words: *confidence 0.9 is not allowed for var-result; allowed: 0.95, 0.975, 0.99*).
- **`result:` names a JSON schema** in the connector's folder of supporting files; every result is validated against it before it is cached or sent. A result that does not match is an error of the connector, never shown as data.
- **Off until approved.** A reactive connector whose `requires:` is not approved loads as *waiting for approval*: Admin → Connectors says so, Health says `DOWN: requirements not approved`, and panels reading its kinds say *This computation is not approved yet*.

### D6. The request and its lifecycle

#### D6.1 The request

The connector SPI's "the id is the question" is kept, but a reactive kind takes **typed parameters** too. A Sutra panel names them with a planned `params:` key on `source:`:

```
panels:
  - use: kpi-tiles@1
    with: { id: headline, rows: $src }
    source:
      kind: var-result
      id: "'DESK:' + $.DESK_ID"                      # the scope, written as an id: readable, linkable, cacheable
      params: { confidence: ctl.confidence, horizon: 1, method: ctl.method }
      as-of: ctl.asof
```

Inside the server every read of a reactive kind becomes one **compute request**:

```
{
  "requestId": "req-01JB4C2Q…",               // ULID, the correlation id: the answer goes back to this and nothing else
  "connector": "var-engine", "kind": "var-result",
  "id": "DESK:RATES-NY",
  "params": { "confidence": 0.99, "horizon": 1, "method": "historical" },
  "asOf": { "businessDate": "2026-10-23", "knownAt": null, "pin": "pin-7f3c…" },     // section 8
  "viewer": { "user": "asha", "roles": ["risk"], "viewSignature": "vs-91a2…" },      // section 9
  "origin": { "type": "panel", "screen": "desk/var-explorer", "panel": "headline" }, // or trigger, api, bi
  "class": "interactive"                     // interactive | event | batch (section 10)
}
```

#### D6.2 The lifecycle

| Step | What happens | Where |
|---|---|---|
| 1. **Receive** | The router sends the read to the connector the kind is routed to, exactly as today. | server |
| 2. **Authorise** | The viewer's roles may open the kind; the viewer holds the planned `compute` power for computed kinds; the scope is within the viewer's entitlements (VAR_GUIDE.md section 9). Refused in words, before any work. | server |
| 3. **Pin the as-of** | The request's as-of is resolved to concrete input versions (the Delta table version of `pnl-vector` for the date, the edit-log watermark), once, and recorded as a **pin** (section D8). | server |
| 4. **Key** | The canonical key: connector, kind, id, normalised params, pin, the viewer's **view signature** (section D9), engine version, environment lock hash. | server |
| 5. **Cache and single flight** | A cached result for the key is answered at once. A request for a key already being computed **joins** that computation (one computation, many waiters, each answered under its own request id). | server, then the key's owner node (section D14) |
| 6. **Admit** | The request takes a slot in its class's queue; a full queue answers *busy, try again* (interactive) or waits (event, batch) (section D10). | server |
| 7. **Read the inputs as the user** | Each `reads:` kind is read through the normal router **as the requesting viewer**: kinds, row entitlements and field masks applied before a byte reaches the computation. Bulk inputs travel as Arrow (the existing `scan` SPI). | server |
| 8. **Compute** | The request, the inputs and a scratch folder go to a worker of the connector's pool. The worker never fetches data itself and holds no credentials. | worker (BI_GUIDE.md) |
| 9. **Progress** | A long computation sends partial results (*desks 7 of 12 done*, a first estimate) on the requester's live channel; the panel paints them, marked *computing 58%*. | worker → server → console |
| 10. **Stamp, validate, cache** | The result is validated against the kind's schema, stamped (pin, computedAt, engine and environment), and cached under its key. | server |
| 11. **Answer** | The result goes to the request id that asked, and to every request id that joined it, each masked for its own viewer. | server |
| 12. **Audit** | Who asked what, with which pin, how long it took, which result hash (section D15). | server |

**Timeout.** An interactive request has a budget (`limits.timeout`, 30 s by default). If the panel's own wait (the view's link budget, `drishti.graph.link-budget`) passes first, the panel is drawn *computing* and fills itself from the live channel when the result lands; the computation is not abandoned because a page was drawn.

**Cancel.** A request is cancelled when every waiter has gone (the panel closed, the page left, a new selection replaced it). A computation already running in a worker is stopped only if it is interactive and nobody else joined it.

### D7. Triggers: computing on events

A reactive connector recomputes **without being asked** when a declared trigger fires, and pushes the new result to the screens that watch it.

| on: | Fires when | The event carries | Typical use |
|---|---|---|---|
| `request` (always on) | a panel, a BI query or an API call reads the kind | the request | every computed kind |
| `write-back` | a change is **applied** in the edit log (APPROVALS_GUIDE.md › The design and its reasons, section 18) | change id, kind, entity, lane, seq, by, the fields changed | an amended trade recomputes the VaR of its book and every node above it |
| `live` | a live connector delivers a message for one of the listed kinds (Kafka, MQ, Redis) | kind, id, the source's offset or sequence | a trade booked in the source system recomputes its book's VaR intraday |
| `data-version` | a static connector's data version moves for a kind and date (a new lake load is published) | kind, business date, old and new version | the overnight vectors land: every cached result of that date is stale, the watched ones recompute |
| `schedule` | a cron expression, in a named zone | the tick time | refresh the watched scopes every 15 minutes in trading hours |

#### D7.1 From an event to the scopes it affects

Each trigger says which results it affects with `affects:`:

- `all`: every cached result of the connector for the event's business date is marked stale; those somebody watches recompute.
- `subscribed`: only results some open screen is watching.
- an **expression** over the event (Rachana-EL, as panels use): `"scopes_of(trade.TT_BOOK)"` returns the keys the event touches. The function `scopes_of` is supplied by the connector (for VaR: the book and its ancestors in the hierarchy, plus every saved ad-hoc scope that contains the trade; VAR_GUIDE.md section 8.4).

Only affected results that **somebody is watching** are recomputed at once; the others are only marked stale and recompute on their next read. A trigger never computes for a screen nobody has open.

#### D7.2 What a trigger may and may not do

- It recomputes **shared** results (a book's VaR as seen by the risk role) and pushes them; it never recomputes a user's private what-if sandbox, which changes only when its owner acts.
- It runs under the **connector's service identity** for reading inputs that every subscriber shares, and the result is then **re-masked per subscriber** before it is pushed (section D9.3). A subscriber whose rights produce a different view signature gets a result computed for that signature (one per distinct signature, not one per person).
- It is **idempotent**: an event already processed (same change id, same Kafka offset, same schedule tick) is ignored (section D8).

### D8. Point-in-time stamping, ordering and idempotence

#### D8.1 The stamp on every result

```
"stamp": {
  "businessDate": "2026-10-23",
  "knownAt": null,                                    // or an instant, for "as known at" reads (section 2)
  "pin": "pin-7f3c…",
  "inputs": {                                         // exactly what the computation read
    "risk-store/pnl-vector":   { "version": "delta:v41", "date": "2026-10-23" },
    "risk-store/scenario-set": { "version": "delta:v12" },
    "edit-log":                { "watermark": { "L17": 3412, "L42": 988 } },   // the last applied change per lane included
    "trade-kafka/trade":       { "offsets": { "p3": 182277 } }
  },
  "trigger": { "type": "write-back", "eventId": "chg-01JB3Z…" },              // absent for a plain request
  "computedAt": "2026-10-23T14:02:13.418Z",
  "engine": { "connector": "var-engine", "entry": "drishti_var.connector:VarEngine", "package": "drishti-var 0.1.0" },
  "environment": "env-sha256:7f3c9a…",                  // the locked environment (BI_GUIDE.md section N5)
  "resultHash": "sha256:…"
}
```

The stamp is what *How this view was built* shows for a computed panel, and what makes a number **reproducible**: the same pin, the same environment and the same engine give the same result hash.

#### D8.2 The pin

When a request (or a screen, section D12.2) is resolved, the server records a **pin**: the concrete version of every input for the business date at that moment. A pin is small (a few hundred bytes), kept for the request's life and, for results that are cached or audited, with the result. Two panels of one screen that share a pin computed on the same data, whatever loads arrive while they run.

#### D8.3 Ordering

- Results for one key are **versioned by their watermark**: the input versions, edit-log seqs and offsets they include. A result is newer than another if every watermark is at least as high and one is higher.
- **A late result never overwrites a newer one.** If an event-driven recompute that started at watermark `L17:3410` finishes after one that started at `L17:3412`, the older is dropped (counted, never shown).
- Events of one **lane** of the edit log are processed in seq order (the lane already orders them, APPROVALS_GUIDE.md › The design and its reasons, section 7); events of one **Kafka partition** in offset order. Across lanes and partitions there is no order to keep: a result simply includes a set of watermarks.

#### D8.4 Idempotence

Every event has an id (the change id, `connector/partition/offset`, `schedule/<tick>`, `data-version/<kind>/<date>/<version>`). The connector keeps the ids of processed events per key for `triggers.dedupe-window` (default 24 h). An event seen again (a redelivered Kafka message, a resent change after a fail-over) is ignored. Recomputing is itself idempotent: the same pin gives the same result.

### D9. Isolation: two users never see each other's answers

The product owner's question (2026-10-24): *if two users write back or ask at the same time, can their responses mix?* No. Five rules make sure of it.

#### D9.1 The rules

1. **Every request and every event has a correlation id** (`requestId`, the change id). A result is delivered only to the request ids that asked for it or joined it, over the requester's own live channel. There is no broadcast of a computed result to "everyone on the page".
2. **The cache key includes the viewer's view signature.** A view signature is a hash of what decides what the viewer may see: the roles' kinds, row entitlements (for VaR: the hierarchy nodes the viewer may see position by position or only in aggregate) and field masks. Two people with the same rights share a cached result (that is the point of the cache); two people with different rights never do.
3. **What-ifs are private sandboxes.** A what-if is a sandbox owned by one user (`sandbox:asha:01JB…`); only its owner can read, change or compute it; a request naming another user's sandbox is refused as *not found* (it does not even confirm the sandbox exists). A sandbox can be shared read-only as a frozen copy, never as a live link to the owner's changes.
4. **Masks are applied per recipient.** A result computed once for a signature is still passed through the recipient's field masks when it is sent (belt and braces: a signature mismatch can only hide, never reveal).
5. **No state survives between requests in a worker.** A worker process handles one request at a time and is reset (or forked fresh) between requests (BI_GUIDE.md section N7); a Python global, a cached object or a temp file of one request is never seen by the next.

#### D9.2 Two traders, two amendments, the same second

Asha (RATES-NY) amends MX-50000001 at 14:02:11.200; Ben (also RATES-NY) books CLY-30000002 at 14:02:11.350. Both changes apply in the edit log (different lanes, `L17:3412` and `L42:988`).

| | **Asha sees** | **Ben sees** | **Everybody watching RATES-NY sees** |
|---|---|---|---|
| Their own change's state | *applied* on her row (her change id) | *applied* on his row | the updated records, with a "changed by" mark |
| **Impact of their own change** | *your change: incremental VaR +1.8m on RATES-NY*, computed against the state just before her event | *your change: incremental VaR −0.4m*, against the state just before his | nothing private |
| **Shared VaR** | one number including **both** changes, in event order, stamped `L17:3412, L42:988` | the same number, the same stamp | the same number, the same stamp |

- The **incremental impact** is attributed per event: the connector computes the vector of the scope before and after each event, in watermark order, and sends each submitter the difference for **their** event, under their change id. Ben never receives Asha's impact, and neither receives the other's private what-ifs.
- The **shared** number is one result for the scope and signature, coalesced over both events (section D10), stamped with both watermarks, so a reader can tell exactly which changes it includes.

#### D9.3 Pushing one recomputed result to many viewers

```
 trigger (event) ─► affected keys ─► for each key: group the subscribers by view signature
                                     ├─ signature A (risk, all positions): compute once ─► mask per subscriber ─► push to A1, A2,
A3
                                     └─ signature B (desk head, aggregate only): compute once ─► mask ─► push to B1
```

Subscribers are the open live channels that read the key (section D1's one subscription per entity, kept per key and signature).

### D10. Coalescing, incremental recompute and back-pressure

#### D10.1 Coalescing bursts

A burst of events for one key (twenty amendments to one book in a second) must not mean twenty computations.

- **Quiet window** (`coalesce.quiet`, default 250 ms): after an event, wait until no new event for the key arrives for this long.
- **Maximum wait** (`coalesce.max-wait`, default 2 s): never wait longer than this after the first event of a burst; compute with what has arrived. A steady stream therefore still yields a result every two seconds.
- **At most one computation per key at a time**; events that arrive during it cause **exactly one more** afterwards (the same rule section D1 uses for view rebuilds, section D1, guarantees).
- Per-event impacts (section D9.2) are still computed per event, because they are cheap (a difference of vectors), even when the shared result is coalesced.

#### D10.2 Incremental recompute

A connector may declare that a kind is **incremental**: given the previous result and the events since, it can produce the new result without starting again. For VaR this is natural: the P&L vector of a scope is a sum, so an amended trade changes the scope's vector by (new vector − old vector), and only the scopes above the trade change (VAR_GUIDE.md section 8.4). The connector's entry point receives `previous` (result plus stamp) and `events`; it may answer *full recompute needed* when the events are too many or of a kind it cannot apply incrementally. Every Nth incremental result (`incremental.verify-every`, default 50) is checked against a full recompute in the background; a difference beyond tolerance is a health warning and the full result wins.

#### D10.3 Back-pressure and fairness

| Class | Who | Queue | When full |
|---|---|---|---|
| `interactive` | a person waiting on a panel or an API call | short, highest priority | *busy, try again in a moment*; the panel shows the last good result, marked stale |
| `event` | triggers | bounded per connector (`limits.event-queue`, default 10,000 keys); keys, not events, are queued, so a burst for one key is one entry | the oldest *unwatched* keys are dropped to *stale, recompute on read*; watched keys are never dropped |
| `batch` | schedules, BI runs, precomputation | lowest priority, a fixed share of workers (`pool.batch-share`, default 25%) | waits |

- **Per-user fairness:** at most `limits.max-inflight-per-user` requests of one user run at once; the rest queue behind other users'.
- **Admission by cost:** a request declares (or the connector estimates) its size (for VaR: the number of positions in the scope); one over `limits.max-scope-positions` is refused before any read, with the size in the message, or sent as a BI run job (HOW_IT_FITS.md §A28) if the caller allows.

### D11. Caching

| Tier | Holds | Key | Invalidated by |
|---|---|---|---|
| **in memory** (per node) | recent results | the key of section D6.2 step 4 | a trigger that affects the key; size and age limits |
| **on disk** (`store: disk`) | expensive results, under `<data dir>/compute/results/<connector>/` | the same | the same; a restart keeps it |
| **inputs** | Arrow batches of inputs read for a pin (the P&L vectors of a desk) | connector, kind, scope, pin, signature | a new data version of the input |

- **The pin is in the key**, so a cached result is never served for another state of the inputs; a new load or an applied edit yields a new pin and therefore a new key. Old keys age out.
- **The engine version and the environment hash are in the key**: upgrading the VaR package or a pinned wheel never serves a result computed by the previous code.
- **Single flight** (one computation per key, others join) holds within a node and, through the key's owner (section D14), across a cluster.
- **Never cached:** a result that failed validation, a partial result, a private sandbox's result beyond the owner's session (sandbox results live with the sandbox, not in the shared cache).
- `purgeCaches()` (Admin → Caches, as for every connector) drops all three tiers.

### D12. One screen, many connectors

A pack already combines connectors: each kind is routed to one connector, and each panel can name its own `source:`. This design makes combining **computed** results first-class.

#### D12.1 An example: the desk VaR screen

```
# config/packs/banking/market-risk/sutras/pages/desk-var.v1.sutra.yaml       (planned)
match: { kind: desk }
title: "VaR · ${$.DESK_ID}"
consistency: snapshot                      # one pin for the whole screen (12.2)
controls:
  - { id: confidence, type: enum, options: [0.99, 0.975], default: 0.99 }
panels:
  - id: headline                           # reactive: var-engine
    use: kpi-tiles@1
    source: { kind: var-result, id: "'DESK:' + $.DESK_ID", params: { confidence: ctl.confidence } }
  - id: books                              # reactive: var-engine, contributions by child
    use: ranked-bars@1
    source: { kind: var-contribution, id: "'DESK:' + $.DESK_ID", params: { by: child } }
    emits: [ { name: select, on: bar-click, payload: { scope: "@.SCOPE_ID" } } ]
  - id: path                               # reactive: the 509 scenario P&Ls of the picked book
    use: series-bars@1
    source: { kind: var-path, id: "@books.select.scope" }
  - id: limits                             # static: core pack, a Delta table
    use: field-list@1
    source: { kind: desk, id: "$.DESK_ID" }
  - id: blotter                            # live: trades ticking from Kafka
    use: row-picker@1
    source: { kind: portfolio-trades, id: "$.DESK_ID" }
    refresh: 10
```

Five panels, three kinds of connector, one screen. Nothing in the pack names a library or a worker.

#### D12.2 One as-of for the whole screen

- `consistency: snapshot` (planned; the default for screens with a computed panel): when the screen is opened the server resolves **one pin** for the business date and every panel's source is read and computed against it. The strip says *as of 2026-10-23 · pinned 14:02:13*. When newer data arrives (a trigger fires for a key the screen shows), panels are **not** silently mixed: the screen shows *Newer data: 3 changes since 14:02 · Update all*, and **Update all** re-pins and refreshes every panel together. A screen that is live (RACHANA_REFERENCE.md › Sutra Next: fragments, events, children, sources, layout and forms, section D9) may set `consistency: snapshot, follow: true`: the screen re-pins by itself, at most once per `coalesce.max-wait`, and all panels move together.
- `consistency: freshest`: each panel shows its newest result as it lands; each panel's header says its own as-of. For monitoring screens where one late panel must not hold the others back.
- The view model carries each panel's pin; a panel computed on a different pin from the screen's (it could not be computed on the screen's pin in time) is marked *computed on newer data* in amber, never presented as consistent.

#### D12.3 Cross-connector joins

Two ways, and the design keeps both:

1. **On the screen**, through events: a click on a book in a reactive panel opens static and live panels for the same book (as the book desk does today). No join engine is needed; each panel reads its own source.
2. **In the reactive connector**, through `reads:`: when the answer itself needs data from several connectors (VaR over vectors from the lake and intraday vectors computed from Kafka trades), the connector reads them all as the user, on one pin, and returns one result. The join is the connector's code and is tested with it.

#### D12.4 Partial failure, per panel

Each panel has its own state, drawn in place, never as an error page:

| State | What the panel shows |
|---|---|
| `computing` | the last good result, dimmed, with *computing 58%* and the partial result if the connector sends one |
| `ready` | the result; the header's freshness from the result's stamp |
| `stale` | the last good result in amber: *stale · var-engine is recomputing after 3 changes* |
| `failed` | the connector named in words: *var-engine could not compute this: the worker ran out of memory (request req-01JB…)*; with the last good result if there is one |
| `refused` | why, in words: *not approved yet, your role cannot see positions of RATES-LDN, the scope has 6.2 million positions; the limit is 5 million* |
| `hidden` | *Hidden for you* (masks), as today |

The other panels are unaffected. *How this view was built* lists each panel's source, pin, state and request id.

### D13. Failure

| What fails | What happens | What the user sees |
|---|---|---|
| a worker crashes or is killed (memory, time) | the request fails; the worker is replaced; the same request is retried **once** on a fresh worker if it is interactive and the crash was not a limit | `failed` with the reason and request id; the last good result |
| the same request crashes workers twice | the key is **quarantined** for `limits.quarantine` (default 10 min): further requests are refused at once instead of crashing more workers | *this computation keeps failing; an administrator has been told* |
| an input connector is down | the request fails with that connector named; triggers from it stop | `failed`: *risk-store isn't answering* |
| the worker pool is exhausted | interactive requests queue up to their timeout; event keys are coalesced; batch waits | `computing`, then `failed: busy` |
| requirements are revoked by an administrator | in-flight requests finish; new ones are refused | `refused: not approved` |
| a result fails its schema | an error of the connector, logged with the request; never cached or shown | `failed: the computation returned an invalid result` |

A connector whose error rate passes `limits.circuit` (default 50% over one minute) opens its **circuit**: requests are answered from the cache (marked stale) or refused, for 30 s, then one probe is let through. Health shows `DEGRADED: circuit open`.

### D14. Several servers (Drishti*net)

- **Who computes a key:** the key's **owner node**, chosen on the Drishti*net ring by the key's hash among the nodes that serve the connector (CLUSTER_GUIDE.md). Any node receiving a request forwards it to the owner (signed, on behalf of the user, as views are forwarded today), so single flight and the cache hold across the cluster.
- **Triggers** are received by every node (an applied edit is published on the cluster; a Kafka consumer group gives each partition to one node) and routed to the owner of each affected key.
- **Pushes** go from the owner to the nodes holding the subscribers' live channels (the existing live relay).
- **A node leaving:** its keys move to the next owner; in-flight computations there are lost and recomputed on the next read or trigger (results are reproducible from their pins, so nothing is wrong, only late). Private sandboxes are kept in the content store (section D5), so they survive a node.

### D15. Observability and audit

- **Admin → Compute** (planned): per reactive connector, its pools and workers (busy, idle, memory, requests served, recycled), queues per class (depth, oldest wait), latency (p50, p95, p99 per kind), cache hit rate, coalescing ratio (events in, computations out), quarantined keys, circuit state, and its approved requirements.
- **Metrics** under `drishti.compute.*` on the server's metrics endpoint, labelled by connector and kind.
- **Tracing:** the request id is carried into the worker and into every log line and metric exemplar; Admin → Compute finds a request by id and shows its timeline (received, pinned, admitted, inputs read, computed, answered).
- **Audit:** every computed answer delivered to a person is an audit entry: who, which kind and id, params, pin, signature, result hash, duration, and the trigger if any. What-if sandboxes record every change and computation. Entries are signed like all audit entries and kept as the site's audit retention says.
- ***How this view was built*** on a computed panel shows the stamp of section D8.1 in words: *VaR computed by var-engine (drishti-var 0.1.0) at 14:02:13 over 18,412 P&L vectors of 2026-10-23 (lake version 41) and 3 changes applied since (L17:3412, L42:988)*.

### D16. The SPI and the APIs

#### D16.1 Java SPI (planned)

The generic `reactive` plugin implements `SourcePlugin` once; what differs per connector is behind a small interface, implemented by the worker bridge (Python, Java worker) or by trusted in-process Java code:

```
// drishti-api, com.ash.drishti.api.compute (planned)
public interface ComputeEngine extends AutoCloseable {
    ComputeManifest manifest();                                  // kinds, params, result schemas, incremental kinds
    ComputeResult compute(ComputeRequest request, ComputeInputs inputs, ProgressSink progress) throws ComputeException;
    default Optional<ComputeResult> incremental(ComputeRequest request, ComputeResult previous,
                                                List<ComputeEvent> events, ComputeInputs inputs) { return Optional.empty(); }
    default Set<String> affected(ComputeEvent event, String expression) { return Set.of(); }   // scopes_of(...)
}
```

`ComputeInputs` hands the engine Arrow batches already read as the user; the engine never sees a connector, a credential or the router.

#### D16.2 HTTP API (planned)

Panels and BI read reactive kinds through the existing view and source paths; nothing new is needed for them. For programs:

| Method and path | Does |
|---|---|
| `POST /api/v1/compute/{connector}/{kind}` | submit a request (`id`, `params`, `asOf`); answers the result inline when ready within the budget, otherwise `202` with the request id |
| `GET /api/v1/compute/requests/{requestId}` | the state and, when done, the result (only to the user who asked) |
| `GET /api/v1/compute/requests/{requestId}/stream` | progress and the result as server-sent events |
| `DELETE /api/v1/compute/requests/{requestId}` | cancel |
| `GET /api/v1/admin/compute` | Admin → Compute's data (administrator) |

A token needs the planned `compute` scope. The CLI gains `drishti.py compute run|status|cancel` (planned).

### D17. Configuration reference

| Key (connector file) | Default | Meaning |
|---|---|---|
| `mode` | from the plugin | `static`, `live`, `on-demand`, `reactive` |
| `kinds.<kind>.params` | — | typed parameters: `type` (`scope`, `decimal`, `int`, `enum`, `date`, `sandbox-ref`, `text`), `default`, `allowed`, `required`, `max` |
| `kinds.<kind>.result.schema` | — | JSON schema each result must match |
| `kinds.<kind>.incremental` | `false` | the engine can update a previous result from events |
| `reads` | — | input kinds, read as the requester through the router |
| `engine.type`, `engine.entry`, `engine.pool` | — | where the code runs (BI_GUIDE.md) |
| `requires` | — | libraries (BI_GUIDE.md section N4) |
| `triggers[]` | — | `on`, `kinds`, `affects`, `recompute`, `cron`, `zone` |
| `triggers.dedupe-window` | `24h` | how long processed event ids are remembered |
| `coalesce.quiet`, `coalesce.max-wait` | `250ms`, `2s` | burst handling |
| `incremental.verify-every` | `50` | a full recompute checks every Nth incremental result |
| `cache.results`, `cache.bytes`, `cache.ttl`, `cache.store` | `20000`, `512MB`, `8h`, `heap` | result cache |
| `limits.timeout`, `limits.timeout-batch` | `30s`, `10m` | budgets |
| `limits.max-inflight-per-user` | `4` | fairness |
| `limits.event-queue` | `10000` | event keys queued |
| `limits.quarantine`, `limits.circuit` | `10m`, `0.5` | failure handling |
| `history.dated` | `true` | a picked business date computes on that date's inputs |

The worker pools themselves are configured once per site, in the planned `config/compute/compute.yaml` (BI_GUIDE.md section N13).

### D18. Build plan

| Step | Delivers | Accepted when |
|---|---|---|
| R1 (**built**) | the `reactive` plugin and `ComputeEngine` SPI; typed `params:` on `source:` (lint, view model); the compute request, key, pin and stamp; cache and single flight on one node; an **in-process Java** test engine | a panel reads a computed kind with params; two panels asking the same key cause one computation; a new data version gives a new key |
| R2 (**built**) | the worker bridge to the Python pool (BI_GUIDE.md steps C1–C3); progress on the live channel; per-panel states | a Python engine computes a result for a panel, with *computing N%* shown, and a crash shows `failed` in that panel only |
| R3 (**built**; `write-back` from the edit log since W6/W7, 2026-10-25, and through the events API) | triggers: `write-back`, `live`, `data-version`, `schedule`; `affects:`; coalescing; ordering by watermark; dedupe | 200 events for one key in a second cause at most 2 computations; an older result never replaces a newer; a redelivered event is ignored |
| R4 (**built**, except per-event impacts: W6/V5) | isolation: view signatures in keys, per-recipient masks on push, per-event impacts, private sandboxes | the two-traders test of section D19 passes |
| R5 (planned) | `consistency: snapshot` and `freshest`; *Newer data* · *Update all* | a screen of five panels on three connectors shows one pin; an applied edit offers *Update all* and re-pins all panels together |
| R6 (planned) | Drishti*net: key owners, forwarding, cluster single flight, trigger routing; Admin → Compute; audit; circuit and quarantine | three nodes, one key requested on each at once: one computation; a node killed mid-computation: the next read recomputes on the new owner |

### D19. Tests

- **Isolation (the product owner's question):** two users with different entitlements request the same scope at the same instant; each receives only the result for their own signature, under their own request id; a third user with the first user's rights shares the first's cached result. Two users submit write-backs to one book within 100 ms; each receives only their own incremental impact; both see one shared result stamped with both watermarks. A request naming another user's sandbox is *not found*.
- **Ordering:** random event streams over many keys and lanes; results for a key are never replaced by one with a lower watermark.
- **Coalescing:** a burst of N events yields at most ⌈burst length / max-wait⌉ + 1 computations per key.
- **Idempotence:** a redelivered Kafka message and a resent change id do not recompute.
- **Incremental equals full:** for VaR, a random sequence of amendments applied incrementally equals a full recompute to 1e-9.
- **Failure:** a crashing engine is retried once and quarantined after the second crash; the circuit opens and closes.
- **Masks:** a field masked for a subscriber never appears in a pushed result.
- **Cluster:** the R6 acceptance tests, plus a network split (results stay correct; late, never mixed).
- **Windows:** everything except the native QuantLib-Java worker runs on the Windows QA machine (BI_GUIDE.md section N12).

### D20. Open questions for the product owner

1. **Default consistency.** Should every screen with a computed panel default to `consistency: snapshot` (one pin, *Update all*), or should `freshest` be the default and snapshot opt-in?
2. **Triggers under the service identity.** Is it acceptable that shared recomputes read inputs as the connector's service identity and then compute once per distinct view signature, or must every recompute run as a named user?
3. **Kafka-triggered intraday VaR?** The `write-back` and `data-version` triggers are needed; is the `live` (Kafka) trigger too, or later?
4. **Where private sandboxes live.** The content store (survives nodes, per user) is proposed; is a time limit wanted (for example sandboxes expire after 30 days unless saved)?
5. **Limits.** Are 5 million positions per interactive request and 4 in-flight requests per user the right starting limits?
6. **In-process Java engines.** Allow trusted, light Java engines inside the server process (with their own classloader), or require every engine to run in a worker?

### D21. Built: R1 to R4

Built on the compute workers C1 to C3 (BI_GUIDE.md). The reference is the reactive connector's README (docs/connectors/types/reactive/README.md); the sample is the `compute-demo` pack (not loaded by default).

| Part | Implementation | Where |
|---|---|---|
| SPI | `ComputeEngine`, `EngineManifest`, `EngineRequest`, `EngineInputs` (documents by input name), `EngineResult` (rows, tables, meta), `EngineProgress`, `EngineException`, `ComputeEvent`, `Viewer`, `ComputeRef` (the id with its parameters). The names differ from section D16.1 (`Engine…` rather than `Compute…`) so they do not clash with drishti-compute's worker types | `drishti-api`, `com.ash.drishti.api.compute` |
| Plugin | `reactive` (built-in, `ReactiveSourcePlugin`), `ReactiveConnector` (the lifecycle), `ReactiveSpec`, `ParamSpec`, `ResultCache`, `Coalescer`, `SeenEvents`, `Watermark`, `Triggers`, `Sandboxes`, `RequestEvents`, `ReactiveHub` (the host the server installs) | `drishti-compute`, `com.ash.drishti.compute.reactive` |
| Connector file | the design's keys at the top level (`mode`, `kinds` with `params`, `reads`, `engine`, `triggers`, `coalesce`, `cache`, `limits`, `history`, `incremental`); refused on any other plugin. Added: `kinds.<k>.compute` (the engine's name for the kind), `engine.split/chunk-rows/reduce` (fan-out), `engine.version`, `limits.wait`, `limits.max-input-rows`, `limits.dedupe-window` (the design's `triggers.dedupe-window`), `reads[].ids` and `{inputs.<as>.<FIELD>}` (a read takes its ids from an earlier one) | `ConnectorFiles` |
| Templates | single braces: `{id}`, `{params.x}`, `{inputs.book.MEMBERS}`, `{payload.BOOK_ID}`, `{entity}`; `${NAME}` stays an environment variable | `Templates` |
| Result rules | `result: { required, types, max-rows }` instead of a JSON schema file (`result.schema`: planned) | `Results` |
| Parameters on a panel | `source: { kind, id, params: { name: expression } }`, only with `id`; lint `DRS-2012` (shape) and `DRS-2057` (not with `where/search`); evaluated as the viewer (a masked value: *not available to your role*) into the id `ID~name=value` | `WiringReader`, `ScreenWire` |
| Panel states | `ready`, `computing` (progress), `stale`, `failed`, `refused` in the view model (`PanelView.compute`) and drawn in place; the console follows `compute:<request>` on the live channel and fetches the panel again when it is done or updated | `ViewModel`, `panels.html`, `drishti_compute.js` |
| Lifecycle | authorise (the viewer's kinds), parameters, pin (the read kinds' data versions and the sandbox version), key (with the view signature: a hash of the roles and the embedding app, `Principal.securitySignature`), cache, single flight, admission (per-user in-flight; the pool's fair queue), inputs read and masked as the viewer, engine, progress, validate, stamp (`STAMP`: pin, inputs, watermark, trigger, engine, where it ran, signature, result hash), cache, audit (`compute.answer`, `compute.failed`) | `ReactiveConnector` |

| Part | Implementation | Where |
|---|---|---|
| Engines | Python in a compute pool (Arrow in and out; fan-out over every worker of the pool and its peers with `split` and `reduce`); Java in the server (Drishti's own always; others with `compute.in-process-java: true`) | `PythonRunner`, `JavaRunner` |
| Triggers | `write-back` and any event through `POST /api/v1/compute/events` (administrators) and `ReactiveHub.publish` (the write-back module's hook: built 2026-10-25, every applied change after its commit, with the watermark `L<lane> -> seq`, replayed per lane from the edit log's cursor; APPROVALS_GUIDE.md › The design and its reasons, section 19); `live`: the entities of its kinds a watched computation read are followed, a new generation recomputes the results that read them; `data-version`: the load signal and a check every 30 s; `schedule`: `cron` (Spring's six fields) or `every`. `affects`: `all`, `subscribed`, id templates, or an expression a Java engine answers. Coalescing, watermark ordering, dedupe as designed | `Triggers`, `Coalescer` |
| Isolation | request ids per read, events to the owner only, view signature in keys, inputs masked per viewer, a recomputed result read again by each watcher through their own view, private sandboxes (`sbx-…`, frozen copies), live views rebuilt on behalf of their viewer | `RequestEvents`, `ViewerExecutor` |
| HTTP and CLI | `POST /api/v1/compute/{connector}/{kind}`, `GET …/requests/{id}/stream`, `DELETE …/requests/{id}`, `POST …/events`, sandboxes, `GET /api/v1/admin/compute/reactive`; `drishti.py compute run`, `cancel`, `event`, `sandboxes`, `reactive` | `ReactiveController`, `tools/computecli.py` |

**Not built yet:** per-event impacts for each submitter (section D9.2, with W6 and V5); the result cache on disk (`cache.store: disk`); `consistency:` and *Update all* (R5); key owners, forwarding, cluster single flight, quarantine and the circuit (R6); a JSON schema file per kind; a `compute` power (any role that may open the kind may compute it); the designer's awareness of parameters (a source with `params:` is kept as written).

**Measured** (a workstation, 2026-10-25; `ReactiveOverheadTest`, `PythonBridgeTest` and the compute-demo screen): a cache hit inside the connector 4.7 µs at p50 (p99 9.6 µs); a miss with an engine that does nothing 47 µs (p99 183 µs); a miss through a Python worker (fork, trivial engine) 2.3 ms; through the server, one computed panel from the cache 2.7 ms, the same as a plain panel (2.6 ms); the compute-demo book screen, two computed sources and two 250-point charts, 5.0 ms. A panel never waits longer than `limits.wait`.

### D22. How the VaR engine plugs in

The VaR engine (VAR_GUIDE.md Part III, steps V0 to V3) becomes the `var-engine` reactive connector (R5/V4) without new machinery:

1. **The file.** `config/connectors/var-engine.yaml` with `plugin: reactive`, the kinds of section D5 (`var-result`, `var-contribution`, `var-path`, `var-history`, `what-if`) and their `params:` (`scope` as `type: scope`, `confidence` as `decimal` with `allowed`, `horizon` as `int`, `method` as `enum`, `whatif` as `sandbox-ref`), and `result:` rules (`required: [VAR, ES]`).
2. **The engine, either of two ways.** - *Java, in the server* (the vector store is Java): a class implementing `ComputeEngine` in `com.ash.drishti…` (so it may run in process), `engine: { type: java, class: …VarComputeEngine }`, with **no** `reads:`: the VaR builder's in-memory vector store is its input, and it must read it **for** `request.viewer()` (the entitlement filter of VAR_GUIDE.md section 9) and **for** `request.asOf()`. `compute` aggregates the scope's vectors and answers `VAR`, `ES` and the path as a table; `incremental` applies a write-back's `delta` vector (`ComputeEvent.payload`); `affected(event, "scopes_of(trade.TT_BOOK)")` returns the book and its ancestors' scope ids from the hierarchy. The vector store's version for the date goes into the pin through `dataVersion` of the kind it serves (`pnl-vector`), so a new load is a new key. - *Python, in a pool* (`compute/drishti-var`): `engine: { type: python, entry: drishti_var.connector:VarEngine, pool: risk }` with `reads: [{ kind: pnl-vector, as: vectors, ids: "{inputs.scope.MEMBERS}" }]` after a read of the scope's members; vectors arrive as fixed-size Arrow vectors; `split: vectors` and `reduce: { tables: { path: [PNL] }, then: { VAR: … } }` spread a bank-wide scope over every worker. Its `requires:` locks numpy and the package and waits for approval.
3. **What-if.** A `sandbox-ref` parameter names the trader's private sandbox; its items (the hypothetical trades) reach the engine as the input `sandbox`; the key holds the owner, so it is never shared. Pricing those items is C4/C5 (QLRisk) or C6 (QuantLib Java).
4. **Triggers.** `{ on: write-back, kinds: [trade], affects: "scopes_of(trade.TT_BOOK)", recompute: [var-result, var-contribution] }` (W6 publishes `edit.applied` to `ReactiveHub.publish`), `{ on: data-version, kinds: [pnl-vector], affects: all }`, and a schedule.
5. **Screens.** The explorer's panels name the kinds with `source: { kind: var-result, id: "'DESK:' + $.DESK_ID", params: { confidence: ctl.confidence } }` and get the computing, stale, failed and refused states for free. The pack routes `var-result: var-engine` beside `pnl-vector: <its store>`.

## D5. The content store: user content every node reaches

Status: **built (Designs; BI content and BI runs; collaboration and notes; embed apps, pack switches and audit; governance proposals; load history), built.** Front-loaded from item 0 of write-back, the one Drishti database. Related: section D6 (the one Drishti database), CLUSTER_GUIDE.md (several servers), CONFIGURATION.md (`config/store/store.yaml`).

### D1. The problem

User content (screen-designer Designs, workspace drafts, BI items, run records, shares, schedules, settings) was saved on the node that served the request. In a Drishti*net cluster, when the console fails over to another node, that node has none of it, and the user sees `404 DRS-5006 no design`. Content must be reachable from every node, and two nodes editing the same item at the same moment must never silently lose one of the edits.

### D2. The interface

One small interface, `com.ash.drishti.identity.content.ContentStore` (drishti-identity, beside the Drishti database code that already owns user state). Documents, not SQL: a key names one item; the store keeps its bytes and an opaque version.

| Call | Does |
|---|---|
| `get(key)` → `Optional<Item{bytes, version}>` | Reads one item. |
| `put(key, bytes, expectedVersion)` → new version | Compare-and-set. `expectedVersion` null means *must be absent* (create); otherwise the item must still have that version. A mismatch throws `409 DRS-5057` (content conflict). |
| `list(prefix)` → keys | Every key starting with `prefix`, sorted. |
| `delete(key, expectedVersion)` → boolean | Compare-and-set delete; false when there was nothing; a stale version is `409 DRS-5057`. |
| `changes(since)` → `Changes{keys, cursor}` | Keys written or deleted after `since` (epoch ms), for polling caches. At least once: pass `cursor` back next time (less a small margin for clock skew across nodes); a key may repeat. Push is a later addition for backends that have it (Postgres `LISTEN`, S3 events). |
| `root()` | Where the store is, as text (the cluster check, section D6). |

Keys are `/`-separated paths such as `designs/<owner>/<id>/design.json`: 1 to 16 segments, each 1 to 120 UTF-8 bytes, any Unicode, no empty, `.` or `..` segment, no control characters, no `\`. A bad key is `400 DRS-5058`. `putLatest(key, bytes)` is a helper (read the version, put, retry a few times) for writes whose last writer may win (sample documents).

### D3. Backends

Chosen in their own file, `config/store/store.yaml` (never `application.yaml`; the file is named by `drishti.store.file`), examples in `config/store.examples/`. Every backend passes the same contract test suite (section D7).

**filesystem (the default).** Root `<data folder>/fstore` (`data/fstore`). An item is one file, `items/<encoded key>`; its version is the SHA-256 of its bytes, so the data file alone is the truth and no read can see bytes and version out of step. A write goes to a visible sibling `<name>~tmp-<uuid>` and is renamed into place; compare-and-set holds a lock file `<name>~lock`, made with create-new (atomic on local disks, NFS v3+ and EFS) and broken when older than 30 s (a crashed writer). Renames and deletes retry with back-off when Windows refuses them because a reader has the file open (the FileSwap pattern). Key segments are encoded (`%XX` for every byte outside `A-Za-z0-9-_.`, a leading `.` and Windows device names too), so names are visible, portable to Windows and never contain `~`, which is reserved for the store's own sidecars. A delete leaves an empty `tombstones/<encoded key>` so `changes` reports it; a later put removes it. Since the version is a content hash, writing back identical bytes keeps the version (harmless: the content is the same). No hidden files, nothing outside the root.

**sqlite and postgres.** One table, `drishti_content (k, v, version, updated_ms, deleted)`, in the two complete Drishti database schema files (`drishti-identity/src/main/resources/db/schema-sqlite.sql` and `schema-postgres.sql`; no migrations, SchemaParityTest keeps them equal). The backend applies the whole schema file at start (idempotent), so it may point at the Drishti database itself (the usual choice) or at a database of its own. The version is a counter; compare-and-set is `UPDATE … WHERE k = ? AND version = ?` (or an `INSERT` racing on the primary key for a create). A delete keeps a tombstone row (`deleted = 1`) so `changes` reports it. Postgres is tested with Testcontainers.

**s3.** Objects `<prefix>/items/<key>`; the version is the ETag. Compare-and-set is the conditional PUT: `If-None-Match: *` for a create, `If-Match: <etag>` for an update; `412` or `409` answers `DRS-5057`. A delete compares the ETag (a HEAD, then `If-Match`) and writes a tombstone `<prefix>/tombstones/<key>`. `changes` lists by LastModified (second resolution). Works with AWS S3 (conditional writes) and compatible stores that honour the headers, such as MinIO; tested against an in-process S3 stand-in that honours the conditional headers, Adobe S3Mock in the test JVM, and the MinIO server binary on Linux (section D7).

**Delta / Iceberg: not primary, design only.** They are poor at single-key compare-and-set and at small hot documents. Later they may be an optional **history/audit sink**: a writer tails `changes`, appends `(key, version, bytes or hash, at, by)` to a Delta or Iceberg table, and BI can query who changed what and when. Not built.

### D4. Configuration

```
# config/store/store.yaml
backend: filesystem             # filesystem | sqlite | postgres | s3
filesystem:
  root: ${drishti.data.dir:./data}/fstore
sqlite:
  url: jdbc:sqlite:${drishti.data.dir:./data}/identity/drishti.db   # the Drishti database itself (the usual choice)
postgres:
  url: jdbc:postgresql://db:5432/drishti
  user: drishti
  password: ${DRISHTI_STORE_PASSWORD:}
  pool: 4
s3:
  bucket: drishti-content
  prefix: prod
  region: us-east-1
  endpoint: ""                  # MinIO or another S3-compatible store
  path-style: false
  access-key: ${DRISHTI_STORE_ACCESS_KEY:}
  secret-key: ${DRISHTI_STORE_SECRET_KEY:}
```

A missing file or key means the default shown. `config/store.examples/` has `laptop.yaml` (filesystem), `cluster-nfs.yaml` (filesystem on a shared mount), `postgres.yaml` and `s3.yaml`.

### D5. Designs: the first user

`drishti.builder.designs.store` is now `content` by default: Designs are kept in the content store as `designs/<owner>/<id>/design.json` and `designs/<owner>/<id>/samples/<hash of name>.json`. `ContentDesignStore` implements the existing `DesignStore`, so `DesignService` (ownership, quotas, expiry, roles) is unchanged. Every Design read carries its store version and every save is a compare-and-set against it, so two nodes saving the same Design at once give one success and one `409 DRS-5057` (changed elsewhere: reload and try again). The older `file` (`data/designs`) and `jpa` stores can still be named, for one node only.

**One-time import at start.** With `store: content`, the server imports what the old stores hold:

- **files** (`drishti.builder.designs.dir`, `data/designs`): each Design and its samples are put with *must be absent*, so an item already in the store is never overwritten and a rerun is harmless. When every Design is in, the folder is renamed `designs-imported-<yyyyMMdd-HHmmss>` (visible, kept, safe to delete later). A failure leaves the folder in place for the next start.
- **the identity database** (`drishti_design`, `drishti_design_sample`, used by `store: jpa`): the same, once; a marker item `system/imports/designs-jpa.json` records it, so a Design deleted later does not come back. The rows are left in place.

Each import logs how many Designs it brought in, skipped (already there) and failed.

### D6. Several servers

Every node of a cluster must use the **same** store configuration. A filesystem store on a cluster needs a **shared mount** (NFS, EFS, SMB) at the same root on every node; a local `data/fstore` per node gives each node its own content again. Each node writes `nodes/<node id>.json` into the store at start (its root as it sees it); a node that finds another member's record with a different root logs a warning (`content store roots differ`): the same share mounted at different paths, or two configurations pointing at one store. A node also logs its store at INFO (`content store: filesystem /mnt/drishti/fstore`). A local store per node cannot see the other nodes' records, so that case is caught by the log line and this rule, not by the check (next step: carry the root in the Drishti*net capability digest and compare it there). Postgres and S3 need no shared disk.

### D7. Tests

`ContentStoreContract` (drishti-identity tests) runs against every backend: concurrent creates and concurrent updates of one key (exactly one wins, the rest `DRS-5057`), conflict codes, list by prefix, delete (stale and current), changes (puts and deletes), binary-safe content (every byte value), a large item (6 MB), Unicode keys, bad keys (`DRS-5058`), and reopening the store keeps everything. Backends: filesystem, sqlite, postgres (Testcontainers), s3 on three S3 simulators:

| Test | Simulator | Runs where | Conditional writes covered |
|---|---|---|---|
| `S3ContentStoreTest` | `FakeS3`, an in-process stand-in | everywhere | create-only, update-if-match, delete-if-match, racing writers |
| `S3MockContentStoreTest` | Adobe S3Mock 4.12.4 in the test JVM (a Maven test dependency: no Docker, no binary) | everywhere, Windows QA included | create-only, update-if-match, delete-if-match; racing creates are **not** atomic on S3Mock, so the race case is skipped there |
| `MinioContentStoreTest` | the MinIO server binary | Linux only | all, when the binary can be had (below) |

Each suite first probes its simulator (`com.ash.drishti.testkit.s3.ConditionalWrites`: a second `If-None-Match: *` create, a stale `If-Match` put, a stale `If-Match` delete, then eight racing creates five times) and prints one line, for example `[S3 conditional writes] S3Mock 4.12.4 (in-process): create-only covered, update-if-match covered, delete-if-match covered, racing writers NOT atomic (skipped)`; a contract case that needs what the simulator does not honour is skipped with that reason, and the stand-in still covers it. The Iceberg native engine's table on S3 has the same pair (`NativeOnS3SimulatorTest`: S3Mock in-process, MinIO binary), beside the Docker-based `NativeOnS3Test`.

MinIO: the test (`com.ash.drishti.testkit.s3.MinioServer`) takes the binary from `DRISHTI_MINIO_BIN`, else from the git-ignored tools cache (`DRISHTI_TOOLS_CACHE`, default `tools/cache/minio/`), else downloads it from `DRISHTI_MINIO_URL` (default: the official release) and checks it against `DRISHTI_MINIO_SHA256` or the published `.sha256sum`. It starts it on a free port in 18750-18759 with its data under `target/` and stops it by PID. It skips with the reason on Windows, offline, or when no binary is served: MinIO stopped serving its community binaries (dl.min.io answers `410 Gone`), so put a binary you trust in `tools/cache/minio/` or point `DRISHTI_MINIO_BIN` at it. How to run all S3 tests: CLI_REFERENCE.md §4.12. `ContentDesignStoreTest`: two in-process nodes on one filesystem root (a Design saved through A is read and updated through B; concurrent saves give one conflict; owners stay isolated) and the import (idempotent, folder renamed, marker for the database).

### D8. Next steps: what is still kept on one node

Already shared between nodes, not through this store: users, roles and API tokens (IdentitySync); `drishti_preference` (settings, saved searches, layouts, pivots, workspaces) and sessions (StateSync, ReplicatedPreferences).

The ordered migration list (the inventory of content kept on one node only, 2026-10-21). Items marked done are in the content store (sections D5, D9 and D10); the rest move one at a time behind the same interface:

1. **Designer drafts** (Designs): done, section D5. Versions, About, form and share live inside the record; ids are random 12-hex.
2. **BI workspaces and items:** done, section D9. `BiStore`, the `drishti_bi_*` tables (workspace, member, item, version, revision, event, user_item, usage, setting).
3. **BI runs:** done, section D9. The `drishti_bi_run*` rows, and results and snapshots under `<data>/bi-runs`.
4. **Notes:** done (section D10.2). `drishti_note` is only the source of the one-time note import; notes live in comment threads.
5. **Collaboration:** done (section D10.3): threads, comments, revisions, mentions, follows, note links, shares and recipients, inbox, outbox, holds and seals. Still per node: the access log (`drishti_access`, who read what), next.
6. **Alerts and monitors:** NOT copied, on purpose (each node evaluates its own).
7. **Embed apps** (`drishti_embed_app`), **pack switches** (`drishti_pack_state`), **audit** (`drishti_audit` and `audit.jsonl`): done (section D10.4).
8. **Files:** governance proposals (`<data>/governance`): done (section D10.5); load history (`<data>/loads`): done (section D10.6). Next, in this order: report outputs (`<data>/reports`), designer bound files (`<data>/dev-sutras`), pack admin state (`<data>/packs/*`, `uninstalled`, `admin/bi-hidden.txt`; a pack jar stays a local file materialised from the store, the store being the truth). UI-written config (`config/connectors`, `config/bi`): done for connectors and data loads overrides, section D11.
9. **BI store Parquet:** a derived cache. Never migrated.

Then: a read cache driven by `changes`, push for Postgres and S3, the root in the Drishti*net capability digest (section D6), and the optional Delta/Iceberg history sink (section D3).

### D9. BI content and BI runs

`drishti.bi.store` and `drishti.bi.runs.store` are `content` by default: BI workspaces, items and their history, and BI runs with their results, live in the content store, so every node of a cluster sees the same BI content and the same runs. `ContentBiStore` implements `BiStore` (now an interface; `JpaBiStore` is the old identity-database store), so `BiService` (workspace membership, item visibility, audiences, governance) is unchanged; `ContentRunRecords`, `ContentRunRecords.Shares`, `ContentRunRecords.Schedules` and `ContentRunStore` implement the runs' `RunRecords`, `RunShares`, `RunSchedules.Store` and `RunStore`, so run visibility, shares and the signature rule on results are unchanged.

**Key layout.** Every segment is escaped (letters, digits, `-` and `.` kept, other bytes `_xx`; past 100 bytes a prefix and a hash); numbers are zero-padded to ten digits so keys sort.

| Key | What |
|---|---|
| `bi/workspaces/<ws>.json` | a workspace |
| `bi/members/<ws>/<user>.json` | one member of a shared workspace |
| `bi/items/<item>.json` | an item with its draft body and catalogue fields |
| `bi/history/<item>/versions/<n>.json` | a published version |
| `bi/history/<item>/revisions/<seq>.json` | a saved state (every save, restore, publish, import) |
| `bi/history/<item>/events/<seq>.json` | a governance event (request, approve, certify, ...) |
| `bi/users/<user>/marks.json` | a person's favourites and last opens, by item |
| `bi/usage/<yyyy-mm-dd>/<node>.json` | the views one node counted that day, by item (`imported.json`: the import's) |
| `bi/settings/<name>.json` | a BI setting |
| `bi/runs/rows/<run id>.json` | a run (the old `drishti_bi_run` row) |
| `bi/runs/shares/<run id>/<recipient>.json` | a share of a run |

| Key | What |
|---|---|
| `bi/runs/schedules/<id>.json` | a scheduled run |
| `bi/runs/results/<result key>/answer.json` | a result, with the signature it was computed for |
| `bi/runs/results/<result key>/meta.json` | its owner, size and last open (the budget and the sweeper read only these) |
| `bi/runs/snapshots/<name>/<version>.json` | a promoted snapshot |

Item documents sit apart from their history (`bi/history/`) so listing the items reads item documents only.

**Safe concurrent edits.** A read of an item or workspace remembers, for the request's thread, the version it returned (30 s); the save that follows is a compare-and-set against it. Two nodes (or two requests) saving the same item give one success and one `409 DRS-5057` (changed elsewhere: reload and try again), beside BI's own `baseEtag` check. A save of a key the thread has not read lately writes over what is there, as the database did. New revision and event numbers, and new snapshot versions, are taken by a create that must not exist yet, so two nodes never take the same number. Marks and the day's views are read-change-write loops on compare-and-set.

**Listing and search.** The catalogue's listings (all items, a workspace's items, the published ones, all workspaces, the runs lists) come from a per-node snapshot of the prefix at most `drishti.bi.store-list-ttl` old (default `1s`); documents whose version did not change are not parsed again, and this node's own writes drop the snapshot at once. A single item, workspace or run is always read from the store. A listing on another node may lag by the TTL; an edit never does (the compare-and-set decides). Next: a snapshot refreshed from `changes` instead of a re-list, once `changes` is cheap on every backend (section D8).

**Events and usage.** Governance events are per-item documents (`events/<seq>`): they are rare, belong to the item's audit trail, and must be the same on every node. Views are append-heavy, so each node counts into its own document per day (`bi/usage/<day>/<node>.json`): no two nodes write the same document, a write is a small compare-and-set on this node's segment, and `views` sums the segments of the days asked.

**Runs across nodes.** A run's status and result read from any node (the row and the result are in the store); `RunForwardFilter` still sends a run's actions to the node that holds it, and is kept for nodes whose runs store is `local`. The lease is a compare-and-set on the run's document: `renew` writes only while the row still names this node, `adopt` only while it is still the dead node's with its lease lapsed, so exactly one node wins (HOW_IT_FITS.md §A28 13.3). A late save from a node that lost a run (a higher attempt on another node) is dropped. A due schedule is taken by one node (`claim`: a compare-and-set on its `nextAt`), so a schedule fires once in a cluster.

**One-time import at start.** With `content`, the server imports, once, the identity database's `drishti_bi_*` rows (marker `system/imports/bi-jpa.json`) and the `drishti_bi_run`, `drishti_bi_run_share`, `drishti_bi_run_schedule` rows with the results and snapshots under `store.dir` (`<data>/bi-runs`; marker `system/imports/bi-runs.json`). Every document is created with "must be absent" (a rerun is harmless; a person's marks already in the store win); the marker keeps an item or run deleted later from coming back. The rows and the files are left in place.

**One node back on the old stores.** `drishti.bi.store: jpa` and `drishti.bi.runs.store: local` put one node back on its identity database and `store.dir`; it then sees only its own BI content and runs (forwarding by node id still reaches the others' runs).

**Tests.** `ContentBiStoreTest` (drishti-identity): two nodes on one filesystem root (a workspace and an item made on A are read, edited and published on B; marks, views and settings; delete), two saves of one item give one conflict, concurrent revisions never share a number, and the import (once; a deleted item stays deleted; nothing counted twice; the rows kept). `ContentRunsTest` (drishti-server): a run submitted on A has its status and result read on B (never across signatures); a dead node's run adopted by exactly one of two nodes through the lease and finished; renew only by the holder and a loser's late save dropped; a share made on A seen on B; a schedule claimed once; the import of rows and the result folder. `RunClusterTest` runs its two servers on one content store root.

### D10. Collaboration, notes and the admin data

Each moved store keeps its interface, so the services above it (rights, limits, retention, holds, seals) are unchanged; a switch per type names the old store for one node only. At start each runs a **one-time import** of the old rows and files: every item keeps its id, number and hashes, an item already in the store is skipped, a marker `system/imports/<what>.json` records the import (so something deleted later does not come back), and the old tables and files are left in place.

#### D10.1 Several documents, one change: `TxContentStore`

Collaboration writes several documents for one action (a share, its recipients' indexes, their inbox rows; a comment, its index entries). Two choices were open: one document per aggregate, or ordered compare-and-set writes with a recoverable intent record. Both are used:

- **One document per thread** (`collab/threads/<id>.json`: the thread, its comments, the sealed revision chain, mentions, followers and note links). A reply is one compare-and-set of that document, so the hash chain can never fork between two nodes, and a conflict is retried on a fresh read (the work has no outside effect).
- **Transactions across documents** (`com.ash.drishti.identity.content.TxContentStore`, what `CollabTx` runs on). Inside `run` every write is buffered and reads see it; a failure writes nothing. The commit checks each document still has the version the work read (else `409 DRS-5057`, nothing written), writes an **intent record** `system/tx/<time>-<uuid>.json` holding every write (key, expected version, bytes) when there is more than one, applies the writes in order (each a compare-and-set), and removes the record. When another node wins one of the writes, the writes already applied are put back and the commit is `409 DRS-5057`. A

process that dies mid-commit leaves the record; `recover()` (at start and every `drishti.identity.refresh-seconds`, on any node) rolls it forward after 60 s: a write whose document still has the expected version is applied, one already there is skipped, one changed since by someone else is left out and logged.
- Index documents (`collab/entities/…`, `collab/comments/…`, `collab/mentions/…`, `collab/shares-sent/…`, `collab/shares-to/…`, `collab/inbox-seq/…`) are written in the same transaction, and readers check every entry against the document it points to, so an index without its document changes nothing.
- **Sequences** (inbox rows, outbox deliveries, holds, proposals) are counter documents (`…/counters/<name>`) advanced by compare-and-set outside any transaction: cluster-wide, never reused, never rolled back (a failed transaction leaves a gap).

#### D10.2 Notes

Notes already live in comment threads (the deprecated `/notes` API is a facade over them), so they moved with collaboration. A note's number is a create of `collab/note-links/<number>`: two nodes adding a note at once cannot take one number (the loser takes the next).

#### D10.3 Collaboration (`drishti.collab.store`: `content`, default; `jpa` or `file` for one node)

| Key | Holds |
|---|---|
| `collab/threads/<thread>.json` | the thread document (above) |
| `collab/entities/<kind>/<sha1 of entity>/<thread>` | an entity's threads |
| `collab/comments/<comment>` | the comment's thread |
| `collab/mentions/<sha1 of target>/<comment>` | who was mentioned where |
| `collab/note-links/<note, 19 digits>` | legacy note number to comment |
| `collab/shares/<share>.json` | a share and its recipients |
| `collab/shares-sent/<sha1 of sender>/<share>`, `collab/shares-to/<sha1 of user>/<share>` | sent and received lists |
| `collab/inbox/<sha1 of user>.json` | a user's inbox rows (newest `inbox.keep`), one read for the bell |
| `collab/inbox-seq/<seq, 19 digits>` | the user of each row, for another node's live hub (`after(seq)`) |
| `collab/outbox/<seq, 19 digits>.json` | one delivery; a claim is a compare-and-set to `sending` with a lease, so two nodes' dispatchers send it once; an expired lease makes it due again |
| `collab/holds/<id, 19 digits>.json` | a legal hold; a release is a compare-and-set |
| `collab/seals/<seal key>-<hash>.json` | tamper-evidence seals (thread heads, comment rows, holds) |
| `collab/counters/inbox`, `outbox`, `holds` | the sequences |

The import reads the identity database's collaboration tables (marker `system/imports/collab-jpa.json`) and, when present, the files under `drishti.collab.dir` (`collab-file.json`). The revision chains keep their hashes, so `GET /admin/collab/verify` still holds.

#### D10.4 Admin data in the identity database

| Data | Switch | Key |
|---|---|---|
| Embed apps (`drishti_embed_app`) | `drishti.embed.store` (`content` / `jpa`) | `admin/embed-apps/<id>.json`; a save is a compare-and-set against the version read, so two administrators editing one application on two nodes get one `409 DRS-5057` |
| Pack switches (`drishti_pack_state`) | `drishti.packs.state-store` (`content` / `jpa`) | `admin/pack-state.json` (every pack's state; nodes re-read it every refresh interval) |
| Audit (`drishti_audit`, and `audit.jsonl` through the earlier import into it) | `drishti.audit.store` (`content` / `jpa`) | `audit/<yyyy-MM-dd>/<node>-<nnnn>.jsonl` and `audit-heads/<node>.json` |

**Audit is append-only**, so no two nodes write one document: each node appends to its own per-day segments (a new segment past 512 KB), one JSON line per event, each chained by hash to that node's previous line; `audit-heads/<node>.json` holds the node's count and newest hash, so removed newest lines show. A read merges every node's segments newest first; the compliance verifier walks each node's chain. The node is `drishti.net.node-id` (else host and port). The table's rows are imported once (marker `system/imports/audit-jpa.json`, written first so only one node copies).

#### D10.5 Governance proposals (`drishti.governance.store`: `content`, default; `file` for one node)

`governance/proposals/P-NNNNNN.json`, numbered from `governance/counters/proposals` (raised past the imported numbers), read from the store every time. A decision is a compare-and-set: two approvers on two nodes cannot both decide one proposal (the second gets `PROPOSAL_CONFLICT`). The folder `<data>/governance` is imported once (`system/imports/governance-files.json`).

#### D10.6 Load history (`drishti.loads.store`: `content`, default; `file` for one node)

`loads/<pack>.json` (the announcements and late flags of one pack), read every time and changed by compare-and-set with a retry on a fresh read, so two nodes receiving announcements at once lose none, a batch announced on one node is a duplicate on the other, and a late notice goes out once. A pack's old file `<data>/loads/<pack>.json` is imported the first time the pack is read (the document's existence is the marker) and left in place. Per-pack overrides saved through Admin are the shared documents `loads/config/<pack>.yaml` (section D11; `drishti.loads.config-store`: `content`, default, or `file`); the folder `<data>/loads/config` is imported once (`system/imports/loads-config.json`) and kept.

#### D10.6a The VaR engine's baskets: people's and the system's

The basket store of the VaR engine (VAR_GUIDE.md sections 5.7 and 5.8) is two scopes of this store, one shape of document, read by every server:

| Key | Holds | Who writes it |
|---|---|---|
| `var/baskets/<owner>/<name>.json` | a person's basket (`BASKET:<owner>/<name>`): its sheet or scope, sharing, versions | its owner; an administrator (audited, the owner told) |
| `var/system-baskets/<ID>.json` | a **system basket** (`PORT:<ID>`), the system scope: id, name, description, owner, type (`hierarchy`, `rule`, `basket`) with its node, rule or members, rolls-up-to, valid-from and valid-to, its version and history | a holder of `var:system-baskets` (the sheet's System basket, the API, an import); audited, the owner told |
| `var/system-baskets-seq` | a counter raised by every system-basket write: each server's cache of the system scope follows it, so an edit on one server is read at the next request on every other | the store, with each write |
| `var/system-baskets-seeded` | written once when a pack's seed was imported into the empty system scope, so a server never seeds it twice | the first server to seed |

The system scope's name is visible (no leading dot or underscore) and no person's basket can take it: people's baskets live under `var/baskets/`, the system's under `var/system-baskets/`. A document that does not read as a system basket is listed (the API's `skipped`) and left out, never an error. The system baskets are no data and no configuration: YAML is only how they are imported and exported (`drishti.py var baskets import | export`); the demo's come from the market-risk pack's seed file, imported once. Moving the store to the one Drishti database (section D6) moves both scopes together, as rows of `drishti_content`.

#### D10.7 Left on each node, on purpose

Alerts and monitors (each node evaluates its own); the BI store's Parquet (a derived cache); users, roles, tokens and preferences (IdentitySync and StateSync already share them).

#### D10.8 Backup

The content store root (`data/fstore`, or the `drishti_content` table, or the bucket) is now the backup of everything above, together with the identity database for what is still there (users, roles, tokens, preferences, access log). Copy it with the server stopped, or take a filesystem snapshot: every item is one file written by rename, so a copy taken while it runs holds whole items, but a multi-document change may be half in (its intent record under `system/tx/` is copied too, and the next start finishes it).

### D11. UI-written configuration: shared overlays (built for load overrides)

The decision (2026-10-21): **Admin updates are shared across the cluster; the files stay the base (GitOps), and every edit made through Admin or the UI is an overlay in the content store that every node reads.** (Option 2 of the four that were listed: one writer node, overlay, store as the truth, a shared config folder.)

**One poller per node.** `ContentWatch` (drishti-identity) calls `changes(cursor)` every `drishti.store.watch-ms` (2 s) and hands the changed keys to the listeners whose prefix they start with; the window overlaps the last one by 2 s of clock skew, and a key is handed over only when its version moved. One walk of the store per interval, however many areas listen.

**Connectors** are not in this overlay: the file in `config/connectors` is the only truth, and an Admin save writes it (with the history and the audit entry). A node's files are its own; each node's digest carries its connector files' hashes and Admin → Health warns when a connector on two nodes differs.

**Data loads overrides** (`drishti.loads.config-store`: `content`, default; `file`): `loads/config/<pack>.yaml`, read on every use, saved by compare-and-set; the folder `<data>/loads/config` imported once (`system/imports/loads-config.json`) and kept.

**`config/bi`**: nothing is written there by the UI (BI content is in the store, section D9), so there is nothing to share.

**Not built yet (next):** the pack admin state (`packs/added.yaml`, `packs/installed/` archives materialised to a local cache on each node, `packs/settings/`, `packs/deploy-history.jsonl`, `packs/samples-mode`, `<data>/uninstalled/`, `<data>/admin/bi-hidden.txt`), with the in-place restart (`Restarter`) triggered on every node when another node installs or uninstalls a pack; report outputs (`<data>/reports`), designer bound files (`<data>/dev-sutras`) and the collaboration access log (`drishti_access`, per-node day segments merged when read, like the audit). The pack state needs the store opened in the environment post-processor (packs load before the Spring context), so each node materialises the shared state to its local folders before the packs are read.

## D6. The one Drishti database (design, not built)

Status: **design, not built** (`DrishtiDatabase`, `config/db/database.yaml`, the stamp and `db copy`); the write-back tables are built (2026-10-25) in the two existing complete schema files, on the identity database's pool (APPROVALS_GUIDE.md › The design and its reasons, section 19). To be built as **item 0 of write-back**, before write-back itself (decided by the product owner). The content store (section D5) was front-loaded from it. Related: APPROVALS_GUIDE.md (the edit log, lanes and overlays that live here), CLUSTER_GUIDE.md (several servers), CONFIGURATION.md (the settings).

### D1. The decision

All relational state that Drishti owns lives in **one database**, the *Drishti database*. Identity already lives in one; the write-back edit log, its lanes, overlays and Drishti-owned data live in the same one, and so does anything relational added later. There is no second store to back up, secure, size or switch.

- **PostgreSQL is the long-term choice** (several servers, a real deployment, backups by the bank's DBAs).
- **SQLite is the default**, for single-node setups: a Windows laptop, a demo machine, a developer. One file, nothing to install.
- **You switch by configuration at any time**, through the planned `drishti.py db copy` (section D6). The code path is the same on both: the same tables, the same SQL (with the few dialect differences kept in the two schema files and one small dialect class).




### D2. What lives in it, and what does not

| Lives in the Drishti database | Does not |
|---|---|
| Identity: users, roles, packs per user, preferences, API tokens, sessions, the audit trail | **DuckDB**, the BI engine: it is compute over the lakes and BI files, rebuilt from them, never a system of record |
| Collaboration: notes, shares, threads, comments, mentions, inbox and outbox | The **lakes** (Delta, Iceberg): source data, written only by the loader (`lake build`, `lake publish`) |
| BI workspaces, items, versions, usage and BI runs only with `drishti.bi.store: jpa` / `drishti.bi.runs.store: local` (one node); by default they live in the content store and the `drishti_bi_*` and `drishti_bi_run*` tables are legacy, read once by the import and kept (section D5 section D9); the content store's table `drishti_content` when `config/store/store.yaml` says `sqlite` or `postgres` (section D5) | **Caches** (RocksDB, layout and fingerprint caches): derived, safe to delete |
| Write-back (built): `drishti_edit_lane`, `drishti_edit_log`, `drishti_edit_request` (changes awaiting approval), `drishti_overlay`, `drishti_owned`, `drishti_edit_version` (each record's version) and `drishti_edit_cursor` (how far the reactive triggers and a follower of an edit primary have read each lane) (APPROVALS_GUIDE.md sections 5, 7, 11 and 19). New tables only: the schema version does not move | **Source systems** (ENDUR, CALYPSO, MUREX, connector databases): Drishti reads them, and writes to them only through a connector's `writes:` |
| Compute approvals (built): `drishti_compute_request` (requests to approve what a connector or pool requires) and `drishti_compute_approval` (the approved items, revocable) (BI_GUIDE.md section 6) | Locked Python environments, staged jars and request scratch folders: files under `<data dir>/compute/` |
| Administrators' switches (built): `drishti_switch` (commands and packs on or off, each with its hybrid-logical-clock stamp and node) and `drishti_switch_log` (every change, with who and why); the system of record that Drishti*net gossip spreads (ADMIN_GUIDE.md (Functions)). New tables only: the schema version does not move | |
| Anything relational added later | Files that are documents by nature (Sutras, packs, connector and net configuration): they stay files under `config/` |

The rule for a new feature: **state Drishti owns and must not lose, with rows and keys, goes in the Drishti database.** A feature never opens its own SQLite file or its own schema.

### D3. SQLite or PostgreSQL

| | SQLite (default) | PostgreSQL (long term) |
|---|---|---|
| For | one server: a laptop, Windows, a demo, a developer | several servers, production, managed backups |

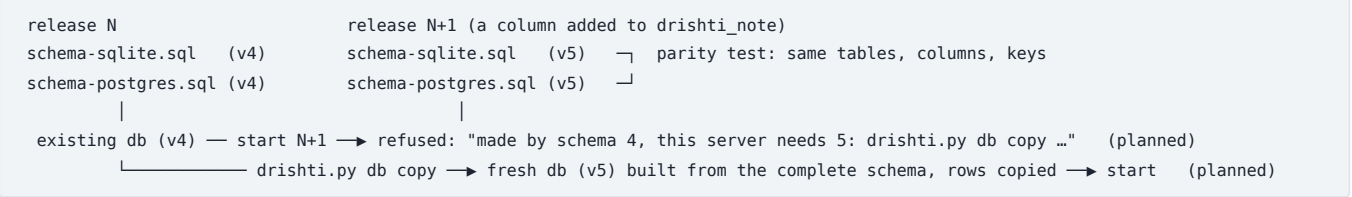
| | SQLite (default) | PostgreSQL (long term) |
|---|---|---|
| Install | nothing: a file under the data folder | a PostgreSQL 14+ server (or RDS / Aurora PostgreSQL) |
| Writers | one at a time (WAL: readers never wait; `busy_timeout`, `IMMEDIATE` transactions) | many |
| Several Drishti servers | each has its own file; nothing is shared (write-back: section D8 and APPROVALS_GUIDE.md › The design and its reasons, section 11) | all share one database: identity, roles and the edit log are cluster-wide |
| Backup | copy the file with the server stopped, or the planned `drishti.py db copy` to a new file | the DBA's backups, point-in-time recovery |

The two are equals in the code: every query is tested on both; neither is a "test" database.

### D4. No migrations: two complete schema files

**No database migrations, ever.** No Flyway, no Liquibase, no numbered `ALTER` scripts, no column fix-ups in Java.

- There are **exactly two schema files**, one per kind, each the **complete** schema: every table, column, key and index Drishti uses. They move from `drishti-identity` to the module that owns the Drishti database (`db/schema-sqlite.sql`, `db/schema-postgres.sql`).
- They are **applied at every start, idempotently**: `CREATE TABLE IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`. A fresh database gets everything; an existing one at the same version is left as it is. A **new table** in a release is simply created by this step.
- A **parity test** reads both files and checks they define the same tables, the same columns (by name, nullability and a mapped type: `TEXT`↔`VARCHAR`, `INTEGER` 0/1↔`BOOLEAN`, `INTEGER PRIMARY KEY AUTOINCREMENT`↔`BIGSERIAL`), the same primary, unique and foreign keys and the same indexes. A change to one file without the other fails the build.
- A change to an **existing table's shape** (a column added, removed, retyped, a key changed) is made in both files, bumps the schema version (section D5), and is reached by an existing database through `db copy` (section D6), never by an `ALTER`.




Why: a complete file is the truth you can read in one place; there is no chain of scripts to replay, reorder or repair, and SQLite's limited `ALTER TABLE` never shapes the design.

### D5. The schema version stamp

A one-row table holds the stamp:

```
CREATE TABLE IF NOT EXISTS drishti_schema (
    id          INTEGER   PRIMARY KEY CHECK (id = 1),
    version     INTEGER   NOT NULL,          -- the schema files' version, written in their first line
    written_by  TEXT      NOT NULL,          -- the Drishti release that built the database
    written_at  TIMESTAMP NOT NULL
);
```

At start, after the idempotent apply:

| The database | The server |
|---|---|
| empty (new) | builds it from the complete schema and stamps it with the files' version |
| stamped with the files' version | starts |
| stamped with another version (older or newer) | **refuses to start**, naming both versions and the command: `the Drishti database was made by schema 4; this server needs schema 5. Stop the servers and run: drishti.py db copy --from config/db/database.yaml --to <new database.yaml>` |
| not stamped (made by an older release) | compares its tables with the schema; if every existing table already has the schema's columns and keys it stamps it and starts, otherwise it refuses as above |

The version moves only when an existing table changes shape; a release that only adds tables keeps it.

### D6. Switching and upgrading: `db copy`

Switching from SQLite to PostgreSQL, from PostgreSQL to another PostgreSQL, or upgrading across a schema version is **one command**:

```
drishti.py db copy --from config/db/database.yaml --to config/db/database.postgres.yaml [--check] [--batch 5000]   # planned, not b
uilt
```

1. **Refuses** while a server uses the source (it checks the cluster's members and the SQLite lock); the copy needs the servers stopped.
2. **Builds a fresh target** from the complete schema of the target's kind (it refuses a target that already has tables, unless `--replace`).
3. **Copies the rows** table by table in foreign-key order, in batches, columns matched by name: a column new in the target takes its declared default; a column the target no longer has is listed and needs `--drop-columns` to proceed. Sequences (PostgreSQL) are set past the highest copied key.
4. **Checks** row counts per table and a checksum of every key column; any difference fails the copy and leaves the source untouched.
5. **Stamps** the target with the schema version and prints the line to put in `database.yaml`.

The source is never changed, so going back is pointing `database.yaml` at it again. `--check` does everything except the writes and reports what would be copied. Admin → Database shows the kind, the version, the size per table and the command for a copy.

### D7. Configuration: `config/db/database.yaml` (planned)

The database has its own file, like `config/connectors/` and `config/net/`, never in `application.yaml`. `application.yaml` keeps only the pointer:

```
drishti:
  db:
    file: ${DRISHTI_DB_FILE:./config/db/database.yaml}   # the Drishti database; ready-made files in config/db.examples/
```

The planned `config/db/database.yaml` (not shipped yet):

```
kind: sqlite                    # sqlite (default, one server) | postgres (several servers, production)
sqlite:
  file: ${drishti.data.dir}/identity/drishti.db     # under the data folder; the same file identity uses today, so an upgrade keeps
it
postgres:
  url: ${DRISHTI_DB_URL:${DRISHTI_IDENTITY_DB_URL:}}            # jdbc:postgresql://db:5432/drishti
  user: ${DRISHTI_DB_USER:${DRISHTI_IDENTITY_DB_USER:}}
  password: ${DRISHTI_DB_PASSWORD:${DRISHTI_IDENTITY_DB_PASSWORD:}}   # only a ${ENV} placeholder; a literal is refused at start
pool:
  size: 20                      # PostgreSQL; SQLite (one writer) keeps 8
  min-idle: 2
  idle-timeout: 10m
  max-lifetime: 30m
  connection-timeout: 10s
  leak-threshold: 60s
  statement-timeout: 5m         # PostgreSQL only
```

- **The identity database settings move here:** `drishti.identity.database-url`, `database-user`, `database-password` and `database-pool-size` leave `application.yaml`. The identity block keeps what is not about the database (passwords, lockout, the seed administrator, `refresh-seconds`). A server that still finds one of the moved keys set starts, uses it, and warns once naming `config/db/database.yaml` (planned); the old `DRISHTI_IDENTITY_DB_*` environment names keep working through the placeholders above.
- `drishti.db.pool` in `application.yaml` stays as the default pool of the **database connectors**; the Drishti database's own pool is the `pool:` above.
- The password is only ever a `${ENV}` placeholder (the same rule as connector files).

### D8. Several servers

| Setup | The Drishti database | Write-back's edit log (APPROVALS_GUIDE.md section 11) |
|---|---|---|
| one server | SQLite (or PostgreSQL) | in the same database |
| several servers, PostgreSQL | **one shared database**: identity and the edit log cluster-wide | in it: the full guarantee |
| several servers on AWS, no PostgreSQL | SQLite per server | S3 conditional writes |
| several servers, neither | SQLite per server | one named *edit primary* holds it; the others forward |

PostgreSQL is what makes a cluster one system for everything Drishti owns; a cluster on SQLite works, and Admin → Edits and Admin → Database say plainly what is not shared.

### D9. Today and the change

| Today | After item 0 |
|---|---|
| `IdentityDatabase` in drishti-identity opens the pool and applies `db/schema-*.sql` (idempotent, "no migrations") | a `DrishtiDatabase` opens the one pool and applies the two complete files; identity, write-back and later features take their connections from it |
| on SQLite three column fix-ups run in Java (`ensureHoldRange`, `ensureTokenScopes`, `ensureShareViewState`) | removed: the columns are in the complete schema; an older database without them is stamped only if it has them, otherwise `db copy` |
| settings in `application.yaml` under `drishti.identity.database-*` | `config/db/database.yaml` (planned); a pointer in `application.yaml` |
| no stamp, no parity test | `drishti_schema` and the parity test |

### D10. Build plan

| Step | What | Accepted when |
|---|---|---|
| 0.1 | `DrishtiDatabase` (pool, dialect, idempotent apply), the schema files moved, `drishti_schema`; identity uses it | every identity test passes on SQLite and PostgreSQL unchanged |
| 0.2 | planned: `config/db/database.yaml`, `config/db.examples/`, the pointer, the moved keys with their warning | an older data folder starts with its users, roles and tokens and no setting changed |
| 0.3 | the parity test; the column fix-ups removed | a column in one file and not the other fails the build |
| 0.4 | planned: `drishti.py db copy` (and `--check`), the refusal on a version mismatch, Admin → Database | SQLite → PostgreSQL → SQLite round trip gives identical rows; a mismatched stamp refuses to start with the command named |
| 0.5 | docs: CONFIGURATION, the admin guide, a tutorial step (UI, CLI, YAML) | the docs tests pass |

The edit-log tables of write-back (APPROVALS_GUIDE.md section 7) are added to both files in W1, on this base (built 2026-10-25, in the files under `drishti-identity` until they move; the parity test covers them).

### D11. Tests

- **Parity:** the two files define the same tables, columns, keys and indexes.
- **Idempotent:** applying the schema twice changes nothing; a new table is created on an existing database.
- **Stamp:** a database stamped with another version refuses to start and names `db copy`; an unstamped one with the right shape is stamped.
- **Copy:** SQLite → PostgreSQL → SQLite keeps every row (counts and key checksums); a dropped column needs `--drop-columns`; a running server blocks the copy.
- **Upgrade:** an older data folder starts with its users and settings; the old environment names still reach PostgreSQL.

### D12. Decisions (product owner, 2026-10-20)

1. **One Drishti database** for all Drishti-owned relational state: identity, the write-back edit log, lanes, overlays and owned data, and anything relational later. PostgreSQL long term; SQLite the default for single-node setups; switch by configuration at any time.
2. **No migrations, ever:** two complete schema files (SQLite, PostgreSQL), applied idempotently at start, kept equal by a parity test.
3. **Switching or upgrading is `drishti.py db copy` (planned):** a fresh database from the complete schema with the rows copied; a schema version stamp makes the server refuse a mismatch and name the command.
4. **Its own configuration file** (planned), `config/db/database.yaml` (`kind: sqlite | postgres`, the SQLite file under the data folder, the PostgreSQL url, user and password as `${ENV}`, the pool); the identity database settings move there; `application.yaml` keeps a pointer.
5. **DuckDB is compute, not this store.**
6. Built as **item 0**.

## D7. Loading: the data inbox and the load rules (internal)

The user-facing behaviour (the lake file, the inbox's folders and cycle, the `.ready` file, routing, business dates, modes, undo) is DATA_LAKE_GUIDE.md; this section keeps only how it is built and why. Requested by the product owner on 2026-11-03 ("an automatic data loader: a `dataload/inbox` and `dataload/outbox` folder ... every 10 minutes ... ingest them using Java into Delta ... a separate Maven module; it can work as a thread in the server"); the load rules decided 2026-11-04; the per-lake folders, the demo move and the `.ready` file's `lake:` added in 2.9.11.d (2026-11-07).

### D1. Reused, never rewritten

| Need | Reused | As |
|---|---|---|
| Read JSON Lines, `.gz`, `.zst`, `.zip`, tars, with every archive guard | `drishti-lakeload`: `Archives`, `TarStream`, `GzipMembers`, `SourceReader` | the inbox's reader |
| Write Delta and Iceberg in parallel, one commit per business date, a table lock, a run log, exit codes 0/1/2/3 | `LakeLoad`, `Loader`, `DeltaTarget`, `IcebergTarget`, `LoadConfig`, `LoadReport` | the inbox's writer, run as a child process |

| Need | Reused | As |
|---|---|---|
| A table's layout | the lake files (`drishti-lakeconfig`: `Lakes`, `LakeFiles`, `PackEntry`) and `PackLayout` | routing |
| Tell the server | `LoadService.announce` (what `POST /api/v1/packs/{pack}/loads` calls), `RefreshCoalescer`, `LoadForwarding` in a cluster | the signal, idempotent per pack, kind, date and batch |
| Schedules, switches, busy protection, temp files | the scheduler, `config/engines/*.yaml`, `AdmissionGate` (`Work.BULK_LOAD`), `TempFiles` | as every engine |

The module `drishti-dataload` only **finds, claims, routes, records and moves** files: `Inbox` (the hand-shake), `Claims` (`inprocess/`, the lock and its heartbeat), `Ledger`, `Routes`, `ReadyNote`, `LoaderProcess` (the child `LakeLoad`, behind the `LoadRunner` contract), `Outbox`, `Recovery`, `Cycle`, `DataInbox` (the schedule, Load now, Retry, the status), `InboxStatus` (the one status source of the Inbox screen), `RowForms`, `Signal`. The server keeps the wiring (`server.dataload.DataLoadConfiguration`, `DataLoadController`). `ArchitectureRulesTest` holds the module off the server, Spring, any table format and any storage: only the loader writes a table.

### D2. Safety by construction

- **One runner**: the owner node only, plus `inbox.lock` (created atomically, a heartbeat; a lock older than three intervals is taken over and logged). **Runs never overlap**: a due cycle while a run holds the lock is recorded as skipped.
- **A file is in exactly one place**: the inbox, one run's `inprocess/<run>/`, the outbox, or failed. Files are renamed into the run's folder before anything is read; the ledger line `claimed` is written first.
- **The load runs as a child Java process** (`-Dloader.main=…LakeLoad` from the server's own jar), with its own heap, `nice`, the server's temp folder and a time limit: a large or malformed file never takes the server's heap; the exit code says what happened.
- **Crash recovery** (`Recovery`): files only claimed go back; a load whose summary says it committed is completed from the summary (never loaded again; the run's `.route.json` says pack, kind and batch); not yet moved: moved; signals in `signals.jsonl` not yet sent: sent. A crash inside the loader before its summary sends the file back and it loads again: replace leaves the same data, append skips the batch's committed dates.
- **Batches**: one loader run per table (pack, kind, connector, table, date, key, lake, format); when a run of several files fails without a commit, each file is loaded again alone so one bad file fails alone. A tar's tables share one `--run-id` (one load id); a failure undoes the committed ones (`LakeUndo.undoAll`).
- **Row forms** (`RowForms`): the first line of a plain, gzip or zstd file is looked at; loader rows and lake rows are refused with the tool that loads them.
- **The warning** for a load into a lake no connector reads (`Cycle.unread`) comes from the packs' `lakesRead()`; null when the server cannot say.
- **The demo move** (`LakeRelocate`, run at start by `drishti.lakes.relocate-on-start` and by `drishti.py lake relocate`): renames only the lake's own top folders, measures every Delta table before and after (version, files, rows from statistics, every named file present), moves everything back on any difference, never deletes, records `relocated.json`.

### D3. The load rules: principles and decisions

The business date comes from outside the data (command line, `.ready`, name, business today), never from a row; a plain file is one table and one date, a tar the one container for several; the `.ready` file is the control file; nothing is guessed; every load can be undone (`_load_id`, `_source_file`). Python (`tools/lake/filedate.py`) and Java (`FileDates`) read one list of example names with their expected outcome, `tools/lake/testdata/file-dates/names.json`, so the two cannot drift. A table made before `_load_id` gains the two nullable columns on its first load (one metadata commit, no file rewritten); its earlier rows count as the oldest load. Our own generators write dated names or tars with date folders; the conversion was checked by counts and checksums per table and date.

| # | Question | Decided |
|---|---|---|
| L1 | Where does the business date come from? | outside the data only |
| L2 | What may one file hold? | a plain file one table and one date; a tar any, as `<table>/<YYYY-MM-DD>/` |
| L4 | Overrides | command line > `.ready` > name; a tar's folders cannot be overridden |
| L5 | Empty files and archives | ignored, nothing touched |
| L8 | An archive that fails part-way | rolled back: all or nothing |
| L9 | New tables | only when asked for and allowed by the inbox |
| L10 | Undo | by load id, from the command line and Admin |
| D1 | `ready.all` | every data file, with the stability check; moved to the outbox after the cycle |
| D2 | `inprocess/`, `failed/`, `ledger/` | added: needed for crash safety, bad files and duplicates |
| D3 | The load inside the server or in a child | a child process |
| D4 | A file with no rule | `failed/` with the reason, never guessed |
| D5 | The same file again | skipped by SHA-256; a changed file with the same name is loaded |
| D7 | Which nodes run it | the owner only, plus the folder lock |

### D4. The Inbox screen and its tests

The screen is mock-up A of three (A (docs/architecture/mockups/dataload-inbox/A.html), B (docs/architecture/mockups/dataload-inbox/B.html), C (docs/architecture/mockups/dataload-inbox/C.html)), chosen 2026-11-03: a status board whose columns follow the folders. It draws only the server's words (`DataInbox.status()` through `InboxStatus`); the page is listed under the engine's console paths, so it is gone while no server runs the inbox. Tests: the hand-shake, claims and the lock (two runners, a stale lock), the ledger, routing, every failure, recovery with a fault injected after each step, and an integration over a temporary data folder and lake. Still planned: the three-node and Windows (NTFS) tests.

## D8. The lake file: one description of every lake

How the lake configuration is built. Every key of the file, with examples, is DATA_LAKE_GUIDE.md section 2.

### D8.1 The rule

**A lake is configuration of its own**, not part of a connector: a Delta or Iceberg lake can be anywhere, and a reader must not own what only a writer decides. One file per lake, `config/lakes/<lake>.yaml`, says where the lake is (`root`: a folder, `s3://…`, or `mem://<store>`), how to reach it (`storage`, `catalog`) and every table in it. **Writers** (the data inbox, `lake build`, the parallel loader, the demo lakes, the official-record publishers) create and write tables exactly as it says; **readers** (the Delta and Iceberg connectors, BI, the pivot engine, Arrow Flight) find tables through it. Nobody else holds a root, a path, a partition or a promoted column, so one table can never be described in several places that disagree.

- **Every table is partitioned by `business_date` first.** It is not a setting; `partition-by` names only what comes after it.
- **Everything specific to a table lives in its entry:** path, format, partitions, promoted columns, vectors, id, date, `latest-by`, load mode, file layout, retention, maintenance, time travel, table properties, and for a `mem://` lake the cap and contract.
- **Where an entry comes from.** A pack ships recommended entries for its kinds (the `lake:` block of `pack.yaml`). When a pack is installed into a lake, or a writer first meets a kind the lake file lacks, the pack's entry is copied into the lake file and said so; from then on the lake file is the truth.
- **The demo lake is a lake like any other** (`config/lakes/demo.yaml`): `lake demo`, `build_demo_lakes.py` and the stack's synthetic lakes write through it.
- **Append keeps the replace-safety.** `load-mode: append` adds rows to the dates a load holds; each load carries a batch id recorded in the table's commit, so a batch already committed is never appended twice (a repeated file, a crash and retry); two rows of one id on one date are resolved by `latest-by` when read.

### D8.2 Who reads it

| Consumer | Uses |
|---|---|
| Delta and Iceberg connectors | `lake: <name>` and their tables (`lake-tables:`); root, paths, format, storage, promoted columns and vectors come from the lake file. A connector file keeps only reading concerns (caches, refresh, which kinds, the source name) |
| `lake build`, the parallel loader (`LoadConfig`) | `--lake` (`target.lake`); partitioning, promoted columns, vectors and file layout from the lake file. A `--partition-by` that disagrees is refused, both named |
| The data inbox | a routing rule names a lake and a kind; a new table is created exactly as the lake file says |
| The in-memory lake | a lake whose root is `mem://<store>`, its cap and contract in the same file (section D9) |
| Admin → Lakes | every lake and table, read-only (`GET /api/v1/admin/lakes/files`) |

A connector may also name tables by path (`root`, `domain`, `table.<kind>`, `layout.<kind>.*`), for tables no lake file describes; both forms are permanent. `drishti.py lake migrate-config` writes lake files from path-form connectors on request.

### D8.3 How it is built

| Part | Where | Why this way |
|---|---|---|
| The reader | module `drishti-lakeconfig` (`Lakes`, `Lake`, `LakeTable`, `LakeCheck`, `LakeFiles`, `ConnectorLake`, `PackEntry`, `LakesCli`) | a module of its own with Jackson as its only dependency: its users sit on both sides of the plugin boundary (the source registry, the loader run alone, the data inbox, the in-memory lake, the server); drishti-api stays dependency-free and drishti-common brings Spring |
| Python | `tools/lake/lakeconfig.py` runs `LakesCli` (`dump`, `table`, `check`, `migrate-config`, `add-entry`) and reads its JSON | one reader, the Java one. The one exception is `tools/lake/memory_caps.py`, which sizes the JVM before any JVM runs and reads two keys (`root`, `memory.max-mb`) |
| Several tables of one kind | an entry is `<domain>.<table>` with `kind:` (`trading.trade_endur`, `kind: trade`); connectors name entries with `lake-tables:`; the loader picks with `target.table` | the demo lake has three trade tables (ENDUR, CALYPSO, MUREX); a kind with several tables is never guessed |
| Readers | `SourceRegistry.ownSettings` expands `lake:` into the settings the plugins already read (`root`, `domain`, `table.<kind>`, `layout.<kind>.columns`, `layout.<kind>.vectors`, `s3.<key>`, `catalog`, `uri`, `warehouse`) | one point every connector's settings pass, whatever file or template they came from; the plugins keep one way of reading. A connector that names a lake and also sets one of those keys is refused |

| Part | Where | Why this way |
|---|---|---|
| Writers | `LoadConfig` (`target.lake`), `LoaderProcess` (the inbox), `lake build --lake NAME [--lake-root DIR]`, `lake demo --lake-name demo`, `build_demo_lakes.py` | `--lake-root` lets a scratch or test lake take a lake's layout |
| Append | `TableTarget.appended`; `DeltaTarget` (`withTransactionId(drishti-batch:<batch>:<date>, 1)`); `IcebergTarget` (`drishti.batch` in the snapshot summary) | the batch is per date, so a crash after some dates' commits retries only the rest. The batch rule exists once, in the Java loader: the Python writer refuses an append into a lake-file append table |
| Memory stores | `MemoryLakeSettings.withLakeStores` | the engine file keeps the switch, the replenish jobs and the compaction policy (the engine's work); the lake file keeps the description |
| Packs | every shipped pack's `lake:` block; its Delta templates `lake: demo` + `lake-tables`; `tools/packlake.py` (`lake_form` for the generators, `view` for the tools that read a template's layout); `PackEntry` | one transformation each way, so the generators and every reader of a template agree |
| Shipped copy | `drishti-lakeconfig/src/main/resources/drishti/lakes/demo.yaml`, checked equal by `settings_catalogue.py` | a server in another working folder (a container, a stack node) knows the demo lake |
| References | nested `${A:${B:x}/delta}`, resolved with the server's settings (`Lakes.useProperties`, installed by `PackEnvironmentPostProcessor`), else a dotted name as a system property or its environment form | the demo root follows the data folder and a `--setting` or test property |
| Official results | `config/lakes/banking/var-official.yaml`, `ccr-official.yaml`; `OfficialReaders` only switches the plugin | the readers follow the publisher without a second description of its store |
| Inbox loader | `Lakes.environment()`: the settings the lake files name, as the server resolves them, handed to the child process | nothing resolved twice |
| Iceberg demo store | `stores.py`: lake `demo-iceberg`, its file beside the connector folder from the packs' entries | no path-form connector left |
| Maintenance | `maintconfig.from_lake_entry` into `maintain.py`'s own keys; lake file over the maintenance file, never under the retention floor | the Delta and Iceberg maintain paths unchanged |

### D8.4 Checks

At start and in `drishti.py lake check`: every connector names a known lake; every table in a lake file matches its table on disk (format, partition columns, promoted columns present); every kind a pack serves from a lake has an entry. A writer refuses to create a table that has no entry. A partitioning that differs from the table on disk is reported, never rewritten: nothing under `data/` is changed by the lake file.

## D9. The in-memory lake: Delta tables that never touch a disk

Some feeds may not, by contract, be saved on a disk. How it is used, every setting and troubleshooting: DATA_LAKE_GUIDE.md section 13; a worked example is tutorial 16, Part D (docs/tutorials/16-load-your-own-data-into-a-new-lake.md#part-d--news-held-in-memory-only).

### D9.1 The idea

A table of a memory-only feed is an ordinary **Delta table** (its `_delta_log` and its Parquet files) whose files live in the server's memory instead of on a disk or in S3. Every Java reader and writer reaches a lake through one interface, `com.ash.drishti.lakeio.Storage` (`LocalStorage` for disk, `S3Storage` for S3); a third implementation, **`MemoryStorage`**, answers paths of the scheme **`mem://`** from off-heap memory under a **hard byte cap**. A server **replenish job** reads the feed (any connector) and writes the table with the Java lake writer, exactly as it writes a disk lake; the Delta connector reads it with `root: mem://…` like any other. When memory runs short the store gives up what it can lose without harm, in a fixed order, and asks the job to refill it; it never writes a byte to disk, and a guard refuses every other way the data could reach one. It is a general lake, not a VaR feature, though VaR may read from it.

### D9.2 Why Parquet on an in-memory file system

| Way to hold it | Memory per row | What it costs | Verdict |
|---|---|---|---|
| **Parquet files in memory** (compressed, dictionary-encoded) | the smallest: the same bytes as on disk, typically 5 to 15 times smaller than the decoded columns for trade-like data | a read decodes the pages it needs, as from disk, with no I/O | **chosen** |
| Decoded Arrow columns | 5 to 15 times more | none to read; every append and every column held whole | only as an optional bounded cache of hot columns |
| DuckDB in-memory tables (`memory_limit`) | in between | spills to temporary files when full, which the contract forbids; a whole table reloads on every change | no |
| Java objects (documents in a map) | 10 to 50 times Parquet | garbage-collection pressure | no |

Keeping the format brings the rest for free: time travel and versions, atomic commits, the same loaders, the same connector, the same footer and page codecs as the disk lake, and a table can be moved between `mem://` and disk by copying its files when a contract allows it. The measured ratios are in CLUSTER_GUIDE.md 5.8 (docs/admin/BENCHMARK.md#216-bounded-resources-of-one-server).

### D9.3 The memory store: `mem://` on Java 21

`mem://<store>/<path>`: a **store** is one named pool of memory with its own cap.

- **Files as chunks off the heap.** A file is a list of fixed-size chunks (`chunk-kb`, 1 MB by default) from the store's pool, so a file costs at most one partly filled chunk and the pool never fragments. Only the listing is on the heap (a sorted map of names to chunk lists: about 200 bytes a file).
- **The chunk pool** is direct `ByteBuffer`s allocated on demand up to the cap and **reused**: a deleted or evicted file's chunks go back to the free list at once, so the cap is exact and never waits for a garbage collection. Chunks beyond what is in use are released explicitly (`sun.misc.Unsafe.invokeCleaner`, in `jdk.unsupported`) when a cap is lowered or the pool has been idle. No preview feature is used; the pool sits behind one small interface, so a `MemorySegment` implementation could replace it on a later Java.
- **The `Storage` contract, met exactly:** sorted listings (`listFrom`), `status`, positional reads (`open` → a `RangeReader` handing the Parquet reader read-only slices of the chunks; a page spanning two chunks is the only copy), `create` (visible when closed), `createIfAbsent` **truly exclusive** (a concurrent second commit of one Delta version loses, as on local disk), `delete`.
- **The cap is checked on every chunk taken.** A write that would pass it first runs the eviction ladder (9.5); if that frees too little the write fails, the file is dropped, and since a Delta commit is the last file written, the table stays whole at its previous version.
- **The JVM's direct-memory limit** must be the server's own 256 MB plus the stores' caps: the start scripts, the Docker images and the Windows script compute it from the configuration (`tools/lake/memory_caps.py`, `DRISHTI_LAKE_MEMORY_MB`), and Admin → Health turns red when the running limit is below it.
- **A restart empties it**, by design: the replenish jobs refill it.

### D9.4 Replenish jobs and memory efficiency

A replenish job reads a source (a URL, a file, a connector's documents, or a connector entity by entity) and writes a `mem://` table with `LakeWriter`, one Delta commit a run, in mode `replace`, `append` or `merge` on a key; on a schedule, on the source's signal, at start in warm-up order, and soon after an eviction. While a table is empty or refilling its screens say so in words, never a blank or a zero that looks computed. The defaults favour bytes over speed: zstd level 3, dictionary encoding on every string column, rows sorted by the table's key before writing (runs of equal values), `keep-versions: 2` (older files deleted as soon as the next commit lands, not after a retention week), small files merged, a checkpoint every 10 commits, Parquet footers and statistics cached on the heap so a read skips row groups without touching their pages.

### D9.5 The eviction ladder and automated compaction

`max-mb` is a hard limit. When a write needs room the store frees memory in this order and stops as soon as there is enough: (1) old versions past `keep-versions`; (2) the decoded-column cache; (3) **whole tables, least recently read first**, never a pinned table and never the table being written: a table is evicted **whole, never file by file** (a Delta table missing some files would be corrupt), and its job is queued; (4) refuse the write: the commit fails and the table stays at its previous version. The store also joins the server's memory budget (`MemoryBudget`) as an **off-heap pool**, so the overload defences refuse new loads politely before the machine runs short.

**Readers see one version.** A read takes the table's version at its start and keeps its files alive until it ends (a reference count per file; the files are freed when the last reader leaves), so a replace or an eviction in the middle never breaks it.

**Compaction runs by itself.** A compactor per store wakes after every commit and every minute, scores each table by the memory a rewrite would save, and rewrites the best one first, one table at a time, as an ordinary atomic commit:

| Trigger | The rewrite | Why it saves memory |
|---|---|---|
| many small files (appends, Kafka batches) | merges them | one footer and one dictionary per column instead of hundreds, better compression |
| rows deleted or replaced (merge mode) above `dead-rows` | rewrites without them | dead rows stop costing memory |
| appends out of key order, after `resort-after` | rewrites sorted by `sort-by` | runs of equal values: dictionary and RLE shrink most columns by half again |
| old versions and the log | expires versions past `keep-versions`, writes a checkpoint | the log stays a few files |

A rewrite needs room for its new files before the old are freed, so it starts only when `headroom-mb` plus the table's estimated new size is free under the cap: compaction never pushes the store into eviction. At most one rewrite at a time per store, on one low-priority thread. For Iceberg the same compactor uses Iceberg's own rewrite and snapshot expiry.

### D9.6 Never on disk: the guard

A connector whose root is `mem://` is **memory-only**, and so is everything computed from it. One server-side check, `Retention.mayPersist(...)`, sits in front of every place Drishti writes and refuses memory-only data unless that place is allowed: the lake on disk, the connectors' disk caches, scheduled reports, e-mail and share items carrying rows, BI runs' stored results, DuckDB's spill and an outside AI provider are refused or switched off; **a user's download** is allowed, streamed from memory with no temporary file and audited; another node gets answers over Drishti*net only; logs and audit hold ids and counts, never values. The table of every sink is in DATA_LAKE_GUIDE.md 13.6.

- **BI** queries such a table through the server's Arrow reads, not DuckDB's own Delta reader (which needs files on a disk), with DuckDB's `temp_directory` off.
- **Iceberg** tables (a metadata file, manifests, Parquet data files) reach `MemoryStorage` through the same `Storage` interface (`catalog: path`, `warehouse: mem://…`).
- **The console's Python** cannot read another process's memory, and a second copy would double the memory and could disagree: it reads through the server (Arrow streams or Arrow Flight) with the server's entitlements and masks; it writes nothing.
- **The host:** no swap (or encrypted swap), no heap dump on out-of-memory unless to a RAM disk, no JFR recording or core dump to disk, the temporary folder on a RAM disk; Admin → Health checks each.

### D9.7 A cluster: the owner node binds the pack

A memory store is **owned by one node**, which binds the pack whose connectors read it (`node.yaml`: `memory-stores: [news]`), holds the store, runs its jobs and compaction, and answers its pack's reads for the whole cluster, so the memory is spent once. The other nodes do not load the store; a request for the pack's kinds is forwarded to the owner over the encrypted node-to-node link and shown, never cached on their disks, exactly as any pack a peer holds (CLUSTER_GUIDE.md, `peer-only-packs`). The code is `MemoryBinding`: local routing and the digest leave the kinds to the owner.

### D9.8 The module: drishti-memlake

The in-memory lake is one Maven module, **`drishti-memlake`**, in plain Java (no Spring; `ArchitectureRulesTest.memoryLakeIsPlainJava`):

| Package | What is in it |
|---|---|
| `com.ash.drishti.memlake.store` | the storage, `mem://`: `MemoryStores` (the stores of the JVM by name), `MemoryStorage` (one store: the `Storage` contract over chunks), `ChunkPool` (off-heap chunks under the hard cap), `MemoryStoreFullException`, and `MemoryStorageProvider` (the scheme `mem` for the lake engines) |
| `com.ash.drishti.memlake` | the service: `MemoryLake` (stores, tables, schedules, warm-up, the eviction ladder, what Admin shows), `MemoryTable`, `ReplenishSource` (what a job reads), `MemoryCompactor`, `MemoryUpkeep` (old versions expired), `MemoryBinding` (the owner node), `Retention` (the guard), `HostChecks`, `MemoryLakeSettings` (the `drishti.lake.memory.*` values, a plain record) |

**The server keeps only the wiring**, in `com.ash.drishti.server.memlake`: `MemoryLakeConfiguration` binds `drishti.lake.memory.*` onto `MemoryLakeSettings`, makes the beans (the lake, the guard, the controller), hands the lake the server's cron parser and joins it to the memory budget and busy protection; `MemoryLakeController` is Admin → Lakes' REST API. The engine switch (`config/engines/memory-lake.yaml`) acts there too.

**Which way the dependencies go.** `drishti-memlake` depends on `drishti-lakeio` and `drishti-deltalake` (it writes its tables with the lake writer and reads them back with Delta Kernel) and on the engine's connector registry. The lake readers must reach `mem://` without depending on it, which would be a cycle. So `drishti-lakeio` defines a small service interface, **`StorageProvider`** (a scheme, the storage for a path, whether the scheme is memory-only), and **`StorageProviders`**, which finds the providers on the class path once with Java's `ServiceLoader`. `drishti-memlake` lists `MemoryStorageProvider` in `META-INF/services/com.ash.drishti.lakeio.StorageProvider`. The native Delta engine, the lake writer, the Iceberg engine's file access and the Delta and Iceberg connectors ask `StorageProviders` for a path's storage and whether it is memory-only; none names a class of `drishti-memlake` (`ArchitectureRulesTest.lakeReadersDoNotSeeTheMemoryLake`). So: no cycle; one code path for every storage (a later scheme is one more provider); and it **fails closed**: a tool built without the module cannot read or write a memory table at all and gets an error naming the missing module, never a quiet detour through a disk.

### D9.9 Tests

- **The sentinel scan** (`NewsInMemorySentinelTest`): a feed whose values are unique markers, every page, export, report and BI run on it, then the data folder, the temporary folders, the logs and the dumps searched for any marker: none may be found.
- **Equality:** the same table on disk and in `mem://` gives byte-equal Parquet and equal answers in every reader.
- **The `Storage` contract suite** (`StorageContract`) passes on local disk and in memory.
- **The cap:** writes past it evict in the ladder's order and then refuse; the table is always whole at its last version; readers in flight during a replace and an eviction finish on their version.
- **Exclusive commits:** two writers of one version, exactly one wins. **Restart:** empty, warm-up refills in order. **Soak:** chunk reuse over repeated replaces, no growth.

### D9.10 Not built yet

- The sizing workbook's line for the stores; the decoded-column cache (the ladder goes from old versions to whole tables).
- Compaction's detection of a dictionary that fell back to plain encoding (a rewrite rebuilds dictionaries, but the trigger is missing).
- The cluster-wide *its node is restarting* words (a peer's screen shows the usual peer-unavailable words); a standby second owner.
- Iceberg on `mem://` end to end: replenish jobs write Delta only; Iceberg's table commit, catalog and maintenance on `mem://` are not tested.
- The guard's derived-data tag on VaR and exposure captures, official records and write-back from memory-only inputs; the memory-only VaR hot tier (VaR inputs on `mem://` still build their hot file on disk).
- The *refilling* words on the Sutra pages themselves (the connector's health and Admin → Lakes have them).
- Native Memory Tracking compared with the store's own count over a long soak.

## D10. The parallel lake loader (drishti-lakeload)

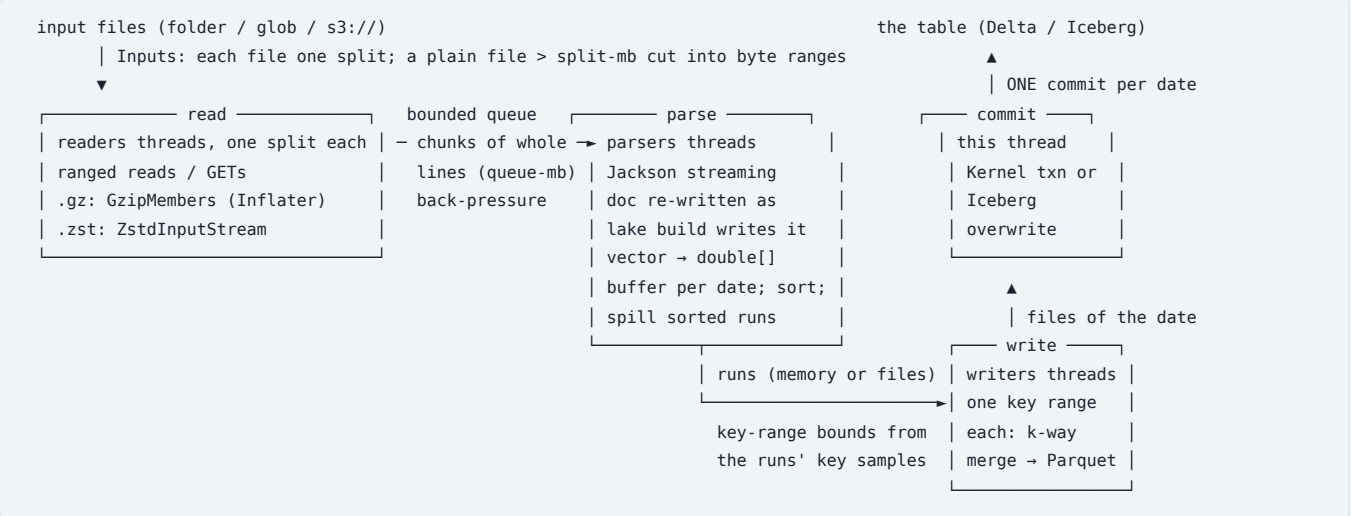
How the Java loader is built: module `drishti-lakeload`, main class `com.ash.drishti.lakeload.LakeLoad`, in the server's exec jar. How to run it: DATA_LAKE_GUIDE.md 3.3; every key of a load file: CLI_REFERENCE.md section 4.13. Its target: the bank's full scenario-PV load (2.3 million positions × about 510 scenarios, about 1.2 billion doubles, 20+ GB of JSON Lines) in at most 10 minutes on a real server, into Delta or Iceberg, from a folder or S3, with memory bounded by configuration whatever the input's size.

### D10.1 Where it lives

| Needs | For | Why not elsewhere |
|---|---|---|
| `drishti-lakeio` | local and S3 reads (`Storage`, `RangeReader`: ranged GETs), S3 settings, the Parquet page codecs | shared by both lake engines already |
| `drishti-iceberg` | the Hadoop-free Parquet writer (`ParquetDataWriter`: `double[]` vectors, UTF-8 text, per-column encodings), catalogs, the Iceberg commit (`Operations.replacePartitions`) | the native Iceberg engine; the Delta files are written by the same writer |
| `drishti-deltalake` | Delta Kernel's transaction and the engine `LakeWriter` commits with (`LakeWriter.engineFor`, `LakeWriter.removals`) | Kernel stays the only Delta committer |
| Jackson (core, YAML), zstd-jni | streaming JSON, the configuration, `.zst` input | already on the server's class path |

Not in `drishti-deltalake` (it would then need the Iceberg engine) nor in the Iceberg plugin (it carries Apache Iceberg Java and Hadoop's S3 file system; the loader needs neither). No third-party dependency of its own.

### D10.2 The pipeline




**Read.** `Inputs` resolves `source.files` (a file, a folder, a glob in the last segments; on S3 a prefix or a glob in the last segment) and sniffs each file's first bytes (gzip `1f 8b`, zstd `28 b5 2f fd`, else plain). A plain file larger than `parallel.split-mb` (256 MB) is cut into byte ranges; a range owns the lines that **start** in it (it reads from one byte before its start, drops the partial line there, and reads past its end to finish its last line). A compressed file is one split, **decompressed as a stream** by the reader that has it, with no temporary copy; `GzipMembers` reads every member of a multi-member gzip (the JDK's `GZIPInputStream` decides whether another member follows from `available()`, which a network stream may answer with 0). Readers (`parallel.readers`, default the cores) take splits largest first, so a big `.gz` starts at once, and cut their streams into chunks of whole lines on one shared queue.

**Parse.** `parallel.parsers` threads take chunks from any split, so one large `.gz` is parsed by every parser. `RowParser` makes one streaming pass per line (Jackson's parser over the bytes, no tree, its fast double parser): the document is written again exactly as `lake build` keeps it (`PyJson`: Python's `json.dumps(ensure_ascii=False, separators=(",", ":"))`, a float as Python's `repr`), the wanted fields are caught on the way (a trie of the id, date, promoted, partition, vector and flattened paths), a top-level vector's numbers go straight into a `double[]` (a null element is NaN, written as null) and the document keeps `{"$vector": "<column>"}` in its place. A row becomes one record in a reusable byte buffer (`RowPlan`), never an object per value.

**Sort.** Each parser buffers its records per business date in 8 MB arenas (`Runs.Buffer`); when its share of the sort buffer is full it sorts each date's records by key (a merge sort over references, an 8-byte key prefix deciding most comparisons) and writes them as a **sorted run** file in the spill folder. What is left after the input becomes in-memory runs. Each run keeps a key sample every 1 MB.

**Write.** Per date: if the date has more runs than the writers can read at once within their memory share, runs are merged into fewer first (`Runs.reduce`: the passes of an external merge sort). The date is then cut into **key ranges** of about equal bytes from the runs' key samples (`Runs.bounds`: about `2 × writers` ranges): the whole distribution, with no second pass over the input and no guess from a first sample. Each range is one task for `parallel.writers` threads: a k-way merge of every run within the range, written as Parquet into files of about `target.layout.file-size-mb`, a new file also when the Delta partition values change and, with `shard-aligned`, at a group boundary (10.6). Files follow the sort order, never the inputs. A file is uploaded as it is written (S3 multipart).

**Commit.** Only the main thread commits, after every file of the date is written (10.5). Every stage is metered (`Meter`: bytes, rows, busy time over its threads, wall time); the summary's `throughput` has read, decompress, parse, sortSpill, mergePasses, write and commit.

### D10.3 Memory

One setting bounds the load whatever its input: **`memory.budget-mb`** (default half of `-Xmx`), shared as

| Share | Default | Holds |
|---|---|---|
| the queue | 15 % (`parallel.queue-mb`) | half for chunks waiting to be parsed (the semaphore that blocks the readers: back-pressure), a quarter for the chunks readers are filling, a quarter for their inflate buffers |
| the sort buffers | 55 % (`memory.sort-buffer-mb`) | the parsers' unsorted records, `budget / parsers` each; past it the parser sorts and spills |
| the writers | 30 % (the rest) | half for reading runs while merging (each writer reads each run of its range through `share / (writers × runs)`, 8 KB to 1 MB; more runs than fit at 64 KB each are first merged into fewer), half for the Parquet row groups being filled |

An explicit `parallel.queue-mb` or `memory.sort-buffer-mb` must leave the writers at least a tenth of the budget. The JVM's own needs (classes, Kernel, the S3 client, the per-row duplicate arrays of about 36 bytes a row, freed before the write) come on top, so `-Xmx` should be the budget plus 1 to 2 GB. The summary's `memory` reports the shares, the peak heap used (sampled every 20 ms), the peak heap after GC and the MB spilled. A 10,000-position load ran in a `-Xmx192m` JVM with a 32 MB budget and wrote the same table as with 7.8 GB.

**Spill space.** Before reading, the loader estimates the spill (the input's uncompressed size, compressed files at 6 to 1 for gzip and 8 to 1 for zstd, less the sort buffers, times 1.1) and stops with a clear message when `memory.spill-dir` has less free space (`memory.check-spill-space: false` to try anyway). The spill folder is a fresh `drishti-lakeload-<pid>-<n>` under `memory.spill-dir` (default `java.io.tmpdir`, which `lake build` points into the run's staging folder): local disk, EBS, or a Fargate task's ephemeral storage.

### D10.4 The tables it writes

Exactly the layout the existing readers expect.

| | Delta | Iceberg |
|---|---|---|
| Columns | `id`, `doc`, `business_date` (partition), the partition columns (partition), the promoted columns (`double` when every value is a number and the field is not an id, else `string`; null on every row: `double`), the vector columns `array<double>` (nullable elements) | `IcebergLayout`: field ids 1 `id` (required), 2 `doc`, 3 `business_date` (required, identity partition), then the partition fields as `string`, the promoted columns (null on every row: `string`), the vector columns `list<double>` (optional elements) |
| Text of a number in a text column | Python's (`json.dumps`: `42`, `1.5`, `1e-05`) | the Iceberg loader's (`42` for a whole number) |
| An existing table | keeps its schema and partitioning: a load column it lacks is left out (`columns.leftOut`), a column the load lacks is null; another partitioning is refused | its columns kept; the missing ones added (one schema commit), as `NativeLayout.open` |
| New table | created by the first date's commit, with `target.properties` | created with the layout's properties (zstd, 1 MB row groups, metrics: `doc` none, `id` full), sort order by id |
| Parquet | `LakeWriter`'s row groups and pages (`DRISHTI_LAKE_ROW_GROUP_KB` 256, `DRISHTI_LAKE_PAGE_KB` 64), Snappy; exact statistics for every scalar column, none for `doc` | the table's properties |
| The vector column | BYTE_STREAM_SPLIT (a double's 8 bytes as 8 streams), ZSTD, no dictionary, no statistics | the same |
| Partition folders and paths | the Delta protocol's: Kernel's write context hands the folder as a URI (`DataWriteContext.getTargetDirectory`), `LakeWriter.targetDirectory` decodes it once, so the folder is the Hive-escaped value Spark writes and `add.path` is that path encoded once; an empty value is null (`__HIVE_DEFAULT_PARTITION__`). Every path Kernel hands a reader is plain and is never decoded again (`LocalPaths.decodeUri` decodes only a `java.nio file://` URI). Examples: DATA_LAKE_GUIDE.md section 14.2 | Iceberg's own (`identity` partition paths) |

BYTE_STREAM_SPLIT is a standard Parquet encoding read by parquet-java, Kernel, Arrow and DuckDB. Where the Python Iceberg path rejects a row (a promoted field holding a list) or drops a value (an object), the loader keeps the JSON text; it makes a column for a field null on every row. A NaN vector element is stored as null. A JSON object with duplicate keys keeps both in the document; a partition value that is an object is its compact JSON.

### D10.5 Commits: one per business date

A reader (the connectors, the VaR loader, the cluster's shards) must see a date whole or not at all, and a re-run must replace the date, never add to it. One commit per date does both; N committers would race on the table's log and could leave a date half-replaced. The commit lists files, never rows, so it is cheap.

- **Delta.** Per date one Kernel transaction (`CREATE_TABLE` for a new table, else `WRITE`), the files written into the folders Kernel names (`Transaction.getWriteContext`), then one `commit` with the add actions (`Transaction.generateAppendActions`) and the remove actions of every earlier file of the date (`LakeWriter.removals`). Kernel's post-commit hooks (checkpoints) run as for `lake build`. On S3 the log entry is an exclusive create (`StorageWriteFileIO`, conditional PUT), so two writers of one version cannot both win.
- **Iceberg.** Per date one `Operations.replacePartitions`: the date's live files out, the new ones in, one snapshot (`overwrite`, summary `drishti.business-dates`), retried on a concurrent commit.

Dates are written one after another, each with every writer thread; a load of 60 dates is 60 commits, each date visible as soon as it is committed. The loader's mode is replace-date (`lake build`'s `overwrite-dates`); an append table takes a batch id (section D8.1).

**Duplicate records** (one id twice on one business date) follow one rule on the Java and the Python path, so both write the same table: the copy with the latest `COMPUTED_AT` (`source.latest-by`; ISO-8601 text, no offset is UTC) is kept, else the last one read in one deterministic order (files sorted by path, then line order; a copy with a timestamp beats one without); the others are dropped, counted per date (`duplicates`) and named (up to ten `duplicateExamples`, `file:line`), never a failure. Each row record ends with its place in the input (split, chunk, line); after the read the ids seen twice (64-bit hashes sorted, exact, no cap; a second hash tells two ids with one first hash apart) are resolved from per-row arrays, and the write stage skips the losers by their place. The Python path stages every row with its place and re-reads only the dates with duplicates (`stage.Stager.resolve`). Rejects are named `file:line`, counted in the file also for byte ranges.

### D10.6 Shard-aligned layout and the load manifest

`target.layout.sort-by` orders each date's rows: the pack's `layout.<kind>.sort-by` if it names one, else **`LEGAL_ENTITY, DESK, id`** (those the rows promote; the id alone when neither is). Every file is a contiguous key range, so its Parquet statistics (and the Delta `stats`, the Iceberg manifest bounds) of `LEGAL_ENTITY` and `DESK` let a reader skip every file outside its legal entities and desks. With `shard-aligned: true` (default) a file is also cut where the group (the sort fields before the id) changes once it holds a quarter of `file-size-mb`: a large desk gets files of its own, small desks share one. Range bounds between writers fall on any key, so a desk may span the last file of one range and the first of the next.

The **load manifest** (`<summary>.manifest.json`, or `report.manifest`) lists, per committed date, the commit's version and every file in key order: `path`, `rows`, `bytes`, `partition` (Delta), and `first` and `last` (the partition columns, the sort fields and `id`). `groupBy` names the shard group. A shard owner may read only the files whose `first..last` range meets its groups; the files' own statistics say the same for a reader without the manifest.

```
{"table": "s3://bank-lake/risk/scenario-pv", "format": "iceberg", "sortBy": ["LEGAL_ENTITY", "DESK", "id"], "groupBy": ["LEGAL_ENTI
TY", "DESK"],
 "dates": {"2026-10-28": {"version": 6012998402214350812, "files": [
   {"path": "s3://…/data/business_date=2026-10-28/2026-10-28-r000-00000-….parquet", "rows": 31840, "bytes": 268170210,
    "first": {"LEGAL_ENTITY": "LE-LDN", "DESK": "DESK-COMM", "id": "P-1000017"}, "last": {"LEGAL_ENTITY": "LE-LDN", "DESK": "DESK-C
OMM", "id": "P-1799862"}}]}}}
```

An Iceberg table sorted by more than the id gets the property `drishti.layout.sort-by`; `lake maintain`'s relayout (which re-sorts days by id) leaves such a table in its order (`IcebergMaintenance.shardSorted`). A Delta table's compaction merges only small files (under `target-file-mb`); a pack layout that names `sort-by` for the kind should name the same columns.

### D10.7 Files and S3

Local paths (Windows paths too: `C:\data\pv\*.jsonl.gz`) and `file:` URIs read through `LocalStorage`; `s3://` (and `s3a://`) through `S3Storage`: a plain object's splits as parallel ranged GETs, a compressed object as a stream of 64 KB to 8 MB reads. Writing: Parquet through the same `Storage` (`S3Output`: multipart upload while the file is written), the Delta log through Kernel's `StorageWriteFileIO`, Iceberg metadata through drishti-iceberg's commit. S3 settings come from `source.storage-options` / `target.storage-options` and **credentials only from the AWS default chain** (environment, profile, instance or task role): a key in the file is refused. A REST catalog's secret is given as `${AN_ENVIRONMENT_VARIABLE}`. No Hadoop anywhere, so it runs on Windows (zstd-jni and snappy-java are JNI libraries inside their jars, no `.exe`).

### D10.8 Failure and clean-up

- A failure before a date's commit leaves that date (and every later one) as it was: its files written so far are in no version or snapshot. They are orphans, removed after their retention by Delta's vacuum or Iceberg's orphan-file removal (`lake maintain`). Dates committed before the failure stay committed, each whole.
- Too many rejects (`rejects.max-pct`) stop the load after the read, before anything is written.
- The spill folder is removed on success, on failure, and when the JVM is stopped (a shutdown hook: Ctrl-C, SIGTERM, a stopped task).
- A reader or parser that fails stops the others at once (the queue never leaves a thread blocked); exit code 1 with the reason in one line (`-Ddrishti.loader.trace=true` prints the stack).
- **The lock:** a local lake gets `lake build`'s lock (shared on `<lake>/_drishti-lake/lock`, exclusive on `locks/<domain>/<table>.lock`; the whole lake on Windows). `lake build` takes a POSIX lock (`lockf`) beside its `flock` on the same files, because Java cannot take `flock`; so a direct loader run and a Python build or `lake maintain` exclude each other (exit 3 for the later one).

### D10.9 Integration with lake build

`drishti.py lake build --engine java` runs the whole build in the loader (`tools/lake/javaload.py`): the build's options become the loader's configuration (a JSON file in the run's staging folder, secrets kept out of it); `--loader-set KEY=VALUE` adds any key. The build holds the lock and writes the run log, the summary (the loader's keys merged in, plus `throughput` and `memory`) and the ownership record. The Python path is the default; `--engine auto` picks the loader only with `--java-above-mb N` (every input JSON Lines of at least N MB, `--mode overwrite-dates`, a jar with the loader). Java is started through `tools/javacmd.py` with `-Djava.io.tmpdir` in the staging folder; the heap is `DRISHTI_LAKE_LOADER_HEAP` (default `4g`). The loader reads JSON Lines only (the data standard); a JSON array stays the Python path's.

### D10.10 Measured, and the 2.3M estimate

`ThroughputTest` loads 10,000 positions × 510 scenarios (53.7 MB of JSON Lines) into Delta: warm, 0.19 to 0.29 s, 34,000 to 52,000 rows/s, about 92 µs of CPU a position over decompress, parse and write. Scaled to 2.3M positions that is about 345 CPU-seconds: about 1 to 2 minutes on a 32 vCPU server with local NVMe, 4 to 5 minutes on 16 vCPU with a slow spill disk, inside the 10-minute target. The stages, the server table, the spill and heap advice: BENCHMARK.md section 7.

### D10.11 Tests

`./mvnw -o -q -pl drishti-lakeload test`:

| Test | What |
|---|---|
| `PythonPathEqualityTest` | `drishti.py lake build` both ways (the Python path, Delta and Iceberg; the loader) on synthetic positions (escapes, non-ASCII, number ids, nulls in vectors, a reject, duplicate records with and without `COMPUTED_AT`, an uncompressed copy beside its `.gz`) and on the market-risk pack's scenario PVs: equal tables, rejects, rows, sample; the Delta and Iceberg connectors read both alike |
| `LakeLoadTest` | one commit per date and a re-run replacing only its date; a load stopped after its first files leaving the table as it was; rejects counted, named, written and `max-pct`; identical tables from one file, byte ranges, one `.gz`, a multi-member `.gz`, six `.gz` files, `.zst`, 1 thread vs many, a 1 MB budget (spills) |
| `S3LoadTest` | S3Mock in the JVM: a glob of `.gz` objects and a plain object read as ranged GETs, written to `s3://` as Delta and as Iceberg, equal to the local load; keys in the file refused |
| `ThroughputTest` | the measurements above, BYTE_STREAM_SPLIT and ZSTD on the vector column, the manifest's key order, a 24 MB budget and a `-Xmx192m` JVM giving the identical table |
| `CodecsTest` | Python's float repr and string escapes, multi-member gzip and a corrupt CRC, globs, sort keys, the pack's layout, the configuration's checks |
| `tools/lake/test_javaload.py` | the build's options as the loader's configuration, secrets kept out of the file, when `auto` picks the loader |
| `tools/lake/test_delta_interop.py` | the loader's Delta tables against delta-rs both ways, with 47 partition values that need escaping, and the features each reader reads (DATA_LAKE_GUIDE.md section 14) |
| `ShardSortedLayoutTest` (Iceberg plugin) | relayout leaves a sort-by table in its order |

### D10.12 Open

- The 2.3M measurement on a real server; the estimate above is extrapolated from 10K.
- A zstd input is one split; a seekable-zstd file could be split into ranges like a plain one.
- S3 reads of a compressed object are sequential per object; a read-ahead would overlap the network and the inflate.
- The read and write phases do not overlap within a date (the sort needs the whole date); dates of a backfill could overlap.

## D11. Arrow Flight: Drishti's data to any client

How the Flight service is built. Turning it on, tokens, the cluster, limits and operations: FLIGHT_GUIDE.md; every request form and the clients: FLIGHT_REFERENCE.md.

### D11.1 Principles

1. **One service, a transport only.** Module `drishti-flight` answers Arrow Flight over the existing query layer (kinds, BI model queries, the pivot engine, VaR and the other measures, the in-memory lake) with the same entitlements, masks, busy protection and audit. No query logic is written twice.
2. **Any client:** Python (`pyarrow.flight`), Java, C#, R, Go; JDBC and ODBC tools through Flight SQL. Browsers keep Arrow over HTTP and live views over Server-Sent Events.
3. **Read only.** Nothing a client sends changes Drishti's data.
4. **One Arrow library.** Apache Arrow Java is used for every Arrow artifact: `ArrowIpc` (compute) and `ArrowIpcWriter` (server) are written with `arrow-vector`; the console passes the server's bytes through and writes no Arrow (`core/arrow_answer.py` keeps `wanted`, `decode`, `pass_on`).

### D11.2 Design

| Question | Design |
|---|---|
| What a request names | **Rachana, with SQL beside it, one layer under both:** a ticket is a Rachana request (the same JSON as `POST /api/v1/pivot/query`, a kind with paths, or a measure call); Flight SQL accepts read-only SQL over models (DSQL), compiled to the same plans |
| What is exposed | kinds, BI models, pivot views, VaR vectors and the in-memory lake; exposure paths, FRTB-SA and SIMM detail planned |
| Who may call | **API tokens only** (scope `data:read` or `data:read:<domain>`), an application's token on behalf of a person, embed tokens; never a browser cookie |
| Port and TLS | its own port (default 18815), **TLS always** (the server's certificate), mutual TLS optional; the engine `flight`, off as shipped (`config/engines/flight.yaml`, Admin → Engines) |
| The cluster | `GetFlightInfo` lists the endpoints (each node's real address); the client reads them in parallel; a part on a node that is down is read from a peer that holds it |
| VaR precision | as the engine computes: scope and node vectors float64; position and trade rows float32 as stored, the schema saying so; a test proves a client's sum of the rows equals the engine's vector within the parity rule |
| Live streams | `DoGet` with `stream: true` (intraday batches, in-memory lake appends), resumable by a sequence number; resume is stateless (VaR: the date's intraday stamp; lake: a cursor column) |
| Limits | per read, from the token's role: rows, bytes, time, Arrow off-heap memory; per node: total Arrow memory; counted by busy protection (`Work.FLIGHT`) |
| Audit | every read: who, on behalf of whom, the request, rows and bytes, the node (`data.flight`); Admin → Data out shows live streams and the last day's readers |
| Java 21 | `arrow-memory` needs `--add-opens=java.base/java.nio=ALL-UNNAMED`: in the server jar's manifest (`Add-Opens`) and the test forks |

### D11.3 How it is built

| Part | Where |
|---|---|
| The service | `FlightService` (TLS always, its own port), `DrishtiFlightProducer`, `FlightBackend` (all the server provides) |
| Tickets | `FlightRequest`: a form (`kind`, `model`, `pivot`, `measure`, `lake`) and a Rachana request |
| The forms | `server.flight`: `KindFlightForm` (Export data's reader, a download's checks), `AnswerFlightForms` (MQ, the pivot engine), `VarFlightForm` (`var.scenarios` float64, `var.parts` float32 for positions and trades), `LakeFlightForm` (a table found through the lake file, section D8) |
| Tokens | `TokenAuthenticator` (the handshake, a session after it) over `ServerFlightBackend.authenticate` |
| The cluster | `FlightNodes`: this node and the live Drishti*net members by their real addresses; the Python client reads parts in parallel and tries the next address of a part |
| Limits | `FlightLimits` per role; Arrow memory per read and per node (`ArrowMemory`, Health's `arrow-flight`) |
| Audit and Admin → Data out | `data.flight` per read; `FlightActivity`; one screen over `GET /api/v1/admin/data-out` with two views: the page (default; mock-up A (docs/architecture/mockups/data-out/A.html): the service, the nodes, live streams with Stop, the last day's readers, tokens and limits, in order of urgency) and the map (mock-up B (docs/architecture/mockups/data-out/B.html): clients and nodes with a line per stream) |
| Flight SQL | `DrishtiSqlProducer` on the same port: each statement a `form: model` request with its SQL, answered by `SqlService` (DSQL) and read through the same producer; one catalog, the models as schemas, their aliases as tables (`ServerFlightSqlCatalog`); the model by a first-line comment or the header `model` |

## D12. Computed and derived attributes

Every number on a market-risk or counterparty-risk page is one of three classes, and the class decides where it is made. This is the rule the packs follow and the mechanisms that apply it; the open steps (the engines still to build, the stored figures still to retire) are in IMPLEMENTATION_PLAN.md, "Computed and derived attributes: open steps".

### D12.1 Three classes, and the rule for deciding which

| Class | What it is | Who makes it | Stored? | Examples |
|---|---|---|---|---|
| **Fundamental** | a fact delivered by a source system or an upstream engine at the grain that engine owns; not reproducible from anything Drishti holds | source systems, upstream pricing and simulation engines | **yes**, with a contract | a trade's `MTM`, a CSA's thresholds, a collateral holding, a limit amount, an exposure path, a P&L vector |
| **Derived** | a deterministic function of fields Drishti holds: a sum, count, max, difference, ratio or pick; no model, no parameters beyond a filter | Drishti, from configuration: a Sutra expression (one document) or a derived kind (across documents) | **no** | a netting set's net MtM = Σ trade MtM; utilisation = used ÷ limit; held collateral = Σ holdings' values |
| **Computed** | needs an engine: scenarios, paths, quantiles, correlations, regulatory parameter tables, any scope, any date, what-if, per-reader entitlements | the engines (reactive connectors, Java or Python) | **no** (only as an immutable capture or a published official result) | VaR, ES, EE, PFE, EPE, SA-CCR EAD, SIMM IM, the FRTB charge, CVA |

The contracts' four data kinds (contracts README section D3 (docs/contracts/README.md#3-four-kinds-of-data)) map onto these: fundamental and input are fundamental; Drishti-owned covers Drishti's derived records (positions) and definitions kept in Drishti (the system baskets, people's baskets); computed is computed. Ask in order; the first "yes" decides:

1. Delivered by a system of record and not reproducible from anything Drishti holds? **Fundamental**: stored, with a contract.
2. A pure function of fields of **one** document? **Derived, in the Sutra** (Rachana-EL), never stored on the document.
3. A pure, additive function of fields of **several** documents of one kind grouped by a key, shared alike by every reader? **Derived kind** (`plugin: derived`).
4. Needs a non-additive statistic, a parameter table, per-path or per-scenario arithmetic, FX by scenario, a join across kinds with rules, an arbitrary scope, a what-if or per-reader entitlements? **Computed** by an engine.

Two corollaries: **a derived attribute over a computed input is computed** (the utilisation of a PFE limit divides by a quantile of summed paths, so it belongs to the limits engine; a Sutra may still show the division of two numbers already on the page as a *check*); **membership is fundamental, totals are not** (which trade is in which netting set is a legal fact; the set's net MtM is a derived total). Stored copies of derived or computed numbers are avoided because they give two numbers for one thing, go stale when the members move, cannot be rebuilt or explained, leak sums over trades a reader may not see, and freeze a method the measure's definition forbids.

### D12.2 The mechanisms

| Mechanism | Example | Runs | Refreshes | Scope | Entitlements | Limits |
|---|---|---|---|---|---|---|
| **Sutra expression** (Rachana-EL in `bind:`, `title:`, `when:`) | `size($.DEFINITION.MEMBERS)` on the system basket page | in the server, as the page is built from one document | every render | one document | the reader's | Rachana's function set |
| **Derived kind** (`plugin: derived`, `DerivedSourcePlugin`) | `counterparty-trades` grouped by `$.COUNTERPARTY` from `trade`; `nset-mtm`, `cpty-mtm`, `stress-scenario-pnl` | in the server, reading every member through the router | once per business date and data version, else every `refresh`; a reader never waits (served stale, refreshed behind) | one page per group key | **built once per date and shared by every reader** | `max-scan` documents per kind; aggregates `count`, `sum`, `avg`, `median`, `min`, `max`, `distinct`, `first`; no join to another kind |
| **Reactive engine** (`plugin: reactive`) | `var-engine`, `exposure-engine` and their what-ifs | Java in the server or Python on the compute pools | on request; cached; recomputed on `data-version` triggers of its input kinds, coalesced | any scope, any date, parameters | **as the requesting user** | timeouts, in-flight limits per user, scope sizes |
| **Engine figures on a stored page** | `VarFigures` (`$.VAR_ENGINE` on position and system basket pages), `ExposureFigures` (`$.EXPOSURE_ENGINE` on netting-set and counterparty pages) | as the page opens | the engine's cache | the page's own entity as a scope | as the reader | the engine's |
| **BI measure** | `MTM: sum(trades.MTM)` in a model | the BI engine over datasets | on load | any slice | dataset row rules and masks | additive measures only; a non-additive figure comes from an engine's published result |

Where each class belongs: fundamental, a stored kind with a contract; derived within one document, a Sutra expression; derived across documents, a derived kind bound by a panel's `source:` (or the measure's engine when per-reader totals or arbitrary scopes are needed); computed, a reactive engine and its figures on the stored page (BI only over an engine's published official results). Never: a page working out a fundamental, a derived figure stored beside its inputs, a computed figure stored on a fundamental record or summed in BI when it does not add up.

### D12.3 Shared definitions

| Symbol | Definition |
|---|---|
| *d* | the business date asked for (`asOf`), else the newest date present |
| **Live**(*t*, *d*) | `TRADE_STATUS` ∈ {`NEW`, `VALIDATED`, `AMENDED`} **and** `MATURITY_DATE` ≥ *d* (or empty): one rule for every figure. A live-status trade past its maturity date is in no total and is counted and listed as a data-quality warning (the exposure engine flags its set `PAST_MATURITY`) |
| MTM(*t*, *d*) | `trade-valuation.MTM` of trade *t* on *d*, USD, dirty, the bank's side, positive = asset |
| NS(*t*, *d*) | the netting set of *t* on *d*: `trade-valuation.NETTING_SET` |

| Symbol | Definition |
|---|---|
| Σ over a set | float64 accumulation; a member with no value is **counted as missing**, never as 0 |
| Fraction | a ratio is stored and passed as a fraction (0.667), formatted as a percentage only on screen |

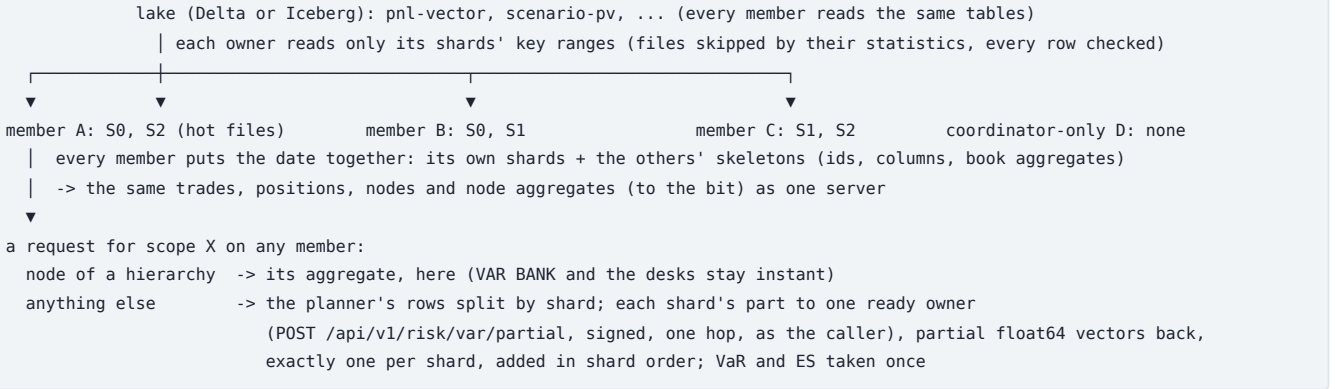
**Sum currency.** Every sum (net and gross MtM, collateral, exposure, limits used, stress P&L, notionals, counterparty and group totals) is made in **USD by default**, the currency said beside it (`SUM_CURRENCY`); each member is converted at the market-data pack's close mid for *d* **before** it is added, never added unconverted, and a member that cannot be converted is counted as missing. Ratios take both amounts in one currency first. A derived kind reads one source kind and cannot join FX, so a derived total over members in more than one currency says *not converted* (`CURRENCY_NOTE`); the converted figure is an engine's. The per-entity override (a netting set or counterparty asking for CAD or EUR) is designed in the open steps.

### D12.4 What the packs build this way

- **`nset-mtm` (`NSMTM`)**, connector `ccr-derived` of the counterparty-risk pack (generated by `tools/packgen/banking/ccr_kinds.py`): one page per netting set from the trade valuations of a live status: net MtM, trade count, gross positive and negative MtM, uncollateralised current exposure, netting benefit, net-to-gross, by product group, the member rows, the past-maturity count and MtM, the sum currency. Fields: its contract (docs/contracts/packs/counterparty-risk.md).
- **`cpty-mtm` (`CPMTM`)**, the same connector: one page per counterparty from the trade valuations directly (a derived kind cannot read another kind of its own connector): net MtM, trade count, gross parts, the netting sets as `group()` rows, uncollateralised CE = Σ over the sets of max(set net, 0).
- **`stress-scenario-pnl` (`SSPNL`)**, connector `mr-derived` of the market-risk pack (`tools/packgen/banking/var_kinds.py`): one page per scenario: the bank's P&L (Σ desks), the desks' P&L, limit use and status, the worst desk, the breaches.
- **Pages** (`tools/packgen/banking/risk_data.py`): the netting set shows *Computed from the trade valuations, beside the stored figures* (computed, stored and difference), net MtM by product group and the live-status trades past maturity; the counterparty shows its computed figures and netting sets; **checks** panels on the collateral balance (held, posted and net after haircut from the positions), the credit limit (headroom, utilisation and its difference from the stored one), SA-CCR (the EAD identity α(RC + multiplier × add-on) and add-on = Σ by asset class) and the stress result (limit use, status, headroom, Σ P&L by book). The derived pages are built once per business date and data version and shared: every user who may open the kind sees them.
- **The data rules** the generators and contracts apply: stress limits are positive magnitudes; collateral balances (`HELD`, `POSTED`, `NET`) and `netting-set.COLLATERAL` are the positions' values after haircut; one CSA per agreement and legal entity (`CSA-<counterparty>-<legal entity>`, named by the netting set, the collateral balance and the margin call; an agreement lists its `CSAS`); CSA terms are numeric with a currency (`CSA_CURRENCY`, `MTA_US`, `MTA_THEM`, `MARGIN_FREQUENCY_DAYS`, `MPOR_DAYS`); maturity buckets are named sets per purpose (`display`, `limit`) in `config/packs/banking/counterparty-risk/config/maturity-buckets.yaml`; every VaR limit lives in `var-limit` (MGMT, LEGAL and limit-letter nodes, and each system basket as hierarchy `PORTFOLIO`; the system baskets themselves are kept in the basket store, section D5, not data), and the hierarchies themselves are built from the trades' promoted columns (no hierarchy feed); a counterparty group's `LIMIT` is its own approved limit; `netting-set.UTILISATION` is not delivered. The retirement guard (`tools/contracts.policy.yaml`, `removed:`) fails the build if a retired field returns.
- **The exposure engine's inputs** (`ExposureLoader`): a netting set's members and their current MtM come from `trade-valuation` (a live status, the set in `NETTING_SET`), not the set's stored `TRADES[]` (a date without valuations reads the stored members and logs it); the collateral held today is the balance's holdings after haircut (held positive, posted negative; a balance without positions gives its `NET`); the limit is the counterparty's `credit-limit.LIMIT`; `trade-valuation` and `credit-limit` are among its data-version triggers. On the test data's 2026-09-30, 55 of 76 sets' derived net MtM equal the stored `NET_MTM`; the other 21 hold 827 live-status trades past maturity, and with those added back all 76 agree to the dollar.

## D13. VaR sharding: how it is built

One date of a large bank is 509 scenario P&L values for every trade: 2 GB of float32 rows a million positions. Sharding spreads a date's rows over the Drishti*net members so each holds and reads from the lake only its part, while every member still answers every scope: adding a fourth member moves only about a quarter of the rows. It is **off by default** (no `sharding:` block in the market-risk pack and no `drishti.var.sharding.enabled`: every server holds whole dates). What a VaR user sees of it is VAR_GUIDE.md section 28; running it is section D14.



**The shard map** (`ShardMap`, `com.ash.drishti.compute.shard`): per business date, the rows counted per value of the shard key (VaR: `LEGAL_ENTITY`, then `DESK`), read as columns with no vector, both trade kinds (`pnl-vector`, `scenario-pv`) together; walked in key order, a shard closed when the next key would take it over `target-shard-mb` (a row's size is the measure's: 4 × scenarios bytes). A desk is never split: a desk larger than the target is a shard of its own. The ranges cover the whole key space (shard 0 also everything before, the last everything after), so every row has exactly one shard. The map is a pure function of the counts: every member reading the same data version computes the same map. Its `digest` names the cut, not the data, so an intraday batch that adds rows without moving a cut keeps it. A source that cannot answer the key as columns gives one shard (the whole date, with a note), owned by `replicas` members.

**Owners by rendezvous hashing, no consensus protocol** (`ShardOwners`): a shard's owners are the top `replicas` members by rendezvous weight (the first 60 bits of SHA-256 of `node|key`, the key the shard's range, so the same desks keep the same owners from day to day while the cut does not move); a second replica goes to another zone when members carry zones; a pinned shard is owned by its pin first. The members that may own shards (`ShardMembership`) are this server and the live, routable Drishti*net members whose capability digest says VaR sharding is on and that are not `coordinator-only`; the view moves only once the eligible members have been the same for `stable-seconds`; a draining member leaves it at once. Ownership is a pure function of shared inputs (the map from the same data, the membership gossip converges on), and a shard is served only by a member that **holds** it at the data version the asker put the date together from, so two members briefly disagreeing cost at most a refused call and a try on the next member, never a wrong number; a quorum and a leader would add nothing to correctness.

**Shard books** (`VarShardBooks`, `VarLoader.loadShard`): an owner reads the date's inputs for its shard's key range only. `SourcePlugin.vectors(kind, column, texts, asOf, KeyRange, sink)` lets a connector skip files whose statistics hold none of the range's keys (Delta: the add action's `minValues`, `maxValues` and `nullCount` of the first key column; Iceberg: the data file's bounds and null count; a statistic not stated, or a Delta string of 32 characters or more, is not used), and the loader checks every row again. The rows go through the same intake as a whole date's and are written as an ordinary hot file (`<hot>/<date>-shard-<i>/inputs-<digest>.vhot`, header flag `RANGE`, a `RANG` section with the map, the shard, its range and the digest of its inputs). A reporting currency's shard book is built from the USD one (`<date>-<CCY>-shard-<i>/`).

**The skeleton** (`ShardSkeleton`): the hot file without its matrices (`META`, `SCEN`, `TIDS`, `TCOL`, `BOOK`, `AKEY`, `AGGS`, `CTOT`, `CELL`, `RANG`, each with its CRC32C; flag `SKELETON`), sent by an owner (`GET /api/v1/risk/var/shards/skeleton`) to members that put the date together without holding the shard; written with positional reads into a buffer and the servlet stream's own `write`, so a slow reader never pins a virtual thread's carrier. Every member holds the whole skeleton (ids, columns, positions, books, nodes and their aggregates), because a question names positions by their attributes and node figures come from aggregates; skeletons are fetched from owners, never rebuilt from the lake by non-owners. Measured at 1,000,000 positions: 0.72 KB a position a date on the heap (BENCHMARK.md section 4.8); at 2.3 million positions about 1.7 GB a date, the same on every member (adding members does not reduce it). Memory-mapping it, dictionary-encoding it, or sharding it too are the ways to shrink it (CLUSTER_GUIDE.md section 5.1).

**Putting a date together** (`ShardAssembly`): the shards' trades side by side in shard order are the whole date's trades in one server's order (a shard is a range of that order), so positions, books, cells and nodes are derived by the same code (`Cells.layout`, `Cells.derive`, `VarBook.nodes`); every book's and cell's aggregate is the one its shard summed (float64, from the lake's rows) and every node's is summed from them bottom-up as `VarBook.Builder` does: **a node's vector is the unsharded date's to the bit**, on every member. The rows stay where they are (`ShardedVectors`: shards held here from their mapped files, the others through their owners). The intake's checks are made again across shards (`VarShardIntake`: one batch id, the date's scenario set, one form and one grain per source system); then the day's intraday updates are read from the lake and applied as one server applies them (a trade's morning row read back by its id, never from a shard). A date with a book (or, with a hierarchy built from columns, a position) under two shards is refused (`DRS-5217`) rather than answered differently from one server.

**Queries: scatter-gather with a completeness check** (`ShardGather`). A scope resolves on the coordinator (any member a request reaches) as the caller. A node is its aggregate, here. For any other set the planner adds the aggregates it can and gathers the rows it needs into one set to add and one to take away; each is summed shard by shard: a shard held here from its mapped rows; else its part (a bitmap of the shard's ordinals) goes to one member, in order: the shard's owners ready at the date's data version, any other member ready at it (an old owner during a handover), then owners not ready yet (they build from the lake). The call (`POST /api/v1/risk/var/partial`) is signed with the cluster key, one hop, with the caller's on-behalf-of token; the owner applies the caller's rights again and answers the float64 sum, labelled with the shard, the map's digest, the shard's data digest and the number of rows; an answer labelled otherwise is refused, a member on another data version answers `409 DRS-5216`. **Exactly one answer per shard**, added in shard order; a shard nobody answers fails the whole sum (`503 DRS-5215`, naming the shard, its range and what each member said): never a partial VaR. What-if, contributions, backtests, history, the official records and the reporting currencies all go through this path. The answer's `provenance.shards` gives `shards`, `heldHere`, `shardsAsked`, `answeredHere`, `answeredByOwners`, `answeredByReplicas` and `nodes`.

**Contributions under load.** A contribution table needs one vector per part; read part by part, a desk of 2,000 positions held elsewhere would be 2,000 calls. The reads are **fetched ahead** (`VarBook.readAhead`, `ShardedVectors.batched`, `VarShardRows.sumAll`): cut at shard boundaries, the local ones left alone, the others grouped by shard and asked once each through `POST /api/v1/risk/var/partials` (up to 1,024 row sets a call as run pairs, every shard's calls side by side on virtual threads), each set's sum the one `/partial` gives, added in the same order, so the numbers are the same to the bit (`ShardReadAheadTest`). The calls of one answer share one deadline, `call-timeout-seconds` from when the answer's context is taken; each member tried for a shard gets its share of the time left, so a stalled owner leaves time for its replica. Measured on three members at 200,000 positions: contributions' p95 0.88 s at 50 users and 2.53 s at 200, no errors, about 230 requests a second (BENCHMARK.md section 4.3).

**Handover.** Readiness travels in the capability digest (service `var-shards`): whether sharding is on, whether the member owns shards or drains, per date the maps held and per shard the data digest it is ready to serve; a change raises the digest's hash in gossip and peers pull it within a round. Every `rebalance-seconds`, once the membership has settled, each member (1) builds from the lake (never from a peer) the shards of dates in use that it owns and does not hold, and announces them (a date is in use while open on any member and for `release-after-seconds` after); (2) releases what `ShardHandover` allows: a shard it no longer owns once **every** owner has announced it ready for `grace-seconds`, an older data version once the newer has been ready here for the grace period, a shard of a replaced cut after the grace period; (3) puts together again a date whose remote shards' owners announce newer data, or nobody announces any more, or whose cut changed; (4) releases for age shard books of dates not in use, outside the newest `hot-dates` and unread for `release-after-seconds`. A replacement date is put together beside the loaded one and swapped in in one step only when complete (`VarService.swap`); until then the loaded date answers, so even with both owners of a shard down `BANK` answers from its aggregate. The old owner's release always waits for the new owner's readiness. An intraday batch moves no shard: each member reads the day's updates from the lake on its own data-load signal; a full run puts the date together again at the new data version. A reporting currency keeps its morning run per date and data version and puts the USD date's intraday layer on it per answer. A date whose updates cannot be read answers from its morning run with a warning until the next signal. `provenance.intraday.batches` lists the batches in one order on every member (by each batch's newest `COMPUTED_AT`, then `BATCH_ID`).

**Measure-agnostic machinery.** Free of Spring and of the VaR engine, in `com.ash.drishti.compute.shard`: `ShardMap` (the cut; the key a prefix of the measure's row order, the row size the measure's), `ShardOwners`, `ShardHandover`, `ShardGather` (scatter-gather with its exactly-once check and the measure's `Combine`); in the server, `com.ash.drishti.server.shard`: `ShardMembership`, `ShardPeers` (readiness in the digest under the measure's service, signed one-hop calls as the caller) and `ShardedMeasure<P>` (its name, digest service, key order, bytes per row, partial type and how two combine). VaR's is `VarMeasure` (`double[]` partials added element by element); the VaR-specific parts are `VarShards`, `VarShardBooks`, `VarShardRows`, `ShardAssembly`, `ShardSkeleton` and `ShardedVectors`. The pivot engine's ordered merge is the same combine, lifted into its core (HOW_IT_FITS.md section A26.6). Owners only sum rows; the coordinator plans and adds, so node aggregates are used once and a node's figures are the same bits (other scopes' summation order differs from one server's within a cent). Both trade kinds are always sharded together; every other kind is read whole by each member.

**Tests.** `ShardMapTest`, `ShardOwnersTest` (a fourth member takes 20 to 30% of first owners and replica slots over 2,000 shards; a member leaving moves only its shards), `ShardGatherTest`, `ShardAssemblyTest` (1,800 trades in six shards: every node's aggregate to the bit, every scope's vector exactly equal), `ShardReadAheadTest`, `VarShardHandoverTest` (a handover played second by second never leaves a shard without a ready holder), `VarShardingSettingsTest`, `VarShardSingleNodeTest` and `VarShardClusterTest`: three real members and a reference server, 13 scopes in USD and EUR (headline, contributions, the P&L path, every member's tree) equal the reference **to the cent** on Delta and Iceberg, with two and three members; an intraday batch applied live equals the reference; the official record has the same figures; while a third member joins and takes shards no request fails or differs; with one owner stopped its replica answers, with both a scope needing the shard answers `503 DRS-5215` and `BANK` still answers, and 15 s later the shard has a new owner.

**Not built.** An Admin → Health "Shards" panel (mock-ups first; until then `GET /api/v1/risk/var/shards` and `drishti.py var shards`); read-ahead for an intraday batch's morning rows and an intraday date's overlay store (read one call each); exact file skipping from a sorted lake loader's per-file key ranges (today the first key column's statistics); exposure and BI sharding on the same machinery.

## D14. VaR sharding for operators

Everything an operator does with VaR sharding; how it works is section D13. Every key below, with its default and environment variable, is in CONFIGURATION.md.

**Switch it on.** In the market-risk pack (no block: off):

```
# config/packs/banking/market-risk/pack.yaml (or the pack's settings): off without it
sharding:
  enabled: true
  key: [LEGAL_ENTITY, DESK]          # ranges cut along this order; a desk is never split ([LEGAL_ENTITY] also allowed)
  kinds: [scenario-pv, pnl-vector]    # the trades' scenario P&L; every other kind is read whole by each member
  target-shard-mb: 2048
  replicas: 2
```

The same five keys under `drishti.var.sharding.*` (application.yaml) win over the block when set (a value that is not valid refuses the start). Sharding needs the tiered memory (`drishti.var.tier.mode: mapped`, the default).

**How many shards, and where.** The number of shards follows from the date's size and `target-shard-mb` (a date of 2.3 million positions at 2,048 MB makes a few shards; a desk is never split, so the largest desk sets the smallest useful target). `replicas: 2` keeps every shard on two members (an owner can stop without a refusal). Placement:

| Setting | Use |
|---|---|
| `coordinator-only` | this member owns no shard: it puts dates together and coordinates queries only (a member for users' requests) |
| `zone-tag` | the member tag prefix naming its zone (`az`: a tag `az:us-east-1a`): a shard's second owner goes to another zone |
| `pins` | a shard (`S3`, or its range) pinned to a member as its first owner, the same on every member (tests) |

| Setting | Use |
|---|---|
| `stable-seconds` | members unchanged this long before shards move to new owners |
| `grace-seconds` | a new owner ready this long before the old one releases a shard |
| `rebalance-seconds` | how often a member looks at the members and its shards |
| `release-after-seconds` | a shard book of a date nobody has open, outside the newest `hot-dates`, unread this long, is released |
| `call-timeout-seconds`, `skeleton-timeout-seconds` | an answer's deadline for its partials; an owner's time to build and send a skeleton |
| `drain` | start drained (takes no shard) |

Every member also needs heap for the date's whole skeleton (about 0.72 KB a position a date, the same on every member) besides its own shards' rows on local disk; the shard books are hot files under the same disk budget.

**Rebalancing.** Nothing to run: when a member joins or leaves and the membership has been stable for `stable-seconds`, the new owners build their shards from the lake and announce them; the old owners release theirs once every new owner has been ready for `grace-seconds`. A fourth member takes about a quarter of the shards.

**Stop a member safely** (a scale-in, a deploy): `drishti.py var shards --drain` (or `POST /api/v1/risk/var/shards/drain {on: true}`, from a task's stop hook), then wait until `GET /api/v1/risk/var/shards` says `"safeToStop": true`. The draining member leaves the owners' view at once, keeps answering and serving its shards until each is handed over. Give a Fargate task's `stopTimeout` at least the time to build its largest shard plus `grace-seconds`. `--undrain` takes it back.

**See it.** `GET /api/v1/risk/var/shards?asOf=` (anyone who may open `pnl-vector`) and `drishti.py var shards [--date D] [--json]`: the settings, the members (owner, in the view, draining, zone, what each announces ready), the shard books held here, and the date's map with each shard's range, rows, owners and ready members. Every answer's `provenance.shards` says how many shards were asked and who answered; `provenance.intraday.batches` which intraday batches it includes. `POST /api/v1/risk/var/partial`, `/partials` and `/shards/skeleton` are the members' own signed channel and `/shards/drain` an administrator's action: all are in `token-never` (no personal API token opens them).

**When something goes wrong.**

| Case | What happens | What to do |
|---|---|---|
| an owner of a shard stops | its replica answers; after `stable-seconds` a new owner builds the shard from the lake | nothing |
| every owner of a shard down | a scope needing its rows answers `503 DRS-5215` (naming the shard and what each member said); node figures (`BANK`, desks) keep answering; once the membership is stable the new owners build it and it answers again | bring a member back, or wait `stable-seconds` plus the shard's build |
| a load lands between members | `409 DRS-5216`, the next member asked; the date is put together again at the new version and swapped in whole | nothing |
| a book (or, with a hierarchy from columns, a position) under two shards | the date is refused, `422 DRS-5217`, naming it | shard by `LEGAL_ENTITY` only, correct the data, or switch sharding off |
| an unsigned call to `/partial` or `/shards/skeleton` | `401 DRS-5051` | check the cluster key on every member |
| Drishti*net off | the server owns every shard; the date is still cut and put together | — |

---

# Part IV · The ViewModel contract

When you open `TRD MX-20000001`, the server does not send HTML and it does not send the trade's raw record. It sends a **ViewModel**: the screen, already laid out and bound, as JSON. The console paints it; a script reads it; a golden test compares it; a live stream patches it. This document defines that JSON: its structure, the rules every value follows, and how to use it. The client-side renderer (EMBED_GUIDE.md, phase 1 built), its distributable JS/CSS library and GenUI depend on it, so it is written as a contract.

The shape is checked by a machine-readable JSON Schema, served at `GET /api/v1/viewmodel/schema` and generated from the Java records, and a test validates real views and live frames against it.

Every example below is real output, abbreviated (lists cut to two items, long text cut). Most come from the all-panels showcase (docs/guides/examples/README.md#all-panels-showcase), the bond trade `DEMO-BOND-1` that holds one panel of every kind; some come from the demo swap `IRS-48213`.

## Contents


1. What it is, and the principles
2. The top level
3. The common types
4. Panels

5. Live: the stream and its patches
6. Entitlements and masks in a view
7. How to use it
8. Compatibility and versioning
9. The JSON Schema

The code: `drishti-engine/src/main/java/com/ash/drishti/engine/view/` (`ViewModel.java`, `PanelData.java`, `Emptiness.java`, `MaskCount.java`, `ScreenGrid.java`, `ViewModelSchema.java`), the binders that fill `data` (`engine/bind/Binder.java`, `ChartBinder.java`, `PivotBinder.java`, `MapBinder.java`), and the live code (`engine/live/PatchDiffer.java`, `Patch.java`, `Frame.java`, `ViewStream.java`; `server/api/StreamController.java`, `FrameMailbox.java`). The decision: ADR-004.

---

## V1. What it is, and the principles

The ViewModel is the **bound layout**: the output of the view pipeline after a Sutra (or inference) has decided what goes where and the binder has evaluated every expression against the entity's document. The pipeline builds it in one pass; the rest of the system only reads it.

| Principle | What it means for a client |
|---|---|
| **Data, no logic** | There is nothing to evaluate: no expressions, no Sutra, no formats. A client walks the JSON and draws it. |
| **Formatted text plus a tone** | Every value a person reads arrives as `text` already formatted by the server (`−412,580`, `3.8500%`, `2026-10-02`) with a `tone` (`neg`, `pos`, …). A client never formats a number, so every client shows `−1,403,091` the same way. The exceptions are the numbers a chart needs to draw: series values, bar lengths, bin edges, coordinates, gauge levels. Each of those travels **beside** its formatted text where a person reads it (a bar has `value` and `text`; a point has `x` and `xText`). |
| **Masks applied on the server** | Values a caller may not see arrive as `•••`, already. A client cannot unmask anything and needs no rule to hide anything. `provenance.masked` says how many were hidden. |
| **Per viewer** | A ViewModel is built for one caller: their masks, the kinds they may open, the business date they asked for, the control values in their query. Two people may receive different JSON for the same entity. |
| **One model, every consumer** | The same JSON is the REST answer, the first event of a live stream, what the console's macros paint, what the Studio preview shows, what golden tests assert, and what live patches replace piece by piece. |
| **Failures stay local** | A panel that cannot bind carries `error` and no `data`; a strip value that fails reads `—`. The view still arrives. |

What it is not: it is not the entity's data (that is `GET /api/v1/entities/{kind}/{id}/raw`), and it is not HTML (the console and the embed API turn it into HTML; see ADR-021).

---

## V2. The top level

`GET /api/v1/views/trade/DEMO-BOND-1` (the showcase, panels and timings abbreviated):


```
{"ref": {"kind": "trade", "id": "DEMO-BOND-1"},
 "mnemonic": "TRD",
 "title": {"pill": "Showcase · US Treasury 4.25% 2038", "id": "DEMO-BOND-1", "with": null},
 "strip": [{"label": "Notional", "text": "USD 25,000,000", "path": "$.CURRENCY"},
           {"label": "MTM (USD)", "text": "+1,209,000", "tone": "pos", "emphasis": true, "path": "$.MTM"}],
 "panels": [{"id": "terms", "kind": "kv", "title": "1 · kv: bond terms", "area": "main", "inferred": false, "empty": false, "span":
8,
             "data": {"fields": [{"label": "Coupon", "text": "4.2500%", "path": "$.TERMS.COUPON"}]}}],
 "keys": [],
 "provenance": {"layout": "Sutra all-panels-showcase v1 + inference", "fingerprint": "b24b…2d27", "source": "test", "generation":
1,
                "fetchedAt": "2026-10-28T17:13:12.366528408Z", "live": true, "businessDate": null,
                "updatedAt": "2026-10-28T17:13:08.913778814Z", "staleAfter": null, "stale": false,
                "sutra": "all-panels-showcase", "masked": 0, "maskedPanels": []},
 "timings": {"fetch": 0.0, "layout": 7.81, "links": 1.46, "bind": 21.66, "total": 30.94}}
```


Every field (record `ViewModel`). *Always* means the server always sends it; *optional* fields are left out when they have no value (the record is written with "leave out nulls").

| Field | Type | Sent | Meaning |
|---|---|---|---|
| `ref` | `{kind, id}` | always | The entity this view shows. |
| `mnemonic` | string | optional | The command-line code of the kind (`TRD`, `NSET`); left out for a kind without one. |
| `title` | Title | always | The title line: pill, identifier, related entity. |
| `strip` | Cell `[]` | always | The header figures, in order. Live `strip` patches address them by index. |

| Field | Type | Sent | Meaning |
|---|---|---|---|
| `panels` | Panel `[]` | always | The panels, in layout order. Each says its column (`area`) and, with a `layout`, its place. |
| `keys` | Key `[]` | always (may be empty) | The function keys. |
| `provenance` | Provenance | always | How the view was built and how fresh it is. |
| `timings` | `{stage: ms}` | always | Milliseconds per stage: `fetch`, `layout`, `links`, `bind`, `total`. Diagnostic; never draw anything from it. |
| `controls` | Control `[]` | optional (left out when empty) | The Sutra's controls (dropdowns, toggles, date ranges, ...) with their options and current values. See Controls and selects. |
| `selects` | Select `[]` | optional (left out when empty) | The panels whose click sets a control. |
| `contract` | string | always | The ViewModel contract this view speaks, `major.minor`: `"1.0"` today (`ViewModel.CONTRACT`). A client refuses a major it does not know (Drishti Render's `DRS-9105`) and ignores what a newer minor adds. A ViewModel from an older server has none: it is `1.0`. See section V8. |
| `layout` | object | optional | The Sutra's screen grid (`areas`, `columns`, `split`, `tabs`, `sections`, `sticky`, `arrange`, `phone`, `tablet`) and each top-level panel's place. Left out when the Sutra has none: the panels then flow in two columns, `main` and `right`. See Area and grid. |
| `phone` | object | optional | What the Sutra says about the phone (MOBILE_DISPLAY.md (docs/guides/BUILD_GUIDE.md#32-designing-for-phones)): `panels` (`{id: {tier, from, rows, columns, sort, series, figures, tab}}`, `from` being `fragment` or `sutra`), `order`, `default` and `summary` (the phone summary line, read against the document). Left out when the Sutra says nothing; a client then uses each panel kind's default tier. A desktop client ignores it. |

---

## V3. The common types

### Cell

One formatted value (record `ViewModel.Cell`). Used by the strip, `kv`, `status` and `provenance` fields, table cells, tab fields, pivot cells and the metric figure. Every field is optional: a cell is written with "leave out defaults", so `false`, `null` and missing are the same.

| Field | Type | Meaning |
|---|---|---|
| `label` | string | The label (a strip figure, a kv field, a table cell's column). |
| `text` | string | The value, formatted. Missing or `—` when the document has no value; `•••` when masked. |
| `tone` | Tone | How to colour it. |
| `link` | Link | The entity the value opens. |
| `emphasis` | boolean | Draw it highlighted (the MTM figure). |
| `path` | string | The document path the value came from (`$.MTM`, `$.TERMS.COUPON`). Diagnostic and for "where does this come from"; live patches address strip cells by index, not by path. |

```
{"label": "MTM (USD)", "text": "+1,209,000", "tone": "pos", "emphasis": true, "path": "$.MTM"}
```

An empty cell is `{}`: the blank cells of a table's total row are written so.

### Link

A navigable reference to another entity (record `ViewModel.LinkView`): `{kind, id, mnemonic}`. `mnemonic` is the target kind's command code, missing when it has none. A client opens it as the view of `kind/id` (the console links to `/v/{kind}/{id}`).

```
{"kind": "counterparty", "id": "CP-NORTHBRIDGE", "mnemonic": "CPTY"}
```

A link the caller may not follow is **removed** before the view is sent (see section V6), so any link present is one the caller may open.

### Tones

A tone is a word, not a colour; the client maps it to its theme (the console: `t-<tone>` classes on tokens in `drishti_tokens.css`) and always pairs colour with a sign or glyph. The values the server sends:

| Tone | Meaning | Comes from |
|---|---|---|
| `pos`, `neg` | a positive or negative figure | `tone: sign` in a Sutra, hbar bars, theme-coloured waterfalls |

| Tone | Meaning | Comes from |
|---|---|---|
| `ok`, `warn`, `bad` | a good, pending or failed status | `tone: status`, the timeline, gain-loss waterfalls |
| `link`, `accent` | a neutral series or highlight | `tone: link/accent`, the first series of a line |
| `muted` | a total, a neutral bar | waterfall totals |
| `inactive` | a value whose command an administrator turned off | entitlements (a table cell whose link was removed) |

The schema lists exactly these for `Cell.tone` and the tones of bars, waterfall steps, histogram markers, timeline events and candles; a client that meets another word should draw the value untoned. A **series'** tone (`line`, `area`) is whatever the Sutra author wrote, usually one of these, and is free text in the schema.

### Title

`{pill, id, with}` (record `TitleView`). `pill` is the kind's caption (`Trade · Interest rate swap`), `id` the identifier shown, `with` an optional related entity as a Cell (`{"text": "Northbridge Capital LLP", "link": {...}}`); `with` is `null` when there is none.

### Keys and actions

`{key, label, action, panel?, link?}` (record `KeyView`). `key` is `F2` to `F12`; `action` says what pressing it does:

| `action` | Field | What the client does |
|---|---|---|
| `panel` | `panel` | Scroll to and focus the panel with that id. |
| `link` | `link` | Open the linked entity. |
| `raw` | | Show the entity's raw document (`GET /api/v1/entities/{kind}/{id}/raw`). |
| `impact` | | Open Impact for the entity (`/impact/{kind}/{id}` in the console). |
| `denied` | | Nothing: the key's target is a kind the caller may not open. Its label ends `(no access)` or `(not active)`. |

```
[{"key": "F2", "label": "Legs", "action": "panel", "panel": "legs"},
 {"key": "F7", "label": "Netting set", "action": "link", "link": {"kind": "netting-set", "id": "NS-NORTH-01", "mnemonic": "NSET"}},
 {"key": "F8", "label": "Impact", "action": "impact"},
 {"key": "F9", "label": "Raw JSON", "action": "raw"}]
```

### Provenance

How the view was built (record `ViewModel.Provenance`). Every field is always sent except `partial` and `missing`.

| Field | Type | Meaning |
|---|---|---|
| `layout` | string | What laid it out: `Sutra irs-vanilla v3 + inference`, or an inference-only label. |
| `fingerprint` | string | A short fingerprint of the document's shape (`4e09…e011`); the layout is cached per Sutra version and fingerprint. |
| `source` | string | The connector that answered (`aero-risk`, `trade-endur`). |
| `generation` | integer | The source's version of the entity. It moves when the data changes; live frames carry it too. |
| `fetchedAt` | ISO-8601 instant | When the document was fetched. |
| `live` | boolean | Whether the source pushes updates for this view. `false` for a past business date and for static sources: a stream then sends one `view` and closes. |
| `businessDate` | `YYYY-MM-DD` or null | The business date the source read; `null` for sources that are not dated. |
| `updatedAt` | ISO-8601 instant or null | When the source last received new data; `null` when it cannot tell. |
| `staleAfter` | ISO-8601 duration or null | The source's freshness threshold. |
| `stale` | boolean | Older than `staleAfter`. The console shows *stale*. |
| `sutra` | string or null | The Sutra's name; `null` when inference alone laid the view out. |
| `masked` | integer | How many values in this view the caller's field masks replaced with `•••` (strip, title and every panel's data); `0` when none. |
| `maskedPanels` | string `[]` | The ids of the panels that hold masked values. |
| `partial` | boolean, optional | Sent only when `true`: a page gathered across Drishti*net with a node down. |
| `missing` | string `[]`, optional | Sent only when not empty: what did not answer, one line per node. |

The `provenance` **panel** kind shows the same facts as label/value lines; a client that wants them as data reads this object.

### Timings

`{"fetch": 3.98, "layout": 2.74, "links": 1.28, "bind": 1.73, "total": 9.74}` (milliseconds, `IRS-48213`). Diagnostic only.

### `empty`, `error` and `denied`

Three panel fields say why a panel shows no data. Draw them in this order:

| Field | Set when | `data` | What to draw |
|---|---|---|---|
| `error` | Binding this panel failed (its data does not fit its Sutra; an expression failed). The text is the reason, often with a code. | missing | The panel's frame and the error, inline. The rest of the view is unaffected. |
| `denied` | The panel reads another entity (`source:`) the caller may not open: `no access to curve`; `not active (turned off by an administrator)` when the command is off; `hidden for you` and `not available to your role` for a value it follows that is masked. | missing | "No access" in words. Never a blank panel. |
| `empty` | The document lacks what the panel asks for (`Emptiness.of`: no rows, no finite series value, every field `—`, ...). Also `true` with `error` and `denied`. | present, but nothing worth drawing | *No data available* (a `links` panel: *No linked entities*; a table's own words when its Sutra says `empty:`, in `data.emptyLabel`). |

`empty` is always sent (`true` or `false`); `error` and `denied` only when set. A panel with `empty: true` may still carry `data` (a table's columns, a kv's labels); a renderer may use it, but should show the empty state.

### Area and grid

Without a `layout`, every panel has `area`: `main` (the wide left column) or `right` (the narrow right one), and the panels flow in `panels` order. Two optional sizes refine this, for either mode:

| Field | Meaning |
|---|---|
| `span` | Width in columns of a 12-column grid (1 to 12); missing = the whole column. |
| `height` | Height in grid rows; missing = as tall as its content. |

With a Sutra `layout:` (RACHANA_REFERENCE.md 13), the top-level `layout` object carries the grid and `layout.place` says where each top-level panel goes: `{panelId: {area, order?, section?, collapsible?}}`. `area` there is a name from `layout.areas` (a panel's `main` or `right` is mapped to the first or last named area). Other keys: `columns` (column widths), `split` (`{between: [a, b], resizable, remember}`), `tabs`, `sections` (`[{id, title, collapsed}]`), `sticky`, `arrange` (whether the reader may rearrange), `phone`, `tablet`. A real example from `ScreenGridTest`:

```
{"areas": ["head head", "body side"], "columns": [8, 4], "arrange": false,
 "place": {"a": {"area": "head"}, "b": {"area": "head"}, "c": {"area": "side", "order": 3, "collapsible": true}}}
```

### Controls and selects

`controls` lists the Sutra's controls (record `ControlView`): `{id, kind, label, placement, panel?, parent?, options: [{value, label}], value, values?, required, remember, min?, max?, step?, range, reads: [panel ids], truncated}`. `kind` is `dropdown`, `multi-select`, `cascading`, `toggle`, `segmented`, `date`, `date-range`, `number-range`, `top-n` or `search`; `placement` is `strip`, `header` (in the header of `panel`) or `side`. A client sends a changed value back as the query parameter `ctl.<id>=<value>` on the view (or stream) URL, and only the panels in `reads` change. A control never offers a masked value. `selects` lists `{panel, by, filter}`: a click on a row (or bar) of `panel` sets control `by`; a row carries the value it sets in `pick`.

### Other panel fields

These belong to Sutra Next features; a basic renderer may ignore them and still draw a correct screen.

| Field | Record | Meaning |
|---|---|---|
| `wire` | `Wire` | Events a click fires (`emits`), `waits` (set while the panel waits on an event: draw that text instead of data), child panel groups (`children`), the `source` kind it reads, `parent` (a child panel's parent). |
| `refresh` | `RefreshView` | `{every, userCanPause}`: fetch the panel again every `every` seconds. |
| `form` | object | The panel's edit form for write-back (fields the viewer may edit, rules, lookups). |
| `provenance` | `PanelProvenance` | Freshness of a panel with its own `source:`: `{source, kind, fetchedAt, updatedAt, staleAfter, stale}`. |
| `compute` | `PanelCompute` | A computed panel's state: `{state: ready\|computing\|stale\|failed\|refused, requestId, connector, message, progress, hasResult, stamp}`. Draw the state in words, over the last result when `hasResult`. A panel's own `source:` that could not be read (the connector failed or did not answer within the fetch timeout, `drishti.sources.fetch-timeout`, 10 s) comes as `failed` with an empty `requestId`, the connector and why: never as an empty panel. |
| `inferred`, `explanation` | | Whether inference built or completed the panel, and why (shown as a tag). |
| `code` | | A short header tag (`REFS`). |
| `key` | | The function key that jumps to it (`F3`). |

---

## V4. Panels

### The common fields

Record `ViewModel.PanelView`. Written with "leave out nulls": `inferred` and `empty` are always sent, the rest when set.

| Field | Type | Sent | Meaning |
|---|---|---|---|
| `id` | string | always | Unique in the view; live `panel` patches replace the panel with this id. |
| `kind` | string | always | One of the 22 kinds below. Decides the shape of `data`. |
| `title` | string | usually | The resolved title. |
| `code`, `key` | string | optional | Header tag; function key. |
| `area` | `main` or `right` | always in practice | The column, when there is no `layout` (see Area and grid). |
| `span`, `height` | integer | optional | Grid width (of 12) and rows. |
| `inferred` | boolean | always | Built or completed by inference. |
| `explanation` | string | optional | Why inference chose it. |
| `data` | object | when bound | The kind's content, below. Missing with `error` or `denied`. |
| `error`, `denied`, `empty` | | see above | |
| `wire`, `refresh`, `form`, `provenance`, `compute` | | optional | See Other panel fields. |

```
{"id": "refs", "kind": "links", "title": "20 · links: linked entities", "code": "REFS", "area": "right",
 "inferred": false, "empty": true, "height": 10, "data": {"links": []}}
```

### The 22 kinds

Verified against `PanelKind` (`drishti-rachana/.../model/PanelKind.java`) and the binders: the grammar has **twenty-two** kinds. (Older documents say twenty; `metric` and `map` came later.) Several kinds share one `data` record:

| `data` record | Kinds |
|---|---|
| `Fields` | `kv`, `status`, `provenance` |
| `Table` | `table`, `ladder` |
| `Tabs` | `tabs` |
| `Chart` | `line`, `area` |
| `Bars` | `hbar` |
| `Links` | `links` |
| `Text` | `markdown` |
| `Gauge` | `gauge` |
| `Surface` | `surface` |
| `Waterfall` | `waterfall` |
| `Histogram` | `histogram` |
| `Scatter` | `scatter` |
| `Candles` | `candlestick` |
| `Graph` | `graph` |
| `Timeline` | `timeline` |
| `Pivot` | `pivot` |
| `Metric` | `metric` |
| `GeoMap` | `map` |

What each kind is for, its Sutra options and how it behaves are in RACHANA_REFERENCE.md; below is only its JSON. In the tables, *always* means the field is always written (it may be `null` where the type says so).

**`kv`**

`{fields: Cell[]}`: label/value pairs in a grid (the console: four columns in `main`, two in `right`).

```
{"fields": [{"label": "Coupon", "text": "4.2500%", "path": "$.TERMS.COUPON"},
            {"label": "Frequency", "text": "Semi-annual", "path": "$.TERMS.COUPON_FREQUENCY"}]}
```


**`table`**

| Field | Type | Sent | Meaning |
|---|---|---|---|
| `columns` | string `[]` | always | Header labels. |
| `numeric` | boolean `[]` | always | Per column: right-align it. |
| `rows` | Row `[]` | always | The rows. |
| `total` | Row | optional | The total row (its blank cells are `{}`). |
| `more` | string | optional | `"10 more trades"`: rows left out beyond the Sutra's `limit`. |
| `search` | boolean | always | Offer a filter box. |
| `pivot` | object | optional | The Pivot tab the Sutra opts into: its fields and opening arrangement; rows come from `GET /api/v1/views/{kind}/{id}/panels/{panel}/records`. |
| `expand` | integer | optional | Tree levels open at first (a table with `children`). |
| `levels` | `[{title, columns, numeric}]` | optional | Drill levels: a row opens into the next level's rows. |
| `root` | string | optional | The first breadcrumb step of a drill table. |
| `emptyLabel` | string | optional | The words to show instead of *No data available*. |

A **Row** is `{cells: Cell[], highlight, path, children?, pick?, crumb?, kidsTotal?, ev?}`: `cells` one per column, `highlight` draw it highlighted, `path` its document path (`null` in a tree's nested rows and the total), `children` nested rows (a tree), `pick` the value a click sets in a control, `crumb` and `kidsTotal` for drill tables, `ev` the row's key for a row-click event. Each cell's `label` repeats its column's.


```
{"columns": ["#", "Event"], "numeric": [false, false],
 "rows": [{"cells": [{"label": "#", "text": "1"}, {"label": "Event", "text": "Booked"}], "highlight": false, "path": "$.LIFECYCLE.E
VENTS[0]"},
          {"cells": [{"label": "#", "text": "2"}, {"label": "Event", "text": "Amended"}], "highlight": false, "path": "$.LIFECYCLE.
EVENTS[1]"}],
 "search": true}
```


A tree (`children`), from the showcase's `tree` panel:


```
{"columns": ["Name", "Trades"], "numeric": [false, false],
 "rows": [{"cells": [{"label": "Name", "text": "DESK-FI · Fixed income"}, {"label": "Trades", "text": "5"}], "highlight": false, "p
ath": "$.BOOK_TREE[0]",
           "children": [{"cells": [{"label": "Name", "text": "BOOK-FI-1 · Govies"}, {"label": "Trades", "text": "3"}], "highlight":
false, "path": null,
                        "children": [...]}]}],
 "total": {"cells": [{}, {"text": "Total"}], "highlight": false, "path": null}, "search": true, "expand": 1}
```


**`ladder`**

The same record as `table`; a ladder is a dated schedule, and `highlight` marks the current row (the next payment, today's settlement).


```
{"columns": ["Pay date", "Rate"], "numeric": [false, true],
 "rows": [{"cells": [{"label": "Pay date", "text": "2025-10-31"}, {"label": "Rate", "text": "4.2500%"}], "highlight": false, "pat
h": "$.SCHEDULE[0]"}],
 "total": {"cells": [{}, {"text": "Total"}], "highlight": false, "path": null}, "search": true}
```


**`tabs`**

`{layout: "tabs" | "columns", tabs: [{title, fields: Cell[], panels?: Panel[]}]}`: one tab per element of a list (`columns` puts them side by side). A tab's `panels` are whole panels bound against that element, with the same shape as top-level panels (they are not in `layout.place`).


```
{"layout": "tabs",
 "tabs": [{"title": "Leg 1 · Pay fixed 3.8500%", "fields": [{"label": "Currency", "text": "USD"}, {"label": "Notional", "text": "5
0,000,000"}]},
          {"title": "Leg 2 · Receive SOFR compounded", "fields": [{"label": "Currency", "text": "USD"}, {"label": "Notional", "tex
t": "50,000,000"}]}]}
```

**`line`**

| Field | Type | Sent | Meaning |
|---|---|---|---|
| `x` | string `[]` | optional | Axis labels, already formatted. |
| `series` | `[{label, values: (number\|null)[], tone}]` | optional | One or more series, `values` aligned with `x`; `null` is a gap. |
| `mark`, `markText` | string | optional | The `x` label to highlight, and its caption (`5Y point 3.60%`). |
| `limit`, `limitLabel` | number, string | optional | A horizontal limit line. |
| `source` | Link | optional | The entity the points came from (a curve); its changes tick this view. |
| `unit` | string | optional | Axis unit. |
| `step` | boolean | optional | Draw as steps (a value holds until the next point). |


```
{"x": ["1M", "3M"], "series": [{"label": "USD-SOFR curve", "values": [3.93, 3.88], "tone": "link"}],
 "mark": "5Y", "markText": "5Y point 3.60%", "source": {"kind": "curve", "id": "USD-SOFR", "mnemonic": "CRV"}, "unit": "%"}
```


**`area`**

The same record as `line`, drawn filled; usually several series and a `limit`.


```
{"x": ["1M", "3M"],
 "series": [{"label": "EE", "values": [800000.0, 1400000.0], "tone": "link"}, {"label": "PFE 95", "values": [1900000.0, 3100000.0],
"tone": "accent"}],
 "limit": 6500000.0}
```


**`hbar`**

`{bars: [{label, value, text, tone, ev?}], shape?, total?}`: bars scaled to the largest absolute `value`; `text` is what to print. `shape: "donut"` draws the same bars as a ring of shares, with `total` (formatted) in the middle.


```
{"bars": [{"label": "3M", "value": -120.0, "text": "−120", "tone": "neg"}, {"label": "1Y", "value": -1450.0, "text": "−1,450", "ton
e": "neg"}]}
```


**`links`**

`{links: [{label, text, link, badge, status}]}`: the linked entities. `status` is `resolved`, `pending` (not fetched within the link budget; a later rebuild fills it), `missing` (no source holds it), `denied` (`badge` is `no access`, `link` removed) or `inactive` (`badge` is `not active`). `badge` is a short summary from the target (`EE 4.1m`), possibly empty.


```
{"links": [{"label": "Book", "text": "RATES-NY-3", "link": {"kind": "book", "id": "RATES-NY-3", "mnemonic": "BOOK"}, "badge": "",
"status": "resolved"},
          {"label": "Netting set", "text": "NS-NORTH-01", "link": {"kind": "netting-set", "id": "NS-NORTH-01", "mnemonic": "NSE
T"}, "badge": "EE 4.1m", "status": "resolved"}]}
```


**`status`**

The same record as `kv`; the values are statuses toned `ok`, `warn` or `bad`.


```
{"fields": [{"label": "Confirmation", "text": "Confirmed", "tone": "ok", "path": "$.CONFIRMATION.STATUS"},
            {"label": "Clearing", "text": "Cleared", "tone": "ok", "path": "$.CLEARING.STATUS"}]}
```


**`provenance`**

The same record as `kv`: *How this view was built* as lines (Layout, Fingerprint, Source, Fetched, ...). On Drishti*net an administrator also sees the node that answered. The structured facts are in the top-level `provenance`.


```
{"fields": [{"label": "Layout", "text": "Sutra irs-vanilla v3 + inference"}, {"label": "Fingerprint", "text": "4e09…e011"}]}
```


**`markdown`**

`{text}`: the Sutra's text with its `${...}` parts filled in. Treat it as text (the console prints it as a paragraph, escaped); do not render it as HTML.

```
{"text": "A buy USD bond. Every panel reads the document in all-panels-showcase.json."}
```

**gauge**

`{value, max, text, label}`, every field always written: a level `value` out of `max`, and `text` to print (formatted from `value / max`). `value` is `null` when the document lacks it or it is masked (`text` is then `–` or `•••`).

```
{"value": 89000.0, "max": 250000.0, "text": "36%", "label": null}
```

**surface**

`{x, y, z, min?, max?, fmt?, unit?, view?}`: `z[row][column]` over `x` (the columns' labels) and `y` (one label per row), `null` where the document has no number; `min` and `max` for the colour scale; `view` is `heatmap` (default) or `3d`. The cell values are numbers; `fmt` names the format the client uses for their labels.

```
{"x": ["90%", "ATM"], "y": ["X6", "Z6"], "z": [[31.2, 28.52], [30.48, 28.02]], "min": 25.17, "max": 31.2, "view": "3d"}
```

**waterfall**

`{steps: [{label, value, from, to, text, tone, total}], unit?}`: floating bars from `from` to `to` (the running total), in order; `total: true` draws a full bar from zero. `tone`: `ok`/`bad` for rises and falls (`colors: gain-loss`, the default) or `pos`/`neg` (`colors: theme`), `muted` for totals.

```
{"steps": [{"label": "Opening MTM", "value": 1250000.0, "from": 0.0, "to": 1250000.0, "text": "+1,250,000", "tone": "ok", "total":
false},
           {"label": "Carry", "value": 18400.0, "from": 1250000.0, "to": 1268400.0, "text": "+18,400", "tone": "ok", "total": fals
e}]}
```

**histogram**

`{bins: [{from, to, count, label}], markers: [{label, value, text, tone}], count, dropped, fmt?, unit?}`: equal-width bins computed on the server (`[from, to)`, the last closed), `label` the range formatted; `markers` are vertical lines (VaR, ES, mean); `count` numbers binned and `dropped` left out (not numbers, or beyond the maximum).

```
{"bins": [{"from": -1165796.0, "to": -1030570.53, "count": 5, "label": "−1.2m to −1.0m"},
          {"from": -1030570.53, "to": -895345.06, "count": 4, "label": "−1.0m to −895.3k"}],
 "markers": [{"label": "VaR 99%", "value": -1097165.0, "text": "−1.1m", "tone": "neg"}], "count": 300, "dropped": 0, "fmt": "compac
t"}
```

**scatter**

`{points: [{x, y, size?, xText, yText, label?, group?, link?}], groups, xLabel, yLabel, sized, more}`: one point per row; `groups` are the distinct groups in first-seen order (a colour each); `sized` says whether points carry `size`; `more` points left out beyond the maximum.

```
{"points": [{"x": 1486851.0, "y": 37.0, "xText": "1.5m", "yText": "37", "label": "RATES-A"},
            {"x": -4871775.0, "y": 43.0, "xText": "−4.9m", "yText": "43", "label": "RATES-B"}],
 "groups": [], "xLabel": "MTM (USD)", "yLabel": "Trades", "sized": false, "more": 0}
```

**candlestick**

`{candles: [{x, open, high, low, close, volume?, openText, highText, lowText, closeText, volumeText?}], volume, last?, change?, tone?, fmt?, unit?}`: oldest first; `volume` says whether candles carry volume; `last` (the last close) and `change` (over the last bar) are formatted, `tone` colours the change. Every candle also carries its prices as text in the panel's `fmt` (`openText` …) and its volume as text (`amount0`), which the Data table prints: a client formats no number (EMBED_GUIDE.md 14.3). A client reading an older server prints the numbers when the texts are missing.

```
{"candles": [{"x": "2026-09-01", "open": 74.0, "high": 74.06, "low": 72.97, "close": 73.2, "volume": 93206.0,
              "openText": "74.00", "highText": "74.06", "lowText": "72.97", "closeText": "73.20", "volumeText": "93,206"},
             {"x": "2026-09-02", "open": 73.2, "high": 74.1, "low": 72.15, "close": 73.61, "volume": 159955.0,
              "openText": "73.20", "highText": "74.10", "lowText": "72.15", "closeText": "73.61", "volumeText": "159,955"}],
 "volume": true, "last": "70.45", "change": "+0.01 (+0.01%)", "tone": "pos", "fmt": "price2"}
```

**graph**

`{nodes: [{id, label, group?, link?, focus}], edges: [{from, to, label?}], layout, more}`: `layout` is `tree` (layered from the roots) or `force`; `focus` marks the node that is this view's entity; an edge to a node not listed is left out; `more` nodes left out beyond the maximum.

```
{"nodes": [{"id": "GRP-NORTHBRIDGE", "label": "Northbridge Holdings", "group": "group", "focus": false},
           {"id": "CP-NORTHBRIDGE", "label": "Northbridge Capital LLP", "group": "counterparty",
            "link": {"kind": "counterparty", "id": "CP-NORTHBRIDGE", "mnemonic": "CPTY"}, "focus": false}],
 "edges": [{"from": "GRP-NORTHBRIDGE", "to": "CP-NORTHBRIDGE", "label": "owns"}], "layout": "tree", "more": 0}
```

**`timeline`**

`{events: [{date, label, detail?, status?, tone?}], more}`: dated events, oldest first; `date` as the document writes it; `tone` (`ok`, `warn`, `bad`) colours the status.

```
{"events": [{"date": "2026-08-31", "label": "Booked", "detail": "Booked by the rates desk from the broker ticket.", "status": "Don
e", "tone": "ok"},
            {"date": "2026-08-31", "label": "Confirmed", "detail": "Matched with the counterparty.", "status": "Done", "tone": "o
k"}], "more": 0}
```

**`pivot`**

| Field | Type | Sent | Meaning |
|---|---|---|---|
| `by` | string | optional | The row field (the corner heading; `BOOK › FAMILY` when nested). |
| `columns` | string `[]` | optional | Column keys, first-seen order. |
| `rows` | PivotRow `[]` | optional | One per row key. |
| `totals` | Cell `[]` | optional | Column totals, then the grand total; missing when totals are off. |
| `agg` | string | optional | The aggregation (`sum`, `count`, …). |
| `heat` | boolean | always | Colour cells by value between `min` and `max`. |
| `min`, `max` | number | optional | The heat scale. |
| `more` | integer | always | Row keys left out beyond the maximum. |
| `levels`, `expand`, `across` | | optional | A nested pivot: the row fields outermost first, levels open at first, the column field. |

A **PivotRow** is `{label, cells: Cell[], values: (number|null)[], total?, path?, group?}`: `cells` formatted and toned, `values` the raw aggregates (for the heat scale and export); in a nested pivot `path` is the keys from the outermost level down and `group: true` marks a subtotal row.

```
{"by": "BOOK › FAMILY", "columns": ["USD", "EUR"],
 "rows": [{"label": "RATES-A", "cells": [{"text": "685.8k", "tone": "pos"}, {"text": "801.1k", "tone": "pos"}], "values": [685763.
0, 801088.0],
           "total": {"text": "1.5m", "tone": "pos"}, "path": ["RATES-A"], "group": true},
          {"label": "bond", "cells": [{"text": "274.9k", "tone": "pos"}, {"text": "−41.3k", "tone": "neg"}], "values": [274901.0, -
41321.0],
           "total": {"text": "233.6k", "tone": "pos"}, "path": ["RATES-A", "bond"], "group": false}],
 "totals": [{"text": "−7.5m", "tone": "neg"}, {"text": "2.0m", "tone": "pos"}], "agg": "sum", "heat": false,
 "min": -2028840.0, "max": 2448996.0, "more": 0, "levels": ["BOOK", "FAMILY"], "expand": 1, "across": "CURRENCY"}
```

**`metric`**

`{value: Cell, delta: Cell|null, unit, caption}`, every field always written: one large figure (`value`, its `label` the panel's label) with an optional change beside it, a short unit and a caption.

```
{"value": {"label": "Mark to market", "text": "+1,209,000", "tone": "pos", "emphasis": true, "path": "$.MTM"},
 "delta": {"text": "−41,000", "tone": "neg", "path": "$.PNL1D"}, "unit": "USD", "caption": "change over the last day"}
```

**`map`**

| Field | Type | Meaning |
|---|---|---|
| `mode` | `world` or `region` | The world, or one country's regions. |
| `highlight` | string | The country to highlight, as the document names it (an ISO code or a name). |
| `region` | string | For `region`: the country whose regions are drawn. |

| Field | Type | Meaning |
|---|---|---|
| `link` | `{kind, template, codes?, note?}` | What a click opens: `template` is an id template (`JUR-{iso3}`), `codes` the codes for which an entity exists (others are not clickable), `note` why nothing is clickable (`no access to economy`). |
| `shade` | `{label, unit?, items: [{code, value, text}], note?}` | Per-country figures that colour the map; `note` why it is not shaded (masked, no access, not read in time). |
| `unit` | string | |

All optional. Shapes come from vendored Natural Earth files; nothing is fetched from outside.

```
{"mode": "world", "highlight": "USA"}
```

---

## V5. Live: the stream and its patches

### The stream

`GET /api/v1/views/{kind}/{id}/stream` (Server-Sent Events, `text/event-stream`) takes the same path, as-of and `ctl.*` parameters as the view. It sends:

| Event | SSE `id:` | Data |
|---|---|---|
| `view` | `0` | The full ViewModel, exactly as `GET /api/v1/views/{kind}/{id}`. Always first, on every connect and reconnect. |
| `frame` | the frame's `seq` | A **Frame**: `{seq, generation, patches: Patch[], latencyMs, p99Ms}`. |
| comment `:hb` | none | A heartbeat every `drishti.live.heartbeat` (15 s) when nothing else was sent. |

`seq` grows by one per frame on this stream (1, 2, 3, ...); `generation` is the entity's source generation after the frame; `latencyMs` is source tick to frame for this frame and `p99Ms` the server's rolling p99 (diagnostic). When the view is not live (`provenance.live` false: a past business date, a static source) the stream sends the `view` event and closes.

### The patches

A frame's `patches` (record `Patch`, from `PatchDiffer` and `ViewStream`):

| `op` | Fields | Apply it by |
|---|---|---|
| `strip` | `index`, `cell` | Replacing strip cell number `index` (0-based) with `cell`. |
| `panel` | `panel` | Replacing the panel whose `id` is `panel.id` with `panel`, a whole panel. Sent when its `data`, `error` or `title` changed. |
| `provenance` | `provenance` | Replacing the view's `provenance`. Sent when the generation or the layout label moved. |
| `deleted` | `at` (ISO-8601 instant) | Saying the entity was deleted at `at`: keep what is shown, grey it, apply no later patches. |
| `restored` | none | Repainting the whole view (fetch it again): a deleted entity came back; the patches after it bring the view up to date. |

These five are all the ops the code has. A real frame (the showcase after its MTM moved; panels abbreviated):

```
{"seq": 1, "generation": 2,
 "patches": [{"op": "strip", "index": 3, "cell": {"label": "MTM (USD)", "text": "−1,300,000", "tone": "neg", "emphasis": true, "pat
h": "$.MTM"}},
             {"op": "panel", "panel": {"id": "built", "kind": "provenance", "…": "…"}},
             {"op": "panel", "panel": {"id": "headline", "kind": "metric", "…": "…"}},
             {"op": "provenance", "provenance": {"layout": "Sutra all-panels-showcase v1 + inference", "source": "test", "generatio
n": 2, "live": true, "…": "…"}}],
 "latencyMs": 0.4, "p99Ms": 1.9}
```

### Applying them, in order

- Apply the patches of a frame **in list order**, frames in `seq` order (SSE delivers them in order on one connection).
- Patches carry values exactly as the first view did: the same masks (every rebuild is made as the caller may see it), the same control values (the ones in the stream's URL).
- A slow client may receive fewer, fuller frames: the server's one-slot mailbox merges pending frames, newest value per strip cell, per panel and per op. A missing `seq` is therefore not an error.
- The view's **structure** never changes through patches: no panel is added, removed or reordered, and the strip keeps its length. A Sutra edited while the view is open re-sends every changed panel as `panel` patches. If a client needs a new structure (another Sutra, other controls), it opens the view again.
- `deleted` freezes the view; ignore everything until `restored`, then fetch the view again.

### Reconnecting

Nothing is replayed. A client that reconnects receives a fresh `view` event and must repaint from it (the browser's `EventSource` reconnects by itself; the console reloads the page on a second `view`). The SSE `id` is not used to resume.

The console does not open one stream per page: each browser holds one channel to the console, which relays the server's streams and adds a rendered `html` field to `panel` patches of non-chart panels (Part III · Data architecture). That channel is the console's own; API clients use the server stream above.

---

## V6. Entitlements and masks in a view

Everything here is done on the server before the JSON leaves; a client only draws what arrives.

| Where | What the caller sees | Set by |
|---|---|---|
| A masked field (`drishti.security.redact`, callers without `raw`) | `"text": "•••"` wherever the value or anything computed from it appears: strip, title, every panel, totals, a gauge (`value: null`, `text: "•••"`), a pivot cell | `Entitlements.redactor` masks the document before binding |
| The count of masks | `provenance.masked` (how many values) and `provenance.maskedPanels` (which panels), so a page can say "some values are hidden" | `MaskCount.of` over title, strip and every panel's data |
| A `links` item to a kind the caller may not open | `status: "denied"`, `badge: "no access"`, no `link` (`inactive` / `not active` when the command is turned off) | `Entitlements.restrict` |
| A table cell linking to such a kind | the text stays, the `link` is removed (tone `inactive` when the command is off) | `Entitlements.restrict` |
| A function key to such a kind | label ends `(no access)`, `action: "denied"`, no `link` | `Entitlements.restrict` |
| The title's related entity | `title.with` keeps its text and loses its link | `Entitlements.restrict` |
| A panel whose `source:` is such a kind | `denied: "no access to <kind>"`, `empty: true`, no `data` | `Binder` (`SourceDeniedException`) |
| A control | never offers a masked value as an option | `ControlView` |
| A map's link or shade from such a kind | `link.note` / `shade.note` say why, no codes or items | `MapSupport` |

A masked view, real (`MTM`, `NOTIONAL` and five other fields masked for the caller):

```
{"strip": [{"label": "Notional", "text": "•••", "path": "$.CURRENCY"},
           {"label": "MTM (USD)", "text": "•••", "emphasis": true, "path": "$.MTM"}],
 "provenance": {"…": "…", "masked": 66, "maskedPanels": ["dv01use", "tree", "grid", "headline"]}}
```

and its gauge: `{"value": null, "max": 250000.0, "text": "•••", "label": null}`. People learn that a linked entity or a value exists without seeing it. A view of a kind the caller may not open at all is refused (`403 DRS-5002`), not sent masked.

---

## V7. How to use it

### Fetch

```
B=http://localhost:18480/api/v1
curl -s $B/views/trade/MX-20000001 | jq -c '{mnemonic, title: .title.id, panels: [.panels[] | {id, kind}]}'
```

- **Who you are.** With security on, `Authorization: Bearer <token>` (a signed JWT or a personal API token; see DEVELOPER_GUIDE, authentication (docs/guides/DEVELOPER_GUIDE.md#114-who-you-are-authentication)); with security off, `X-Drishti-User: <name>` or nobody (`anonymous`). The answer is masked for that caller.
- **Which day.** `X-Drishti-As-Of: 2026-09-29` or `?asOf=2026-09-29` (a snapshot: `provenance.live` is `false`), and `X-Drishti-Known-At` / `?knownAt=` for time travel (DEVELOPER_GUIDE, business date (docs/guides/DEVELOPER_GUIDE.md#115-which-day-the-business-date)).
- **Which layout.** `?sutra=<name>` (another Sutra of the kind) or `?design=<id>` (one of your Designs).
- **Controls.** `?ctl.<id>=<value>`.
- **Errors** are Problem Details with a `code`: `404 DRS-1001` (no such entity), `403 DRS-5002` (may not open the kind), `502 DRS-1003`, `504 DRS-1004` (error code table (docs/guides/API_REFERENCE.md#error-code-table)).

### Stream

```
curl -s -N $B/views/trade/MX-20000001/stream
```

The same headers and parameters. See section V5 and Part III · Data architecture.

### Preview an unsaved Sutra

`POST /api/v1/studio/preview` with `{yaml, kind, id}` (a real entity) or `{yaml, kind, id, document}` (pasted JSON) answers a ViewModel built from that Sutra, masked for the caller; `422 DRS-2001/DRS-2002` with `problems` when the Sutra is bad. This is how Sutra Studio and the Build workbench show a draft, and how a renderer can be tested against any Sutra without saving it.

### A minimal render loop

```
view = GET /api/v1/views/{kind}/{id}             # or the first "view" event of the stream
paint(view):
    draw title (pill, id, with as a link when with.link)
    for i, cell in view.strip: draw cell (label, text, tone, emphasis, link)
    grid = view.layout or two columns (main, right)
    for panel in view.panels:
        place panel by layout.place[panel.id] or panel.area; size by span and height
        frame = header(panel.title, panel.code, panel.key)
        if panel.wire.waits:   draw text panel.wire.waits
        elif panel.error:      draw error panel.error
        elif panel.denied:     draw "no access" in words
        elif panel.empty:      draw data.emptyLabel or "No data available"
        else:                  RENDERERS.get(panel.kind, fallback)(panel.data)
    draw keys (by action)

stream = SSE /api/v1/views/{kind}/{id}/stream
on "view":  if not first: paint(event.data)       # a reconnect: repaint from scratch
on "frame": for p in frame.patches (in order):
    strip      -> replace strip[p.index] with p.cell
    panel      -> replace the panel with id p.panel.id, re-render it
    provenance -> replace view.provenance
    deleted    -> freeze and grey the view, say when; ignore patches until restored
    restored   -> fetch the view again and repaint
```

`fallback` draws an unknown kind as its title and *This panel kind is not supported by this client* (see section V8).

### How the console maps a kind to a renderer

The console is the reference renderer. `drishti-console/web/templates/_macros/panels.html` has one macro per kind (first paint, server side, Jinja); `pbody(p)` dispatches on `p.kind`, after the `waits`, `empty` and `denied` checks above. Charts are drawn in the browser by `static/js/drishti_charts.js` (ECharts) from the same `data`, embedded in the page.

| Kind | Macro (panels.html) | Drawn by |
|---|---|---|
| `kv`, `status` | `fields(p.data.fields)` | HTML |
| `provenance` | inline `<dl class="prov">` (links the Sutra and the connector) | HTML |
| `table`, `ladder` | `pivotable(p)`, or `drill(p)` when `data.levels` | HTML |
| `tabs` | `tabs(p)` (inner panels through `panel()`) | HTML |
| `line`, `area` | `chart(p)` | drishti_charts.js |
| `hbar` | `bars(p.data)`, or `donut(p)` when `shape` is `donut` | HTML / drishti_charts.js |
| `surface` | `surface(p)` | drishti_charts.js (ECharts GL for `3d`) |
| `links` | `links(p.data)` | HTML |
| `markdown` | `<p>{{ p.data.text }}</p>` | HTML |
| `waterfall`, `histogram`, `scatter`, `candlestick`, `graph` | one macro each | drishti_charts.js |
| `timeline` | `timeline(p.data)` | HTML |
| `map` | `geomap(p)` | drishti_charts.js |
| `pivot` | `pivot(p.data)` | HTML |
| `metric` | inline | HTML |
| `gauge` | inline (`value / max` as a bar) | HTML |

Live: `static/js/drishti_live.js` applies `strip` patches in place (text and tone, then a flash), hands `line` and `area` panels new data (`drishti.updateChart`), swaps every other panel for the console-rendered `html` the channel adds, and handles `provenance`, `deleted` and `restored` as in section V5.

Elements 1.x (`<drishti-view>`) and the embed API v1 receive HTML rendered by the same macros, not the ViewModel (EMBED_GUIDE.md, ADR-021); Elements 2.0 and the embed API v2 receive the ViewModel itself and draw it with Drishti Render (EMBED_GUIDE.md › Drishti Elements: the design, section 19).

The client-side renderer (EMBED_GUIDE.md section 7) ports every row of this table to `static/render/<version>/m/` and is compared with these macros by `tests/test_render_parity_browser.py` and, on the console's own page in both modes, `tests/test_render_console_browser.py`. In phase 2 (built) the console's entity page draws with it when a client mode (since retired) is on, and Elements 2.0 do; the macros stay as the `server` path and are retired later (ADR-022). On a client page live panel patches come over a `vm:` subscription as panels, with no HTML, and are applied by the renderer.

---

## V8. Compatibility and versioning

### What exists today

- **The contract version.** Every ViewModel carries `"contract": "1.0"` (major.minor; `ViewModel.CONTRACT`, a required field of the schema, checked by `ViewModelSchemaTest`). The minor moves for additive changes (a field, a kind, a tone); the major for anything that breaks a client (a rename, a removal, a changed meaning). Drishti Render declares the majors it draws (`CONTRACTS = [1]`) and refuses another with `DRS-9105`; a ViewModel without `contract` (an older server) is 1.0. The server's own version is at `GET /api/v1/about`. The JSON Schema served at `GET /api/v1/viewmodel/schema` is the shape of the server you are talking to.
- **Additive changes only, in practice.** Fields have been added over releases (`controls`, `layout`, `wire`, `form`, `compute`, `provenance.masked`, ...) and none renamed or removed. Every addition is optional, so an older client keeps working.
- **Clients must ignore unknown fields.** The console does (Jinja reads only what it names). Do not validate a live server's answer with a schema from an older server and `additionalProperties: false`; validate against the schema the same server serves.
- **A new panel kind needs a fallback.** `metric` and `map` were added after the first twenty. A client must draw an unknown `kind` as a titled panel that says it is not supported, never fail the view.
- **New tones and new words.** A client that meets a tone, `action`, `status` or `op` it does not know should draw the value plainly (no tone), skip the key, show the status text as is, and ignore the patch, respectively.

### Proposed

**The client-side renderer is built (phase 1 and phase 2, 2026-10-30;** EMBED_GUIDE.md, ADR-022): Drishti Render (`static/render/1.0.0/`) draws every kind of section V4 from this JSON, applies section V5's patches, and is compared with the console's macros by a parity suite over every pack Sutra; the console's entity page draws with it behind a client mode (since retired), and Elements 2.0 receive this JSON from the embed API v2. Items 1 (on every ViewModel; not on frames yet) and 2 are built, 3 and 4 are not; item 5 is ADR-022. The proposals, which turn this document into a published contract:

1. **A contract version on every ViewModel and frame**: a top-level `"contract": "1.0"` (major.minor). The minor moves for additive changes (a field, a kind, a tone); the major for anything that breaks a client (a rename, a removal, a changed meaning). The schema's `$id` would carry the same version (`.../viewmodel-1.0.json`). **Built on every ViewModel**; frames and the schema's `$id` are not stamped yet.
2. **The renderer library declares the majors it understands** and refuses (with a clear message) a major it does not know, instead of drawing a wrong screen.
3. **A kind registry in the schema**: each kind with the version it appeared in, so a renderer can say "needs 1.3".
4. **Deprecation before removal**: a field to be removed stays for one minor with `"deprecated": true` in the schema.
5. **The client-side renderer revisits ADR-021's decision** ("no second renderer"): done, ADR-022 (one renderer, in the browser; the console moves onto it in phase 2), with parity tests that compare its output with the console's macros for every kind, every pack Sutra and the edge cases of this document (EMBED_GUIDE.md section 14).

---

## V9. The JSON Schema

### Served

`GET /api/v1/viewmodel/schema` (read only, `application/schema+json`, any signed-in caller) answers the JSON Schema (draft 2020-12) of the ViewModel. Its root is `{"$ref": "#/$defs/ViewModel"}`; `$defs` holds every record: `ViewModel`, `Cell`, `LinkView`, `PanelView`, `Provenance`, the 18 panel `data` records, their parts, `Tone`, and `Frame` and `Patch` for the live stream.

```
curl -s $B/viewmodel/schema | jq -c '{title, ref: ."$ref", defs: (."$defs" | keys | length)}'
curl -s $B/viewmodel/schema | jq -c '."$defs".PanelView.allOf[0]'
```

```
{"title":"ViewModel","ref":"#/$defs/ViewModel","defs":55}
{"if":{"required":["kind"],"properties":{"kind":{"const":"kv"}}},"then":{"properties":{"data":{"anyOf":[{"$ref":"#/$defs/Fields"},
{"type":"null"}]}}}}
```

### How it is produced

`ViewModelSchema.build()` (`drishti-engine/.../view/ViewModelSchema.java`) **generates** it from the Java records by reflection, the way `RachanaSchema` builds the Sutra schema served at `GET /api/v1/rachana/schema`:

- Every component of every record becomes a property; a field added to a record is in the schema at once.
- Each record's Jackson inclusion decides **required** and **nullable**: a component always written is required (and may be `null` if it is a reference); one left out when null, empty or default is optional and never `null`. That is why `PanelView.empty` is required and `PanelView.error` is not.
- Numbers may be `null`: the server writes NaN and the infinities as `null` (`JsonConfiguration`).
- Every record is closed (`additionalProperties: false`) **for this server**: see section V8 before using that against another version.
- Hand-written on top, because types cannot say them: the `data` record of each kind (`DATA_BY_KIND`, an `if/then` per kind on `PanelView`), the closed word lists (`Patch.op`, `KeyView.action`, `LinkItem.status`, `PanelCompute.state`, `Tabs.layout`, `Graph.layout`, `Surface.view`, `GeoMap.mode`), the tones, `PanelView.kind` (from `PanelKind`), and the fields the server always sends although the record allows them to be missing (`ViewModel.ref`, `title`, `strip`, `panels`, `keys`, `provenance`, `timings`; `PanelView.id`, `kind`; `Patch.op`).

**How it is kept true**

`ViewModelSchemaTest` (`drishti-server`) validates real ViewModels, serialised by the server's own `ObjectMapper`, against the served schema:

- the all-panels showcase (all 22 kinds; the test fails if a kind the schema names is missing from it);
- the reference views `IRS-48213`, `FXS-20931`, `CFT-77120` and `NS-NORTH-01`;
- the chart kinds on a broken document (`ChartPanelsTest`'s), and every kind with nothing to show (`empty: true`);
- a masked view (`provenance.masked` > 0) and a panel denied by its `source`;
- live frames: `strip`, `panel` and `provenance` patches from `PatchDiffer`, and `deleted` and `restored`;
- the schema itself: a `data` record for every `PanelKind` and every `PanelData` record used; and a negative case (an unknown field, an unknown kind, a gauge carrying a table's data, rows that are not a list must all fail).

No JSON Schema library is in the build, so the test carries a small validator of exactly the keywords the schema uses (`$ref`, `type`, `properties`, `required`, `additionalProperties`, `items`, `enum`, `const`, `anyOf`, `allOf`, `if/then`); it fails on any keyword it does not know, so the schema cannot outgrow it unnoticed. Any standard draft 2020-12 validator accepts the same schema.

---

# Part V · Shared screens

Four screens that replace several today, chosen by the product owner from mock-ups, followed by the controls screens (section S6) and the editors (section S7). Each folds duplicates into one and deletes the old in the same release (*lean & streamlined*). The mock-ups are in mockups/shared-screens/ and open in a browser; the editors they lead to are in EDITORS.md.

## Contents


1. One Build home
2. One Approvals queue
3. One Explain for every number
4. The trader home
5. Build checklist
6. Controls screens
7. The editors


## S1. One Build home

Mock-up: e1-build-home.html, option **A** as the home, **B** as the first-visit view, **C**'s lineage columns in A's lists.

- **One rail** grouped **Data** (connectors, lakes, tables and models, Python tables) · **Screens** (Sutras, packs, the UI kit) · **Analysis** (reports and boards, pivot views, functions) · **Work** (my drafts, waiting for review, recently changed), each with a count.
- The middle lists the chosen kind: name, kind, status with versions, **feeds / fed by** (lineage), last change and by whom; opening an item goes to its editor (EDITORS.md's three-pane shell).
- **First visit** (or by choice): four task tiles, *Bring data in · Shape it · Show it · Package it*, with "continue where you left off" and "waiting for you".
- One search across everything a person may build; + New opens the guided steps; what a person may build follows their role.
- Studio, BI Studio, the workbench and the pack tools redirect here and their separate homes are deleted.

## S2. One Approvals queue

Mock-up: e2-approvals.html, option **A**, with **B**'s *due soon* items pinned at the top.

- **One inbox** of everything waiting for me, of any kind: Sutras, tables and models, reports, trade corrections, pivot functions, VaR and exposure sign-offs, users and roles, API keys, captures. Filters: Data · Screens · Risk · Access; *Mine sent* and *Done*.
- **Due soon** (a sign-off with a deadline) pinned on top.
- The chosen item shows **what changes** (a diff, with the maker's reason), **its effect if approved** (e.g. a scope's VaR and limit use before and after, the reports that refresh) and **the rules** that allow me to decide.
- Approve; reject with a reason; or **ask a question** (a thread on the item). The maker can never approve their own. Everything audited; a count on the top bar; the per-kind queues redirect here and are deleted.
- A team board by stage (proposed, questions, approved, rejected) only if asked for later.

## S3. One Explain for every number

Mock-up: e3-explain.html, option **A**, with **B** as its preview and **C** through a link.

- Every computed number on every screen (VaR, ES, components, parametric VaR, exposure, FRTB-SA, SIMM, the cubes, Pivot cells, BI measures) has one Explain.
- **Preview** (hover, or `?`): a one-line card with the answer and links.
- **Drawer** (click, or `E`) on the right, the page still visible, four tabs, the same everywhere: **Formula** (with the actual values), **Inputs & lineage** (down to rows), **Why it moved** (since the previous date, steps that add up), **Workbook** (the live-formula Excel); plus checks (inputs complete, equal to the official capture).
- **Full page** from the drawer, for auditors and printing.
- Computed and explained on the server; a stored figure says it is stored. Today's explain panels (How calculated, the ? pop-ups, Why it moved and the others) fold into it and are deleted.

## S4. The trader home

Mock-up: e4-trader-home.html, option **A**, with **C**'s feed as a column and **B** one click from a book.

- **Five headline figures**: today's P&L (live), VaR against its limit, the biggest risk, breaches (and near ones), approvals for me.
- **My books**: live P&L, VaR and a limit-use bar each; a click opens the book position by position (P&L, sensitivities, VaR contribution).
- **Markets I watch**, live; **what changed since this morning** (new and amended trades, market, with *Why it moved*); a **live feed** column (trades booked, market moves, limits approached, approvals waiting).
- What a person sees follows their desk and role; every figure is computed on the server and ticks through the live hub, never in the browser; every number opens the Explain; the command line is always at the top. It is the Trading lens's home (configuration, not a new product); a pre-deal *what if I add this trade* box joins it, drawn from the Java pre-deal service (built: `POST /api/v1/risk/exposure/predeal`, EXPOSURE_GUIDE.md section 18.9) when its screen is built on the Elements libraries.

## S5. Build checklist

Each item ships with a click test for every button, its guide section, its tutorial steps (UI, command line and YAML) and its help entry, and deletes what it replaces.

| # | Screen | Replaces | Done |
|---|---|---|---|
| 1 | One Build home (rail, lists with lineage, first-visit tiles, one search) | Studio, BI Studio, the workbench, the pack tools' homes | ☑ (2.9.11.e: `/build`; BI Studio's own home `/bi` redirects to it, its four lists the home's For you strip) |
| 2 | One Approvals queue (inbox, due soon, diff, effect, rules, questions) | the per-kind approval queues | ☑ (built in 2.9.11.e: `/approvals`, `/api/v1/approvals`; the access kinds and the effect in risk numbers are under construction) |
| 3 | One Explain (preview, drawer with four tabs, full page) | the five explain panels | ◐ built for VaR, ES, parametric, exposure, the MR cube's cells and the Pivot's Why did it move? card (`GET /api/v1/explain`, `drishti_explain.js`); BI measures and Sutra metrics in the under-construction registry (`explain-*`) |
| 4 | The trader home (headline figures, books, markets, what changed, feed) | the generic home for the Trading lens | ☐ |

## S6. Controls screens: run status and sign-off, the breach queue, number honesty

The screens of the controls a risk function runs every day, chosen by the product owner from mock-ups (2026-11-05), built in 2.9.11.f (trust, RBAC, security and controls; the security rules themselves are settled in that release's design discussion). The mock-ups are in mockups/controls/; people, roles and API keys are in EDITORS.md sections 5 and 6; the Approvals queue and the Explain drawer they use are in Part V · Shared screens.

### S6.1 Run status and sign-off

Mock-up: f1-run-signoff.html, option **A** for a day's run, **B** as the overview, **C** as a tab.

- **A day's run** (VaR, exposure) as five steps: **inputs arrived → complete → checks → computed → signed off**, each green, amber or waiting. The checks are listed (positions not priced and their share of gross, a day-on-day move over its threshold with *Why it moved*, the backtest zone).
- **The sign-off panel**: who prepared it, who signs, by when, the signer's comment, and what signing means: the figures and their capture become **Official**.
- **Overview**: a grid of business dates × measures, each cell computed, awaiting sign-off, signed (by whom, when) or **restated**; a cell opens the day's run. **Timeline**: everything that happened to the run, as a tab of the day's run.
- Rules: "Official" only after sign-off; the signer is never the person who prepared it; warnings and the comment are stored with the capture; a change after sign-off is a restatement with its reason and a second sign-off; an overdue sign-off reaches the Approvals queue as *due soon*; everything audited.

### S6.2 One breach and escalation queue

Mock-up: f2-breaches.html, option **A** as the risk-control screen, **B** in the Approvals inbox, **C** as a tab.

- **One list** of open breaches and near-breaches (VaR limits and credit limits); the chosen one is a **case**: the figures with Explain and *Why it moved*, the **escalation ladder** (who was told when, the next level and when it is reached), the actions taken, and how it may be resolved: back under the limit, an approved temporary increase (four eyes), or a waiver with a reason.
- A breach escalated to a person appears at the top of **their Approvals inbox**.
- A **heat map** (limits × days, coloured by use) as a tab, for trends building toward a breach.
- Rules: **credit-limit use is computed from the engine, never stored**; near-breaches shown from a configurable threshold (e.g. 85%); escalation by time and level, configured; built on the one alert engine (today's separate alert lists fold into it); everything audited.

### S6.3 Number honesty: computed, stored, official

Mock-up: f3-honesty.html, mixed by context.

| Status | Means | Headline tiles and cards | Tables, trees, the pivot |
|---|---|---|---|
| **Computed** | by Drishti, from its inputs, now | badge `COMPUTED` | green dot |
| **Stored** | a source's figure as delivered, not recomputed | badge `STORED · as delivered` | amber dot |
| **Official** | signed off (section S1) | badge `OFFICIAL · signed <time>` | blue dot |

- One legend per screen with dots; colour is never the only cue (each dot's tooltip says the word).
- The Explain drawer's first line states the status, with the source and the time.
- The server sends each figure's status; the console only displays it; a guard test fails if a figure is drawn without one.
- Exports carry a status column. A stored figure whose computation is under construction keeps the registry's chip (UNDER_CONSTRUCTION.md).

### S6.4 Build checklist

Each item ships with a click test for every button, its guide section, its tutorial steps and its help entry, and deletes what it replaces.

| # | Screen | Replaces | Done |
|---|---|---|---|
| 1 | Run status and sign-off (the day's run, the grid, the timeline) for VaR and exposure | the capture pages' partial status | ☐ |
| 2 | One breach and escalation queue (cases, ladder, resolve, inbox, heat map) | the separate alert lists for limits | ☐ |
| 3 | Number honesty (badges, dots, legend, the drawer's first line, export column, guard test) | unlabelled stored figures | ☐ |

## S7. The editors: one pattern for every object people author

How Drishti's authoring screens look and behave: tables and virtual tables, semantic models, Python-driven tables and models, people and roles, and API keys. One shell and one set of rules, so a person who has learned one editor knows them all. The product owner chose each design from mock-ups (2026-11-05); the mock-ups are in mockups/editors/ and are opened in a browser.

Which release builds what: tables, virtual tables, semantic models and Python-driven tables and models come with the BI foundation; people, roles and API keys come with trust, RBAC and security, whose rules are settled in that release's design discussion (this document fixes only their screens).

### S7.1 The shell every editor shares

Mock-up: g1-editor-shell.html, option **C** as the home, option **B** for every + *New*.

- **Three panes.** Left: the objects (tables, models, Python tables) with search. Middle: the editor, with the **live YAML below it**, the line being edited highlighted. Right: a **live preview with checks** (rows, totals computed by the pivot engine, warnings, impact).
- **A Visual / YAML / Both toggle**, remembered per person. YAML edits validate as typed and show back in the visual side; a layman can keep the YAML collapsed.
- **The header**: where you are, the name, the status with versions (*draft v4*, *approved v3*), and Validate, Preview, History, Impact and **Propose for review**.
- **+ New** opens guided steps in plain language (option B); when the object exists it opens in the three-pane editor.
- **As built**: the shell is drishti-designer's `DesignerShell` and the YAML its `YamlPane` over drishti-inputs' `CodeEditor` (ELEMENTS.md section 8.9), the same for every designer: the table, model and Python editors, the Build workbench's canvas and YAML, the BI report designer's canvas, undo and YAML. Its look is mock-up g7's: **studio** (option A, the default: the left rail's tabs, the canvas with the YAML docked under it, the inspector, the preview), **side by side** (B: the canvas and the YAML side by side, the inspector a card by the selection) and **terminal** (C: a command line on Ctrl+K and the function keys F3 Validate, F4 YAML, F5 Preview, F7 History, F8 Impact, F9 Propose); each person's layout, mode and whether the YAML is open are kept by the server. A YAML edit is read on the server after a pause and each problem is marked at its line (a sign in the gutter, the list under the editor); the visual side keeps the last valid version.

### S7.2 Tables and virtual tables

Mock-up: g2-tables.html, option **A** as the editor, option **C** as its Text toggle.

- **The source and its look-ups as rows**; **columns as a checklist** with the name here, where it comes from, its type, and computed columns as Rachana formulas. A look-up brings one row of another table by a key (many to one: `related(table.COLUMN, on = key)`). A plain table is the same editor with one source and no look-up.
- **Tables use look-ups; models join** (product owner, 2026-11-06): a table enriches its rows with look-ups and never multiplies them; combining tables row by row (one to many, several facts) is a semantic model's relationships (section S3).
- **Text toggle**: the same table as one Rachana statement (`table … from … select … where …`, look-ups as `related(…)` columns) with type-ahead.
- **A look-up across connectors shows its cost** (rows each side, memory, time) and is refused above a limit until the reader filters.
- The diagram (option B) is not an editing mode; it may appear later as a read-only picture of the look-ups and a model's relationships.

### S7.3 Semantic models

Mock-up: g3-models.html, option **A** as the editor, **B** as the Overview and the + *New* steps, **C** for a measure's details and the Access tab.

- **Tabs**: Overview, Tables, Fields, Measures, Hierarchies, Row rules, Masks, Access, each with a count.
- **Overview** answers four questions: what it is about, what people look at it by, what numbers it gives, who may see what. The same four questions are the + *New model* steps.
- **A measure's ?** opens its card: the formula, a plain explanation, a worked test value, and where it is used.
- **Access** is a matrix of roles against rows, columns (masked or not) and editing.
- Whether a total adds up is the engine's answer, never the author's: VaR, ES and ratios are computed per total.

### S7.4 Python-driven tables and models

Mock-up: g4-python.html, option **A** as the editor, **B** (the Notebook) to explore, **C** (templates) for common cases; all three end in A.

- **Code beside its output contract**: the columns, types, keys and rules the code must return. The inputs, when it runs (on demand and cached per business date, or on each load of an input), and its compute pool and limits are on the same page; a sample run and its checks on the right.
- **Notebook → Turn into table**: cells explored on the server become a build function, and the contract is filled in.
- **Templates** (join and derive, resample, statistics per group, a file or feed from an approved source) write the Python and show it.
- A Python **model** is the same, with its measures declared in the contract.
- **Where it runs.** The sample run, the Notebook's cells and the table's rows when read all run on the server's compute pool `python`, in its strict sandbox: the inputs read as the person running it (row rules and masks), pinned libraries, the code's time and memory limits, no network (an approved source is read by the server and handed in as an input), files only in its own folder, every run audited; the rows are checked against the contract and a mismatch is refused in words (BI_GUIDE.md section N19). Where the server cannot run Python safely (compute off, Windows) the buttons are off and say why.

### S7.5 People and access

Mock-up: g5-people.html; all three options, as the three tabs of **Admin → People & access**.

- **People**: a searchable list; a person's page with sign-in (directory, MFA, last seen), roles, scope (packs, desks, lens), powers, API keys, recent activity, and **what this person sees**.
- **Roles**: every role against every power on one page; built-in roles (from configuration) are locked.
- **Check access**: *can this person do this?* answered yes or no **with the reason** (the role, the rule and the attribute that decided it) and how to change it.

### S7.6 API keys

Mock-up: g6-apikeys.html; all three options.

- **My keys** (everyone): what each key acts as (its owner, never more than the owner's rights), its scopes in words, its limits (only these models, only from this network), expiry and **last use** (when, from where, what), Rotate and Revoke.
- **+ New key** in steps: purpose, what it may do, limits and expiry, then the secret **shown once** (Drishti keeps only a fingerprint).
- **Admin → API keys**: every key with owner, scopes, expiry, last use and flags (expiring, unused, used from outside the network), bulk revoke, and the key policy (maximum lifetime, unused keys revoked, a rotation warning).

### S7.7 Rules every editor follows

- **Nothing goes live without review**: a change is proposed and approved by another person (four eyes), versioned and audited.
- **Impact before change**: every editor shows what uses the object (models, Sutras, reports, boards, keys) before it is changed.
- **One grammar**: every formula, filter, row rule and computed column is Rachana, typed in drishti-inputs' code editor (one line for a field, the whole text for the statement) with the function catalogue and the editor's names as its completion.

- **Built from Drishti Elements, never by hand** (ELEMENTS.md section 3.1): every pane of fields is drishti-inputs' `Form` (live, with the formula widget for a Rachana field: the table's name, rows and look-ups, the model's Overview and masks, the Build workbench's inspector through drishti-designer's `Inspector`); the rows of a table of settings (a model's tables, relationships, measures, hierarchies, row rules, fields and access matrix; a table's source and look-ups and its columns checklist) are drishti-grid with cell editing (`E.rowGrid`): Enter, F2, a double click or typing edits a cell, Enter saves, Escape cancels, Ctrl+Z undoes; a select of the offered values, a checkbox that turns at once, and a formula cell that is drishti-inputs' one-line Rachana code editor completing from the editor's names and the functions (the grid's `code` editor). Each edit, paste or undo is one change of the document; a row's ?, Remove and the engine's "adds up?" are cells the editor draws.
- **One entitlement service**: row rules and masks are enforced on the server for every reader (BI, Pivot, VaR, Flight, the API).
- **Computed on the server**: previews, totals and Python runs happen on the server; the console only displays.
- **Under construction**: an editor or a part of one that depends on unbuilt work is hidden or marked from the registry (UNDER_CONSTRUCTION.md).

### S7.8 Build checklist

Each item ships with its tests (a click test for every button), its guide section, its tutorial steps (UI, command line and YAML) and its help entry.

| # | Editor | Built in | Done |
|---|---|---|---|
| 1 | The three-pane shell, the Visual / YAML / Both toggle, the header, + New steps | BI foundation | ☑ |
| 1a | The shell on the designer kit (DesignerShell, YamlPane) in g7's three layouts; every designer on the kit (ELEMENTS.md 12.3) | Drishti Elements | ☑ shell, YAML; ☐ the editors' own panes |
| 2 | Tables and virtual tables (grid, Text toggle, cross-connector cost) | BI foundation | ☑ |
| 3 | Semantic models (tabs, Overview, measure cards, Access matrix) | BI foundation | ☑ |
| 4 | Python-driven tables and models (code and contract, Notebook → table, templates, the sample run on the compute pool) | BI foundation | ☑ |
| 5 | People & access (People, Roles, Check access) | Trust, RBAC and security | ☐ |
| 6 | API keys (My keys, + New key, Admin overview) | Trust, RBAC and security | ☐ |

## Part VI · Decisions

The architecture decisions, one paragraph each: what was decided, why, and its status. Each was decided by the product owner; a decision is amended, never rewritten, and a later one that replaces part of an earlier one says so. The short table of the key decisions is section A21.

**ADR-001 · The backend is one Spring Boot application** (accepted). Drishti ships only as `drishti-server`, never embedded; Maven modules are internal building blocks, each contributing its beans through its own `@Configuration`; domain classes stay constructor-injected and unit-test without a Spring context; the plugin SPI (`drishti-api`) stays Spring-free. *Why:* one deployable, with Spring's configuration, metrics and lifecycle everywhere; embeddability was given up because no user needs it. ArchUnit enforces one-way module dependencies and the Spring-free SPI.

**ADR-002 · A layout is a Sutra plus inference** (accepted). A view's layout is the merge of an optional declarative Sutra and inferred panels, cached by the pair (Sutra and version, shape fingerprint). *Why:* hand-built screens per product do not scale; unknown data renders on day one, the inference cost is paid once per data shape and warm requests only bind values.

**ADR-003 · A Sutra is YAML with a closed expression language** (accepted; amended by ADR-017). Bindings use Rachana-EL, a closed, side-effect-free language (paths, filters, ternary, arithmetic, a few functions) compiled to closures. *Why:* layouts need bindings, conditions and small computations without becoming programs; they stay reviewable, diffable and safe. Adding a function is a decision of its own.

**ADR-004 · The server returns a view model** (accepted). The server returns a JSON ViewModel, the bound layout, not HTML; its structure is Part IV, its JSON Schema `GET /api/v1/viewmodel/schema`. *Why:* the same model feeds the console, API clients and golden tests, and live patches address ViewModel paths.

**ADR-005 · Server-sent events for live updates** (accepted). Live views use Server-Sent Events with `Last-Event-ID` resume, not WebSockets. *Why:* the channel is one-way and proxy-friendly, browsers reconnect by themselves, and the client never needs to push over it.

**ADR-006 · A vendored front end without a build pipeline** (accepted). Every library is vendored: no CDN, no npm, no inline script (strict CSP). *Why:* desks can be air-gapped and a JavaScript build chain adds weight; the console works offline and upgrades mean copying files deliberately.

**ADR-007 · Build gates in tests** (accepted, amended). `drishti-it` holds `LicenseHeaderTest`, `SourceFileSizeTest` (2000 lines, UX templates exempt) and `ArchitectureRulesTest`; `tools/license_headers.py --fix` inserts headers; the parent POM is `spring-boot-starter-parent` (no separate BOM). Error Prone runs in every compile (its error-level checks, with `NonAtomicVolatileUpdate`, `LockNotBeforeTry` and `JavaTimeDefaultTimeZone` raised to errors; an intentional exception carries `@SuppressWarnings` with the reason)

and Spotless checks at `verify` what `.editorconfig` promises (no unused imports, no trailing whitespace, a final newline, spaces not tabs; `./mvnw spotless:apply` fixes it); no full reformatter, which would rewrite the wider-line style. *Why:* header, size and style rules must not depend on developer discipline; a plain `./mvnw verify` enforces them.

**ADR-008 · Rachana is the grammar, a Sutra is a layout** (accepted; amended by ADR-017). **Rachana** is the language, its schema, its expression language (Rachana-EL) and the module `drishti-rachana`, configured under `drishti.rachana.*` and documented in RACHANA_REFERENCE.md; a **Sutra** is one versioned layout written in it (`sutras/<domain>/<name>.v<N>.sutra.yaml`), modelled by `Sutra` and loaded by `SutraRegistry`. *Why:* both names mean what the product concept and the mock-ups say (`Sutra irs-vanilla v3 + inference`).

**ADR-009 · Users live in the server** (accepted). Users live in `drishti-identity` behind the `UserStore` interface (PBKDF2-HMAC-SHA256 hashes); the console verifies a sign-in through `POST /api/v1/auth/login` with a service token and receives only the profile; administration goes through `/api/v1/admin/**` (the `admin` role); every event is audited; an empty store seeds a development administrator and the console warns until its password changes. *Why:* full user management (create, disable, reset, audit, lockout) needs one shared, writable source of truth however many consoles there are; a database-backed store and single sign-on map onto it without changing the API.

**ADR-010 · Industries are domain packs** (accepted). Everything industry-specific lives in a pack (`config/packs/<domain>/<name>/`): its `pack.yaml` vocabulary beside its Sutras, hints, formats, samples, starters and guides. `drishti-packs` makes the enabled packs the lowest-precedence property source, so no core module depends on a pack and site configuration always wins; pack clashes are start-up errors. *Why:* "any domain" must be true of the product, not only the engine; a new industry is content and configuration, and an unknown domain still renders, more plainly, until its pack adds vocabulary.

**ADR-012 · Every read is for a business date; history lives in Delta Lake** (accepted). The top bar carries a business date: **Live** (the default; the current business date on the configured calendar) streams, a **picked date** is a static snapshot; the console only stores the choice and sends it as `X-Drishti-As-Of`. `AsOf(businessDate, knownAt, live)` reaches every read through dated overloads of `fetch`, `reverse` and `search` that default to the undated ones; a dated source declares `SourceCapabilities.dated` and stamps `Provenance.businessDate`, and an undated source answering for a past date shows "No data held for " rather than passing current data off as history. For a picked date dated sources are tried first. Delta Lake is read through Delta Kernel (Java, no Spark): one table per kind, `(id, doc)` rows partitioned by `business_date`; snapshot tables take the newest partition on or before the date within a lookback, effective tables the last change on or before it, and `knownAt` uses time travel. Named connectors run a plugin several times (one lake per domain); the file plugin reads dated folders, the JDBC plugin binds `:asOf`. *Why:* risk and trading users read end-of-day data, and sources hold history very differently; history then costs no code per domain.

**ADR-013 · Sutras go live through review** (accepted). With governance on (the default) a save is a proposal (the text, the author's note and the live text it was written against); an approver approves (written and hot-reloaded) or rejects with a reason; nobody approves their own (`drishti.governance.four-eyes`); approval is refused when the live Sutra changed after the proposal; proposals are kept and every step is audited. Sutras committed through version control are reviewed there. *Why:* a Sutra decides what every user sees, and a change of that reach needs a second pair of eyes and a record. Single-person setups switch four-eyes or governance off.

**ADR-014 · Single sign-on, verified by the server** (accepted). OpenID Connect, authorization code flow with PKCE; the console hands the ID token and nonce to `POST /api/v1/auth/oidc` (its service identity only), and the server verifies signature (JWKS from the issuer), issuer, audience, authorised party, expiry, not-before, issued-at and nonce, with JDK cryptography only (RS, PS and ES families; never `none` or shared-secret HMAC). Groups become roles through `drishti.security.oidc.role-map`; users are created on first sign-in; a user disabled in Drishti stays out; every sign-in, provisioning and refusal is audited; password sign-in stays for break-glass administrators. *Why:* banks sign staff in through a corporate identity provider, and a compromised console must not be able to sign anyone in.

**ADR-015 · Packs inherit, and the more specific pack wins** (accepted). `extends: [a, b]` (with `requires:` read the same way) inherits everything the parents bring; precedence is C3 linearisation (the child over its parents, the rightmost parent over those to its left, a shared ancestor once); an inconsistent order or a cycle stops the server; overrides are global, logged and shown under Admin → Health; a kind belongs to exactly one pack and is never overridden; a more specific pack's Sutra with the same `name@version` replaces the parent's, and site Sutra folders still win. *Why:* real domains layer (counterparty risk is trading and market data, with more), so a pack must be able to specialise its parent without forking it.

**ADR-016 · One console, many servers; the user picks one and signs in to it** (accepted, built). The console's `servers:` catalogue names each server (URL, its own token secret, its own single sign-on, `listed` or reachable only by `/connect/<id>`); `backend.url` is the one-entry form. The user picks a server and signs in to it; each server keeps its own users, roles, packs and audit and decides alone. The chosen server is kept per browser (cookie `drishti_server`); each server's session has its own signed cookie (`drishti_session_<id>`), so switching back needs no new sign-in; links name their server (`/connect/<id>?next=…`, `?srv=<id>`); the picker reads each server's public `GET /public/about` concurrently. In the console every per-user cache and every task started for a person (live channels, pack jobs, uploads, AI limits) is keyed and scoped by (server, user) through `core/request_context.py`; a live channel re-checks its session every `auth.recheck_seconds` and ends with `DRS-5010` for a signed-out or disabled user. *Why:* organisations run several servers side by side, open and exclusive, while the console stays open to all; one secret per server limits a leak to one server, and a server id not in the catalogue is refused. A federated session, a console per server and exclusive packs on one server were considered and rejected.

**ADR-017 · Sutras are YAML, through and through** (accepted; supersedes ADR-011, amends ADR-003 and ADR-008). One file `<name>.v<N>.sutra.yaml`, first key `rachana: 1` (missing or unknown: `DRS-2009`); prose in plain-text `description` and `notes` keys, never evaluated, and nothing derivable written down; every unknown key an error with line and column (`DRS-2011`); a `.sutra.md` or plain `.yaml` in a Sutra folder reported (`DRS-2004`) with the fix; the JSON Schema served at `GET /api/v1/rachana/schema`, generated from the

grammar with this server's kinds and formats; `tools/rachana/md_to_yaml.py` converts old files; a complete Sutra in a document is a `yaml` block starting `rachana: 1`, parsed by the tests. *Why:* two dialects of one language had to be learned, tooled and tested twice, ordinary YAML tooling could not see a Markdown Sutra, and the prose went stale; an incompatible grammar now means a new `rachana:` version with a converter.

**ADR-018 · Packs come from a signed, versioned registry** (accepted, built). A registry is plain files (`index.json` and one zip per pack version) over HTTPS or a folder; every archive is signed with Ed25519 by its publisher (`tools/packreg`), and a server installs only from publishers in `drishti.packs.registry.trusted-keys`; installing checks size, SHA-256, signature, paths and unpacked size and that `pack.yaml` names the same pack and version, then installs into `drishti.packs.installed-dir` with the usual pack check and in-place restart; the replaced version is kept for **Roll back**; archives are reproducible; plain `http:` is refused; installs are audited. *Why:* packs carry Sutras, connectors and roles, so a pack from the wrong hands can point a connector at the wrong data or widen a role. Maven or PyPI packages, git checkouts and checksums without signatures were rejected.

**ADR-019 · Pricing is a QuantLib service behind the Drishti server** (proposed; a later candidate, not in a planned release). A separate stateless service, `drishti-quant` (Python with QuantLib's wheels), reachable only from the server; the server is its only client (checks a `quant` power and the right to open the entity, applies redaction, assembles market data for the date, caches by entity version, date, model and scenario, records each pricing); packs declare how products are priced (`quant:` in `pack.yaml`). *Why:* a model NPV, Greeks and scenarios need curve building and calibration that a browser cannot do, and a native crash must not touch the server. QuantLib in the server or in WebAssembly was rejected.

**ADR-020 · Collaboration records; the notice is a link, never data** (accepted, built; the store revised). Shares and comments live in the content store by default (`drishti.collab.store: content`; `jpa` or `file` for one node), so every cluster member sees them (section D5). Delivery is a transactional outbox (the record, in-app notices, mail rows and the access-log row in one transaction; a dispatcher claims rows under a lease, renders each mail for its recipient at send time and retries). A share stores the note and a pin (date, "known at", generation, source), never a copy of the data, so every reader gets the view built for their own rights; masked values typed into a note are recorded as spans and shown `•••`; mail carries the kind, the id and the note as that recipient may read it. Comments are never overwritten: each revision is a row hashed with the one before (SHA-256 per thread); retention, legal holds and the compliance export are an operations matter. *Why:* people discuss data elsewhere and lose the date and the context, and sometimes leak values; a separate store, a stored snapshot and an in-memory queue were rejected.

**ADR-021 · Embedding: reports drawn by the viewer's own code, the embed token on every call** (accepted; its first decision, views as HTML over the wire, superseded by ADR-022). A report in `<drishti-report>` is drawn by the same code as the console's viewer, served with CORS under `/embed/v1/js/`; the host's backend exchanges its credentials and user for a short-lived, read-only token (RFC 8693), verified on every call with the user's row rules and masking always on; which hosts may show which kinds and reports is the host application's `kinds` list; the console serves the library and the npm package is the same bytes (immutable exact versions with SHA-384 `integrity.json`, a committed record that fails the build when a module changes under an unchanged version); the element API and the embed API are semantic-version contracts. *Why:* hosts want Drishti inside their own pages, signed in as their own users, without an iframe; one renderer of a report means every fix reaches embedded reports with no second change.

**ADR-022 · One renderer, in the browser** (accepted, built). Views are rendered in the browser from the ViewModel by one library, Drishti Render (plain ES modules, no build step, vendored dependencies only); there is only ever one renderer; the sanitiser's rules are kept while building (no markup parsed from data, the same allow-lists, no `eval`, one Trusted Types policy); navigation stays in the host's container by default; the same `render()` draws a static, a live and a streamed ViewModel (skeleton, panel-add, panel-replace, patch, done, error). Parity is a test (`tests/test_render_parity_browser.py`). *Why:* the ViewModel became a published contract, hosts want screens without the console's HTML, and GenUI needs screens that arrive panel by panel; the payload is smaller than HTML. Keeping HTML over the wire, a build toolchain and a renderer per framework were rejected.